

SQL Server 2022

Apprendre à administrer un serveur de base de données

Ce livre s'adresse à toute personne désireuse d'apprendre à administrer un serveur de **base de données** SQL Server 2022 **administrateur de base de données** mais aussi **développeur**.

Il présente les différents éléments nécessaires à cette administration ainsi que l'ensemble des manipulations à réaliser par l'administrateur, depuis l'**installation** jusqu'aux opérations de **sauvegarde** et de **restauration**, en passant par la gestion de l'**espace disque**, la gestion des **utilisateurs**, la gestion de la **réplication**.

Les différents outils permettant l'**optimisation** du serveur sont présentés, tels que l'**analyseur de**

performances, SQL Profiler et l'assistant de paramétrage de base de données.

Les différentes opérations sont réalisées depuis SQL Server Management Studio et en Transact SQL.

Des **exercices avec leurs corrigés** sont proposés au lecteur pour une mise en pratique immédiate des concepts présentés. Des éléments complémentaires sont en téléchargement sur cette page.

Auteur(s)

Jacques POIRIER

Certifié Microsoft Certified Trainer (MCT) sur Windows et sur SQL Server depuis la version 6.5, **Jacques POIRIER** est formateur et Consultant chez ENI Service depuis plus de 20 ans et DBA du groupe ENI depuis 2020. Au quotidien, il dispense des formations dans les

domaines des systèmes et des réseaux, des bases de données avec SQL Server ainsi que de la Business Intelligence (avec SSIS, SSAS et SSRS). Il accompagne des stagiaires dans leur montée en compétences depuis de nombreuses années et assiste également les clients sur des besoins d'audit et/ou d'optimisation de serveurs SQL Server. Il gère également l'ensemble des serveurs de bases de données du groupe ENI. Cette expérience, couplée à une parfaite connaissance des produits, confère à son discours pédagogique une clarté et une pertinence qui profitent à tous.

Réf. ENI : RI22SQLA | ISBN : 9782409039744

Introduction

Cet ouvrage présente l'administration de SQL Server et, plus particulièrement, offre une introduction précise aux différents concepts d'administration d'une base de données.

Les actions nécessaires à la réalisation des tâches d'administration quotidiennes sont étudiées en termes de concept, puis l'implémentation de la fonctionnalité telle qu'elle est mise en place dans SQL Server est détaillée. Dans le cadre des opérations les plus fréquentes, le choix est fait de présenter les manipulations au travers de l'interface graphique SQL Server Management Studio, mais aussi les instructions Transact SQL associées. Dans le cadre d'opérations ponctuelles, seul le mode opératoire via l'interface graphique est proposé.

Ce livre s'adresse particulièrement à toute personne désirant appréhender l'administration de bases de données au travers de SQL Server. Il n'est pas nécessaire d'avoir des compétences relatives à

l'administration de bases de données avant d'aborder ce livre.

Pour exploiter parfaitement les différentes instructions présentes dans cet ouvrage, une connaissance du Transact SQL et donc du langage SQL est souhaitable. Cette connaissance du Transact SQL permettra au lecteur de mieux comprendre et exploiter les différentes instructions Transact SQL présentées tout au long de ce livre. D'autre part, cette connaissance permettra également de mieux comprendre comment peuvent être utilisées les données et, bien entendu, de faire les bons choix en termes d'administration. Il faut donc bien savoir comment sont utilisées les données présentes dans la base pour les administrer au mieux.

Après avoir détaillé les choix possibles lors de l'installation, les possibilités offertes en termes de stockage, de sécurité, de sauvegarde, de restauration, de tolérances de pannes et de répartition des données sont présentées. Par la suite, un chapitre ouvre des pistes vers l'optimisation du serveur.

Introduction

Depuis quelques versions déjà, SQL Server est arrivé à une certaine maturité en ce qui concerne la gestion des données relationnelles, et les évolutions les plus notables sont sur la partie BI mais aussi sur l'intégration de SQL Server à Windows Azure (ces deux points n'étant pas l'objet de ce livre).

Cependant, cela ne signifie en aucun cas qu'il n'y a pas d'évolution au niveau de SQL Server.

Tout d'abord, en ce qui concerne la distribution, l'installation de l'outil client SQL Server Management Studio est complètement détachée de l'installation du moteur de base de données. Ainsi, SQL Server Management Studio doit être téléchargé, gratuitement, à partir du site web de Microsoft.

En complément de l'évolution de quelques commandes, les outils progressent également avec un souci d'intégration. Ainsi, SQL Server Profiler est déprécié, donc existe toujours, mais se voit remplacé

par les événements étendus.

De même, le magasin de requêtes permet de mieux suivre l'évolution des temps d'exécution des requêtes ainsi que de s'assurer de la bonne santé du serveur avec des indicateurs comme le temps de consommation processeur, le nombre de lectures logiques, le temps d'exécution des requêtes...

SQL Server propose aussi les tables temporelles, ce qui permet de suivre l'évolution des données dans le temps. La durée de rétention de l'historique est gérée par SQL Server ainsi que la bascule des informations de la table des données vers la table d'historique.

Toute la technicité nécessaire à la mise en place de cette fonctionnalité est assurée par SQL Server.

Présentation de SQL Server

Le but de ce chapitre est de présenter SQL Server dans sa globalité et d'acquérir un aperçu de SQL Server dans son ensemble, à savoir :

- Comprendre la notion de SGBDR et le mode de fonctionnement client/serveur.
- Présenter les composants de SQL Server et les plates-formes d'exécution.
- Présenter l'architecture d'administration et de programmation.
- Présenter la notion de base de données et les bases installées sur le serveur SQL.

SQL Server est un SGBDR (système de gestion de base de données relationnelle) entièrement intégré à Windows, ce qui autorise de nombreuses simplifications au niveau de l'administration, tout en offrant un maximum de possibilités.

1. Qu'est-ce qu'un SGBDR ?

SQL Server est un système de gestion de base de données relationnelle (SGBDR), ce qui lui confère une très grande capacité à gérer les données tout en conservant leur intégrité et leur cohérence.

SQL Server est chargé :

- de stocker les données,
- de vérifier les contraintes d'intégrité définies,
- de garantir la cohérence des données qu'il stocke, même en cas de panne (arrêt brutal) du système,
- d'assurer les relations entre les données définies par les utilisateurs.

Ce produit est complètement intégré à Windows et ce à plusieurs niveaux :

- Observateur d'événements : le journal des applications est utilisé pour consigner les erreurs générées par SQL Server. La gestion des erreurs est centralisée par Windows, ce qui facilite le diagnostic.

- **Analyseur de performances** : par l'ajout de nouveaux compteurs, il est facile de détecter les goulots d'étranglement et de mieux réagir, pour éviter ces problèmes. On utilise toute la puissance de l'Analyseur de performances, et il est possible au sein du même outil de poser des compteurs sur SQL Server et sur Windows et ainsi d'être à même de détecter le vrai problème.
- **Traitements parallèles** : SQL Server est capable de tirer profit des architectures multiprocesseurs. Chaque instance SQL Server dispose de son propre processus d'exécution et des threads Windows ou bien des fibres (si l'option est activée) sont exécutés afin d'exploiter au mieux l'architecture matérielle disponible. Chaque instance SQL Server exécute toujours plusieurs threads Windows. Pour prendre en charge tous les processeurs présents sur le système, le paramètre de configuration **max degree of parallelism** doit conserver la valeur 0. Il s'agit de la valeur par défaut. Pour empêcher la génération de plan d'exécution parallèle, il suffit d'affecter la valeur 1 à ce paramètre. Enfin en lui affectant une valeur comprise entre 2 et le nombre de processeurs, il est possible de limiter le degré de parallélisme. La valeur maximale supportée par ce paramètre est 64.
- **Sécurité** : SQL Server est capable de s'appuyer intégralement sur la sécurité gérée par Windows, afin de permettre aux utilisateurs finaux de ne posséder qu'un nom d'utilisateur et un seul mot de passe. Néanmoins SQL Server gère son propre système de sécurité pour tous les clients non Microsoft.
- **Les services Windows** sont mis à contribution pour exécuter les composants logiciels correspondant au serveur. La gestion du serveur (arrêt, démarrage et suspension) est facilitée et il est possible de profiter de toutes les fonctionnalités associées aux services de Windows (démarrage automatique, exécution dans le contexte d'un compte d'utilisateur du domaine...).
- **Active Directory** : les serveurs SQL et leurs propriétés sont automatiquement enregistrés dans le service d'annuaire Active Directory. Il est ainsi possible d'effectuer des recherches dans Active Directory pour localiser les instances SQL Server qui fonctionnent.

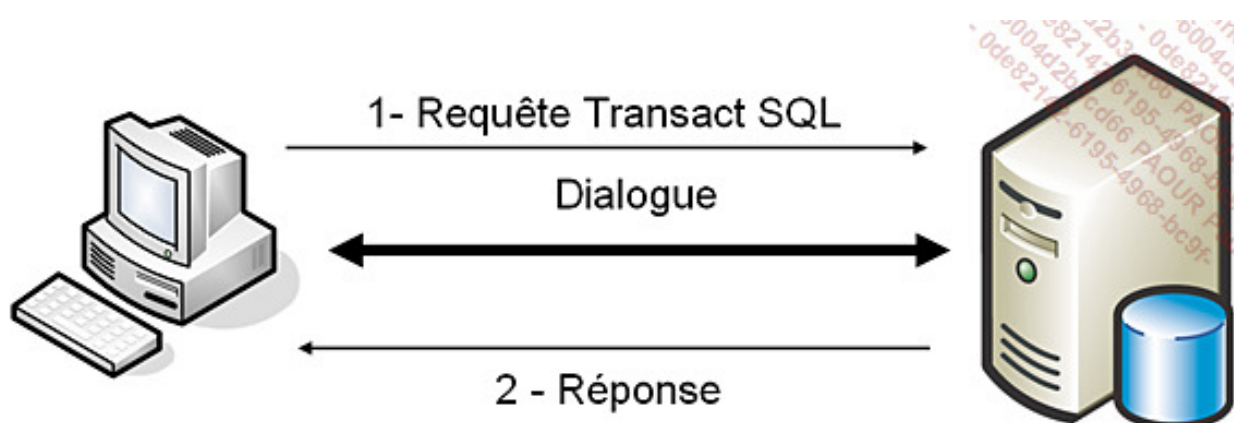
SQL Server peut gérer deux types de bases de données différentes :

- **Les bases OLTP** (*OnLine Transactional Processing*) qui correspondent à des bases utilisées par les applications courantes des utilisateurs (ERP, CRM...). Les données sont en lecture/écriture.
- **Les bases OLAP** (*OnLine Analytical Processing*) qui contiennent des données

agrégées sous forme de cube multidimensionnel dans un but d'aide à la décision, par exemple. Les données contenues dans des bases OLAP sont rechargées régulièrement à partir de données contenues dans une base OLTP. Ce type de bases est géré par le service SSAS (*SQL Server Analysis Service*), qui ne fait pas l'objet de cet ouvrage.

2. Mode de fonctionnement client/serveur

Toutes les applications qui utilisent SQL Server pour gérer les données, s'appuient sur une architecture client/serveur. L'application cliente est chargée de la mise en place de l'interface utilisateur. Cette application s'exécute généralement sur plusieurs postes clients simultanément. Le serveur, quant à lui, est chargé de la gestion des données, et répartit les ressources du serveur entre les différentes demandes (requêtes) des clients. Les règles de gestion de l'entreprise se répartissent entre le client et le serveur.



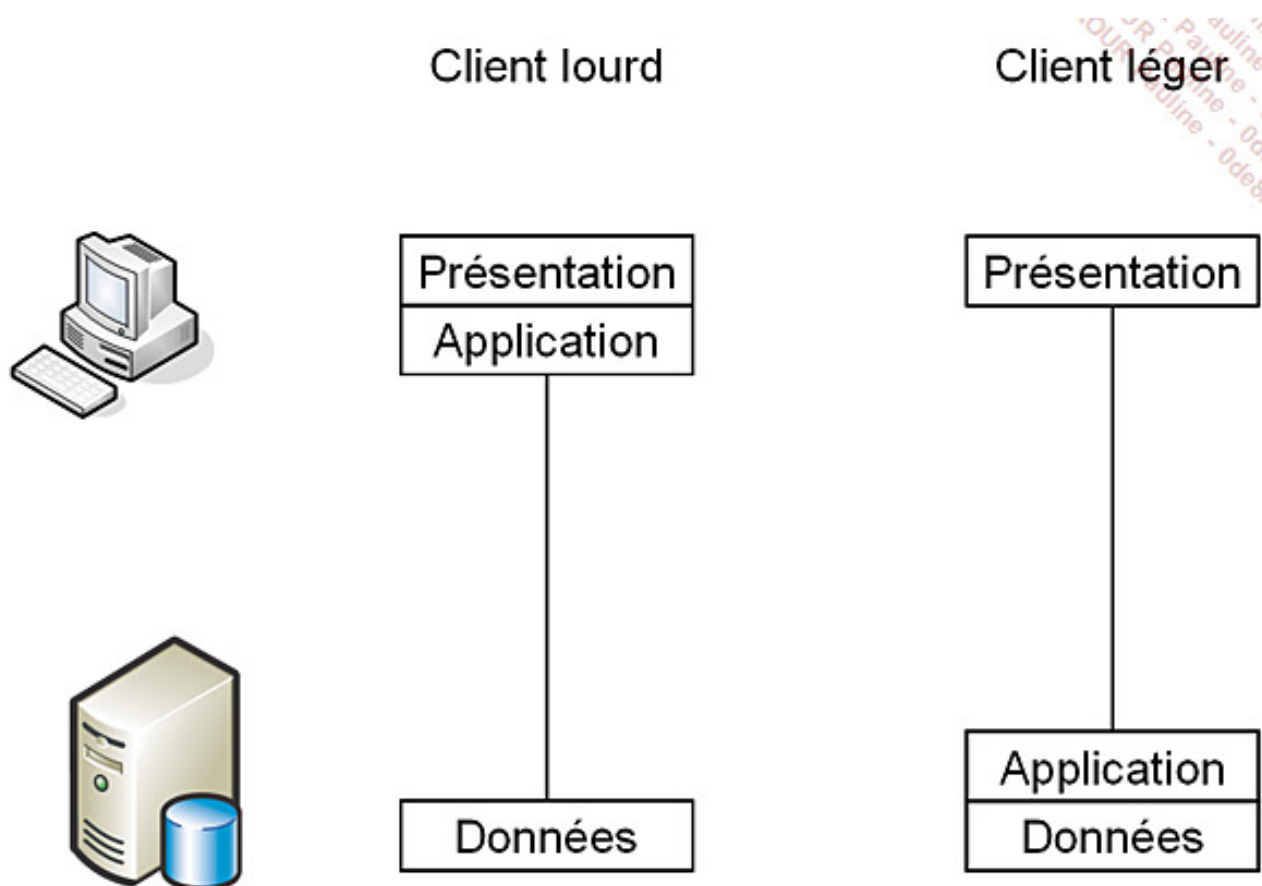
Mode de fonctionnement client/serveur

On peut distinguer trois cas :

- Les règles sont entièrement implémentées sur le client, appelé alors **client lourd**. Cette solution permet de libérer des ressources au niveau du serveur, mais les problèmes de mise à jour des clients et de développement d'autres applications se posent.
- Les règles sont entièrement définies sur le serveur, le client est alors un **client léger**. Cette solution permet d'obtenir des clients qui possèdent peu de

ressources matérielles, et autorise une centralisation des règles ce qui rend plus souples les mises à jour. Cependant de nombreuses ressources sont consommées sur le serveur et l'interaction avec l'utilisateur risque d'être faible, puisque l'ensemble des contraintes est vérifié lorsque l'utilisateur soumet sa demande (requête) au serveur.

- Les règles d'entreprises sont définies sur une tierce machine, appelée **Middle Ware**, afin de soulager les ressources du client et du serveur, tout en conservant la centralisation des règles.



L'architecture client/serveur permet un déploiement optimum des applications clientes sur de nombreux postes tout en conservant une gestion centralisée des données (le serveur), ce qui rend possible le partage d'informations à l'intérieur de l'entreprise.

Il est bien sûr possible d'avoir plusieurs applications clientes sur le même serveur de base de données. Cette possibilité offre de nombreuses fonctionnalités, mais il faut toutefois veiller à ce que la charge de travail sur le serveur ne soit pas trop importante au regard des capacités de la machine. Cette architecture client/serveur est respectée par tous les outils

permettant d'accéder à des informations contenues par le serveur SQL, donc les outils d'administration, même s'ils sont installés sur le serveur.

Toutes les demandes en provenance des clients vers le serveur, doivent être écrites en Transact SQL. Ce langage de requête de base de données respecte la norme ANSI SQL. Le SQL fournit un ensemble de commandes pour gérer les objets et manipuler les données dans les bases. Le Transact SQL est enrichi de nombreuses fonctionnalités, non normalisées, afin d'étendre les possibilités du serveur. Il est ainsi possible de définir des procédures stockées sur le serveur.

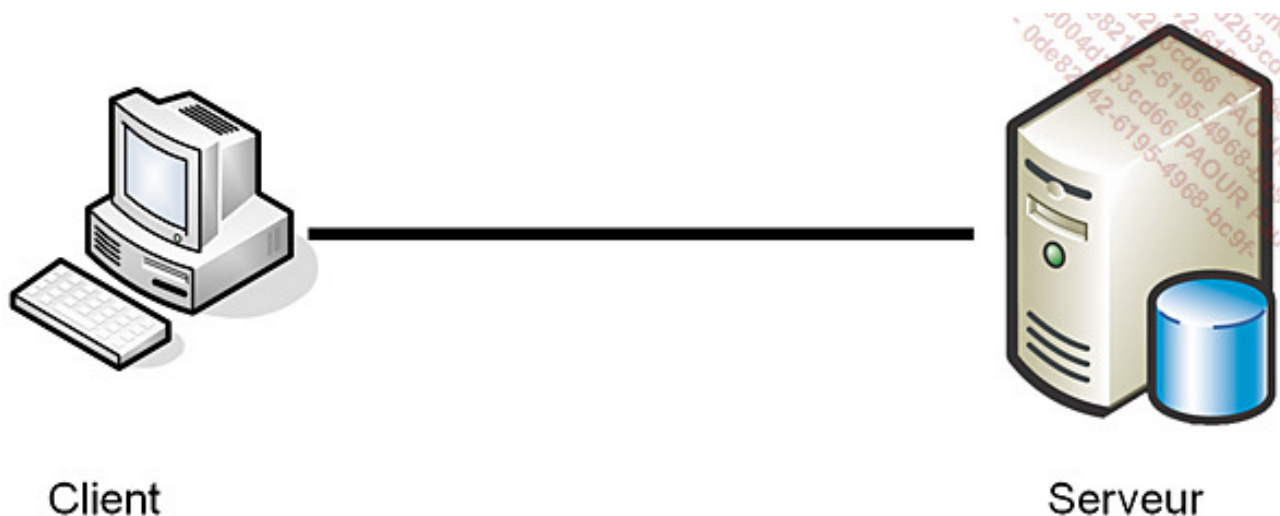
3. Les plates-formes possibles

Il est important de distinguer deux cas : d'un côté les plates-formes possibles pour le client et de l'autre les plates-formes pour le serveur.

Les plates-formes clientes présentées ici sont les postes sur lesquels les outils d'administration SQL Server peuvent être installés. Il ne s'agit pas des postes qui hébergent une application qui se connecte à une instance SQL Server pour gérer les données.

D'une façon synthétique, les outils clients d'administrations peuvent être installés sur tout système d'exploitation Windows Server ou poste de travail.

Par contre, pour la partie serveur, les disponibilités en termes de plates-formes sont fonction de l'édition SQL Server choisie. Néanmoins pour héberger une instance de base de données en production, il est nécessaire de disposer d'un serveur performant et fiable. Une plate-forme Windows Server est donc recommandée. L'édition de Windows sera choisie en fonction des contraintes imposées par l'édition SQL Server sélectionnée et des contraintes liées à l'environnement technique. L'installation d'une instance sous Windows Client sera réservée à des postes nomades.



Dans le cas où le client héberge une application spécifique, la gamme des plates-formes est considérablement élargie grâce, en particulier, au pilote jdbc qui permet d'accéder à une instance SQL Server depuis une application écrite en Java. La gamme est encore élargie dans les cas d'une application ASPX qui propose une interface Internet. Un simple navigateur Internet permet alors de lancer l'application.

4. Les composants de SQL Server

Le moteur de base de données de SQL Server ou Database Engine est composé de plusieurs logiciels. Certains s'exécutent sous forme de services alors que d'autres possèdent une interface utilisateur graphique ou en ligne de commande.

Composants serveur

SQL Server s'exécute sous forme de services Windows. Suivant les options d'installation choisies, il peut y avoir plus de services. Les principaux services sont les suivants :

- SQL Server : c'est le moteur de base de données à proprement parler. Si ce service n'est pas démarré, il n'est pas possible d'accéder aux informations. C'est par l'intermédiaire de ce service que SQL Server assure la gestion des requêtes utilisateurs. Dans la console services de Windows, ce service est référencé sous le nom « SQL Server (MSSQLSERVER) » pour l'instance par défaut et « SQL Server (*nomInstance*) » dans le cas d'une instance nommée.
- SQL Server Agent : ce service prend en charge l'exécution de tâches planifiées, la

surveillance de SQL Server et le suivi des alertes. Il est directement lié à une instance de SQL Server. Il est référencé dans le gestionnaire de service sous le nom « SQL Server Agent(MSSQLSERVER) » pour l'instance par défaut et « SQL Server Agent(*nomInstance*) » dans le cas d'une instance nommée.

- Microsoft Full Text Search : ce service propose de gérer l'indexation des documents de type texte stockés dans SQL Server et gère également les recherches par rapport aux mots-clés.

Il est possible d'installer plusieurs instances SQL Server sur le même poste, comportant chacune sa liste de services (au minimum le moteur de base de données et l'agent SQL Server).

Outils de gestion

Les réalisations des tâches d'administration sont possibles par l'utilisation d'outils. Ces outils possèdent pour la plupart une interface graphique conviviale et d'utilisation intuitive. Cependant, les tâches administratives doivent être réfléchies avant leur réalisation. L'utilisation de certains outils suppose que le composant serveur correspondant est installé.

Ces outils sont :

- SQL Server Management Studio pour réaliser toutes les opérations au niveau du serveur de base de données.
- Gestionnaire de configuration SQL Server pour gérer les services liés à SQL Server.
- SQL Server Profiler pour suivre et analyser la charge de travail d'une instance SQL Server.
- Assistant paramétrage du moteur de base de données pour permettre une optimisation du fonctionnement du serveur de base de données.

Tous les outils et le fonctionnement de SQL Server sont richement documentés.

Les composants

Les différentes briques logicielles fournies par SQL Server s'articulent toujours autour du moteur de base de données relationnelles qui traite de façon performante les informations stockées au format relationnel et au format XML.

- SQL Server Integration Service (SSIS) est un outil d'importation et d'exportation de données facile à mettre en place tout en étant fortement paramétrable.
- La réplication des données sur différentes instances permet de positionner les données au plus près des utilisateurs et de réduire les temps de traitement.
- Le langage de statistique R est maintenant intégré à SQL Server. C'est-à-dire qu'il est possible d'interroger des données stockées dans une base SQL Server directement depuis le langage R. Ce langage de statistique s'adresse plus particulièrement aux bases de données décisionnelles, qui ne sont pas décrites dans ce livre.
- L'intégration du CLR dans SQL Server permet de développer procédures et fonctions en utilisant les langages VB.NET et C#. L'intégration du CLR ne vient pas se substituer au Transact SQL mais se présente comme un complément afin de pouvoir réaliser un codage simple et performant pour l'ensemble des fonctionnalités qui doivent être présentes sur le serveur.

CLR

L'intégration du CLR (*Common Language Runtime*) à SQL Server, permet d'augmenter considérablement les possibilités offertes en termes de programmation. La présence du CLR ne remet pas en cause le Transact SQL. Chacun est complémentaire. Le Transact SQL est parfait pour écrire des procédures ou fonctions pour lesquelles il y a un traitement intensif des données. Au contraire, dans le cas où le volume des données manipulées est faible, le CLR permet d'écrire simplement des traitements complexes, car il bénéficie de toute la richesse du CLR.

Le CLR permet également de définir ses propres types de données ou bien de nouvelles fonctions de calcul d'agrégat.

Enfin, le CLR permet aux développeurs d'applications de développer des procédures et fonctions sur SQL Server tout en conservant leurs langages favoris (VB.NET ou C# par exemple), et donc sans avoir besoin de maîtriser le Transact SQL.

Dans le cas où le code est écrit depuis Visual Studio, l'intégration de la version compilée dans SQL Server et le mappage CLR-Transact SQL est réalisé de façon automatique. Il est possible de réaliser le développement en dehors du Visual Studio mais l'intégration à SQL Server sera faite de façon manuelle, ce qui est une tâche fastidieuse.

PowerShell

SQL Server, comme tous les serveurs de Microsoft, intègre complètement PowerShell comme langage de scripting. Contrairement à d'autres serveurs, SQL Server dispose de peu de cmdlets spécifiques car PowerShell va donner la possibilité d'exécuter des instructions au format Transact SQL, qui est déjà le langage de script utilisé par SQL Server. Mais comme PowerShell repose sur le framework .NET, il est également possible de mener les opérations d'administration au travers de la bibliothèque SMO.

Pour ces deux raisons, les commandes PowerShell ne sont pas présentées en plus de chaque commande Transact SQL. Par contre, dans le chapitre Outils complémentaires, une partie est dédiée à PowerShell et cmdlets spécifiques à SQL Server.

Architecture

1. Administration

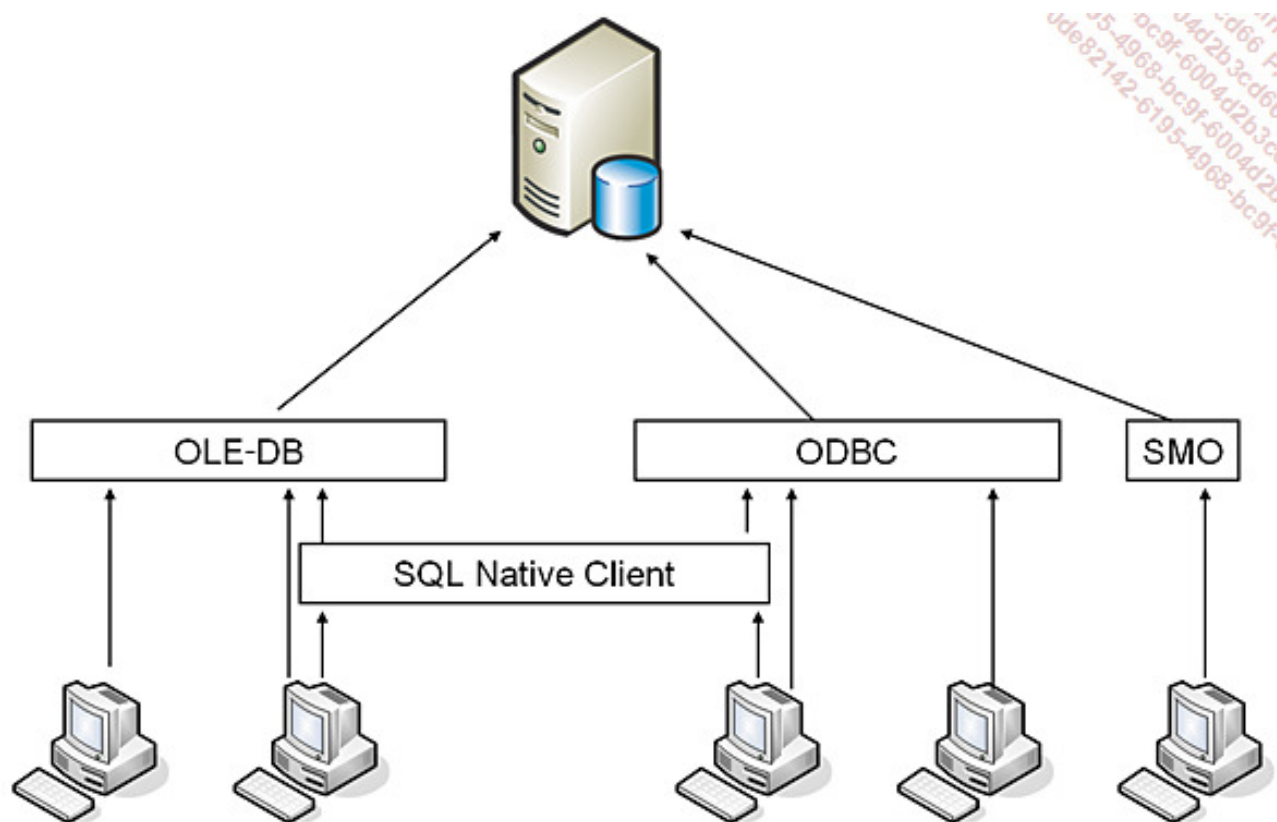
Le langage naturel de SQL Server est le Transact SQL. Il est donc nécessaire de lui transmettre les instructions dans ce langage. Comme ce langage n'est pas forcément naturel pour l'utilisateur, il est possible de composer l'instruction de façon graphique par SQL Server Management Studio, puis de provoquer son exécution sur le serveur à l'aide des boutons **OK**, **Appliquer...**

Même si seules les tâches administratives sont traitées dans cet ouvrage, une bonne connaissance du Transact SQL et des possibilités offertes par ce langage est plus que souhaitable pour le lecteur qui désire se diriger vers l'administration de base de données.

Il est donc possible d'écrire des scripts Transact SQL pour exécuter des opérations administratives sous forme de traitement batch.

2. Programmation

Le développement d'applications clientes pour visualiser les données contenues dans le serveur peut s'appuyer sur différentes technologies.



La DLL SQL Native Client est une méthode d'accès aux données qui est disponible aussi bien en utilisant la technologie OLE-DB ou bien ODBC pour accéder aux données. Avec cette nouvelle API, il est possible d'utiliser l'ensemble des fonctionnalités de SQL Server comme les types personnalisés définis avec les CLR (UDT : *User Defined Type*), MARS (*Multiple Active Result Sets*) ou bien encore le type XML.

SQL Native Client est une API qui permet de tirer pleinement profit des fonctionnalités de SQL Server et de posséder un programme qui accède de façon optimum au serveur.

Il est toujours possible d'utiliser les objets ADO pour accéder à l'information. Ce choix est plus standard car un programme accédant à une source de données ADO peut travailler aussi bien avec une base Oracle que SQL Server, mais ne permet pas la même gestion de toutes les fonctionnalités offertes par SQL Server. L'API SQL Native Client permet l'écriture de programmes clients optimisés mais uniquement capables d'accéder à des données hébergées par un serveur SQL Server.

SQL Native Client sera adopté comme modèle d'accès aux données dans les nouveaux programmes écrits en C# ou VB.NET qui souhaitent travailler avec SQL Server mais aussi dans les programmes existants lorsque ces derniers souhaitent travailler avec des

éléments spécifiques à SQL Server, comme le type XML, par exemple.

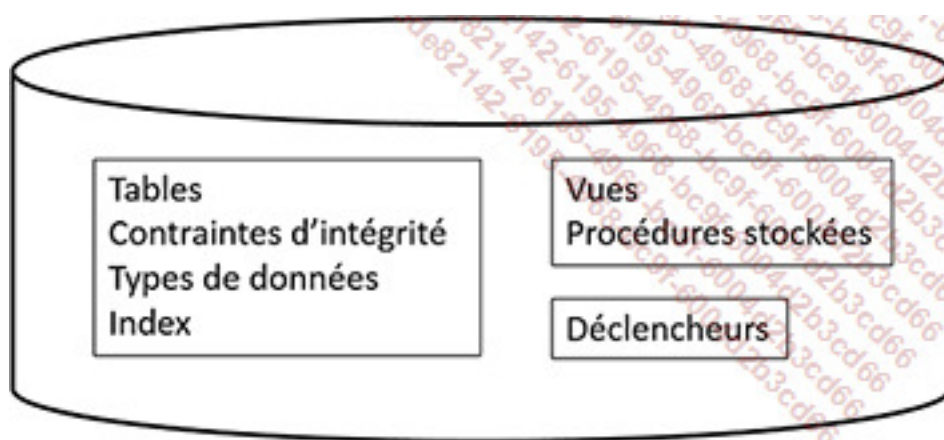
Ce modèle de programmation correspond à une application client qui souhaite gérer les données. Dans le cas où l'application souhaite être capable de faire des opérations d'administration, il est alors nécessaire d'utiliser la bibliothèque SMO.

Base de données SQL Server

1. Objets de base de données

Les bases de données contiennent un certain nombre d'objets logiques. Il est possible de regrouper ces objets en trois grandes catégories :

- Gestion et stockage des données : tables, types de données, contraintes d'intégrité et index.
- Accès aux données : vues et procédures stockées.
- Gestion de l'intégrité complexe : déclencheur (procédure stockée s'exécutant automatiquement lors de l'exécution d'un ordre SQL modifiant le contenu d'une table : **INSERT**, **UPDATE** et **DELETE**). Le déclencheur est toujours associé à une table et à une instruction SQL. Il permet de mettre en place des règles d'intégrité complexes à cheval sur plusieurs tables ou de maintenir des données non normalisées.



Objets de base de données

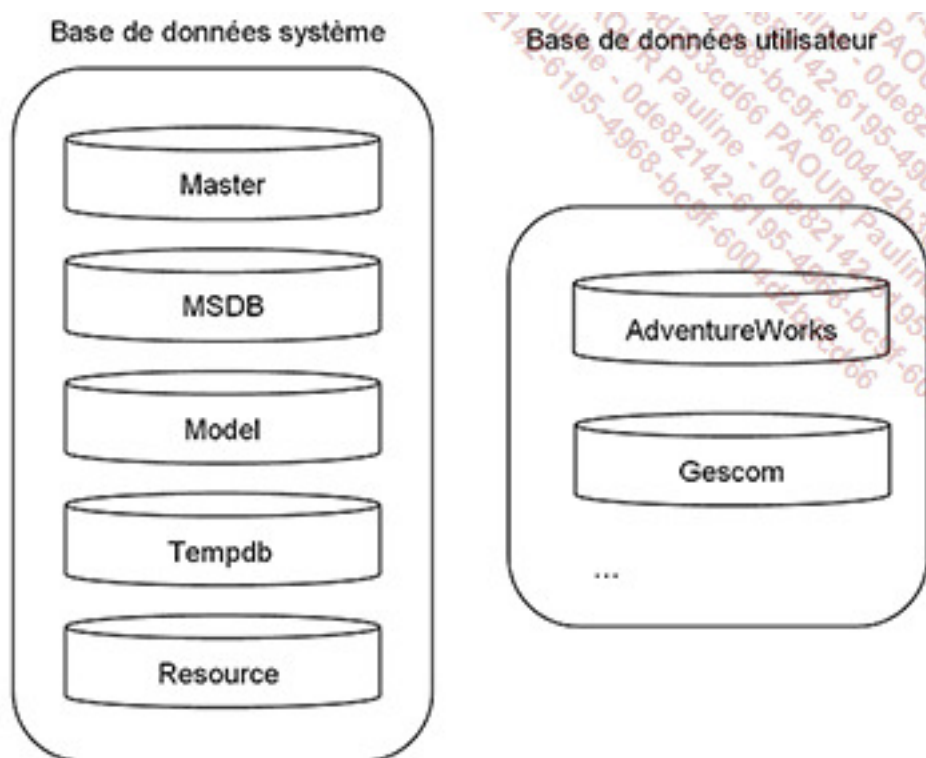
Nom complet des objets

La règle appliquée pour nommer les objets permet une parfaite identification. Le nom complet est composé comme suit : *serveur.nomBase.schéma.objet*. Pour toutes les requêtes, il est conseillé de nommer les objets au minimum avec *schéma.objet*, même si le

schéma est celui par défaut, à savoir dbo. Ceci évitera une phase de recherche dans l'exécution des requêtes.

2. Bases de données système et tables système

La quasi-totalité des informations de configuration du serveur et des bases de données utilisateur est stockée dans SQL Server. Il existe donc des bases de données système et, sur chaque base utilisateur, quelques tables système. L'insertion et la mise à jour de données dans ces tables ne s'effectuent jamais directement, mais via des commandes d'administration Transact SQL ou des procédures stockées système.



Organigramme des bases de données



Les noms des bases de données et des tables système sont fixés et connus par SQL Server.

Master

C'est la base de données principale de SQL Server. L'ensemble des données stratégiques pour le bon fonctionnement du serveur y est stocké (comptes de connexion, options de configuration, l'existence des bases de données utilisateurs et les références vers les fichiers qui composent ces bases...). Cette base sera donc sauvegardée régulièrement, donc intégrée à un plan de sauvegarde.

Model

Cette base sert de modèle lors de la création d'une base de données utilisateurs : on peut dire qu'un « copier-coller » de cette base va être fait pour créer notre base. Elle contient donc l'ensemble des objets système présents dans toute base utilisateur (tables, procédures stockées...). Il est éventuellement possible d'y rajouter des éléments, mais cela va compliquer considérablement sa gestion.

Tempdb

La base **Tempdb** est un espace temporaire de stockage partagé. Il permet de gérer les tables temporaires locales ou globales, les tables de travail intermédiaires pour faire des tris par exemple, mais aussi les jeux de résultats des curseurs. La base **Tempdb** est recréée, avec sa taille initiale, lors de chaque démarrage de l'instance. Ainsi, aucune information ne peut être conservée de façon persistante à l'intérieur de la base **Tempdb**. Les objets temporaires sont, quant à eux, supprimés lors de la déconnexion de leur propriétaire.

Les paramètres physiques de la base tempdb peuvent être spécifiés lors de l'installation d'une instance SQL Server. Par défaut, cette base est composée de fichiers de 8 Mo (1 par cœur de processeur).

MsdB

Elle est la base de données du service SQL Server Agent. Ainsi, elle contient des informations sur son paramétrage, sur les travaux, les alertes, les opérateurs de messagerie...

Resource

Cette base en lecture seule contient la définition de tous les nouveaux éléments définis à partir de SQL Server 2014. Les objets système y sont définis bien que logiquement ils

apparaissent dans le schéma de l'utilisateur **sys**.

Cette base de données ne peut pas être incluse dans un processus de sauvegarde.

Bases de données utilisateur

Les bases de données utilisateurs vont héberger les données fournies par les utilisateurs. Les bases présentes sur le schéma précédent (AdventureWorks et Gescom) sont les bases d'exemples utilisées dans la documentation officielle de SQL Server et dans cet ouvrage.

3. Les tables système

Les tables système sont toujours présentes dans SQL Server. Il est possible d'y accéder en lecture uniquement afin de récupérer des informations sur la configuration du serveur ou d'une base. Il est préférable d'y accéder via des vues système contenues dans les schémas **sys** ou **INFORMATION_SCHEMA**.

Les tables système sont utilisées directement par le moteur de SQL Server. Il convient de faire attention lors du développement d'applications qui utilisent SQL Server et accèdent directement à ces tables. En effet, comme la structure de ces tables évolue avec les versions de SQL Server, si certaines applications accèdent de façon directe aux tables système, on peut se trouver dans l'impossibilité de migrer vers une nouvelle version de SQL Server tant que l'application n'a pas été réécrite.



SQL Server ne prend pas en compte les déclencheurs qui pourraient être définis sur les tables système car ils peuvent gêner le bon déroulement de certaines opérations.

Dans le tableau ci-dessous, quelques vues système sont référencées :

Vue système	Objectif
sys.server_principals	Liste des connexions définies sur le serveur.
sys.messages	Une ligne pour chaque message ou avertissement défini sur le serveur, et ceci pour chaque langue prise en charge.
sys.databases	Une ligne pour chaque base de données (système et utilisateur) présente sur le serveur.
sys.configurations	Chaque ligne correspond à une option de configuration. L'interrogation de cette vue permet de connaître la valeur de chaque option de configuration.
sys.database_principals	Une ligne pour chaque utilisateur défini au niveau de la base courante.
sys.columns	Une ligne pour chaque colonne des tables, vues et paramètres des procédures stockées.
sys.objects	Une ligne pour chaque élément défini dans la base de données courante.
sys.database_files	Cette vue retourne une ligne par fichier composant la base de données depuis laquelle elle est appelée. Cette vue système est donc spécifique à chaque base et renvoie un jeu de résultats spécifique à chaque base.

4. Extraction de métadonnées

Pour interroger les données contenues dans les tables système, même s'il est possible de le faire directement par une requête de type **SELECT**, il est préférable de passer par l'utilisation de procédures stockées, de fonctions système et de vues système.

Procédures stockées système

Les procédures stockées système permettent d'interroger les tables système et connaître ainsi l'état du serveur, de la base... Mais elles peuvent également être utilisées pour effectuer des opérations de configuration.

De nombreuses procédures stockées système sont présentes au niveau du serveur mais ne sont pas documentées au niveau de la documentation en ligne de SQL Server. Il ne s'agit pas d'un oubli, cela permet d'identifier facilement une procédure stockée présente dans la version courante et dont Microsoft ne garantit pas la présence dans une version future de SQL Server.

Pour interroger les tables système, il existe de nombreuses procédures stockées. Elles commencent toutes par **sp_**. Parmi toutes les procédures stockées, citons :

Procédure stockée	Description
sp_help [nom_objet]	Informations sur l'objet indiqué.
sp_helpdb [nom_base_données]	Informations sur la base de données indiquée.
sp_helpindex nom_table	Informations sur les index de la table indiquée.
sp_helplogins [nom_connexion]	Informations sur la connexion indiquée.
sp_who et sp_who2	Liste des utilisateurs actuellement connectés.
sp_lock	Liste des verrous actuellement posés.

Exemple

Lister les bases de données du serveur :

```
exec sp_helpdb
```

Ou :

```
select * from sys.databases
```

Le catalogue

SQL Server propose des vues système qui permettent d'obtenir des informations système. Toutes ces vues sont présentes dans le schéma **sys**.

La structure des vues système peut être amenée à s'enrichir dans les futures versions de SQL Server. Il est donc nécessaire de ne pas utiliser le caractère générique * dans une instruction **SELECT** mais plutôt de préciser le nom des colonnes. En procédant de cette manière, le script sera insensible aux changements de structure éventuels de la vue.

Afin de naviguer au mieux dans ces vues, elles sont regroupées par thèmes :

- Haute disponibilité (AlwaysOn)
- Suivi des changements
- CLR
- Collection de données (vues spécifiques à la base msdb)
- Espaces de stockage
- Gestion des mails (Database Mail)
- Miroir de base de données
- Fichiers
- Endpoints
- Propriétés étendues
- Serveurs liés
- Messages d'erreur
- Objets

- Partitions
- Règles (Policy-Based)
- Gouverneur de ressources
- Requêtes
- Types de données scalaires
- Schémas
- Sécurité
- Service Broker
- Configuration du serveur

Fonctions système

Les fonctions système sont utilisables avec des commandes Transact SQL. Il est ainsi possible de récupérer des valeurs concernant la base de données sur laquelle on travaille, sur le serveur ou sur les utilisateurs.

Citons-en quelques-unes :

Fonctions système	Paramètre	Description
DB_ID	Nom	Retrouve l'identificateur de la base de données.
USER_NAME	ID	Retrouve le nom de l'utilisateur à partir de son identifiant.
COL_LENGTH	Nom de la table et de la colonne	Longueur de la colonne.
STATS_DATE	Identifiants de la table et des statistiques	Date de dernière mise à jour des statistiques.
DATALENGTH	Expression	Longueur de l'expression.

Exemple

Dans l'exemple ci-après, la fonction **DB_ID** est présentée dans deux cadres d'utilisation différents. Utilisée sans paramètre, cette fonction permet de connaître l'identifiant de la base de données sur laquelle on travaille. Utilisée en fournissant un paramètre, cette fonction permet de connaître l'identifiant de cette base de données.

```
select db_id() as 'base courante', db_id('gescom') as gescom
```

Schéma d'information

Il s'agit d'un ensemble de vues qui proposent une visualisation des paramètres de façon indépendante du type de serveur de bases de données utilisé, car ces vues sont conformes à la définition du standard ANSI SQL pour les schémas d'information.

Les vues du schéma d'information :

CHECK_CONSTRAINTS

Visualise l'ensemble des contraintes de type **CHECK** définies dans la base de données.

COLUMN_DOMAIN_USAGE

Visualise l'ensemble des colonnes définies sur un type de données utilisateur.

COLUMN_PRIVILEGES

Visualise l'ensemble des privilèges accordés, au niveau colonne, pour l'utilisateur courant ou pour un utilisateur de la base de données.

COLUMNS

Visualise la définition de l'ensemble des colonnes accessibles, dans la base de données courante, à l'utilisateur actuel.

CONSTRAINT_COLUMN_USAGE

Visualise l'ensemble des colonnes pour lesquelles il existe une contrainte.

CONSTRAINT_TABLE_USAGE

Visualise l'ensemble des tables pour lesquelles au moins une contrainte est définie.

DOMAIN_CONSTRAINTS

Visualise l'ensemble des types de données utilisateurs, qui sont accessibles par l'utilisateur courant et qui sont liés à une règle.

DOMAINS

Visualise l'ensemble des types de données utilisateurs, qui sont accessibles par l'utilisateur courant.

KEY_COLUMN_USAGE

Visualise l'ensemble des colonnes pour lesquelles une contrainte de clé est définie.

PARAMETERS

Visualise les propriétés des paramètres des fonctions définies par l'utilisateur et des procédures stockées accessibles à l'utilisateur courant. Pour les fonctions, les informations relatives à la valeur de retour sont également visualisées.

REFERENTIAL_CONSTRAINTS

Visualise l'ensemble des contraintes de clés étrangères définies dans la base de données courante.

ROUTINE_COLUMNS

Visualise les propriétés de chaque colonne renvoyée par les fonctions accessibles à l'utilisateur courant.

ROUTINES

Visualise l'ensemble des procédures stockées et des fonctions accessibles à l'utilisateur courant.

SCHEMATA

Visualise les bases de données pour lesquelles l'utilisateur courant possède des autorisations.

SEQUENCES

Permet de connaître les différentes séquences avec le détail des paramètres de chaque séquence.

TABLE_CONSTRAINTS

Visualise les contraintes de niveau table, posées dans la base de données courante.

TABLE_PRIVILEGES

Visualise l'ensemble des privilèges accordés à l'utilisateur actuel dans la base de données courante et l'ensemble des privilèges accordés par l'utilisateur actuel à

d'autres utilisateurs de la base de données courante.

TABLES

Visualise l'ensemble des tables de la base de données courante pour lesquelles l'utilisateur actuel dispose de droits.

VIEW_COLUMN_USAGE

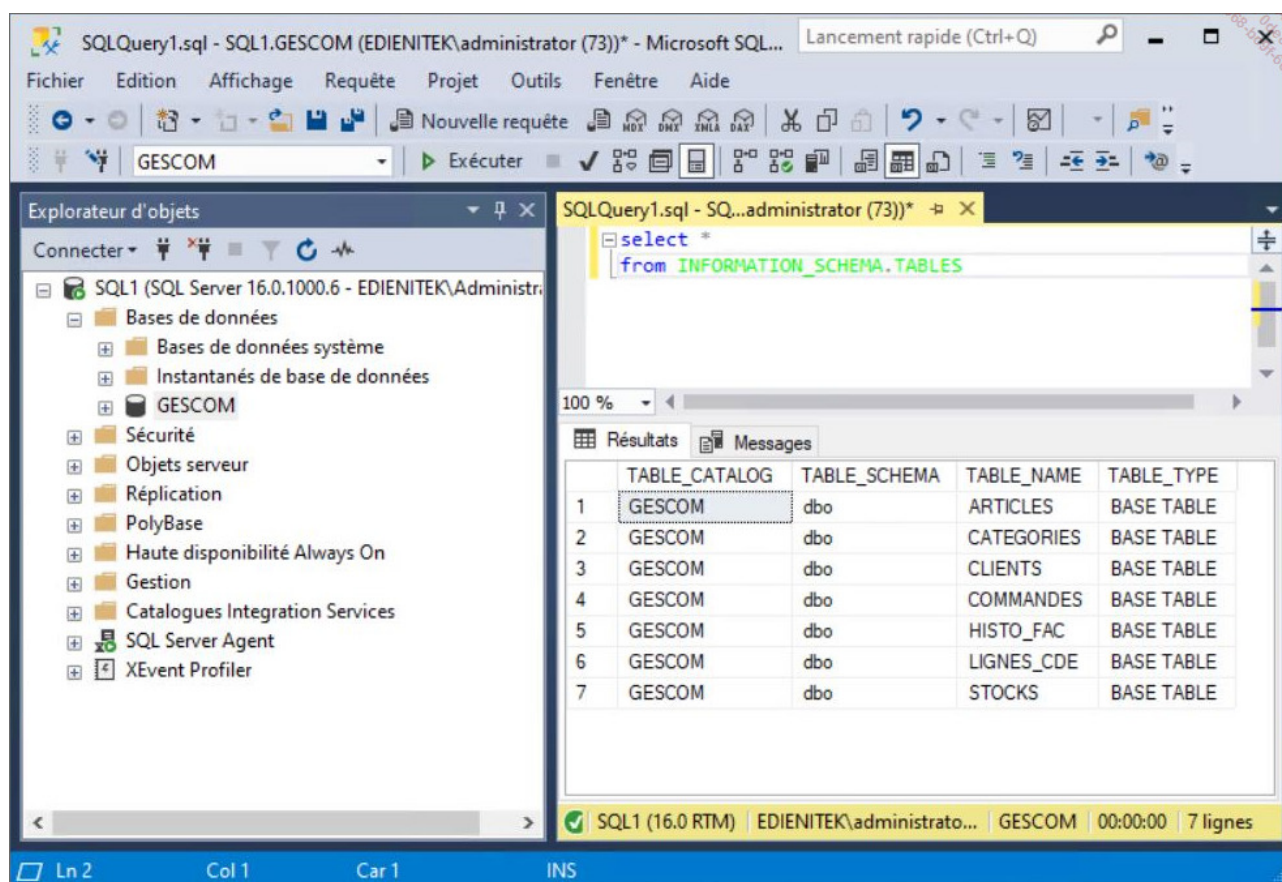
Visualise l'ensemble des colonnes de tables qui sont utilisées dans la définition d'une ou plusieurs vues.

VIEW_TABLE_USAGE

Visualise l'ensemble des tables de la base de données courante qui sont utilisées lors de la définition de vues.

VIEWS

Visualise l'ensemble des vues accessibles à l'utilisateur actuel dans la base de données courante.



Exemple : pour lister les tables de la base de données courante

5. Les tâches de l'administrateur

L'administrateur de bases de données a pour objectif principal de gérer les tâches courantes concernant les serveurs de base de données. Il doit également participer à l'optimisation de son fonctionnement avec l'équipe de développement des applications.

Voici les principales tâches de l'administrateur :

- Gérer les services SQL Server.
- Gérer les instances SQL Server.
- Mettre en place le processus de sauvegarde et de restauration.
- Configurer une disponibilité des données en accord avec la politique d'entreprise.
- Gérer les configurations réseau.

- Importer et exporter des données.

En plus des compétences système que doit posséder l'administrateur pour être capable de gérer au mieux l'instance SQL Server, il est important pour lui de connaître les possibilités offertes par SQL Server pour l'automatisation des tâches avec SQL Agent.

Pour mesurer le résultat de son travail et comparer les différents choix de configuration qu'il peut être amené à faire, l'administrateur doit être en mesure d'utiliser les outils et méthodes liés à SQL Server.

Enfin et c'est sans doute un point crucial, l'administration de la base doit être inscrite dans un processus plus global qui intègre l'administrateur dès la conception de la base, de façon à effectuer les meilleurs choix en termes d'architecture dès la conception. L'administrateur pourra ainsi intervenir sur la création de la base et les choix faits comme, par exemple, le nombre de groupe de fichiers à utiliser, les index, les vues, les procédures stockées à définir de façon à optimiser le trafic réseau mais aussi pour simplifier la gestion des droits d'accès. C'est également l'administrateur qui va pouvoir conseiller sur le partitionnement ou non d'une table.

Installation de SQL Server

L'installation de SQL Server permet de présenter les différentes éditions de SQL Server ainsi que de détailler les options d'installation possibles. Un point tout particulier sera mis en place pour détailler l'exécution des services associés à SQL Server (logiciels serveur). Une fois le serveur installé, il faut s'assurer que l'installation s'est déroulée correctement puis il faut configurer le serveur et certains outils clients pour réaliser les différentes étapes d'administration.

Bien que facile à réaliser, l'installation de SQL Server doit être une opération réfléchie. En effet, de nombreuses options d'installation existent et leur choix doit correspondre à un réel besoin, ou permettre de couvrir une évolution future du système.

1. Les éditions de SQL Server

SQL Server est disponible sous la forme de plusieurs éditions. Chaque édition se distingue par des caractéristiques spécifiques. En fonction des possibilités souhaitées, le choix se portera sur une édition ou bien une autre.

SQL Server propose trois éditions principales qui vont permettre de répondre à la plus grande majorité des cas présents en entreprise. En complément de ces solutions, il existe deux éditions qui permettent de répondre à des besoins précis en termes de disponibilité ou bien d'usage.

Même si cela peut poser des problèmes d'optimisation, il est possible d'installer plusieurs instances de SQL Server sur le même serveur. Cette notion d'instance permet d'optimiser l'utilisation des licences tout en cloisonnant la gestion des différentes instances.

Enfin, la mise en place de SQL Server sur un serveur Windows Core possède des caractéristiques et des spécificités non abordées dans ce livre.

Entreprise

L'édition Entreprise est la plus complète. Elle propose l'ensemble des fonctionnalités disponibles avec SQL Server. Cette édition est conçue pour être capable de gérer des

volumes très importants en termes de données et de transactions avec de nombreux utilisateurs connectés.

Cette édition dispose également des fonctionnalités avancées en ce qui concerne les opérations de business intelligence et de haute disponibilité.

Standard

Cette édition, plus simple que l'édition Entreprise, a pour objectif de répondre aux besoins d'une entreprise qui cherche un moteur de base de données performant et qui n'a pas besoin des fonctionnalités spécifiques de l'édition Entreprise.

C'est principalement au niveau des solutions de haute disponibilité que les écarts sont présents. La gestion des très gros volumes de données est également moins performante que dans l'édition Entreprise.

Cette édition représente un bon choix car elle permet une évolution facile vers d'autres éditions tout en conservant l'outil d'administration SQL Server Management Studio.

Express

L'édition Express de SQL Server possède la particularité d'être utilisable en production sans qu'il soit nécessaire de s'acquitter d'une licence de SQL Server. Cette version n'est pas une version dégradée de SQL Server, mais il s'agit bien du moteur SQL Server pleinement fonctionnel. Il n'existe pas de limite quant au nombre d'utilisateurs connectés. Les seules limitations qui existent concernent le volume de données : 10 Go et le fait que le moteur ne puisse pas exploiter plus de 1,4 Go de mémoire. Il est toutefois raisonnable de penser que lorsqu'une application atteint ces limites, l'entreprise possède les moyens nécessaires pour acquérir une version complète de SQL Server.

Cette édition Express est à conseiller sans modération auprès des développeurs d'applications car il sera possible de migrer très simplement vers une édition supérieure de SQL Server.

Ce type d'édition est également bien adapté pour les applications autonomes. En effet, l'édition Express peut être installée sur une plate-forme Windows utilisateur, comme par exemple l'ordinateur portable d'un commercial qui emmène avec lui la base de tous les articles de l'entreprise. Cette base peut être mise à jour par le processus de réplication de SQL Server qui permet de synchroniser les données au niveau du catalogue et de les injecter dans le système d'information de l'entreprise.

Cette édition est également utile dans la cadre d'une application monoposte qui nécessite une gestion robuste et fiable des données. Choisir d'utiliser SQL Server pour ce type d'application permet de laisser ouvert le passage vers une gestion multi-utilisateur.

Developer

En plus de toutes ces éditions de production, SQL Server propose également une édition Developer. Cette édition Developer comprend l'ensemble des fonctionnalités proposées par l'édition Entreprise. Toutefois, avec une édition Developer, la mise en production n'est pas légale. Comme son nom l'indique, la version Developer permet à l'équipe de développement d'application de faire ses tests sur une base pleinement fonctionnelle sans pour autant être dans l'obligation d'acquérir une licence de production.

L'édition Developer est disponible gratuitement en téléchargement depuis le site de Microsoft.

Utiliser cette édition est un bon moyen de tester les différentes fonctionnalités de SQL Server.

Web

Centrée sur la gestion de données, cette édition permet d'offrir un moteur de base de données à destination des sites web à faible coût. Les possibilités en termes d'administration sont réduites. Cette édition est limitée aux seuls centres de services.

2. Déroulement de l'installation

Comme pour de nombreux outils, l'installation est découpée en deux phases bien distinctes. La première consiste à demander à l'utilisateur le paramétrage exact de l'installation qu'il désire effectuer. Cette étape est relativement rapide. En effet, en fonction des choix de l'utilisateur, des éléments et des options sont chargés ou non. Les choix effectués au cours de cette phase de questions-réponses sont extrêmement importants car certains choix ne peuvent être faits que lors de l'installation du moteur SQL Server.

Le processus d'installation concerne le moteur de base de données. L'installation de la console d'administration SQL Server Management Studio est faite de façon indépendante après téléchargement du fichier d'installation depuis le site de Microsoft.



Les étapes détaillées pour une installation du moteur sont également vraies en ce qui concerne l'installation d'une nouvelle instance.

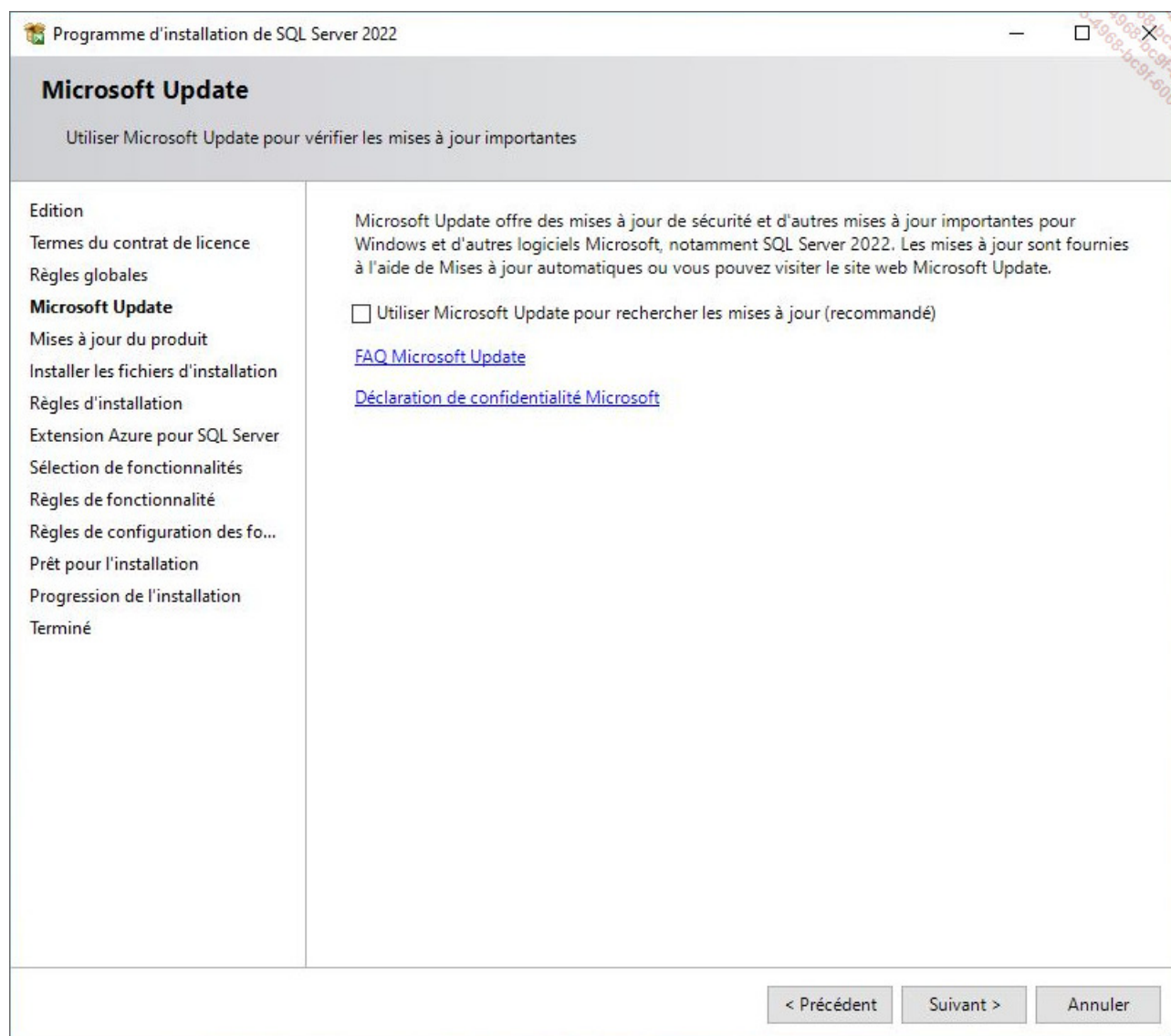
Après saisie de la clé du produit et acceptation du contrat de licence, les fichiers de support du programme d'installation de SQL Server sont installés.

Le processus d'installation de SQL Server commence par exécuter un certain nombre de règles afin de valider la configuration de la plate-forme.

Si ces règles ne sont pas parfaitement respectées il en ressort soit des avertissements, soit des erreurs. Un avertissement permet de préciser que bien qu'il soit possible

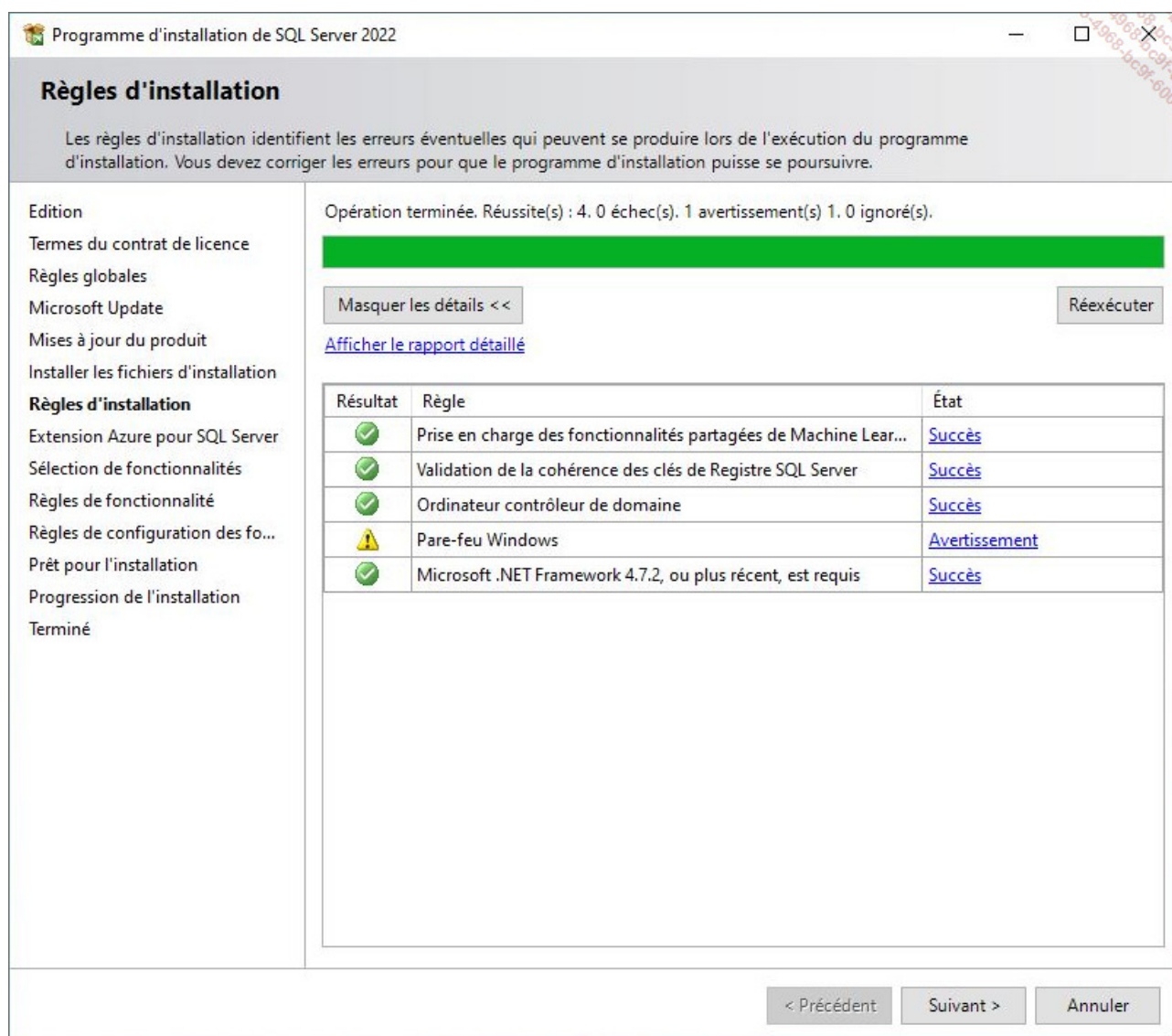
d'installer une instance SQL Server, certains composants ne pourront pas être installés.

Le processus d'installation vérifie ensuite s'il y a des mises à jour à effectuer au niveau du produit.



Quelques règles d'installation sont ensuite vérifiées. Attention, l'installation de SQL Server n'ouvre pas les ports nécessaires à son bon fonctionnement sur le pare-feu Windows.

Afin que le serveur soit accessible à distance, il faudra créer les règles nécessaires :



a. Choix des composants

Avant de procéder au paramétrage de l'installation, SQL Server demande de sélectionner les composants qui vont être installés sur le poste local.

Il s'agit de sélectionner finement les composants à installer. Il ne s'agit pas de cocher toutes les options, mais bien de sélectionner les composants qui vont être effectivement utilisés. En limitant le nombre de composants installés, la surface d'attaque du système est réduite et l'on évite également une surcharge du système par des composants non utilisés. Bien entendu, si certains composants n'ont pas été sélectionnés lors de l'installation initiale et que le besoin s'en fait sentir par la suite, il est possible de les ajouter.

Programme d'installation de SQL Server 2022

Sélection de fonctionnalités

Sélectionnez les fonctionnalités de Developer à installer.

Edition
 Termes du contrat de licence
 Règles globales
 Microsoft Update
 Mises à jour du produit
 Installer les fichiers d'installation
 Règles d'installation
 Extension Azure pour SQL Server
Sélection de fonctionnalités
 Règles de fonctionnalité
 Configuration de l'instance
 Configuration du serveur
 Configuration du moteur de ba...
 Règles de configuration des fo...
 Prêt pour l'installation
 Progression de l'installation
 Terminé

Fonctionnalités :

Fonctionnalités de l'instance

- ☒ Services Moteur de base de données
 - ☐ Réplication SQL Server
 - ☐ Machine Learning Services et extensions de langage
 - ☐ Extraction en texte intégral et extraction sémantique de
 - ☐ Data Quality Services
 - ☐ Service de requête PolyBase pour données externes
- ☐ Analysis Services

Fonctionnalités partagées

- ☐ Data Quality Client
- ☐ Integration Services
 - ☐ Scale Out Master
 - ☐ Scale Out Worker
- ☐ Master Data Services

Fonctionnalités redistribuables

Description du composant :

Inclut le moteur de base de données, lequel constitue le service principal pour le stockage, le traitement et la protection des données. Le moteur de base de données permet un accès

Configuration requise pour les composants sélectionnés :

Déjà installé(s) :
 Windows PowerShell 3.0 ou versio

À installer depuis un média :

Espace disque nécessaire

Lecteur C : 994 Mo requis, 50709 Mo disponibles

Sélectionner tout Désélectionner tout

Répertoire racine de l'instance : C:\Program Files\Microsoft SQL Server\

Répertoire des fonctionnalités partagées : C:\Program Files\Microsoft SQL Server\

Répertoire des fonctionnalités partagées (x86) : C:\Program Files (x86)\Microsoft SQL Server\

< Précédent Suivant > Annuler

Pour chacun des composants, il est possible de définir son emplacement physique sur le disque dur. Pour accéder à cet écran de configuration, il faut sélectionner le bouton  .

En fonction des éléments sélectionnés, il peut être nécessaire de procéder à l'installation de composants complémentaires. L'installation de ces composants peut être intégrée au processus d'installation de SQL Server ou bien doit être effectuée de façon indépendante.

b. Nom de l'instance

MS SQL Server offre la possibilité d'installer une ou plusieurs instances du moteur de base

de données sur le même serveur. Chaque instance est complètement indépendante des autres installées sur le même serveur et elles sont gérées de façon autonome. Lorsque plusieurs instances sont présentes sur le même serveur, il est nécessaire d'utiliser le nom de chacune d'entre elles pour les distinguer. La première instance installée est généralement l'instance par défaut et elle porte le même nom que le serveur sur lequel elle est installée.

Programme d'installation de SQL Server 2022

Configuration de l'instance

Spécifiez le nom et l'ID d'instance de l'instance de SQL Server. L'ID d'instance devient partie intégrante du chemin d'installation.

Edition
Termes du contrat de licence
Règles globales
Microsoft Update
Mises à jour du produit
Installer les fichiers d'installation
Règles d'installation
Extension Azure pour SQL Server
Sélection de fonctionnalités
Règles de fonctionnalité
Configuration de l'instance
Configuration du serveur
Configuration du moteur de ba...
Règles de configuration des fo...
Prêt pour l'installation
Progression de l'installation
Terminé

☒ Instance par défaut

☐ Instance nommée : *

ID d'instance :

Répertoire SQL Server : C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER

Instances installées :

Nom de l'instance	ID d'instance	Fonctionnalités	Edition	Version
MSSQLSERVER	MSSQL15.MSSQLS...	SQLEngine	Developer	15.0.2000.5
< Composants part...		Conn		15.0.2000.5

< Précédent
Suivant >
Annuler

Les autres instances porteront alors un nom. Après le choix du nom de l'instance à installer, le processus d'installation vérifie les capacités disque du système.



Un même serveur ne peut héberger qu'une seule instance par défaut.

Chaque instance est parfaitement identifiée par son nom. Le nom de l'instance respecte les règles suivantes :

- Le nom de l'instance est limité à 16 caractères.
- Les majuscules et les minuscules ne sont pas différenciées.
- Le nom de l'instance ne peut pas contenir les mots DEFAULT et MSSQLSERVER, ainsi que tout autre mot réservé.
- Le premier caractère du nom de l'instance doit être une lettre (A à Z).
- Les autres caractères peuvent être des lettres, des chiffres ou bien le trait de soulignement (_).
- Les caractères spéciaux tels que l'espace, l'antislash (\), la virgule, les deux points, le point-virgule, la simple cote, le "et" commercial (&) et l'arobase (@) ne sont pas autorisés dans le nom d'une instance.

c. Les services SQL Server

En fonction des choix faits lors de l'installation, il peut y avoir de nombreux services créés, les plus courants étant :

- SQL Server Database Services
- Agent SQL Server
- Analysis Services
- Reporting Services
- Integration Services
- Recherche de texte intégral
- SQL Server Browser (sert uniquement pour se connecter à des instances nommées et est connecté au port UDP 1434)

Certains de ces services sont liés à l'instance SQL Server installée. C'est par exemple le

cas pour SQL Server Database Services et l'Agent SQL Server. Certains services comme Reporting Services sont directement liés à des utilisations particulières de SQL Server, ici c'est le cas de la Business Intelligence.

Dans le cadre de cet ouvrage, ce sont principalement les services SQL Server Database Services et l'Agent SQL Server qui vont être étudiés.

Instance	Nom pour se connecter à l'instance	Nom du service
Par défaut	<i>Nom Serveur</i>	SQL Server (MSSQLSERVER)
Nommée	<i>Nom Serveur \ NomInstance</i>	SQL Server (<i>NomInstance</i>)

Compte de service

Les logiciels côté serveur s'exécutent sous forme de services. Comme tous les services, pour accéder aux ressources de la machine, ils utilisent le contexte d'un compte d'utilisateur. Très souvent, les services s'exécutent dans le contexte du compte Local System. Ce compte permet d'obtenir toutes les ressources de la machine locale, mais il ne permet pas d'accéder à des ressources du domaine.

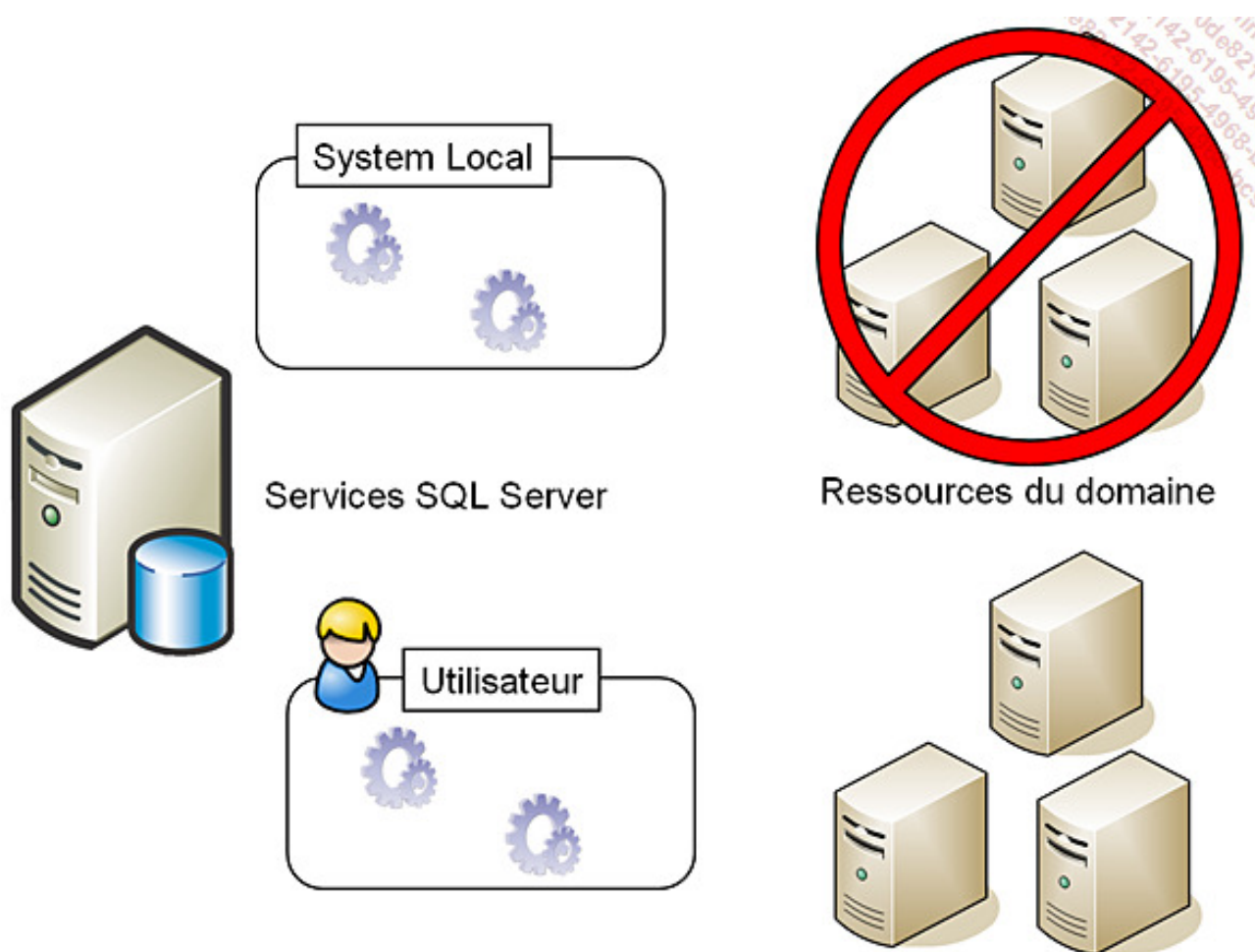
Les deux services, que sont MS SQL Server et SQL Server Agent doivent être capables d'accéder à des ressources du domaine, afin de pouvoir utiliser toutes les fonctionnalités proposées par SQL Server (gestion des tâches planifiées, réplication...). Lors de l'installation, il est possible de préciser le compte du domaine (utilisateur standard ou compte managé) qui sera utilisé pour les deux services. Celui-ci devra être membre du groupe des administrateurs locaux du serveur SQL Server. **Les préconisations en termes de sécurité sont d'utiliser un compte managé.** L'avantage de ce type de compte est que l'on ne connaît pas son mot de passe, qui est géré par Windows (environ une centaine de caractères et renouvelé tous les mois !).

Pour simplifier les opérations de gestion, il est recommandé d'utiliser le même compte Windows pour les deux services.



Le même compte pourra également être utilisé pour le service de recherche de texte intégral.

Si l'entreprise comporte plusieurs serveurs SQL dans plusieurs domaines, il est préférable que chaque serveur SQL Server s'exécute en utilisant un compte de service lui étant dédié.



Compte d'ouverture de session

Création d'un compte managé et paramétrage

La création d'un compte managé se fait via PowerShell et nécessite l'installation du module PowerShell pour Active Directory (installé avec les consoles d'administration Active Directory).

Dans une console PowerShell lancée en mode administrateur, il suffit d'exécuter le code

suivant (sqlservice correspondant au nom du compte managé et SQL1 au compte d'ordinateur du serveur SQL Server) :

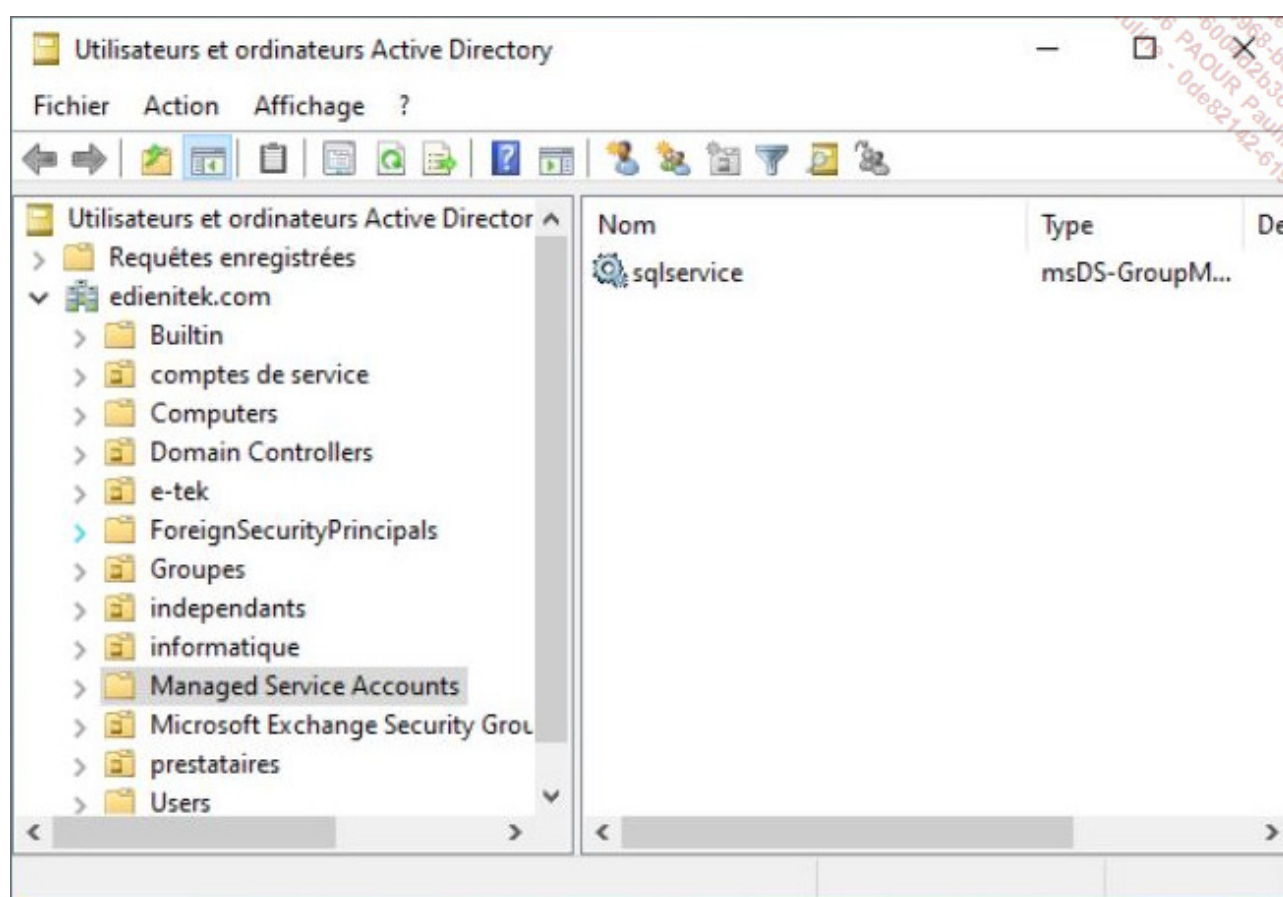
```
Add-KdsRootKey -EffectiveTime ((get-date).addhours(-10))
```

```
New-ADServiceAccount -Name sqlservice -DNSHostName sqlservice.edienitek.com -PrincipalsAllowedToRetrieveMa
```

```
Add-ADComputerServiceAccount -identity SQL1 -ServiceAccount sqlservice
```

à lancer sur le serveur SQL Server

```
Install-ADServiceAccount -Identity sqlservice
```



Il est possible de paramétrer les services de SQL Server de façon à ce qu'ils démarrent automatiquement lors du démarrage de Windows. L'avantage de cette option est qu'il n'est pas nécessaire d'ouvrir une session sur le poste en tant qu'administrateur, pour lancer les services du SGBDR. Lors de chaque démarrage de Windows, les bases gérées par le SGBDR sont immédiatement accessibles.

Ces deux options, **Comptes de services** et **Démarrage automatique de service**, peuvent être fixées lors de l'installation de SQL Server ou bien une fois que SQL Server est installé au travers des outils d'administration.

Programme d'installation de SQL Server 2022

Configuration du serveur

Spécifiez les comptes de service et la configuration du classement.

Édition
Termes du contrat de licence
Règles globales
Microsoft Update
Mises à jour du produit
Installer les fichiers d'installation
Règles d'installation
Sélection de fonctionnalités
Règles de fonctionnalité
Configuration de l'instance
Configuration du serveur
Configuration du moteur de ba...
Règles de configuration des fo...
Prêt pour l'installation
Progression de l'installation
Terminé

Comptes de service Classement

Microsoft conseille d'utiliser un compte distinct pour chaque service SQL Server.

Service	Nom du compte	Mot de passe	Type de démarrage
SQL Server Agent	EDIENITEK\sqlservice\$		Automatique
Moteur de base de données SQL ...	EDIENITEK\sqlservice\$		Automatique
SQL Server Browser	NT AUTHORITY\LOCAL S...		Désactivé

☐ Accorder le privilège Effectuer des tâches de maintenance en volume au service du moteur de base de données SQL Server

Ce privilège permet l'initialisation instantanée des fichiers en évitant la mise à zéro des pages de données. Cela peut entraîner la divulgation d'informations en autorisant l'accès au contenu supprimé.

[Cliquez ici pour des détails](#)

< Précédent Suivant > Annuler

d. Paramètres de classement

La langue par défaut de l'instance de SQL Server va avoir une incidence directe sur le classement sélectionné.

Il est possible d'installer SQL Server quelle que soit la langue définie au niveau du système d'exploitation.

Il faut veiller à ne pas confondre la page de code et le classement.

La page de code est le système de codage des caractères retenus. La page de code permet d'identifier 256 caractères différents. Compte tenu de la diversité des caractères utilisés d'une langue à l'autre, il existe de nombreuses pages de code. Pour pouvoir prendre en compte des données exprimées dans différentes langues, il est possible d'utiliser le système Unicode. Ce système de codage des caractères permet d'utiliser deux octets pour coder chaque caractère. Le système Unicode permet donc de coder 65536 caractères différents ce qui est suffisant pour coder la totalité des caractères utilisés par les différentes langues.

Cette solution avec les pages de codes n'est acceptable que si l'on stocke dans la base de données uniquement des données en provenance d'une seule langue. Or, avec la montée en puissance des applications de commerce électronique, les bases de données vont de plus en plus souvent contenir des informations (comme les nom, prénom et adresse des clients) en provenance de différentes langues. Pour pouvoir supporter les caractères spécifiques à chaque langue, il faut utiliser le type de données **Unicode**. Les données de type Unicode sont conservées dans les types **nchar** et **nvarchar**. Au niveau de l'application cliente, qui permet la saisie et l'affichage des informations, le type de données Unicode doit également être supporté.



L'espace réclamé par le type de données Unicode est deux fois plus important que pour les données non Unicode. Mais, ce léger désavantage est largement compensé par le temps gagné lors de l'affichage des données sur le poste client. En effet, avec le type de données Unicode, il n'est plus nécessaire de réaliser un mappage entre la page de code utilisée dans la base de données pour stocker l'information, et la page de code utilisée sur le poste client pour afficher l'information.

Le classement correspond à un modèle binaire de représentation des données et qui permet de définir des règles de comparaisons ainsi que des règles de tris. Par exemple, le classement permet de définir sur la langue française comment les caractères accentués doivent être pris en compte lors d'opération de comparaison et de tris.

Il existe par défaut trois types de classements :

- Les classements Windows qui s'appuient sur les paramètres de langues définies au niveau Windows. En s'appuyant sur ce type de classement, les ordres de tris, de comparaisons s'adaptent automatiquement à la langue du serveur.
- Les classements binaires sont réputés pour leur rapidité de traitement. Ils sont basés sur le code binaire utilisé pour enregistrer chaque caractère d'information au format unicode ou non.
- Les classements SQL Server sont présents pour assurer une compatibilité ascendante avec les versions précédentes de SQL Server. Il est donc préférable de ne pas faire ce choix dans le cadre d'une nouvelle installation.

Un classement doit nécessairement être précisé lors de la création de l'instance, il deviendra le classement par défaut de l'instance et sera utilisé par les bases de données **master**, **msdb**, **tempdb**, **model** et **distribution** (attention, ce classement ne peut pas être modifié sans réinstaller SQL Server).

Lors de la création des bases de données utilisateurs, il sera possible de préciser un autre classement par l'intermédiaire de la clause **COLLATE**. Dans ce cas, les bases utilisateurs et système (en particulier tempdb) n'utiliseront pas le même classement, ce qui pourra poser des problèmes lors de l'exécution de requêtes utilisant des tables temporaires (préfixées avec le caractère #).



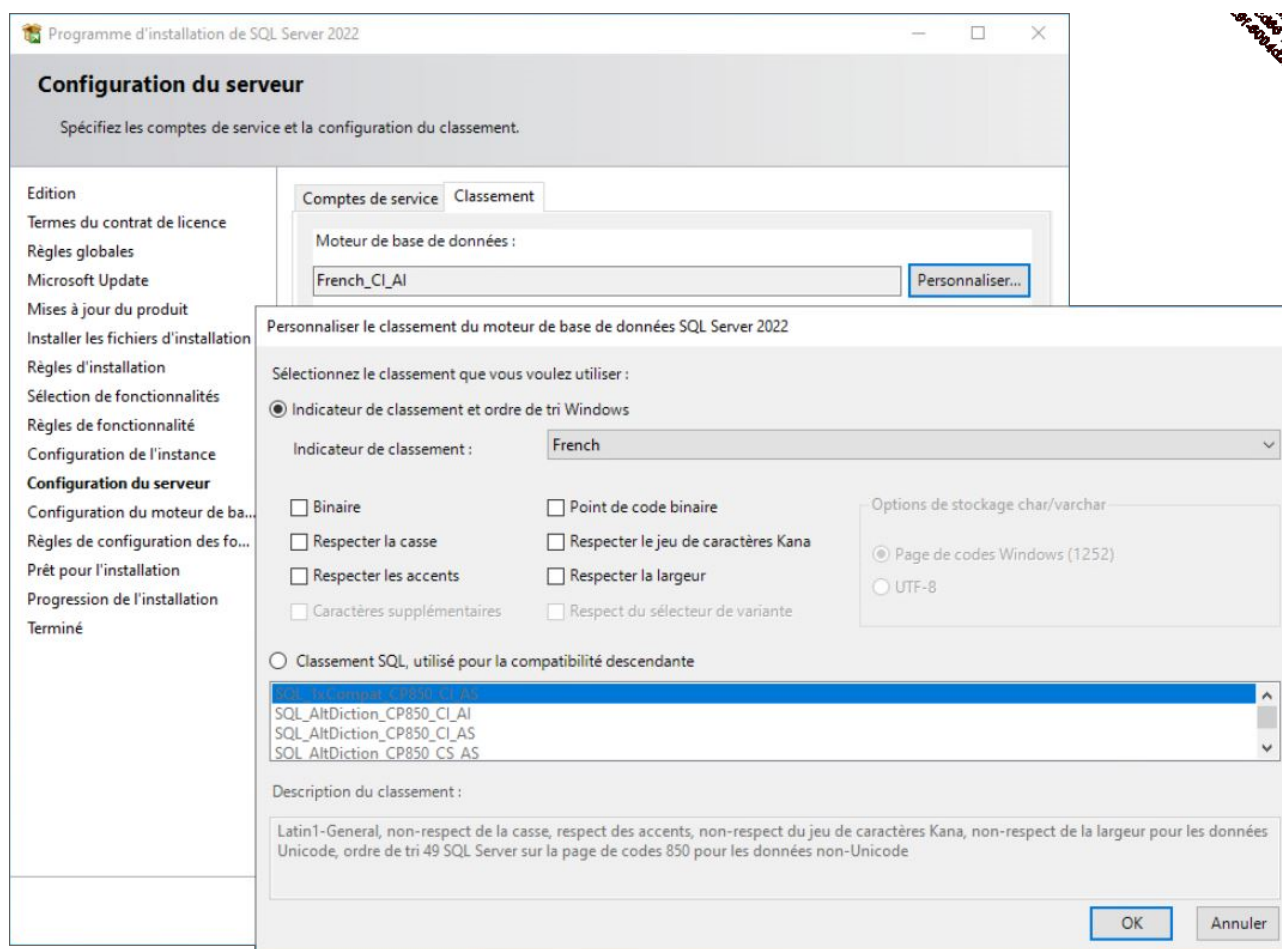
Comme les classements contrôlent l'ordre de tri des données Unicode et non Unicode, il n'est pas possible de définir des ordres de classement incompatibles.

Le nom des différents ordres de tri est toujours structuré de la même façon : le nom de la langue, par exemple *French* suivi de plusieurs couples de caractères représentant les options relatives à l'ordre de tri. Ces options sont les suivantes :

_CS	Soit <i>Case Sensitive</i> , donc la casse est respectée, autrement dit l'ordre de tri fait la distinction entre majuscules et minuscules.
_AS	Soit <i>Accent Sensitive</i> , la distinction entre le caractère accentué ou non accentué est conservée.
_KS	Soit <i>Kana Sensitive</i> , donc maintient la différence entre les jeux de caractères japonais.
_WS	Soit <i>Width Sensitive</i> , l'ordre de tri considère comme différents le même caractère codé en unicode (nchar ou nvarchar) sur 2 octets et celui codé par rapport à une page ASCII sur 1 octet.



Ces options peuvent apparaître avec un I à la place du S. Le I signifie Insensitive et permet de préciser que l'ordre de tri ne fait pas de distinction sur ce critère.



Fixer les paramètres de classement

Le choix du classement dépendra des applications et des bases de données gérées par le serveur SQL Server. Cependant, un choix insensible à la casse et aux accents (French_CI_AI par exemple) semble correspondre à la majorité des cas (par défaut sensible aux accents : CI_AS) : ainsi, le résultat des requêtes ne dépendra pas de la saisie ou non des accents par l'utilisateur depuis l'application cliente. Et si toutefois ce résultat de requête doit ponctuellement différencier les accents et/ou la casse, il est toujours possible d'utiliser la clause **COLLATE** dans une requête **SELECT**.

Il est possible de connaître le classement adopté au niveau du serveur par l'intermédiaire de la fonction **SERVERPROPERTY** :

```
Select serverproperty('collation');
```

Il est également possible de connaître l'ensemble des classements disponibles sur le serveur en utilisant la fonction `fn_helpcollations()` :

```
Select * from sys.fn_helpcollations();
```



ATTENTION : le respect de la casse s'applique aussi bien aux identificateurs qu'aux données. Si l'on choisit l'ordre de tri binaire avec le respect de la casse, alors toutes les références aux objets doivent être réalisées en utilisant la même casse que celle spécifiée lors de la création de ces objets.

e. Mode d'authentification

La gestion des comptes d'utilisateur peut s'appuyer intégralement sur les comptes des utilisateurs Windows mais il est aussi possible de définir des comptes d'utilisateur et de les gérer intégralement au sein de SQL Server. Dans ce cas, il est nécessaire de spécifier le mot de passe de l'utilisateur SQL Server qui possédera les privilèges d'administrateur SQL Server.

Il est très fortement recommandé d'utiliser le seul mode d'authentification Windows et de ne recourir au mode mixte que dans les seuls cas où cela est strictement nécessaire. Dans ce dernier cas, il est alors indispensable de saisir un mot de passe fort pour l'utilisateur **sa** (administrateur de l'instance SQL Server en cours d'installation).

Comme ce type de configuration peut être corrigé après l'installation de l'instance, en cas de doute, il est préférable de ne retenir que le seul mode d'authentification Windows.

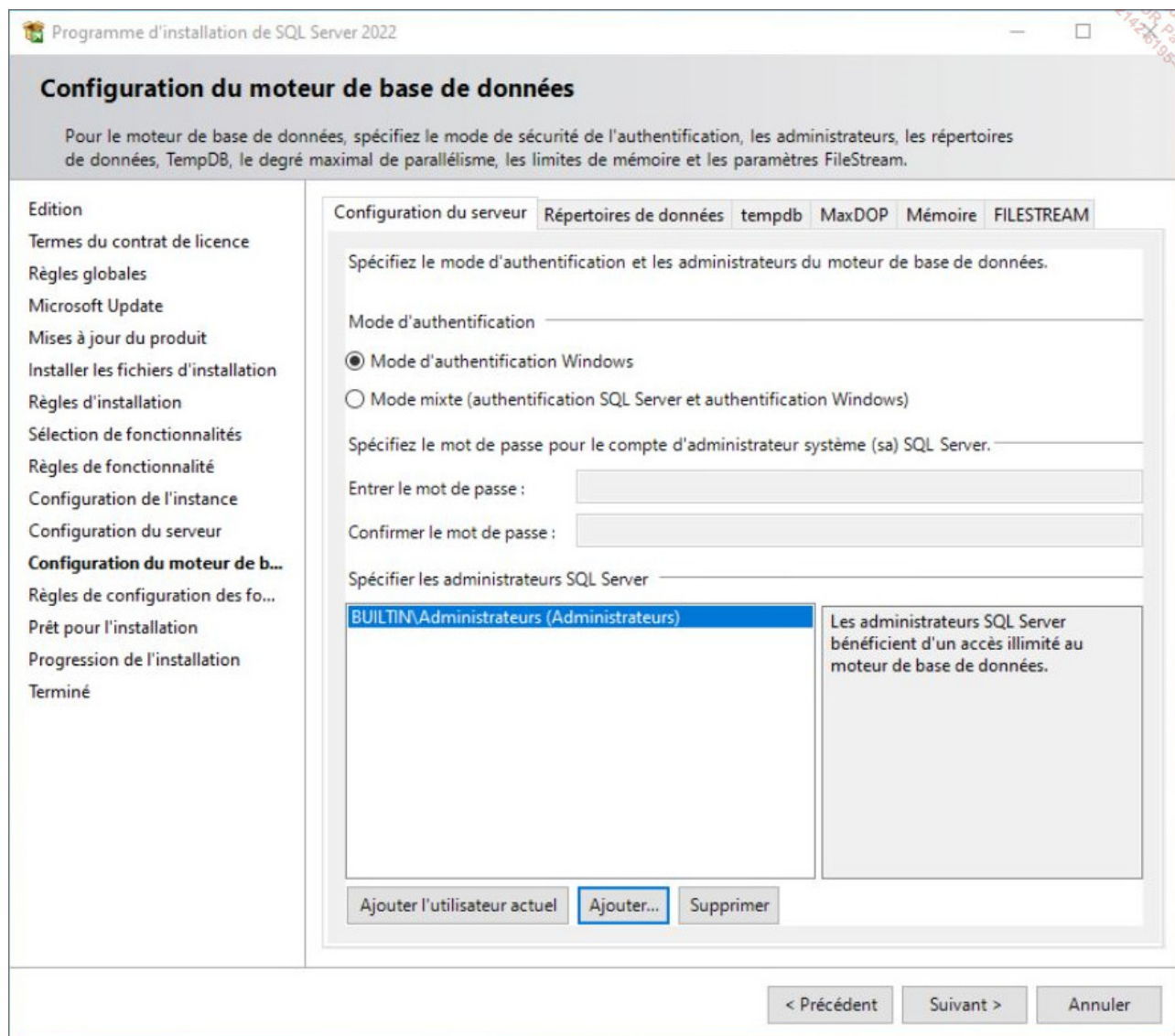
Lors de l'installation, il faudra spécifier au minimum un administrateur pour l'instance, par exemple le groupe local des administrateurs.

f. Configuration du moteur de base de données

La configuration du moteur de base de données permet de spécifier le mode sécurité choisi, l'emplacement des fichiers de données, ainsi que l'activation ou non de l'option

FILESTREAM.

Ces différents choix sont personnalisables au travers des onglets correspondants. Par exemple, pour bénéficier dès la fin de l'installation de l'option **FILESTREAM**, il est nécessaire d'activer ce choix depuis l'onglet correspondant.



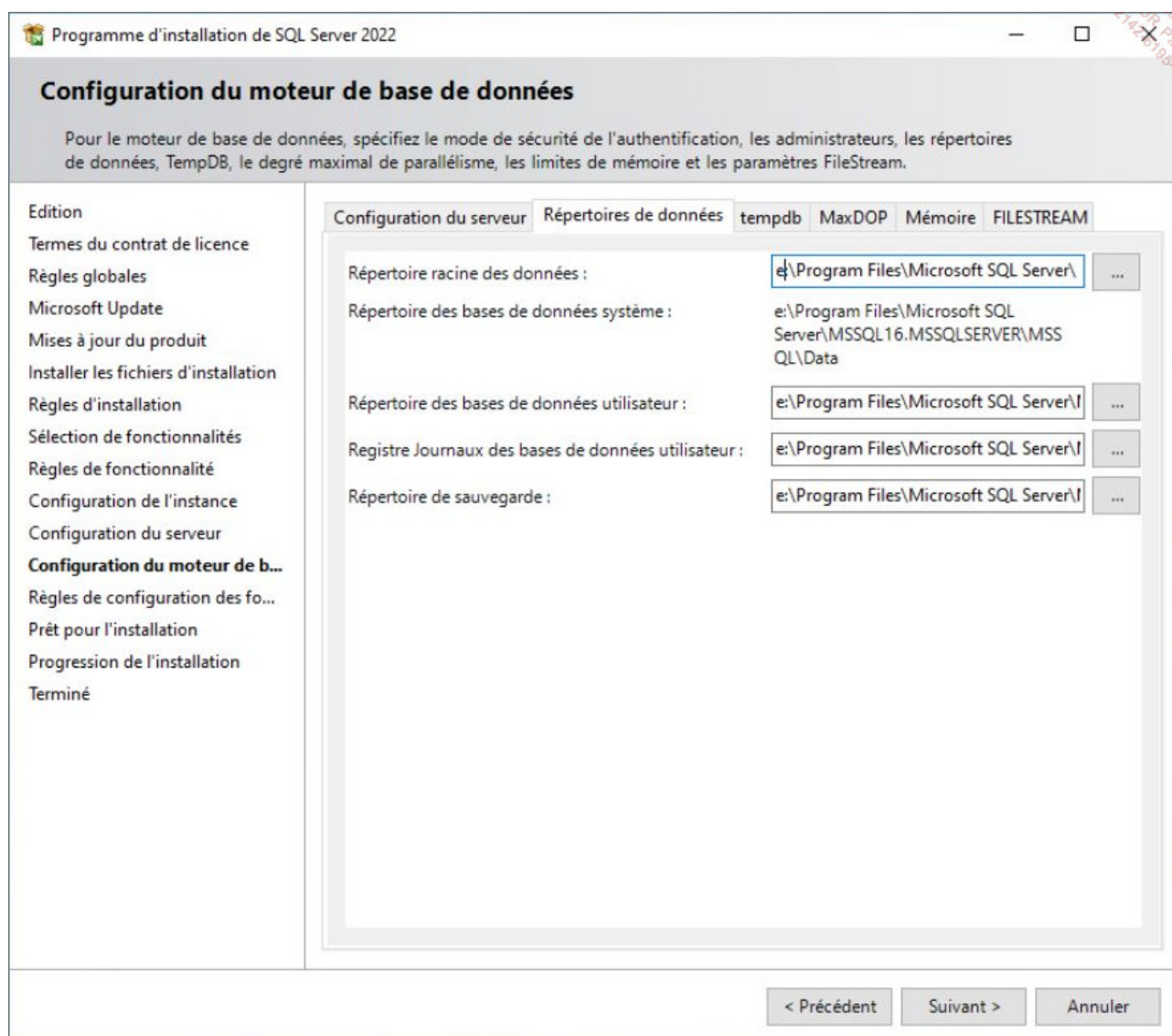
Ces différents choix peuvent être modifiés/outrepassés par la suite lors de la configuration du serveur ou bien lors de la gestion des fichiers relatifs à une base de données.

Pour une sécurité plus élevée au niveau des utilisateurs, il est recommandé de s'appuyer sur le contexte de sécurité Windows et d'interdire la sécurité SQL Server. Cela évitera, par exemple, que des développeurs codent en dur un nom et un mot de passe dans une application ou bien dans un fichier de configuration.

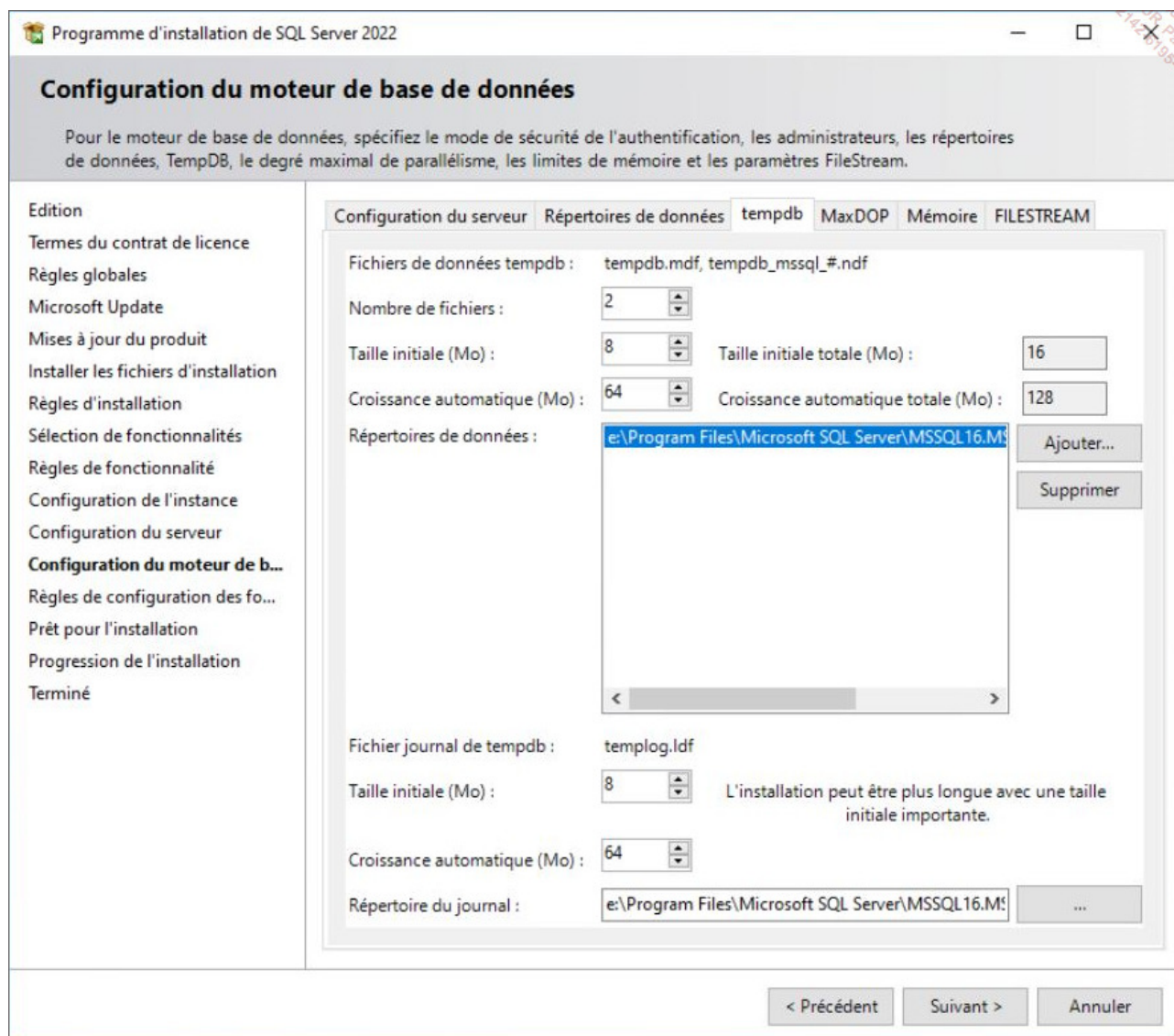


Pour les instances SQL Express, une connexion correspondant au groupe BULTIN\Users est automatiquement ajoutée. Ainsi, tous les utilisateurs du poste peuvent se connecter à l'instance. Il est bien sûr possible de supprimer cette connexion sans que cela pose de problème quant au bon fonctionnement de l'instance.

C'est également à ce niveau qu'il est possible de définir le dossier par défaut pour les fichiers de données des bases utilisateurs et système. En effet, le processus d'installation propose par défaut de stocker ces informations dans le dossier **Program Files**. Il est donc très fortement recommandé de spécifier à cet endroit un emplacement sur un volume séparé pour contenir les fichiers des bases de données utilisateurs. Ce volume devra être formaté avec une taille d'unité d'allocation de 64 ko.



C'est également à ce niveau qu'il est possible de configurer la base tempdb. Celle-ci devra être composée d'autant de fichiers qu'il y a de cœurs de processeur. Ce nombre de fichiers ne peut toutefois être supérieur à 8. Tous les fichiers devront avoir la même configuration (taille, croissance automatique...) et devront être placés sur un volume autre que C: et formaté également avec une taille d'unité d'allocation de 64 ko.



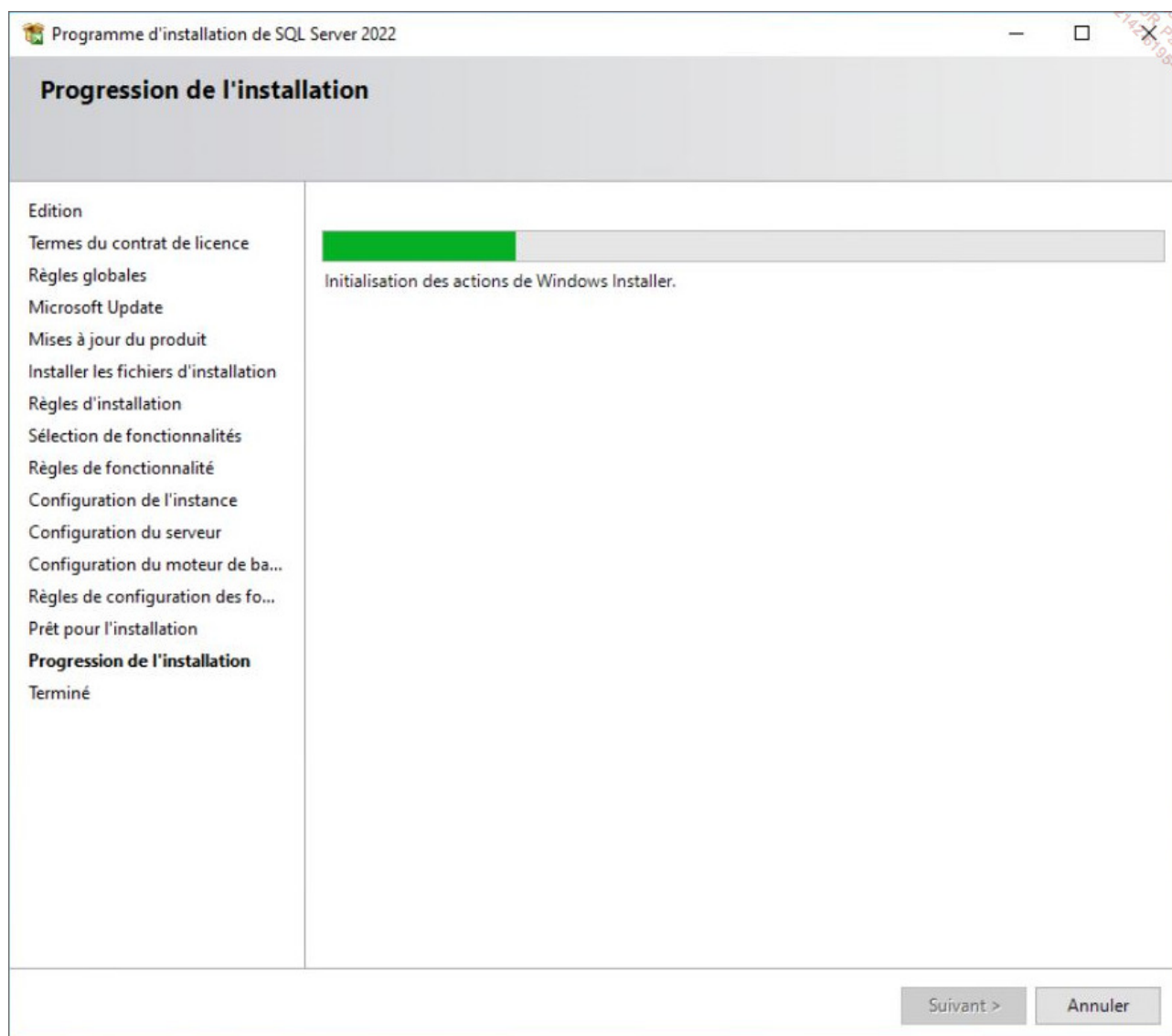
Les options **MaxDOP** (degré maximum de parallélisme) correspondant au nombre maximum de cœurs utilisés pour l'exécution d'une requête, de **mémoire minimum et maximum** de l'instance peuvent être configurées dès l'installation mais restent modifiables par la suite.

L'option **FILESTREAM** peut facilement être mise en place sur une instance déjà installée. Mais bien évidemment, si lors de l'installation on sait que cette option doit être mise en place, alors il est préférable de l'activer dès le processus d'installation.

g. Synthèse du processus d'installation

Les règles d'installation sont vérifiées afin de s'assurer que rien ne viendra bloquer le

processus d'installation, puis le rappel des éléments qui vont être installés est proposé. Après validation de ce dernier écran, l'installation proprement dite débute.



3. Gestion du réseau

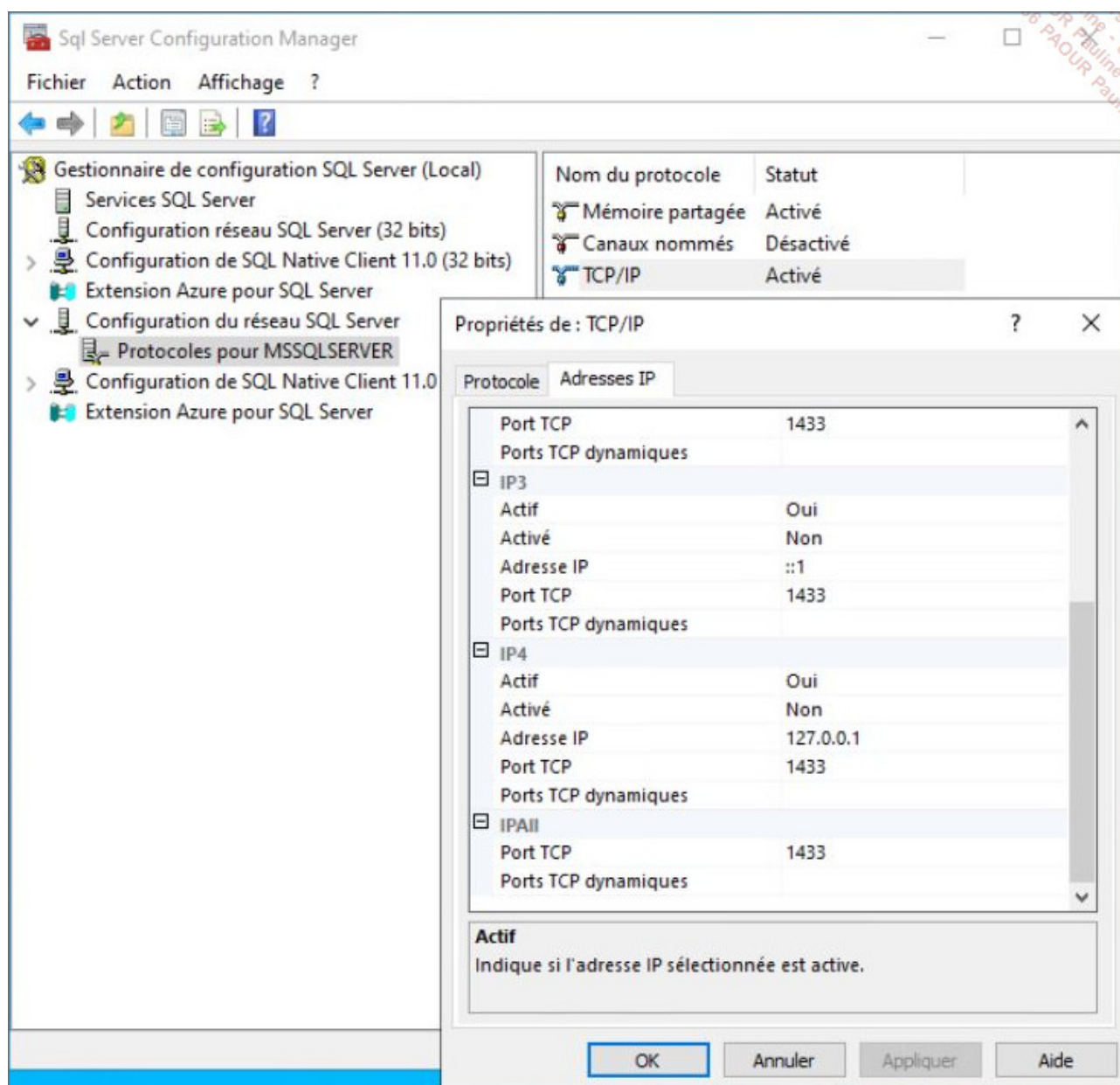
SQL Server utilise des bibliothèques réseau afin d'assurer la gestion de la transmission des paquets entre le serveur et le client. Ces bibliothèques réseau, existant sous forme de DLL (*Dynamic Link Library*), procurent toutes les opérations nécessaires pour établir le dialogue entre le serveur et le client même si ces deux processus se trouvent sur le même poste.

Ces bibliothèques réseau sont utilisées par l'application via le mécanisme IPC ou Communication Inter Processus.

Un serveur peut être à l'écoute, simultanément, de plusieurs bibliothèques et accepte donc des demandes provenant de clients dialoguant avec des protocoles réseau différents. La seule obligation pour que le serveur puisse répondre au client est que la bibliothèque réseau correspondant à celle du client doit être installée sur le serveur.

Une fois que les bibliothèques réseau sont installées sur le serveur, il faut encore configurer les net library afin que le serveur puisse les prendre en compte.

La gestion du réseau entre le poste client et le serveur passe principalement par TCP/IP. Attention à bien l'activer après l'installation du serveur.



Bibliothèques disponibles :

Canaux nommés

Les canaux nommés sont désactivés pour toutes les éditions de SQL Server. Leur utilisation se limite au dialogue entre les outils graphiques et le service SQL Server sur le poste serveur.

L'instance par défaut ouvre le canal de communication nommé `\\.\pipe\sql\query`.

Sockets TCP/IP

Ce protocole permet d'utiliser les sockets Windows traditionnels.

Pour pouvoir utiliser correctement le protocole TCP/IP, il est important de préciser le numéro de port auquel SQL Server répond. Par défaut, pour l'instance par défaut, il s'agit du port 1433, numéro officiel attribué par l'IANA (*Internet Assigned Number Authority*) à Microsoft. Il est également possible d'utiliser un proxy. Dans ce cas c'est l'adresse du proxy qui sera précisée lors de la configuration du protocole TCP/IP.

Le port 1433 est utilisable par SQL Server si aucune autre application ou processus ne tente de l'utiliser simultanément.

Pour les instances nommées, il est nécessaire de paramétrer un port disponible sur le serveur SQL et créer les règles nécessaires au niveau des pare-feu. C'est le service SQL Server Browser (UDP 1434) qui fournit le numéro de port correspondant de l'instance au client qui souhaite s'y connecter.



Dans le cas où SQL Server est configuré pour utiliser un port dynamique, le numéro du port est susceptible de changer à chaque démarrage de SQL Server.

4. Mode de licence

Le mode de licence est directement lié à l'édition choisie et à son mode d'utilisation. Il existe différents modes de licence qui sont soit par serveur associé à des licences d'accès clients, soit par processeur.

L'édition Entreprise ne propose qu'un mode de licence par processeur, l'édition Business Intelligence est disponible uniquement en mode licence par serveur, tandis que l'édition standard propose les deux modes.

Les licences par processeur prennent en compte les cœurs du processeur et il s'agit plus exactement d'une licence par cœur utilisé par SQL Server. Au niveau de la licence, il ne peut y avoir moins de quatre cœurs sur ce type de licence.

Le tableau ci-dessous résume les différentes possibilités de licence par rapport à l'édition.

	Édition		
	Entreprise	Standard	Business Intelligence
Licence par cœur	X	X	
Licence par serveur		X	X

Dans le cas de l'utilisation d'une licence serveur, il est nécessaire de compléter par des licences d'accès clients (utilisateur ou périphérique).

Lorsque les serveurs de base de données sont virtualisés, il est nécessaire de disposer d'une licence par serveur (pour chaque serveur virtualisé) et de licence d'accès clients, ou bien d'une licence par cœur virtuel pour profiter sans limite des possibilités de virtualisation sur un serveur.



La gestion des licences est un élément sensible, et il convient de s'assurer que la solution retenue est conforme aux règles édictées par Microsoft. Seule une synthèse de ces règles est présentée ci-dessous.

Licence par cœur

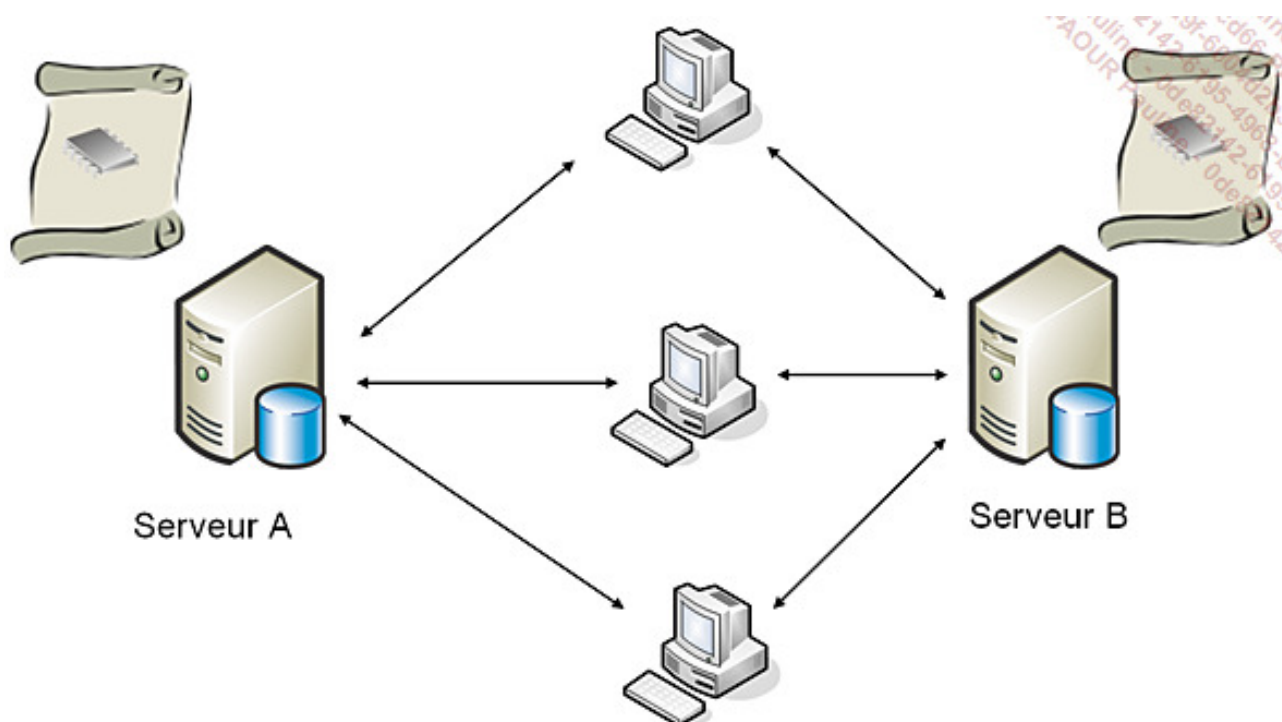
Avec ce mode de gestion des licences, c'est chaque cœur de processeur du serveur qui se voit attribuer une licence d'utilisation de SQL Server. Ainsi, un nombre illimité d'utilisateurs peuvent se connecter au serveur sans qu'il soit nécessaire pour eux de posséder une licence SQL Server. Mis au point au départ pour les applications de type Internet/intranet, ce mode de licence est également utilisable avec n'importe quel type d'application. Son principal avantage réside dans une gestion simplifiée des droits d'utilisation.

Exemple : une entreprise dispose d'un serveur SQL et 10 stations de travail doivent accéder à ce serveur. Les stations de travail se répartissent en 2 groupes de 5 postes :

travail de jour et travail de nuit. Les postes de ces groupes ne peuvent se connecter au serveur que pendant les horaires précisés et il ne peut pas y avoir de chevauchement. Combien de licences d'accès à SQL Server sont nécessaires en mode de licence par cœur de processeur ?

Réponse : une licence d'accès est nécessaire. Elle est gérée par le serveur et autorise les stations de travail à venir se connecter simultanément sur le serveur SQL.

Pour acquérir une licence couvrant la totalité du serveur, il faut acquérir des licences pour couvrir tous les cœurs processeurs présents sur le serveur physique.



Mode de licence par processeur

Licence par utilisateur

Avec ce mode de gestion des licences, chaque utilisateur doit disposer d'une licence pour travailler avec SQL Server. Ce mode de gestion va être intéressant si les utilisateurs peuvent se connecter à SQL Server par l'intermédiaire de différents outils.

L'utilisation d'un serveur d'application qui gère éventuellement un pool de connexions ne réduit pas le nombre de licences nécessaires. C'est chaque utilisateur physique qui a nécessité une licence d'accès.

Une même licence d'accès client ou CAL (*Client Access License*) permet d'accéder à de multiples serveurs. Le niveau de licence du client doit correspondre au serveur le plus exigeant.

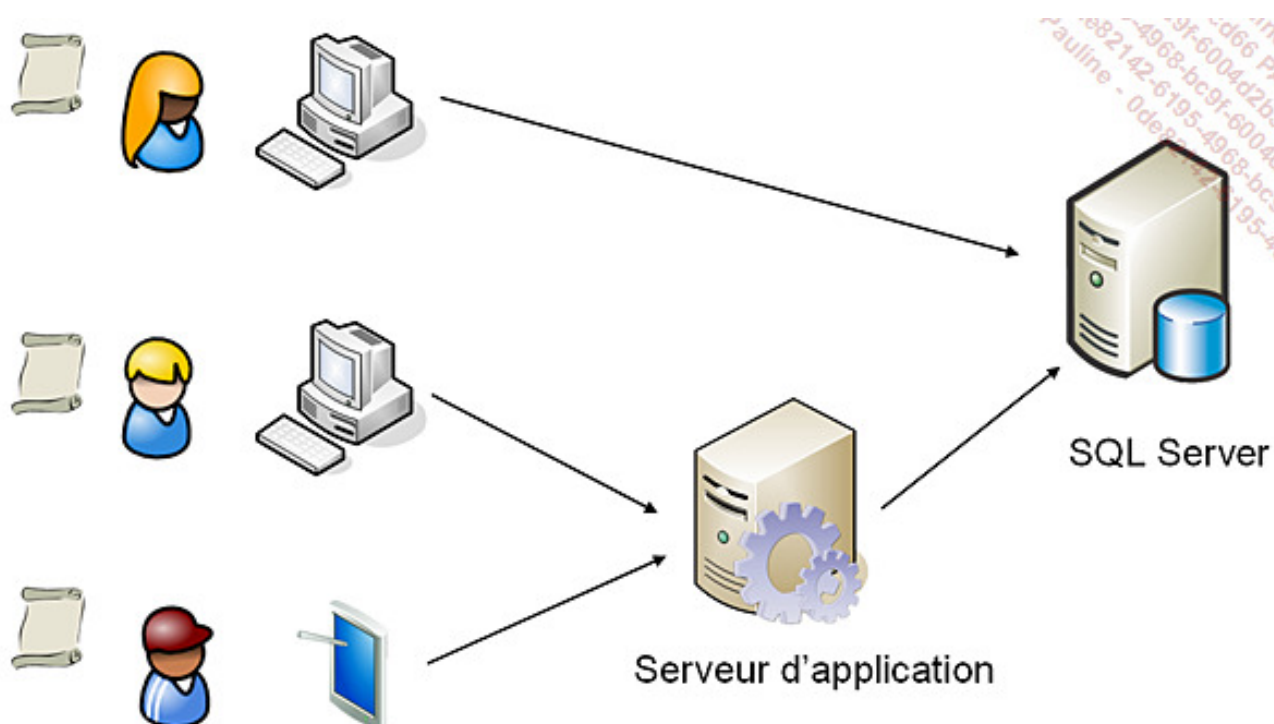
Dans le mode de gestion des licences par utilisateur, il est, en plus, nécessaire d'acquérir une licence de type serveur pour la machine sur laquelle l'instance SQL Server est installée.



Les utilisateurs exprimés ici sont des utilisateurs physiques. Ils peuvent, même si cela n'est pas recommandable, utiliser tous le même utilisateur de base de données.

Exemple

Dans l'exemple suivant, trois utilisateurs différents se connectent au serveur. Cette solution nécessite donc trois licences d'accès client.



Une licence d'accès SQL Server version n permet également d'accéder à un serveur SQL Server version n-1. La réciproque n'est bien entendu pas valide.

Licence par poste

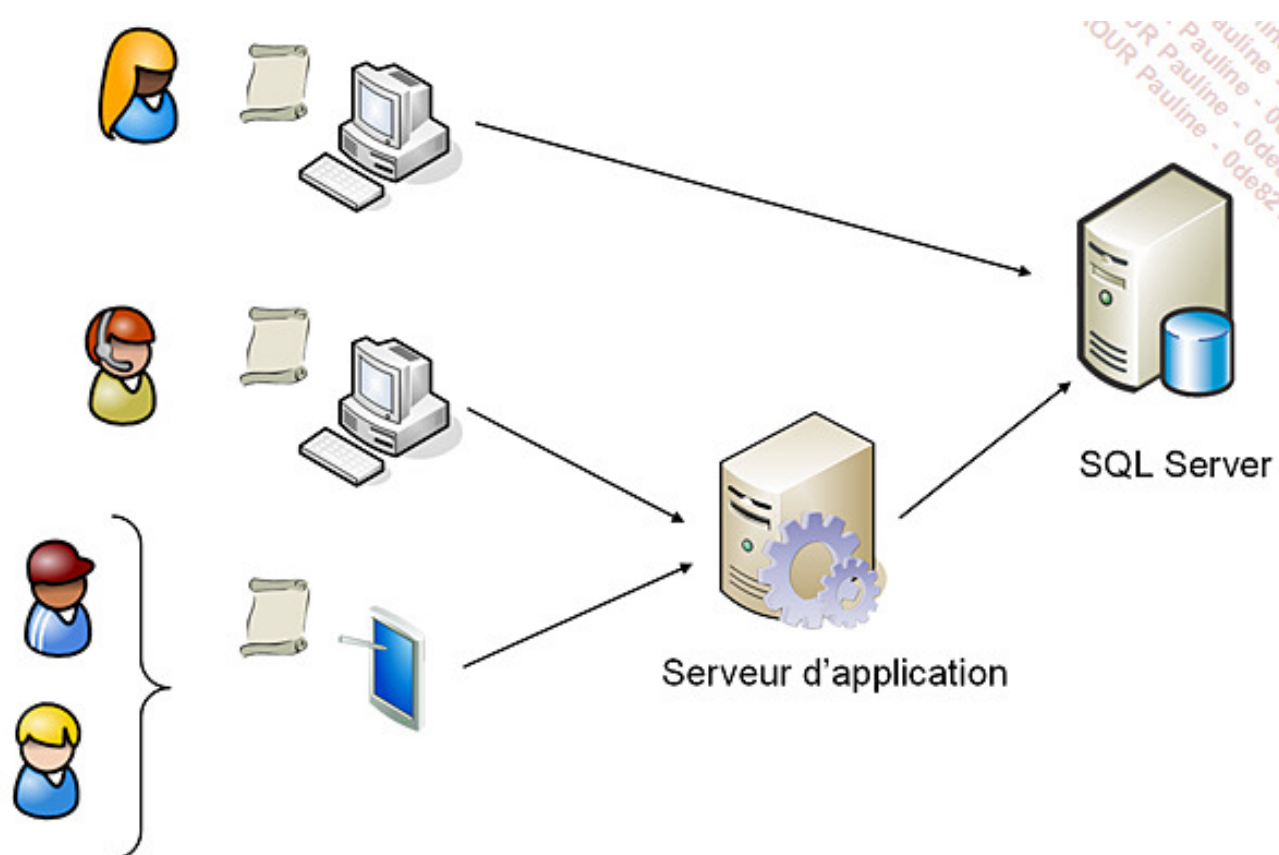
Avec la licence par poste (ou par siège), c'est chaque périphérique qui établit une connexion à un serveur SQL Server qui possède une licence. Cette licence n'est pas liée au nombre d'utilisateurs qui peuvent être amenés à utiliser le poste. Ainsi, si plusieurs utilisateurs se partagent le même poste de travail, ils peuvent utiliser la même licence d'accès.

Muni de cette licence, un poste peut se connecter à plusieurs instances SQL Server. La licence doit être compatible avec l'instance SQL Server la plus exigeante.

Dans le mode de gestion des licences par poste, il est, en plus, nécessaire d'acquérir une licence de type serveur pour la machine sur laquelle l'instance SQL Server est installée.

Exemple

Dans l'exemple suivant, deux utilisateurs se partagent le même périphérique. Il est donc nécessaire de disposer de trois licences d'accès par poste.



5. SQL Server et la virtualisation

Dans le cas de SQL Server, il existe deux cas bien distincts de virtualisation. Le premier consiste à virtualiser des serveurs, et dans ce cas il faut disposer de licences pour chaque machine virtuelle. L'autre cas où la virtualisation est utilisée correspond à des environnements de type cloud privé. Seul le premier cas est abordé ici.

Dans le cas où les serveurs SQL Server sont virtualisés pour permettre une gestion plus aisée des machines, c'est le même mode de gestion de licences que celui présenté pour les serveurs physiques qui s'applique. Dans le cas d'une licence par cœur, tous les cœurs affectés à la machine virtuelle doivent disposer d'une licence SQL Server. Dans le cas des licences d'accès, rien ne change entre la machine physique et la machine virtuelle.

6. Exécuter le programme d'installation

Le programme d'installation situé sur le DVD de SQL Server s'exécute automatiquement dès que le DVD est inséré dans la machine. Si la procédure d'installation ne démarre pas automatiquement, il convient de demander l'exécution du programme SETUP.EXE situé à la racine du DVD.

Installation automatique

Il est possible de réaliser une installation automatique.

La procédure d'installation automatique passe par un usage en ligne de commande et permet de préciser dans la ligne de commande les valeurs des différents paramètres. Même si la définition de cette ligne de commande peut être relativement fastidieuse et complexe dans le cas où des installations de SQL Server doivent être exécutées de façon répétée ou bien de façon automatique, le gain de temps est significatif.

Cependant lors d'une installation en mode interactif de SQL Server, tous les choix faits au niveau des options d'installation se trouvent enregistrés dans un fichier de configuration nommé ConfigurationFile.ini présent dans le journal de l'installation (soit pour SQL Server 2022 dans C:\Program Files\Microsoft SQL Server\160\Setup Bootstrap\Log) et dans le sous-dossier correspondant au jour de l'installation. Si le processus d'installation est exécuté plusieurs fois le même jour, il est possible de distinguer les dossiers grâce à

l'horodatage.

Ce mode de fonctionnement en installation interactive génère un fichier de paramètres permet de configurer rapidement une installation automatique.



Quel que soit le mode d'installation sélectionné, il est toujours nécessaire d'accepter le contrat de licence sauf à quelques exceptions près comme par exemple si vous utilisez un mode de licence par volume.

Syntaxe

```
setup.exe /ConfigurationFile=nomCompletFichier.ini
```

Les options du programme **setup.exe** vont différer en fonction du type d'installation souhaité. Par exemple, dans le cas où SQL Server est installé dans une image en vue d'un déploiement, il existe des options particulières relatives à la préparation de l'image (sysprep).

7. Les bases d'exemple

Lors de l'installation du moteur de base de données, aucune base d'exemple n'est mise en place par défaut. En effet, sur un serveur de production, les bases d'exemples ne sont pas nécessaires et ne peuvent qu'introduire des failles de sécurité. Toutefois, sur un serveur de test ou bien de formation, ces mêmes exemples prennent tout leur sens.

Pour SQL Server, Microsoft propose plusieurs exemples de bases de données : TSQL, la « famille » des bases AdventureWorks. Toutes ces bases sont téléchargeables depuis le site de Microsoft, que ce soit au format fichiers de sauvegarde (.bak) ou script d'installation Transact SQL.

Les exemples de la documentation en ligne de Transact SQL sont essentiellement basés

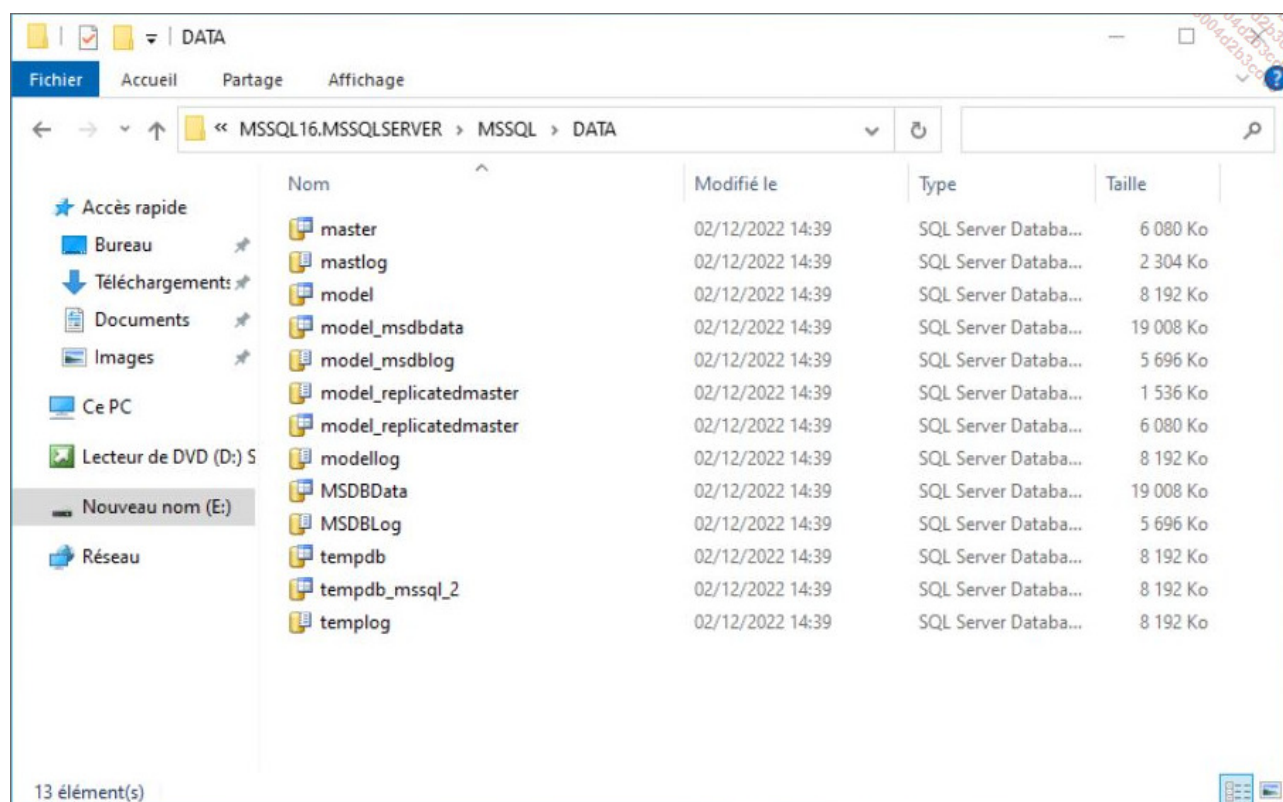
sur la base AdventureWorks et ses dérivés (datawarehouse, etc.).

Vérification de l'installation

Comme pour tout produit, il est important de s'assurer que l'installation s'est correctement exécutée et que le serveur est maintenant opérationnel.

1. Vérifier les éléments installés

Par l'intermédiaire de l'explorateur de fichier, il est possible de vérifier que l'arborescence qui regroupe tous les fichiers nécessaires au bon fonctionnement du moteur de base de données est bien définie. Si l'installation est exécutée avec les paramètres par défaut, alors cette arborescence est c:\Program Files\Microsoft SQL Server\160. Les fichiers de données se trouvent quant à eux dans le dossier c:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\DATA sauf si un chemin autre a été défini lors de l'installation.



Au niveau des fichiers, il est possible de consulter le journal d'installation (summary.txt) qui est présent dans le dossier c:\Program Files\Microsoft SQL Server\160\Setup

Bootstrap\LOG. La lecture de document s'avère principalement utile lorsque le processus d'exécution se termine en échec pour mieux cerner l'origine du problème. Ce fichier summary est généré par chaque exécution du programme d'installation. Une nouvelle exécution du programme d'installation va renommer (avec horodatage) le fichier summary existant afin de conserver la trace de toutes les installations.

2. Vérifier le démarrage des services

Pour un usage classique de la base, les deux services MS SQL Server et SQL Server Agent doivent être démarrés et positionnés en démarrage automatique. Le service MS SQL Server représente le moteur de la base de données et tant que ce service n'est pas démarré il est impossible de se connecter au serveur et de travailler avec les données qu'il héberge. Le service SQL Server Agent prend à sa charge, entre autres, l'exécution et la gestion de toutes les tâches planifiées. Pour gérer les services, il est bien sûr possible de le faire avec les outils proposés en standard par Windows, mais SQL Server propose également ses propres outils.

Le service SQL Server CEIP (MSSQLSERVER) peut être arrêté via la console services de Windows, car il ne sert qu'à remonter des informations d'utilisation à Microsoft.

3. Les mises à jour (Cumulative Updates)

Régulièrement, Microsoft met à disposition des mises à jour de SQL Server appelées *Cumulative Updates*. Celles-ci doivent être installées afin d'avoir les derniers correctifs logiciels et mises à jour de sécurité.

La liste des dernières mises à jour pour toutes les versions de SQL Server se trouve actuellement à l'adresse suivante : <https://learn.microsoft.com/en-us/sql/database-engine/install-windows/latest-updates-for-microsoft-sql-server?view=sql-server-ver16>

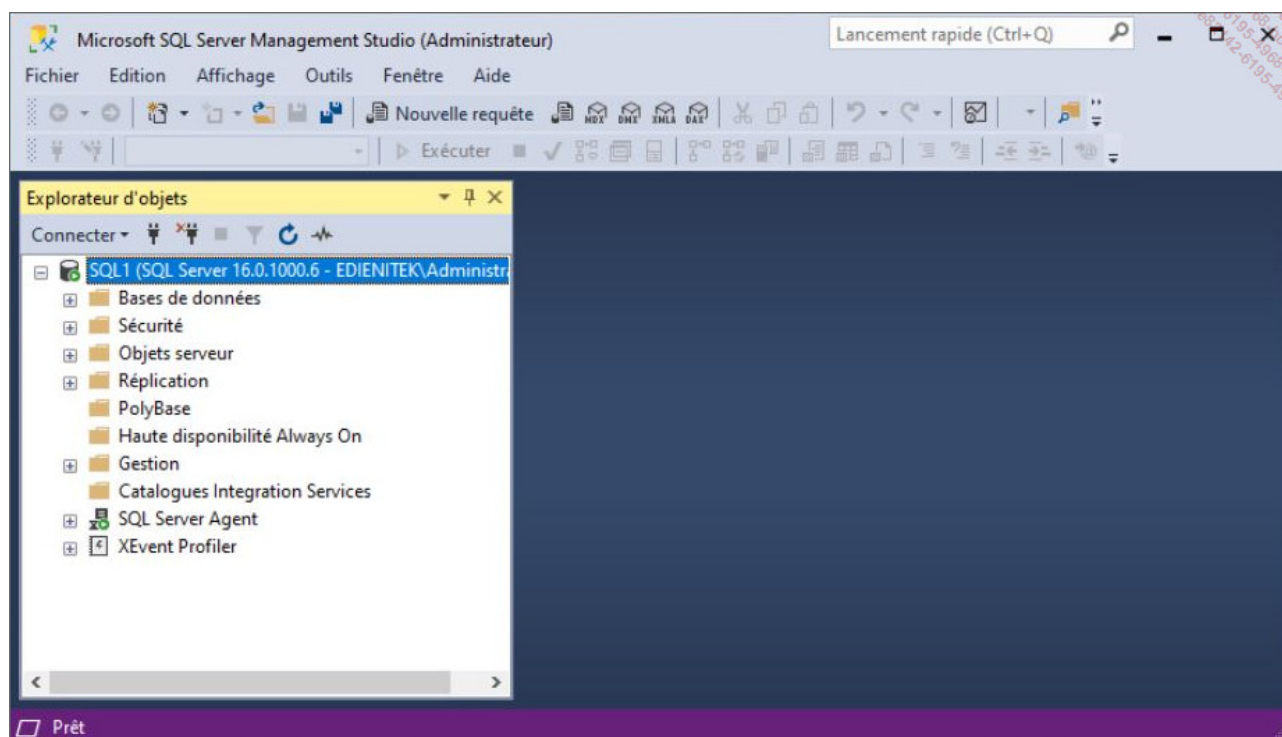
Les outils

SQL Server dispose de nombreux outils. Ces outils sont complémentaires et chacun d'entre eux est adapté à un type de problème ou d'action. Il est important de posséder une vue d'ensemble sur l'objectif visé par chacun de ces outils afin de savoir lequel employer pour résoudre un problème bien précis. Il est possible de faire ici une analogie avec le bricolage, si vous ne possédez qu'un tournevis vous allez essayer de tout faire avec, alors qu'au contraire, si votre caisse à outils est bien garnie il y a de forte chance d'y trouver l'outil qui répond exactement au problème rencontré. Pour SQL Server l'approche est similaire. Il est bien entendu possible de faire quasiment tout en Transact SQL, mais SQL Server propose des outils graphiques autres pour permettre de solutionner des problèmes bien précis. L'utilisation de la plupart d'entre eux va être détaillée tout au long de cet ouvrage. Un regard global permet cependant d'avoir une meilleure vision de l'intérêt présenté par chacun.

SQL Server Management Studio

Il s'agit de l'outil principal de SQL Server et il est destiné aussi bien aux développeurs qu'aux administrateurs.

SQL Server Management Studio (SSMS) est la console graphique d'administration des instances SQL Server. Il est possible d'administrer plusieurs instances locales et/ou distantes depuis cet outil. SQL Server Management Studio est également l'outil principal des développeurs de bases de données qui vont l'utiliser pour définir les scripts de création des tables, des vues, des procédures, des fonctions, des déclencheurs de base de données...



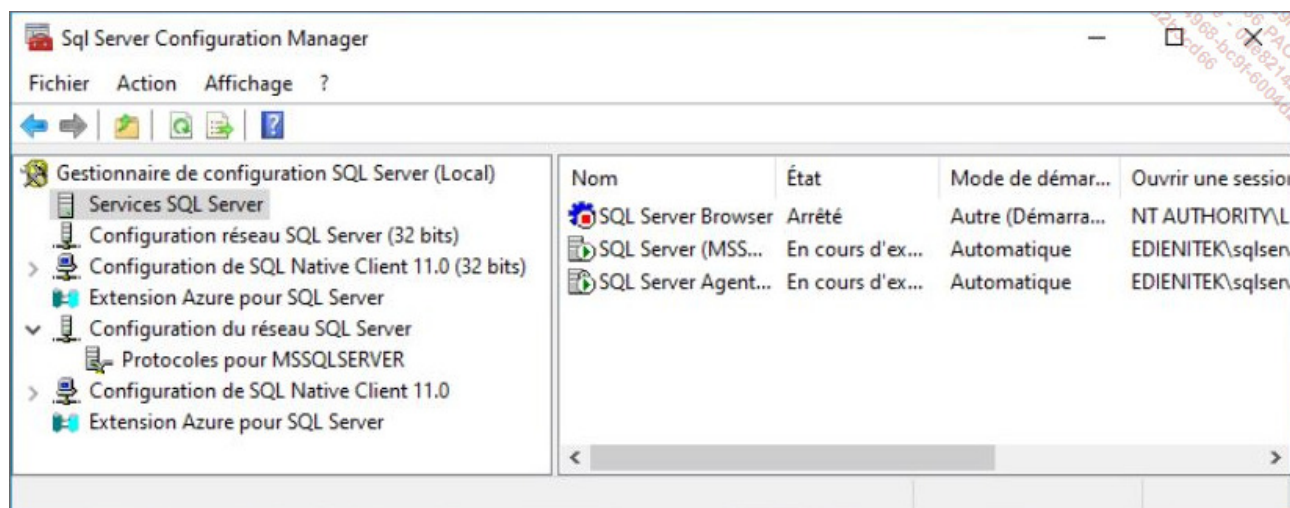
Cet utilitaire peut être démarré depuis la ligne de commande par l'intermédiaire de l'application **ssms**.

SSMS propose un éditeur de scripts SQL et Transact SQL évolué qui intègre la coloration syntaxique afin de distinguer clairement et facilement les mots-clés, les chaînes de caractères, les commentaires. SSMS propose également l'autocomplétion qui permet le complément automatique des mots-clés lors de l'écriture de requête. Cette autocomplétion est activée avec le traditionnel appui sur les touches [Ctrl] [Espace]. En plus des mots-clés, les références aux tables, colonnes, nom de procédures, fonctions sont disponibles au travers de l'autocomplétion. Cette fonctionnalité permet de gagner un temps précieux lors de l'écriture des scripts en limitant les erreurs de saisie.

Afin de faciliter la bonne gestion des bases de données, SQL Server Management Studio propose la génération de rapports. Ces rapports permettent d'avoir une vue globale et synthétique d'un ou de plusieurs éléments de la base ou du serveur. SQL Server propose un certain nombre de rapports prédéfinis mais il est possible de définir ses propres rapports en complément.

[Gestionnaire de configuration SQL Server](#)

Le gestionnaire de configuration de SQL Server permet de gérer l'ensemble des éléments relatifs à la configuration des services et du réseau côté client et côté serveur.



La configuration des services

Les différents services relatifs à SQL Server peuvent être administrés directement depuis cet outil, qui est à préférer à la console services de Windows car des actions supplémentaires sont effectuées sur SQL Server afin de garder une cohérence sur la configuration. En plus des opérations classiques d'arrêt et de démarrage, il est possible de configurer le type de démarrage (automatique, manuel, désactivé) ainsi que le compte de sécurité au sein duquel le service doit s'exécuter.

Propriétés de : SQL Server (MSSQLSERVER)

Groupes de disponibilité Always On Paramètres de démarrage Avancé

Ouvrir une session Service FILESTREAM

Ouvrir une session en tant que :

☐ Compte intégré :

☒ Ce compte :

Nom du compte : EDIENITEK\sqlservice\$ Parcourir

Mot de passe : *****

Confirmer le mot de passe : *****

État du service : En cours d'exécution

Début Arrêter Suspendre Redémarrer

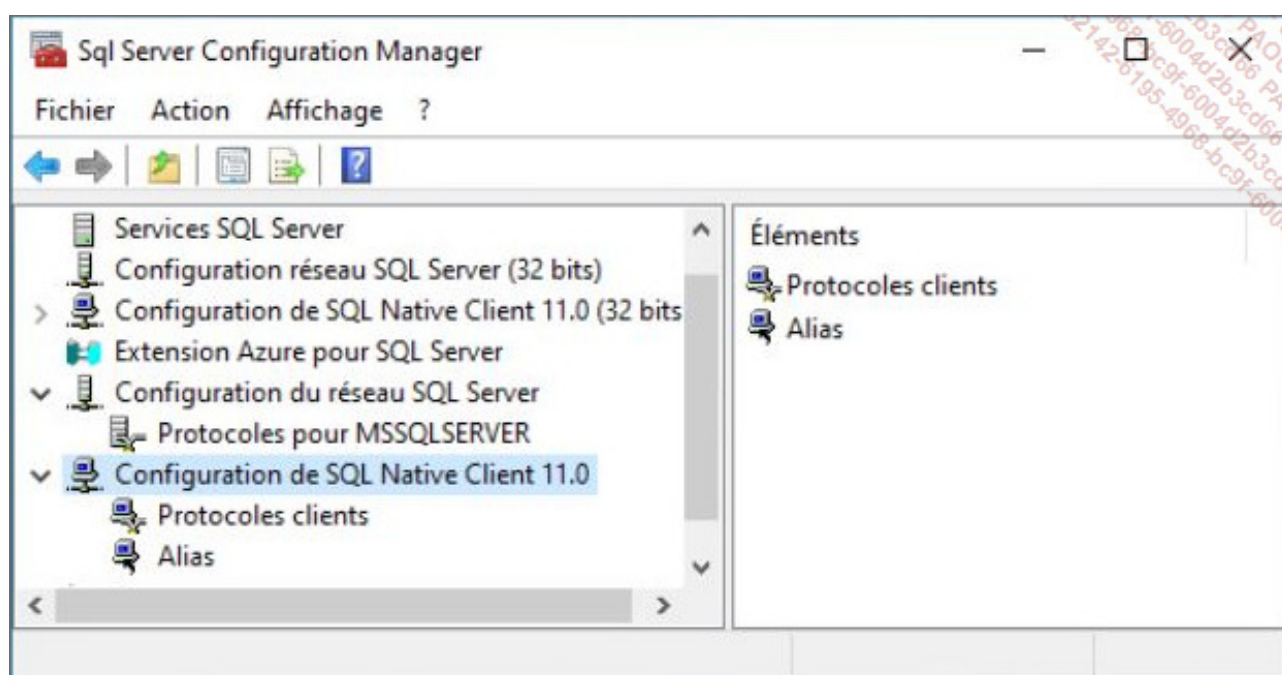
OK Annuler Appliquer Aide

Configuration de réseau SQL Server

Le gestionnaire de configuration permet également de gérer quels sont les protocoles pris en charge au niveau du serveur. Il est également possible à ce niveau de modifier les propriétés spécifiques à chaque protocole comme le numéro du port d'écoute du protocole TCP/IP.

Configuration de SQL Native Client

Cette fois-ci la configuration porte sur les outils client installés localement et plus exactement de définir précisément les protocoles à leur disposition pour entrer en contact avec le serveur, mais aussi, lorsque cela s'avère nécessaire, la possibilité de définir des alias. Cette fonctionnalité est particulièrement intéressante lorsque le nom du serveur est enregistré dans une application et qu'il n'est pas possible ou bien facile de modifier cet enregistrement. La définition d'un alias permet de rediriger toutes les demandes vers un serveur distinct.



Assistant Paramétrage de base de données

Cet assistant permet, entre autres, à partir d'une charge de travail capturée avec SQL Profiler de valider ou non la structure de la base de données. En fin d'analyse, l'assistant va conseiller la création/suppression d'index ou bien le partitionnement de tables afin d'obtenir des gains de performances.

SQLCmd

SQLCmd est un utilitaire en ligne de commande qui permet d'exécuter des scripts SQL. Cet outil va être mis à contribution lors de la demande d'exécution de tâches administratives concernant SQL Server à partir de Windows.

Cet outil permet de se connecter à une instance locale ou non de SQL Server.

L'authentification à cette instance peut être effectuée en s'appuyant sur le mode de sécurité Windows ou bien SQL Server.

SQLCmd permet d'exécuter des scripts SQL que ce soit des instructions du DML (*Data Manipulation Language*) ou du DDL (*Data Definition Language*).

bcp

Il s'agit d'un utilitaire en ligne de commande qui permet d'extraire facilement et rapidement des données depuis la base vers un fichier ou bien de faire l'opération inverse.

sqlps

Permet d'exécuter l'environnement PowerShell spécifique à SQL Server. Cet utilitaire est encore présent pour des raisons de compatibilité mais il est maintenant recommandé d'intégrer les extensions PowerShell pour SQL Server dans l'environnement PowerShell du serveur Windows.

sqldiag

Cet utilitaire de diagnostic peut être exécuté afin de fournir des informations au service support.

sqllogship

Cette application permet de préparer un fichier journal par sauvegarde ou copie avant de l'envoyer vers une autre instance SQL Server.

Tablediff

Comme son nom l'indique, cet utilitaire permet de comparer le contenu de deux tables. Il s'avère particulièrement utile pour résoudre les problèmes de synchronisation qui peuvent apparaître dans le cadre d'une réplication de fusion.

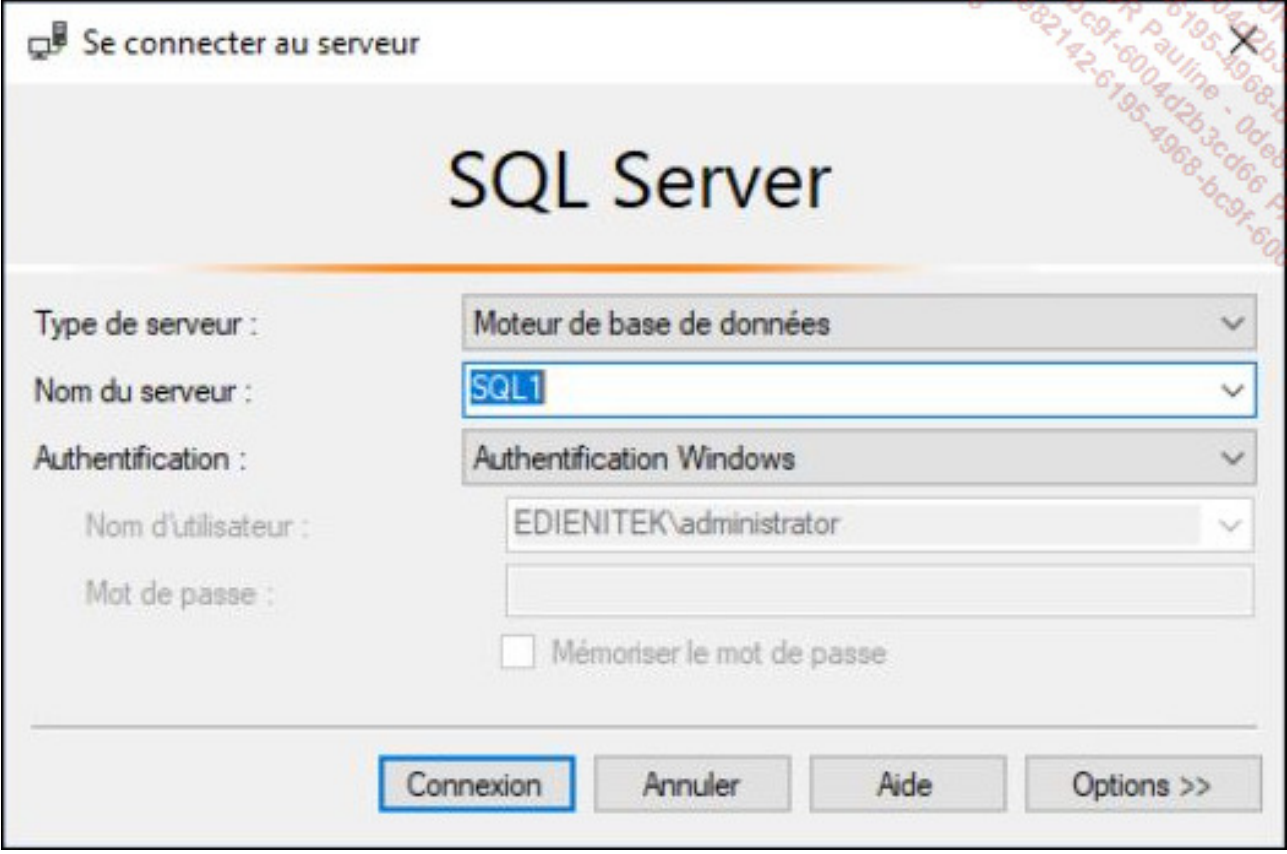
sqlservr

Souvent peu connu, cette application permet de démarrer SQL Server non plus comme un service, mais comme une application. En tant qu'application en ligne de commande, il existe de nombreux commutateurs permettant de définir un certain nombre de paramètres actifs au démarrage de l'application. Par exemple, le commutateur `-f` permet de démarrer SQL Server avec une configuration minimale, c'est-à-dire en ignorant la configuration définie. Ce type de démarrage en mode minimum permet parfois de reprendre la main

suite à un choix de configuration non souhaité.

Utiliser un outil client pour se connecter à SQL Server

Pour établir la première connexion au serveur, plusieurs outils sont disponibles. En mode graphique, il convient de lancer SQL Server Management Studio. Au lancement de l'outil, la fenêtre suivante de connexion est présentée.



Se connecter au serveur

SQL Server

Type de serveur : Moteur de base de données

Nom du serveur : SQL1

Authentification : Authentification Windows

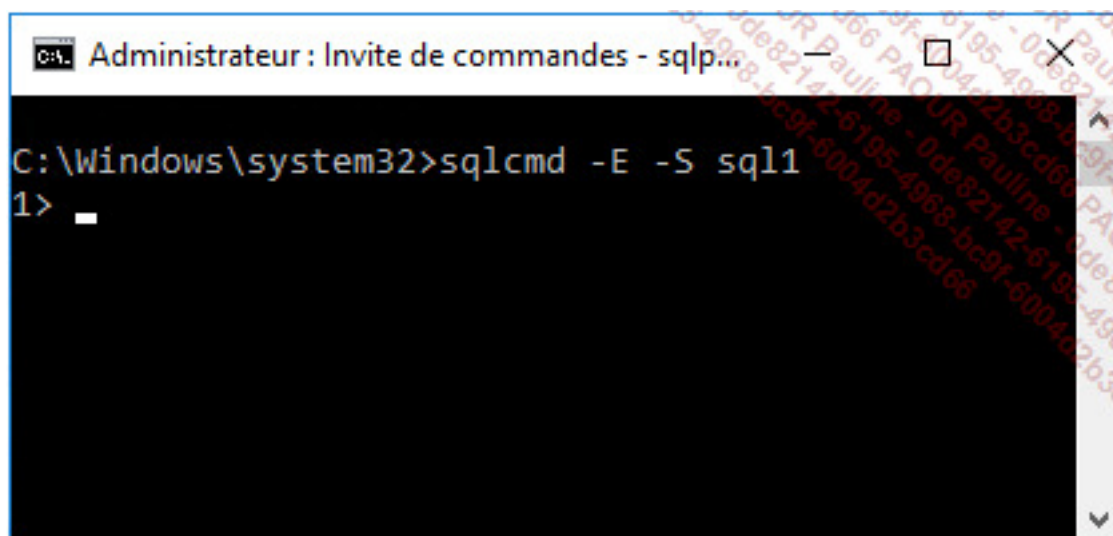
Nom d'utilisateur : EDIENITEK\administrator

Mot de passe :

☐ Mémoriser le mot de passe

Connexion Annuler Aide Options >>

Il est possible d'établir la connexion depuis un outil en ligne de commande comme sqlcmd. L'écran suivant permet de se connecter au serveur en utilisant le mode de sécurité Windows.



```
C:\Windows\system32>sqlcmd -E -S sql1
1> _
```


La configuration

Avant de mettre en service le serveur SQL, en le rendant accessible par tous les utilisateurs, il est important de réaliser un certain nombre d'opérations de configuration du serveur et des outils d'administration client.

1. Les services

Les différents composants serveur s'exécutent sous forme de service. Il est donc nécessaire que ces services soient démarrés afin de pouvoir travailler avec le serveur. Ces services peuvent être gérés avec le gestionnaire de configuration de SQL Server mais ils peuvent également être gérés comme tous les services Windows.

Depuis le gestionnaire de configuration, il est simple de visualiser l'état du service ainsi que de modifier ses propriétés.

Le gestionnaire de configuration est aussi identifié par le terme SQL Server Configuration Manager. SQL Server Configuration Manager n'est pas une application mais un composant de la mmc.

Comme tous les services Windows, ils peuvent être gérés de façon centrale au niveau du serveur Windows.

Enfin, il est possible d'agir sur ces services directement en ligne de commandes par l'intermédiaire des commandes **net start** et **net stop**. Lors d'un démarrage en ligne de commande, il est possible d'outrepasser la configuration par défaut du service en spécifiant la configuration à utiliser sous forme de paramètres. Par exemple, l'option `m` (`net start mssqlserver m`) permet de démarrer le serveur en mode mono utilisateur.

En cas de problème de démarrage, il est possible de démarrer le serveur SQL Server en tant qu'application à l'aide de l'exécutable `sqlservr.exe`. L'utilisation de cet exécutable permet de démarrer l'instance en ne tenant pas compte de toutes les options de configuration définies.

[Les différents états des services](#)

Démarré

Lorsque le service MSSQL Server est démarré, les utilisateurs peuvent établir de nouvelles connexions et travailler avec les données hébergées en base. Lorsque le service SQL Server Agent est démarré, l'ensemble des tâches planifiées, des alertes et de la réplication est actif.

Suspendu

Si le service MSSQL Server est suspendu, alors plus aucun nouvel utilisateur ne peut établir une connexion avec le serveur. Les utilisateurs en cours ne sont pas concernés par une telle mesure. La suspension du service SQL Server Agent désactive la planification de toutes les tâches ainsi que les alertes.

Arrêté

L'arrêt du service MSSQL Server désactive toutes les connexions utilisateur et déclenche un processus de CHECKPOINT (l'ensemble des données validées présentes en mémoire sont redescendues sur le disque dur et le point de synchronisation est inscrit dans le journal). Un tel mécanisme permet d'assurer que le prochain démarrage du serveur sera optimal. Cependant le service attend que toutes les instructions en cours soient terminées avant d'arrêter le serveur. L'arrêt du service SQL Server Agent désactive l'exécution planifiée de toutes les tâches ainsi que la gestion des alertes.

2. SQL Server Management Studio

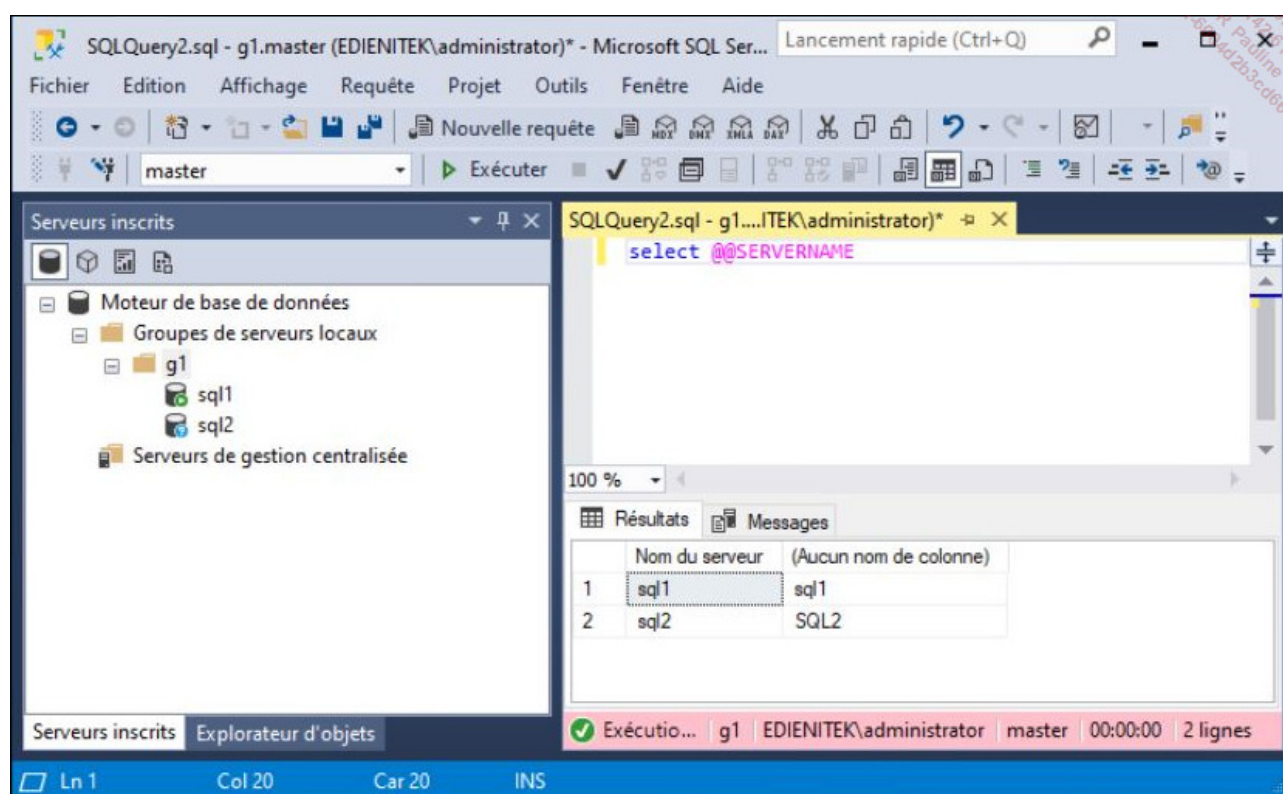
SQL Server Management Studio est l'outil de gestion graphique de SQL Server qui permet de réaliser les tâches administratives et toutes les opérations de développement. L'utilisation du même outil permet de réduire la distinction entre les deux groupes d'utilisateurs que sont les administrateurs et les développeurs. En partageant le même outil, il est plus facile de connaître ce qu'il est possible de faire d'une autre façon.

[Inscrire un serveur](#)

La fenêtre **Serveurs inscrits** permet de créer des groupes de serveurs inscrits dans SQL Server Management Studio. Si cette fenêtre n'est pas visible, il est possible de demander son affichage par le menu **Affichage - Serveurs inscrits** ou par le raccourci-clavier [Ctrl][Alt] **G**.

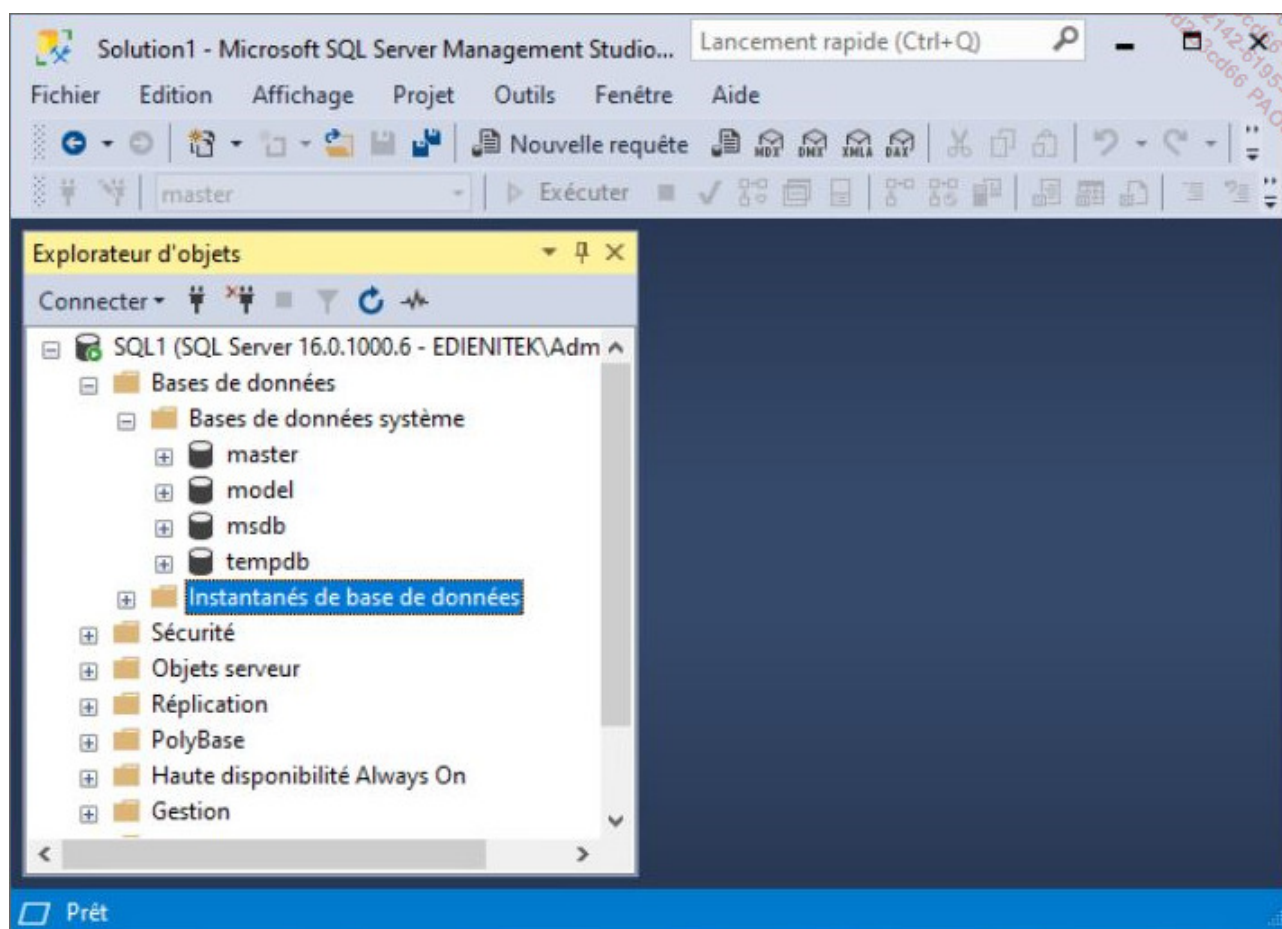
Les serveurs sont regroupés selon les besoins des administrateurs (applications, emplacement géographique...).

Il est ensuite possible d'exécuter des requêtes sur tous les serveurs d'un groupe en même temps. Pour cela, il faut sélectionner le groupe avant de cliquer sur le bouton **Nouvelle requête**. Le nom du groupe est indiqué en bas, dans la barre d'état, à la place du nom du serveur. Le résultat de la requête est affiché pour chaque serveur du groupe :



Pour faciliter le travail de l'administrateur, les bases de données système sont regroupées dans un dossier. Ainsi, ce sont les bases de données des utilisateurs qui sont visibles au premier plan. Cette séparation permet de ne pas encombrer la console par les bases système qui n'ont d'importance que pour SQL Server.

Pour un affichage plus fonctionnel, vous pouvez utiliser l'explorateur d'objets qui est présent sur la partie gauche de SSMS. Si l'explorateur d'objets n'est pas présent, il est possible de l'afficher par le menu **Affichage - Explorateur d'objets** ou la touche [F8].



Le même type de séparation est effectué sur les bases entre les tables système et les tables créées par les utilisateurs. Ce sont ces dernières qui contiennent les informations et sur lesquelles tous les efforts d'administration vont porter.

3. Configuration du serveur

Avant d'ouvrir plus librement l'accès au serveur et de permettre aux utilisateurs de venir travailler sur le serveur, il convient de surveiller attentivement les deux points suivants :

Mot de passe de l'administrateur

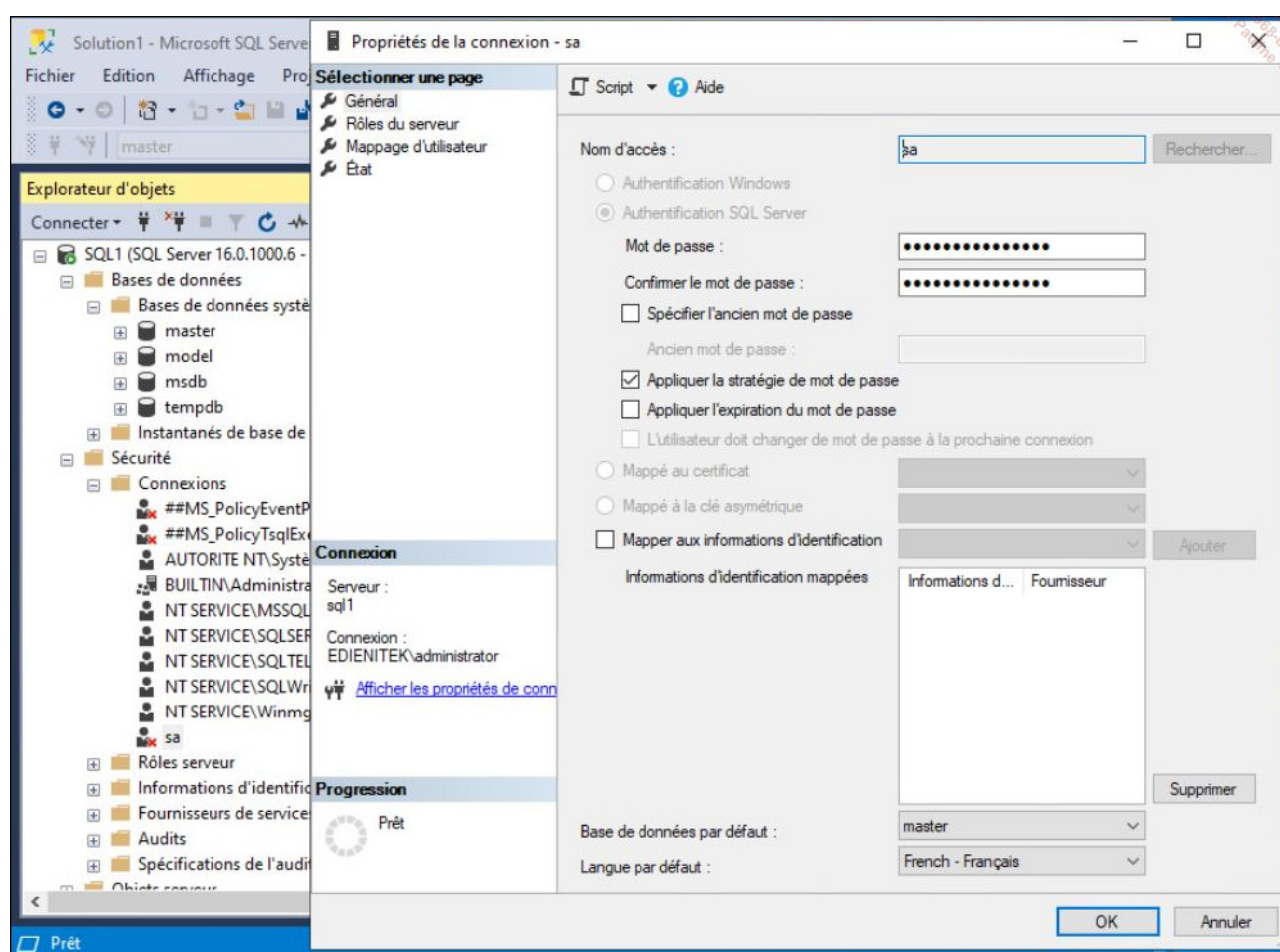
Cette préoccupation concerne uniquement les serveurs qui sont configurés en mode de sécurité mixte. Si ce choix a été fait au cours de l'installation, il faut s'assurer que le mot de passe de l'administrateur SQL Server (sa) est suffisamment fort. Si ce n'est pas le cas, il

est alors nécessaire de le modifier.

Si lors de l'installation, seul le mode de sécurité Windows a été activé, alors la connexion **sa** est non active. Il est nécessaire de définir un mot de passe fort avant d'activer la connexion. Cependant la connexion ne pourra être utilisée que si le serveur est configuré en mode de sécurité mixte.

Deux possibilités se présentent.

Par SQL Server Management Studio



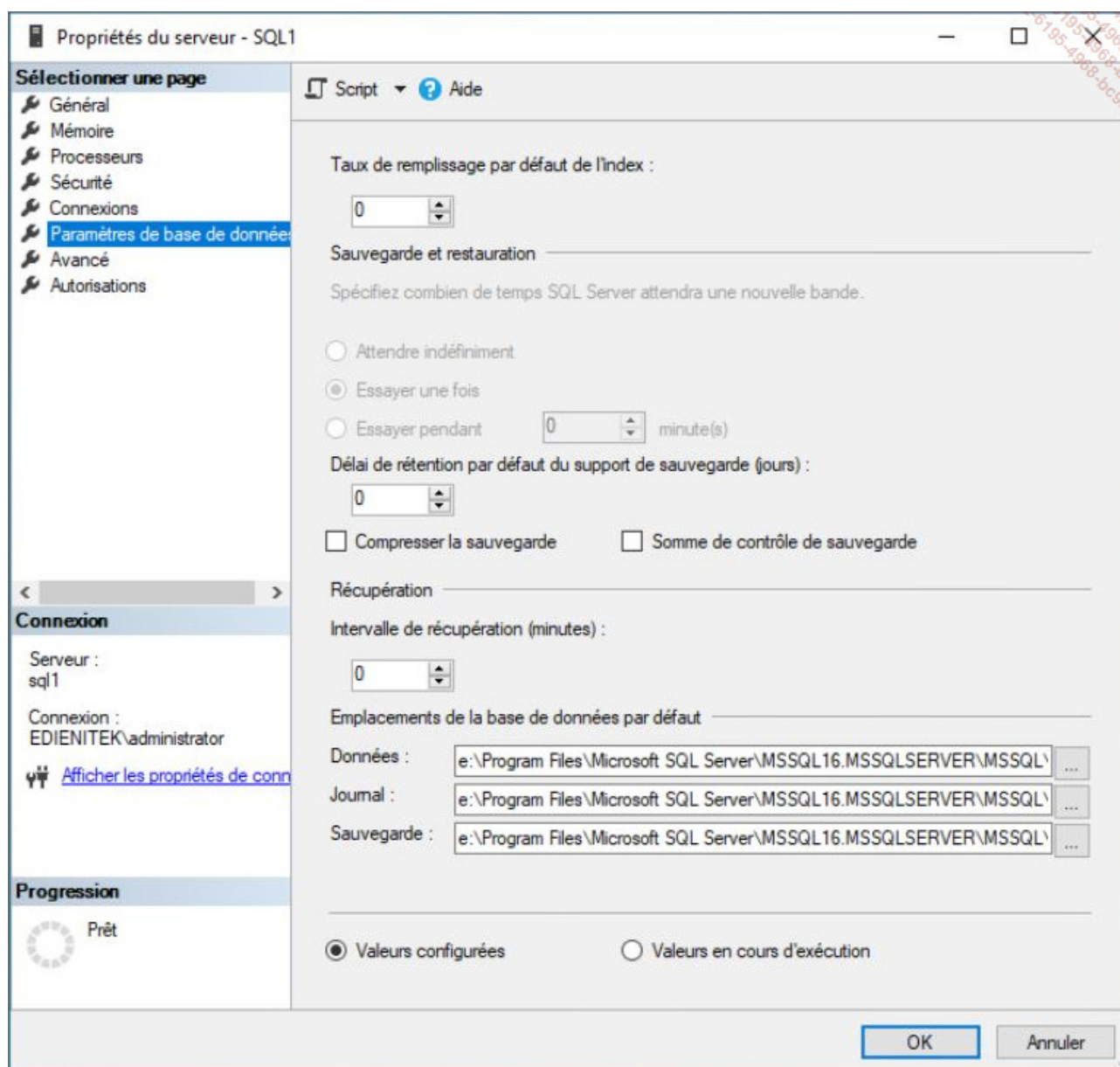
Par le Transact SQL

```
USE [master]
GO
```

```
ALTER LOGIN [sa] WITH PASSWORD=N'Pa55w.rd'  
GO
```

Gestion des ressources

Les ressources de la machine sont gérées dynamiquement. Cette gestion automatique des ressources permet d'offrir les meilleures fonctionnalités possibles du serveur tout en réalisant un minimum de tâches administratives. Toutefois, il peut parfois être intéressant de gérer manuellement certaines ressources afin de réaliser une optimisation de l'utilisation des ressources du serveur. Quelques-uns de ces réglages peuvent être réalisés au moyen de SQL Server Management Studio.



Pour accéder à la totalité des paramètres du serveur, il faut utiliser la procédure stockée `sp_configure`.

```
EXEC sys.sp_configure N'show advanced options', N'1'
go
RECONFIGURE
GO
EXEC sys.sp_configure N'min server memory (MB)', N'2048'
GO
RECONFIGURE
```

Si une option est modifiée à l'aide de la procédure `sp_configure`, elle ne prendra effet que lors du prochain redémarrage du serveur SQL. Il est possible d'appliquer la modification de façon immédiate en exécutant la commande `RECONFIGURE`.

4. La gestion du processus SQL Server

Pour le système d'exploitation, chaque application s'exécute sous forme de processus. Chaque processus dispose de ces propres threads qui correspondent aux unités de travail que le système d'exploitation doit soumettre au processeur. A un processus correspond toujours au moins un thread.

Chaque instance SQL Server gère elle-même ses propres threads et gère leur synchronisation sans passer par le noyau Windows. L'objectif de SQL Server est de répondre efficacement et rapidement à des demandes de montées en charge brusque et soudaine. Afin d'être toujours disponible, SQL Server gère son propre pool de threads dont le nombre maximal est contrôlé par le paramètre `max worker threads`. Avec la valeur par défaut (0) SQL Server se charge de gérer lui-même ce nombre de threads mais il est également possible de fixer le nombre maximum de threads. Ces threads vont avoir pour objectif de traiter les requêtes des utilisateurs. Étant donné qu'un utilisateur ne travaille pas 100 % de son temps sur le serveur, il doit lire ou bien modifier les données avant de soumettre une nouvelle requête, il est préférable pour SQL Server de partager un même thread entre plusieurs utilisateurs.



La valeur maximale de ce paramètre était de 255 avec SQL Server 2000. Aussi, dans le cadre d'une migration de serveur est-il recommandé de positionner cette valeur à 0.

Enfin, pour ordonnancer l'exécution des différents threads, Windows affecte à chaque processus une priorité variant de 1 (le moins prioritaire) à 31 (le plus prioritaire). Cette gestion de priorité ne concerne pas les processus système. Par défaut, le processus SQL Server reçoit un niveau de priorité égal à 7, c'est-à-dire un processus normal. Il est possible de donner une priorité supérieure par l'intermédiaire de l'option de configuration

`priority boost`. Cette option peut s'avérer particulièrement intéressante lorsque plusieurs instances SQL Server s'exécutent sur le même poste et que l'on souhaite en favoriser une.

```
EXEC sys.sp_configure N'show advanced options', N'1'
go
RECONFIGURE
GO
EXEC sys.sp_configure N'priority boost', 1
GO
RECONFIGURE
```

Pour connaître les valeurs des différentes options de configuration sur l'instance, il est possible d'interroger la vue `sys.configurations`. Cette vue contient, entre autres, les colonnes suivantes :

<code>name</code>	Représente le nom de l'option.
<code>value</code>	Valeur actuellement configurée ; c'est cette valeur qui sera prise en compte au prochain démarrage de l'instance.
<code>value_in_use</code>	Valeur avec laquelle l'option est présente sur l'instance en cours d'exécution.

5. La gestion de la mémoire

Par défaut, SQL Server gère dynamiquement la quantité de mémoire dont il a besoin. Il est d'ailleurs recommandé de conserver une gestion dynamique de la mémoire qui permet une répartition optimale de la mémoire entre les différents processus s'exécutant sur le serveur.

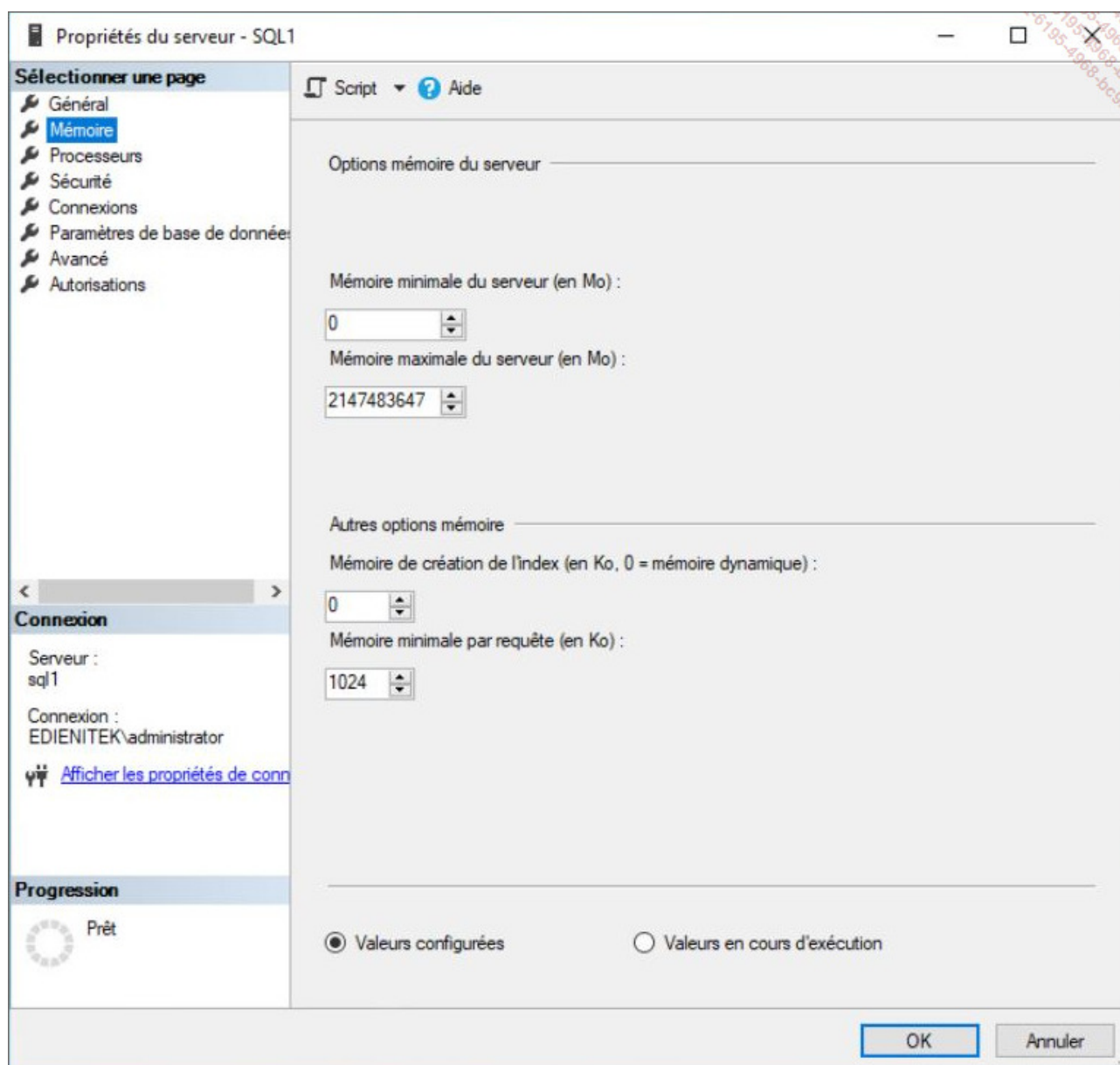
Il est important que le serveur dispose d'une quantité de mémoire suffisante car cela

permet de minimiser le nombre de lectures physiques et de favoriser les lectures logiques. Plus ces dernières sont nombreuses, meilleurs sont les temps de réponse du serveur. Le ratio entre ces deux types de lectures peut être obtenu au travers de l'Analyseur de performances. Ce point est abordé au chapitre Outils pour l'optimisation - L'analyseur de performances (moniteur système).

Pour permettre la gestion dynamique de la quantité de mémoire utilisée, SQL Server s'appuie sur l'API (*Application Programming Interface*) de gestion de la mémoire de Windows (Query Memory Resource Notification) afin d'acquérir le maximum de mémoire sans pour autant priver le système de la quantité de mémoire qui lui est nécessaire.

Cette gestion dynamique peut être limitée en utilisant les paramètres `min server memory` et `max server memory`. L'instance SQL Server, même si elle est peu utilisée, conservera toujours la quantité de mémoire spécifiée par `min server memory`. En cas de charge de travail, il est possible d'acquérir de la mémoire, sans jamais dépasser la valeur spécifiée par `max server memory`.

Il est conseillé de conserver les valeurs par défaut afin de laisser SQL Server gérer l'utilisation de la mémoire.



6. La documentation en ligne

Lors de l'installation de SQL Server, la documentation en ligne n'est pas installée sur le serveur, mais il est fait référence au site web <https://learn.microsoft.com> qui héberge la documentation en ligne de SQL Server. Cette solution permet de garantir le fait que l'on accède toujours à la dernière version de la documentation de SQL Server. Dans le cas où l'on doit administrer des serveurs de différentes versions, cela permet d'avoir un point unique pour la documentation de toutes les versions de SQL Server.

Le service de texte intégral

Le service de recherche de texte intégral a pour objectif d'améliorer la pertinence et la vitesse des requêtes menées sur les champs qui contiennent des textes de grandes dimensions, c'est-à-dire sur des colonnes de type char, nchar, varchar, nvarchar, varbinary. Dans le cas où la table possède de nombreuses lignes, une requête avec l'opérateur like peut prendre plusieurs minutes pour s'exécuter. Si la colonne dispose d'un index de texte intégral, alors l'extraction des lignes demande quelques instants.

La mise en place d'un tel service, utilisé pour l'indexation, l'interrogation et la synchronisation nécessite la présence d'une clé unique (ou clé primaire) sur toutes les tables qui sont inscrites en vue d'une recherche de texte intégral. L'index de texte intégral conserve une trace de tous les mots significatifs qui sont employés ainsi que leur emplacement.

Pour que la recherche soit pertinente et rapide, seuls les mots chargés de sens doivent être indexés. Pour identifier les mots dénués de sens, SQL Server va utiliser une liste des mots vides. Cette liste est conservée directement dans la base de données.

Cette liste des mots vides peut être modifiée librement afin d'y ajouter les mots vides spécifiques à une entreprise ou bien un contexte de travail. Par exemple, le nom de la société peut être considéré comme un mot vide car il y a de fortes chances qu'il apparaisse très fréquemment.

Le service de recherche de texte intégral s'appuie sur quelques termes bien précis pour décrire sa mise en place et son fonctionnement :

- Index de texte intégral : stocke les informations relatives aux mots significatifs. C'est à partir de ces informations que les recherches sont effectuées.
- Catalogue de texte intégral : associé à une instance de SQL Server, le catalogue peut contenir de 0 à n index.
- Analyseur lexical : en fonction de la langue et de ses règles lexicales, l'analyseur lexical va définir les jetons.
- Jeton : il s'agit d'une chaîne de caractères (bien souvent des mots) repérée par l'analyseur lexical.
- Générateur de formes dérivées : en fonction de la langue, le générateur de formes dérivées permet de gérer les différentes formes que peut prendre un

terme, comme par exemple sa conjugaison pour un verbe ou bien son accord pour un nom ou un adjectif.

- Filtre : cet élément permet d'extraire le texte à partir d'un fichier spécifique (.doc par exemple) et d'enregistrer ce texte dans une colonne de type varbinary(max) par exemple.
- Analyse ou Alimentation : il s'agit du processus qui permet d'initialiser et de maintenir à jour l'index.
- Mots vides : il s'agit d'une liste de mots qui ne sont pas porteurs de sens pour les recherches en mode texte. Cette liste de mots est spécifique à chaque langue. L'objectif recherché est de réduire le nombre de mots à traiter en éliminant tous les mots de liaison, les articles ou des termes dénués de sens par rapport au contexte, par exemple le nom de l'entreprise ou bien un acronyme couramment utilisé.

Ce service est disponible pour toutes les bases hébergées sur le serveur.

Implémentation

La recherche linguistique sur des données texte n'est possible que sur les tables activées pour la recherche de texte intégral. La recherche linguistique, contrairement à l'opérateur **LIKE** qui agit sur les caractères, effectue une comparaison sur les mots et sur les expressions.

La mise en place d'une recherche de texte intégral dans une base de données impose d'effectuer les opérations suivantes :

- Préciser les tables et les colonnes qui doivent être inscrites dans la recherche de texte intégral.
- Réaliser l'indexation des données des colonnes inscrites et remplir les index de texte intégral avec les mots porteurs de sens.
- Exécuter les requêtes sur les colonnes inscrites pour la recherche de texte intégral.
- S'assurer que toutes les modifications effectuées sur ces colonnes ont été propagées jusqu'aux index.

Ces index ne sont pas remplis en temps réel mais plutôt de façon asynchrone car :

- Le temps d'indexation est généralement beaucoup plus long.

- Les recherches de texte intégral sont, en règle générale, beaucoup moins précises que les recherches en mode standard.

Mise en place

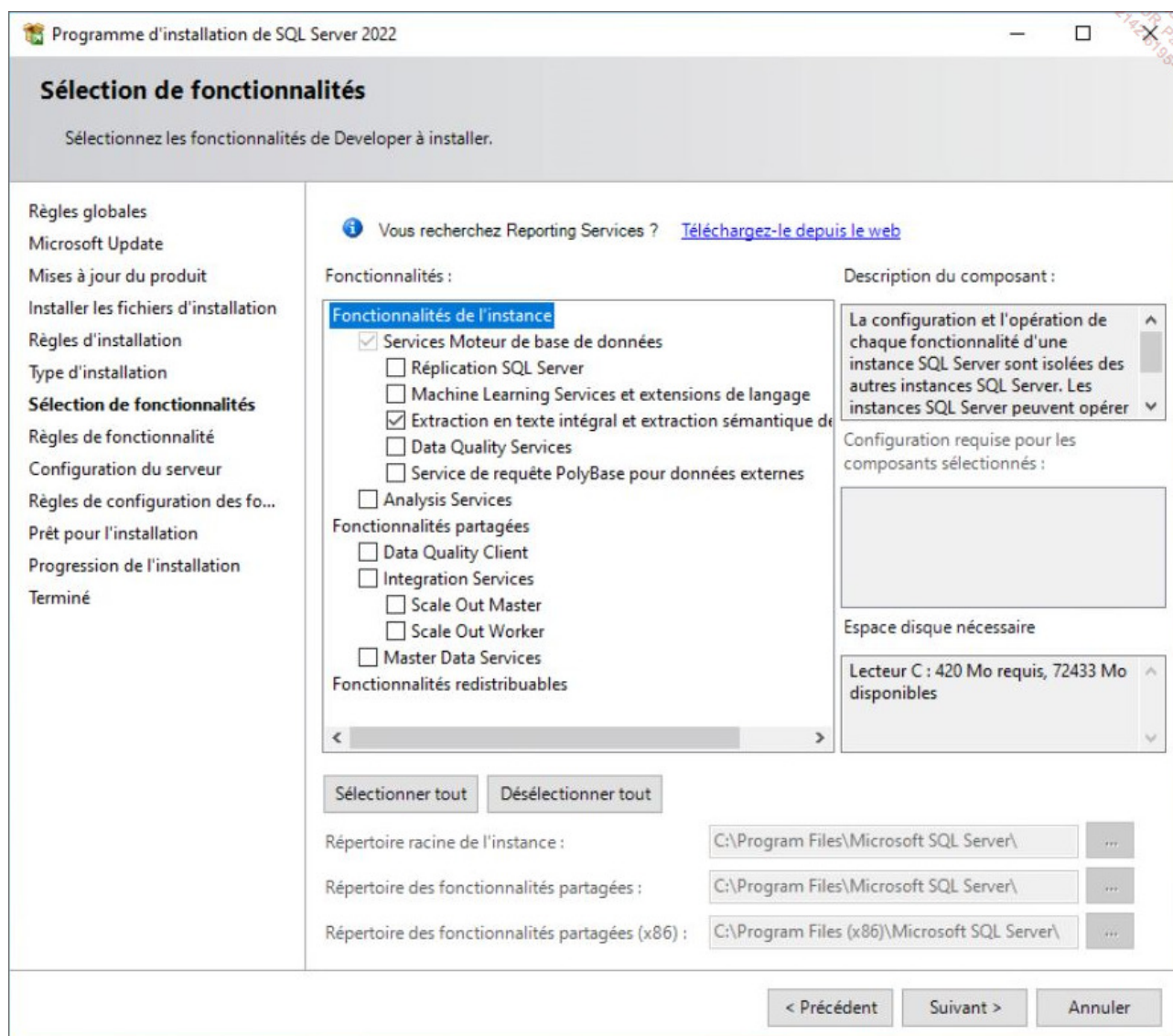
Le service de recherche de texte intégral n'est pas forcément installé avec le moteur SQL Server car c'est un composant optionnel de l'installation de SQL Server.

Dans le cas où ce service n'est pas installé, il est nécessaire de relancer le processus d'installation de SQL Server mais en demandant cette fois de modifier l'instance pour y ajouter le service en question.

Exemple

Demander la modification de l'instance existante pour y ajouter des fonctionnalités

Après avoir sélectionné l'instance sur laquelle on applique cet ajout de fonctionnalités, il est nécessaire de sélectionner le composant nommé **Extraction en texte intégral et extraction sémantique de recherche**, comme illustré ci-dessous.



C'est ce même processus d'ajout de fonctionnalités qui sera à appliquer lorsque la fonctionnalité n'est pas présente dans l'installation d'origine de SQL Server.

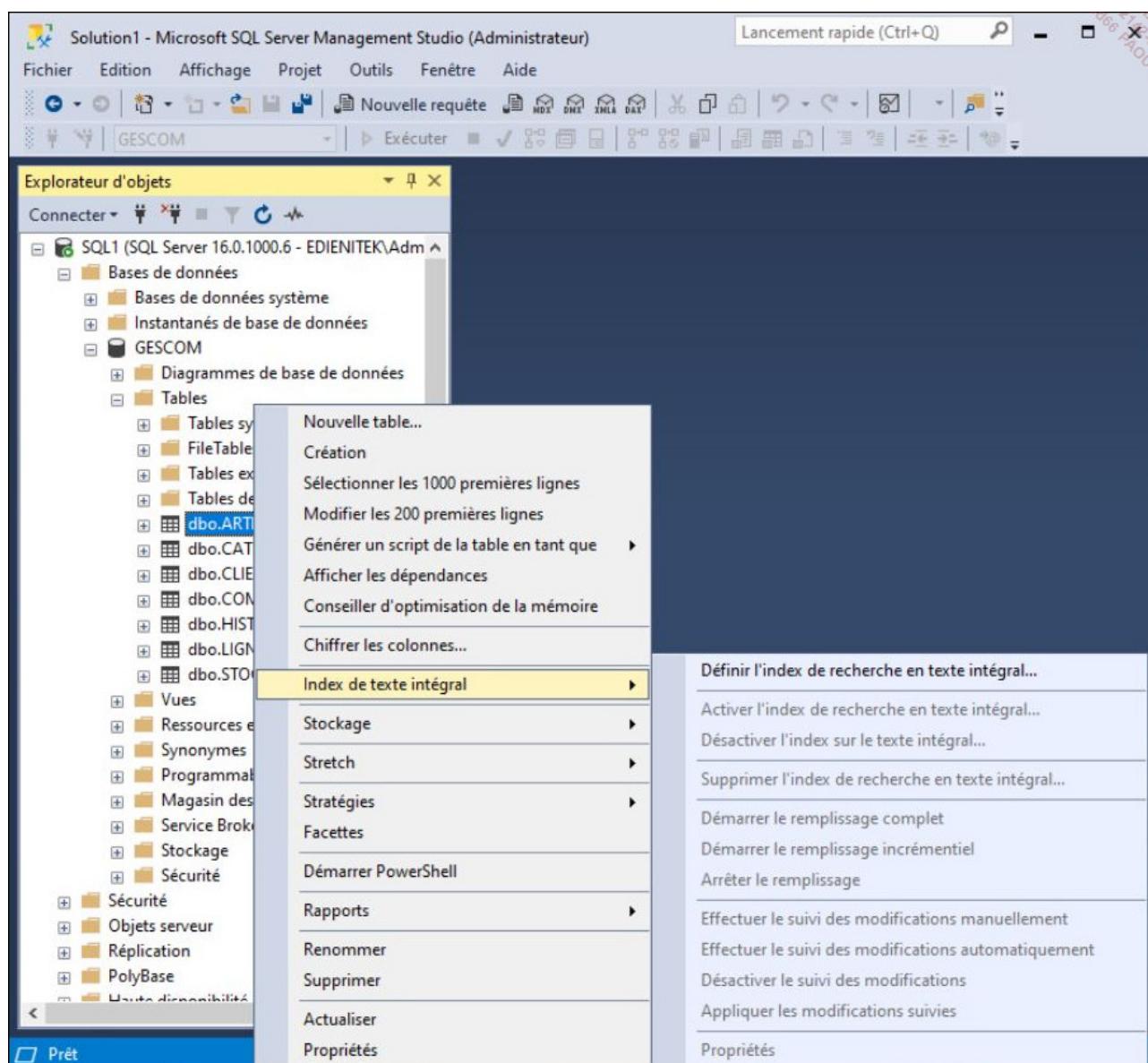
La mise en place sur service de texte intégral peut être effectuée depuis SQL Server Management Studio sous forme de scripts Transact SQL.

Voici les étapes de travail à réaliser :

- Créer un catalogue.
- Créer un ou plusieurs index qui utilisent ce catalogue.
- Définir la liste des mots vides.

La mise en place du service de texte intégral peut être réalisée soit en mode graphique

depuis SQL Server Management Studio, soit par l'intermédiaire de scripts Transact SQL, soit par l'intermédiaire de l'assistant Indexation de texte intégral. Compte tenu du fait que la création de ce type d'index est une opération ponctuelle, l'utilisation de l'assistant peut s'avérer fort utile. Cet assistant est accessible depuis le menu contextuel associé à la table sur laquelle l'index va être défini.



Cette mise en place peut également être réalisée étape par étape par l'intermédiaire de commandes Transact SQL.

1. Le catalogue

Lors de sa création initiale l'index de texte intégral va être un consommateur important au niveau des lectures et écritures sur le disque dur. Il est donc nécessaire que l'index soit défini sur un système de fichiers performant. L'index de texte intégral va donc être défini sur un catalogue qui lui est propre. Le catalogue est obligatoirement défini sur la même base que l'index. Si un index est toujours défini sur un catalogue, un catalogue peut contenir un ou plusieurs index. Lorsque la table indexée contient de très nombreuses lignes, il est souhaitable que l'index de texte intégral soit défini sur son propre catalogue. Par contre, si les tables possèdent un nombre raisonnable de lignes, alors il est possible de définir un nombre raisonnable d'index sur un même catalogue.

Afin que le catalogue soit optimisé, il est souhaitable de regrouper sur un même catalogue des index qui possèdent une fréquence de mise à jour semblable.

Le catalogue va être géré avec les instructions Transact SQL `CREATE FULLTEXT CATALOG`, `ALTER FULLTEXT CATALOG` et `DROP FULLTEXT CATALOG`. Seule la création d'un catalogue avec l'instruction `CREATE FULLTEXT CATALOG` est détaillée ici.



Il n'est pas possible de définir des catalogues sur les bases master, model et tempdb.

```
CREATE FULLTEXT CATALOG nomCatalogue
WITH ACCENT_SENSITIVITY = {ON|OFF}
[AS DEFAULT]
[AUTHORIZATION nomPropriétaire]
```

`nomCatalogue`

Nom du catalogue. Chaque catalogue porte un nom unique. Le nom est limité à 120 caractères et respecte les règles de nommage des identifiants dans SQL Server. Le nom du catalogue sera utilisé pour nommer le fichier. Le nom du fichier doit également être unique.

`ACCENT_SENSITIVITY`

Permet de préciser si le catalogue doit distinguer ou non les caractères accentués.

`AS DEFAULT`

Le catalogue devient le catalogue par défaut pour les index de texte intégral qui peuvent être définis par la suite.

`nomPropriétaire`

Dans le cas où le créateur du catalogue n'est pas le propriétaire, il est possible d'indiquer le nom du propriétaire.

Exemple

Dans l'exemple suivant, un catalogue de texte intégral est défini sur la base Gescom.

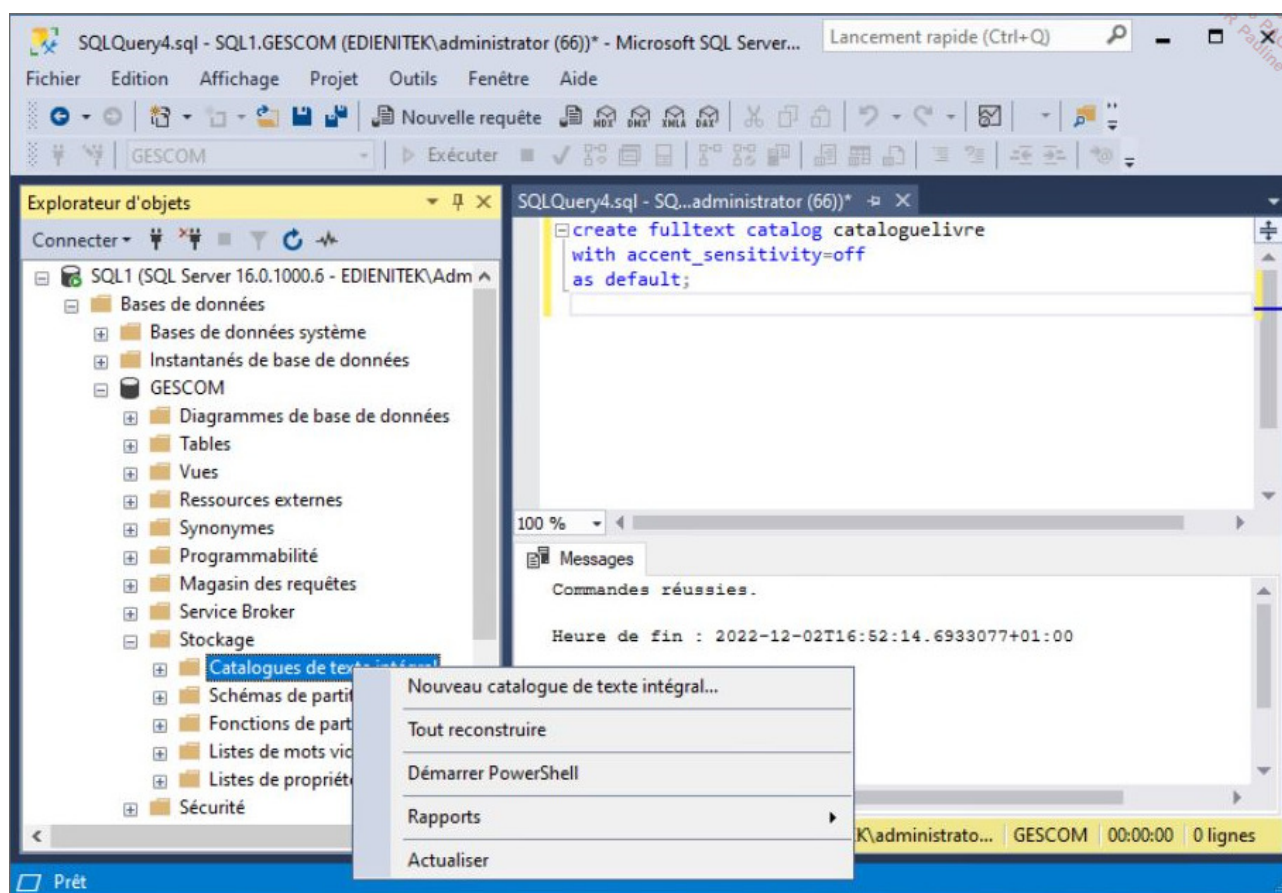
```
create fulltext catalog cataloguelivre
with accent_sensitivity=off
as default;
```



Les options `ON FILEGROUP` et `IN PATH` sont encore présentes au niveau de l'instruction, mais sont sans effet lorsque l'on travaille sur une instance SQL Server 2012.

Pour effectuer les opérations de modification et de suppression, il faut utiliser les instructions `ALTER FULLTEXT CATALOG` et `DROP FULLTEXT CATALOG`.

Il est également possible de réaliser ces opérations depuis la console SQL Server Management Studio en sélectionnant l'option **Nouveau catalogue de texte intégral** depuis le menu contextuel associé au nœud **Stockage - Catalogue de texte intégral** dans l'explorateur d'objets.



L'index de texte intégral

L'index unique utilisé par l'index de texte intégral pour référencer les lignes d'informations devra être le plus compact possible afin de limiter la charge de travail. Une donnée de type entière (int) convient parfaitement.

L'index de texte intégral va pouvoir être défini avec l'instruction `CREATE FULLTEXT INDEX`.

Ce type d'index peut bien sûr être défini sur des colonnes qui contiennent des données de type texte mais également des colonnes de type `varbinary(max)` qui stockent des textes directement dans un format de données spécifique (par exemple .doc).

```
CREATE FULLTEXT INDEX ON nomTable
[(nomColonne [TYPE COLUMN typeDocument]
[LANGUAGE indicateurDeLangue] [...])]
KEY INDEX nomIndex
```

```
[ON nomCatalogue]
[WITH CHANGE_TRACKING
{MANUAL | AUTO | OFF [, NO POPULATION]]}
[,STOPLIST={OFF|SYSTEM|nomListeMotsVides}]
```

nomTable

Il s'agit du nom de la table sur laquelle l'index de texte intégral est défini.

nomColonne

Nom de la colonne ou des colonnes qui participent à l'index.

typeDocument

Lorsque la colonne indexée est de type `varbinary(max)` et contient un document, cette colonne permet de connaître le type du document.

indicateurDeLangue

Permet de spécifier la langue dans laquelle les informations sont enregistrées dans la colonne indexée. Cet indicateur n'est utile que dans le cas où la langue utilisée pour stocker les informations est différente de la langue par défaut définie au niveau de SQL Server. Par exemple, pour indexer une colonne qui contient des textes en anglais alors que SQL Server est configuré avec le français comme langue par défaut.

nomIndex

Nom de l'index unique qui sera utilisé pour référencer les lignes d'informations dans la table.

nomCatalogue

Nom du catalogue utilisé par l'index. Si aucun nom de catalogue n'est spécifié, alors c'est le catalogue par défaut qui est utilisé (celui créé avec la clause `AS DEFAULT`).

WITH CHANGE_TRACKING

Cette clause permet de spécifier comment les modifications apportées aux colonnes indexées sont reportées dans l'index.

STOPLIST

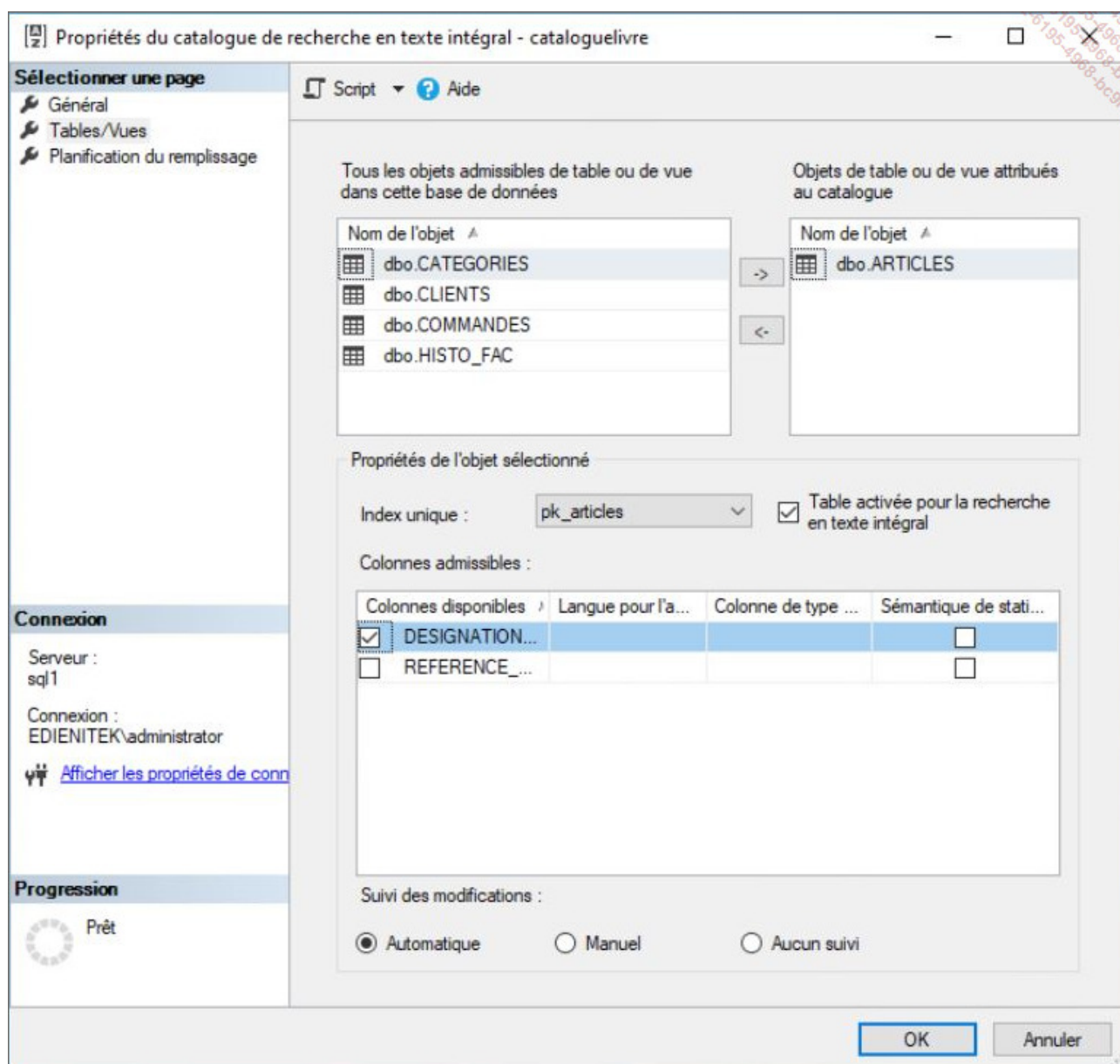
Cette option permet de préciser la liste de mots vides associés à l'index, soit aucune (**OFF**), soit la liste par défaut (**SYSTEM**), soit une liste personnalisée dont il faut donner le nom.

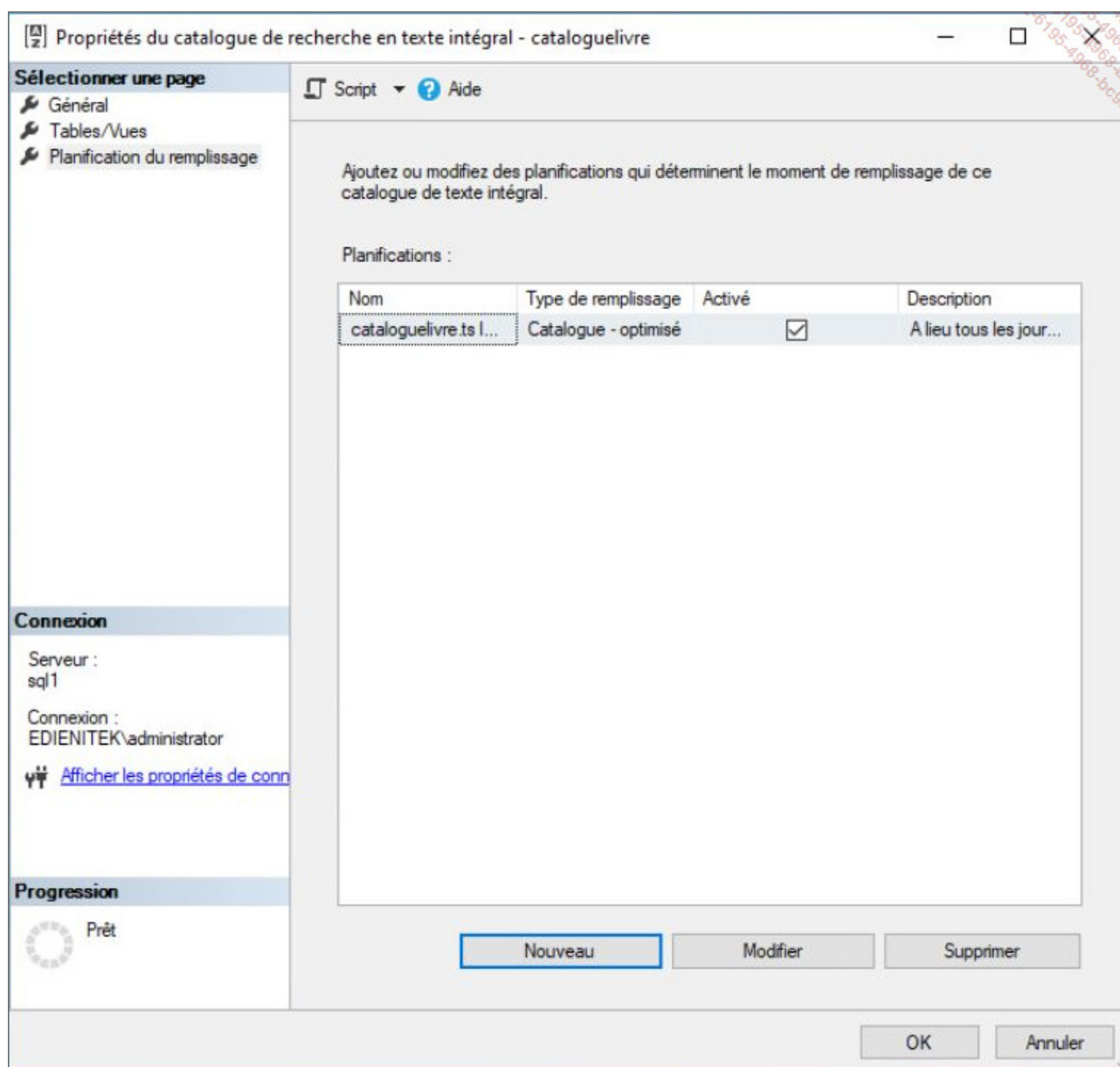
Exemple

Dans l'exemple présenté ci-dessous, la colonne désignation de la table des articles est indexée. Cet index utilise le catalogue **catalogueLivre** et suit les modifications de façon automatique.

Depuis SQL Server Management Studio, il est possible d'afficher les détails de cet index par l'intermédiaire des propriétés du catalogue.

Les propriétés du catalogue sont accessibles en sélectionnant **Propriétés** depuis le menu contextuel associé au catalogue.





Exemple

Il est également possible de définir l'index depuis l'instruction Transact SQL **CREATE FULLTEXT INDEX** comme illustré ci-dessous :

```
create fulltext index on dbo.articles(designation_art)
key index pk_articles
on cataloguelivre;
```

2. La liste de mots vides

Cette liste permet de définir quels sont les mots vides de sens et donc à ne pas prendre en compte au niveau de l'index.

Cette liste de mots vides est disponible uniquement à partir de SQL Server 2008. Donc la base de données doit être en niveau de compatibilité 100 ou supérieur (160 pour SQL Server 2022) pour être en mesure de l'utiliser.

Syntaxe

```
CREATE FULLTEXT STOPLIST nomListeMotsVides  
[FROM {nomListeMotsVidesSource|SYSTEM STOPLIST}]  
[AUTHORIZATION propriétaire];
```

nomListeMotsVides

Il s'agit du nom affecté à la liste de mots vides en cours de création.

nomListeMotsVidesSource

Il s'agit de l'identifiant d'une liste de mots vides dont le contenu va être copié dans la liste de mots en cours de création.

SYSTEM STOPLIST

Avec cette option, la liste des mots vides est définie à partir de la liste située dans la base resource.

propriétaire

Il s'agit cette fois de spécifier explicitement qui va être le propriétaire de cette liste. Cette option n'est nécessaire que dans le cas où l'exécutant de la commande n'est pas le propriétaire de l'index.

[Exemple](#)

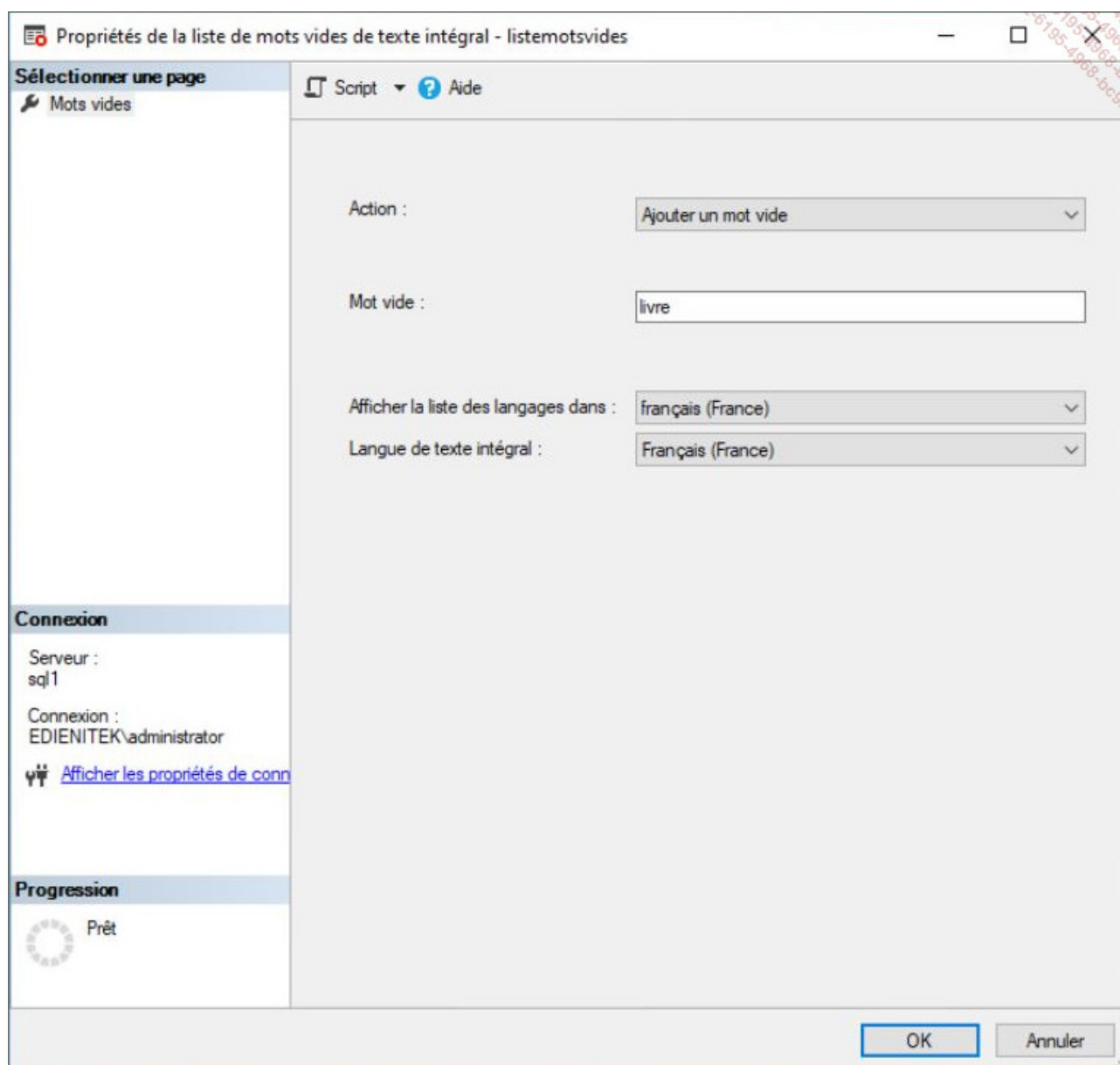
L'exemple suivant montre la création d'une liste de mots vides depuis un script Transact SQL.

```
CREATE FULLTEXT STOPLIST listemotsvides  
FROM SYSTEM STOPLIST;
```

Cette liste peut également être gérée depuis SQL Server Management Studio à partir du nœud **Stockage - Liste des mots vides de texte intégral**.

Exemple

Le mot livre est ajouté à la liste.



Une fois définie, cette liste de mots vides peut également être modifiée à l'aide de l'instruction `ALTER FULLTEXT STOPLIST` et supprimée avec `DROP FULLTEXT STOPLIST`. La commande `ALTER` permet de modifier la liste aussi bien en ajoutant qu'en supprimant des mots dénués de sens.

Syntaxe

```
ALTER FULLTEXT STOPLIST listeMotsVides
ADD 'nouveauMot' LANGUAGE langue;
```

listeMotsVides

Représente l'identifiant de la liste à modifier.

nouveauMot

Correspond au nouveau terme dénué de sens. Les termes ajoutés ainsi représentent des mots spécifiques à un vocabulaire métier qui est présent trop souvent pour rendre l'indexation efficace.

langue

Représente la langue pour laquelle ce mot est défini. Le codage des différentes langues enregistrées dans SQL Server est disponible en interrogeant la table **sys.syslanguages**.

Exemple

Dans l'exemple suivant, le mot *livre* est considéré comme dénué de sens pour la langue française.

```
ALTER FULLTEXT STOPLIST [listemotsvides]
ADD 'livre' LANGUAGE 'French';
GO
```

3. Initialiser l'index

Après la création de l'index, il est nécessaire de planifier la valorisation de l'index avec les données. Cette opération n'est pas faite de façon instantanée, car elle peut nécessiter une forte charge de travail pour le serveur. Il est donc préférable de planifier le remplissage de l'index à une période de moindre activité.

Il existe trois façons différentes d'initialiser et de tenir à jour l'index complet, basé sur les modifications ou sur une base de temps régulière.

L'initialisation complète de l'index est sans doute la façon la plus naturelle de travailler

avec les index car tous les termes sont indexés de façon globale, soit lors de la création de l'index, soit basé sur l'horodatage incrémentiel.

Avec le mode de remplissage basé sur les modifications, SQL Server garde une trace de toutes les données ajoutées ou bien modifiées. Le report de ces modifications vers l'index de texte intégral peut être effectué de façon manuelle, sous la forme de script avec une exécution planifiée, ou bien de façon continue.

Enfin, le remplissage basé sur l'horodatage incrémentiel, s'appuie quant à lui sur une valeur de type timestamp. Lors de l'ajout d'information dans la table, la colonne de type timestamp est valorisée. Si la table ne possède pas de colonne de type timestamp, il n'est pas possible de mettre en action ce type de remplissage. À la fin du processus de remplissage, la valeur timestamp courante est conservée dans les métadonnées ainsi lors du prochain processus de remplissage, il sera possible de prendre en compte uniquement les données les plus récentes.

Il est possible de définir cette opération à partir de la fenêtre des propriétés du catalogue.

Cette opération peut également être faite en Transact SQL avec l'instruction `ALTER FULLTEXT INDEX ON nomTable START FULL POPULATION` pour une initialisation complète. Cette instruction possède différentes options qui permettent, par exemple, d'activer ou de désactiver l'index.

Utilisation au sein des requêtes

L'utilisation des index de texte intégral s'effectue au moyen de deux prédicats : `CONTAINS` et `FREETEXT` qui peuvent être utilisés dans n'importe quelle clause `WHERE`. Il existe également deux fonctions Transact SQL qui ramènent un ensemble de lignes et peuvent être utilisées dans une clause `FROM` d'une requête `SELECT`, à savoir, `CONTAINSTABLE` et `FREETEXTTABLE`.

```
select *  
from dbo.ARTICLES  
where contains(DSIGNATION_ART,'Microsoft')
```

4. Retrouver les informations relatives aux index de texte intégral

Toutes les informations relatives à ces index peuvent être retrouvées en interrogeant les différentes vues du catalogue système.

Parmi toutes ces vues, celles présentées ci-dessous sont fréquemment utilisées.

`sys.fulltext_index_catalog_usages`

permet d'obtenir la liste des catalogues définis.

`sys.fulltext_index_columns`

permet d'identifier les colonnes qui participent à un index de texte intégral.

`sys.dm_fts_index_population`

permet de recueillir les informations de remplissage sur les index actifs de type texte intégral.

`sys.fulltext_indexes`

permet de connaître les index de texte intégral définis dans la base courante.

`sys.fulltext_languages`

permet de connaître la liste des langues prises en charge par SQL Server au niveau des index de texte intégral.

`sys.dm_fts_index_keywords_position_by_document`

permet de connaître la position d'un mot-clé dans un document.

Les mots vides peuvent être consultés en interrogeant les vues `sys.fulltext_stoplists`, `sys.fulltext_stopwords` et `sys.fulltext_system_stopwords`.

Exercice : installer une nouvelle instance

1. Énoncé

Créez une nouvelle instance de SQL Server. Cette instance sera nommée Livre et l'emplacement par défaut des fichiers de données sera c:\donnees\Livre. Cet emplacement s'appelle le répertoire racine des données. Ensuite, SQL Server organise sa propre arborescence pour les fichiers de données, les fichiers journaux et les fichiers de sauvegarde.



Pour plus de simplicité dans les noms de dossiers, il est préférable de ne pas utiliser les caractères accentués.

Pour cette instance, vous pouvez utiliser une édition Developer.

En plus du moteur de base de données, il est nécessaire d'installer les services de réplication.

Le service SQL Server Agent associé à cette instance sera configuré en mode de démarrage automatique. Il en sera de même pour le service SQL Server Browser, car ce service est nécessaire pour accéder aux instances nommées de SQL Server.

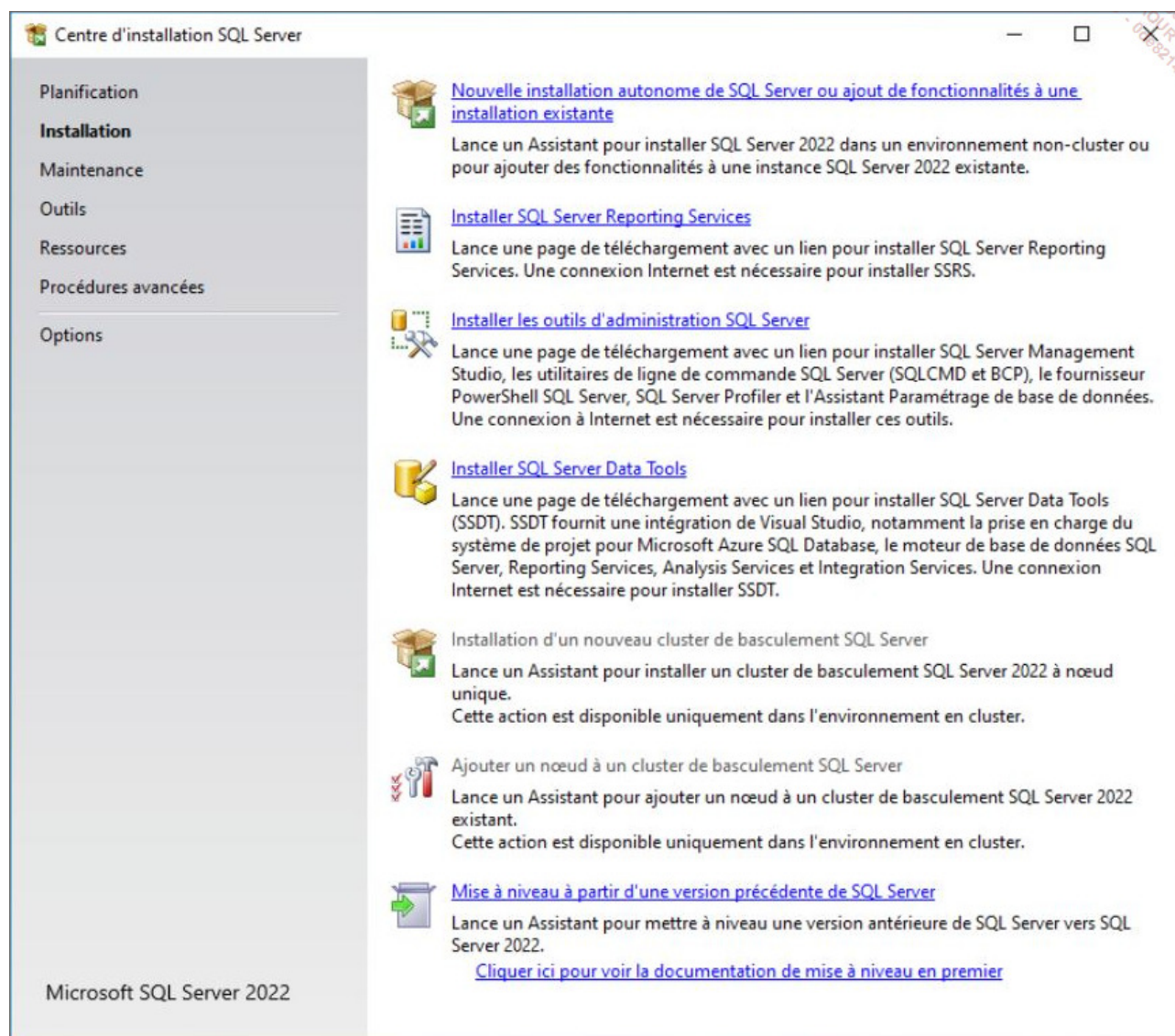
En ce qui concerne le classement, il est recommandé de choisir French_CI_AI, c'est-à-dire un classement insensible à la casse et aux accents.

Le mode d'authentification Windows est choisi comme mode de sécurité et l'utilisateur qui effectue cette installation est ajouté en tant qu'administrateur SQL Server.

Il est recommandé d'activer la fonctionnalité **FILESTREAM**. Cette fonctionnalité va permettre de stocker en dehors de la structure de base de données les fichiers encombrants (tels que les images, vidéos et sons) et donner un accès plus rapide à ces fichiers.

2. Corrigé

Depuis l'outil d'installation de SQL Server, vous pouvez réutiliser le setup.exe à la racine du DVD d'installation. Il faut pour cela demander l'installation d'une nouvelle instance.



Après le contrôle des prérequis, l'installateur vous demande de préciser votre choix : l'ajout de fonctionnalités sur une instance existante ou bien l'installation d'une nouvelle instance.

Programme d'installation de SQL Server 2022

Type d'installation

Effectuez une nouvelle installation ou ajoutez des fonctionnalités à une instance existante de SQL Server 2022.

Règles globales
Microsoft Update
Mises à jour du produit
Installer les fichiers d'installation
Règles d'installation
Type d'installation
Edition
Termes du contrat de licence
Sélection de fonctionnalités
Règles de fonctionnalité
Règles de configuration des fo...
Prêt pour l'installation
Progression de l'installation
Terminé

☒ Effectuer une nouvelle installation de SQL Server 2022
Sélectionnez cette option si vous voulez installer une nouvelle instance de SQL Server ou des composants partagés.

☐ Ajouter des fonctionnalités à une instance existante de SQL Server 2022
MSSQLSERVER
Sélectionnez cette option si vous souhaitez ajouter des fonctionnalités à une instance existante de SQL Server. Vous pouvez par exemple ajouter les fonctionnalités Analysis Services à l'instance qui contient le moteur de base de données. Les fonctionnalités d'une instance doivent être de la même édition.

Instances installées :

Nom de l'instance	ID d'instance	Fonctionnalités	Edition	Version
MSSQLSERVER	MSSQL16.MSSQLS...	SQLEngine,SQLEn...	Developer	16.0.1000.6

< Précédent Suivant > Annuler

Pour plus de facilité, il faut choisir l'édition Developer, qui présente le double avantage d'ouvrir toutes les fonctionnalités et d'être gratuite, ce qui est très bien pour un apprentissage. Cependant, cette édition ne correspond pas à une mise en production.

Programme d'installation de SQL Server 2022

Edition

Sélectionnez l'édition de SQL Server 2022 que vous voulez installer.

Règles globales
Microsoft Update
Mises à jour du produit
Installer les fichiers d'installation
Règles d'installation
Type d'installation
Edition
Termes du contrat de licence
Sélection de fonctionnalités
Règles de fonctionnalité
Règles de configuration des fo...
Prêt pour l'installation
Progression de l'installation
Terminé

Sélectionnez une édition de SQL Server à installer. Vous pouvez choisir d'utiliser une licence SQL Server que vous avez déjà achetée en saisissant la clé du produit ou choisir le paiement à l'utilisation via Microsoft Azure. Vous pouvez également spécifier une édition gratuite de SQL Server : Developer, Evaluation ou Express. L'édition Evaluation possède le plus grand nombre de fonctionnalités de SQL Server, telles que documentées dans SQL Server Books Online, et est activée avec une expiration de 180 jours. L'édition Developer n'a pas de date d'expiration, possède le même ensemble de fonctionnalités que l'édition Evaluation, mais est uniquement destinée au développement d'applications de base de données hors production. Pour passer d'une édition installée à une autre, exécutez l'assistant de mise à niveau d'édition.

☒ Spécifiez une édition gratuite :

Developer

☐ Utiliser la facturation avec paiement à l'utilisation via Microsoft Azure :

Avertissement : pour activer cette option, vous devez disposer d'un abonnement Azure actif que vous devrez fournir ultérieurement avec un groupe de ressources, une région Azure et un ID de client. Pour plus d'informations, consultez : <https://aka.ms/ArcEnabledSqlPAYG>.

Standard

☐ Entrez la clé de produit (Product Key) :

- - - -

☐ J'ai une licence SQL Server avec Software Assurance ou SQL Software Subscription

☐ J'ai une licence SQL Server uniquement

< Précédent Suivant > Annuler

L'étape suivante consiste à sélectionner les fonctionnalités à installer qui, dans le cas présent, sont le moteur de la base de données et la répllication.

Programme d'installation de SQL Server 2022

Sélection de fonctionnalités

Sélectionnez les fonctionnalités de Developer à installer.

Règles globales
 Microsoft Update
 Mises à jour du produit
 Installer les fichiers d'installation
 Règles d'installation
 Type d'installation
 Edition
 Termes du contrat de licence
Sélection de fonctionnalités
 Règles de fonctionnalité
 Configuration de l'instance
 Configuration du serveur
 Configuration du moteur de ba...
 Règles de configuration des fo...
 Prêt pour l'installation
 Progression de l'installation
 Terminé

Fonctionnalités :

Fonctionnalités de l'instance

- ☒ Services Moteur de base de données
 - ☒ Réplication SQL Server
 - ☐ Machine Learning Services et extensions de langage
 - ☐ Extraction en texte intégral et extraction sémantique de
 - ☐ Data Quality Services
 - ☐ Service de requête PolyBase pour données externes
- ☐ Analysis Services

Fonctionnalités partagées

- ☐ Data Quality Client
- ☐ Integration Services
 - ☐ Scale Out Master
 - ☐ Scale Out Worker
- ☐ Master Data Services

Fonctionnalités redistribuables

Description du composant :

La configuration et l'opération de chaque fonctionnalité d'une instance SQL Server sont isolées des autres instances SQL Server. Les instances SQL Server peuvent opérer

Configuration requise pour les composants sélectionnés :

Déjà installé(s) :

- Windows PowerShell 3.0 ou version s
- Microsoft Visual C++ 2017 Redistribu

Espace disque nécessaire

Lecteur C : 994 Mo requis, 71730 Mo disponibles

Sélectionner tout Désélectionner tout

Répertoire racine de l'instance : C:\Program Files\Microsoft SQL Server\

Répertoire des fonctionnalités partagées : C:\Program Files\Microsoft SQL Server\

Répertoire des fonctionnalités partagées (x86) : C:\Program Files (x86)\Microsoft SQL Server\

< Précédent Suivant > Annuler

Cette instance porte le nom de Livre.

Programme d'installation de SQL Server 2022

Configuration de l'instance

Spécifiez le nom et l'ID d'instance de l'instance de SQL Server. L'ID d'instance devient partie intégrante du chemin d'installation.

Règles globales
Microsoft Update
Mises à jour du produit
Installer les fichiers d'installation
Règles d'installation
Type d'installation
Edition
Termes du contrat de licence
Sélection de fonctionnalités
Règles de fonctionnalité
Configuration de l'instance
Configuration du serveur
Configuration du moteur de ba...
Règles de configuration des fo...
Prêt pour l'installation
Progression de l'installation
Terminé

☐ Instance par défaut
☒ Instance nommée : *

ID d'instance :

Répertoire SQL Server : C:\Program Files\Microsoft SQL Server\MSSQL16.

Instances installées :

Nom de l'instance	ID d'instance	Fonctionnalités	Edition	Version
MSSQLSERVER	MSSQL16.MSSQLS...	SQLEngine,SQLEn...	Developer	16.0.1000.6

Les services Agent SQL Server et Browser sont mis en démarrage automatique et le classement est défini à French_CI_AI.

Programme d'installation de SQL Server 2022

Configuration du serveur

Spécifiez les comptes de service et la configuration du classement.

Règles globales

Microsoft Update

Mises à jour du produit

Installer les fichiers d'installation

Règles d'installation

Type d'installation

Edition

Termes du contrat de licence

Sélection de fonctionnalités

Règles de fonctionnalité

Configuration de l'instance

Configuration du serveur

Configuration du moteur de ba...

Règles de configuration des fo...

Prêt pour l'installation

Progression de l'installation

Terminé

Comptes de service

Classement

Microsoft conseille d'utiliser un compte distinct pour chaque service SQL Server.

Service	Nom du compte	Mot de passe	Type de démarrage
SQL Server Agent	NT Service\SQLAgent\$LI...		Automatique ▼
Moteur de base de données SQL ...	NT Service\MSSQL\$SLIVR...		Automatique ▼
SQL Server Browser	NT AUTHORITY\LOCALS...		Automatique ▼

☐ Accorder le privilège Effectuer des tâches de maintenance en volume au service du moteur de base de données SQL Server

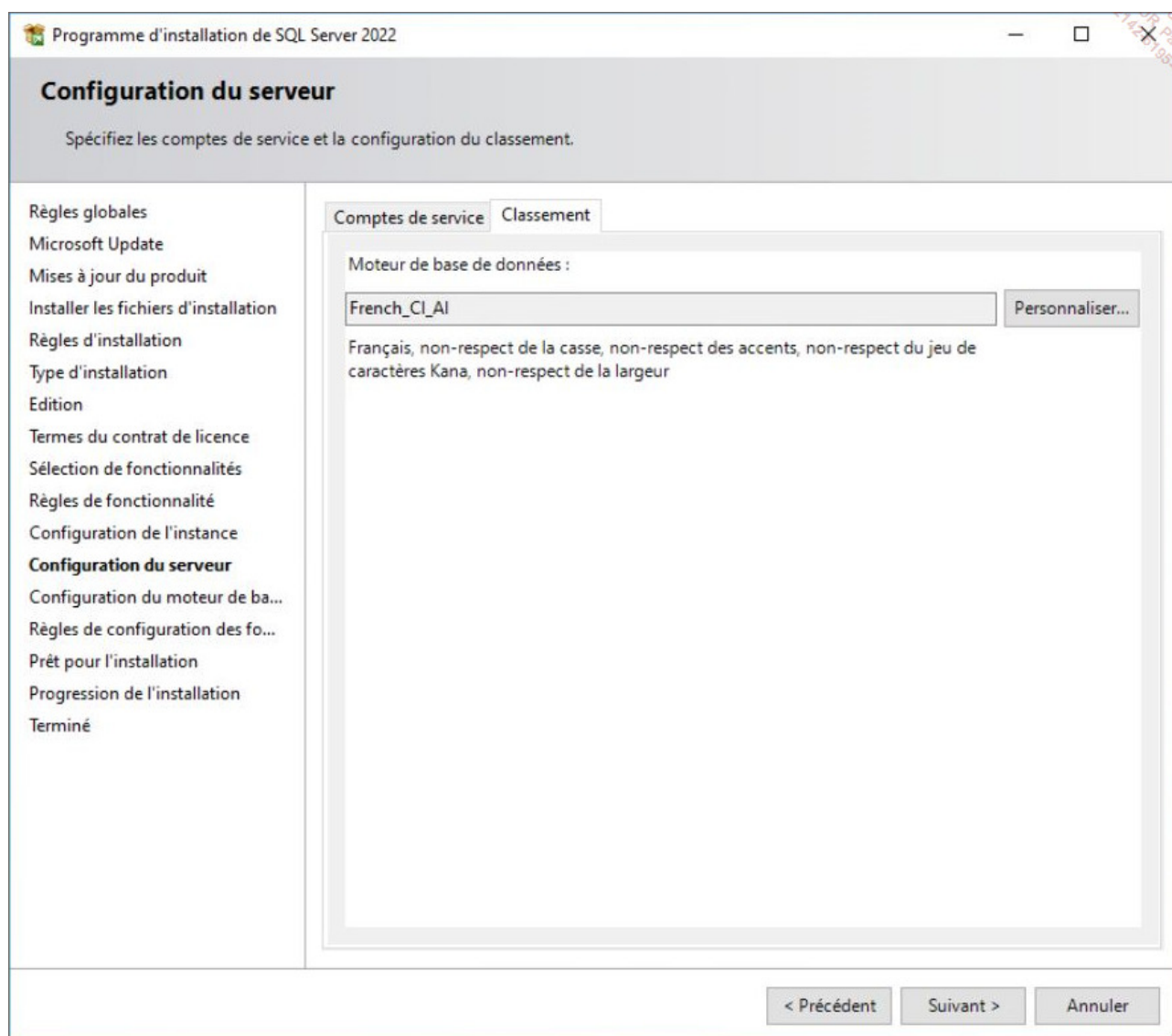
Ce privilège permet l'initialisation instantanée des fichiers en évitant la mise à zéro des pages de données. Cela peut entraîner la divulgation d'informations en autorisant l'accès au contenu supprimé.

[Cliquez ici pour des détails](#)

< Précédent

Suivant >

Annuler



Le compte actuel est ajouté en tant qu'administrateur SQL Server et le dossier de stockage des données par défaut est configuré.

Programme d'installation de SQL Server 2022

Configuration du moteur de base de données

Pour le moteur de base de données, spécifiez le mode de sécurité de l'authentification, les administrateurs, les répertoires de données, TempDB, le degré maximal de parallélisme, les limites de mémoire et les paramètres FileStream.

- Règles globales
- Microsoft Update
- Mises à jour du produit
- Installer les fichiers d'installation
- Règles d'installation
- Type d'installation
- Edition
- Termes du contrat de licence
- Sélection de fonctionnalités
- Règles de fonctionnalité
- Configuration de l'instance
- Configuration du serveur
- Configuration du moteur de b...**
- Règles de configuration des fo...
- Prêt pour l'installation
- Progression de l'installation
- Terminé

Configuration du serveur | Répertoires de données | tempdb | MaxDOP | Mémoire | FILESTREAM

Spécifiez le mode d'authentification et les administrateurs du moteur de base de données.

Mode d'authentification

☒ Mode d'authentification Windows

☐ Mode mixte (authentification SQL Server et authentification Windows)

Spécifiez le mot de passe pour le compte d'administrateur système (sa) SQL Server.

Entrer le mot de passe :

Confirmer le mot de passe :

Spécifier les administrateurs SQL Server

EDIENITEK\administrator (Administrator)

Les administrateurs SQL Server bénéficient d'un accès illimité au moteur de base de données.

Ajouter l'utilisateur actuel | Ajouter... | Supprimer

< Précédent | Suivant > | Annuler

Programme d'installation de SQL Server 2022

Configuration du moteur de base de données

Pour le moteur de base de données, spécifiez le mode de sécurité de l'authentification, les administrateurs, les répertoires de données, TempDB, le degré maximal de parallélisme, les limites de mémoire et les paramètres FileStream.

Règles globales
Microsoft Update
Mises à jour du produit
Installer les fichiers d'installation
Règles d'installation
Type d'installation
Edition
Termes du contrat de licence
Sélection de fonctionnalités
Règles de fonctionnalité
Configuration de l'instance
Configuration du serveur
Configuration du moteur de b...
Règles de configuration des fo...
Prêt pour l'installation
Progression de l'installation
Terminé

Configuration du serveur

Répertoires de données

tempdb

MaxDOP

Mémoire

FILESTREAM

Répertoire racine des données : C:\donnees\livre ...
Répertoire des bases de données système : C:\donnees\livre\MSSQL16.LIVRE\MSSQL\Data
Répertoire des bases de données utilisateur : C:\donnees\livre\MSSQL16.LIVRE\MSSC ...
Registre Journaux des bases de données utilisateur : C:\donnees\livre\MSSQL16.LIVRE\MSSC ...
Répertoire de sauvegarde : C:\donnees\livre\MSSQL16.LIVRE\MSSC ...

< Précédent
Suivant >
Annuler

Et la fonctionnalité **FILESTREAM** est activée.

Programme d'installation de SQL Server 2022

Configuration du moteur de base de données

Pour le moteur de base de données, spécifiez le mode de sécurité de l'authentification, les administrateurs, les répertoires de données, TempDB, le degré maximal de parallélisme, les limites de mémoire et les paramètres FileStream.

Règles globales

Microsoft Update

Mises à jour du produit

Installer les fichiers d'installation

Règles d'installation

Type d'installation

Edition

Termes du contrat de licence

Sélection de fonctionnalités

Règles de fonctionnalité

Configuration de l'instance

Configuration du serveur

Configuration du moteur de b...

Règles de configuration des fo...

Prêt pour l'installation

Progression de l'installation

Terminé

Configuration du serveur | Répertoires de données | tempdb | MaxDOP | Mémoire | **FILESTREAM**

☒ Activer FILESTREAM pour l'accès Transact-SQL

☒ Activer FILESTREAM pour l'accès aux E/S de fichier

Nom de partage Windows :

☒ Autoriser les clients distants à avoir un accès aux données FILESTREAM

< Précédent

Suivant >

Annuler

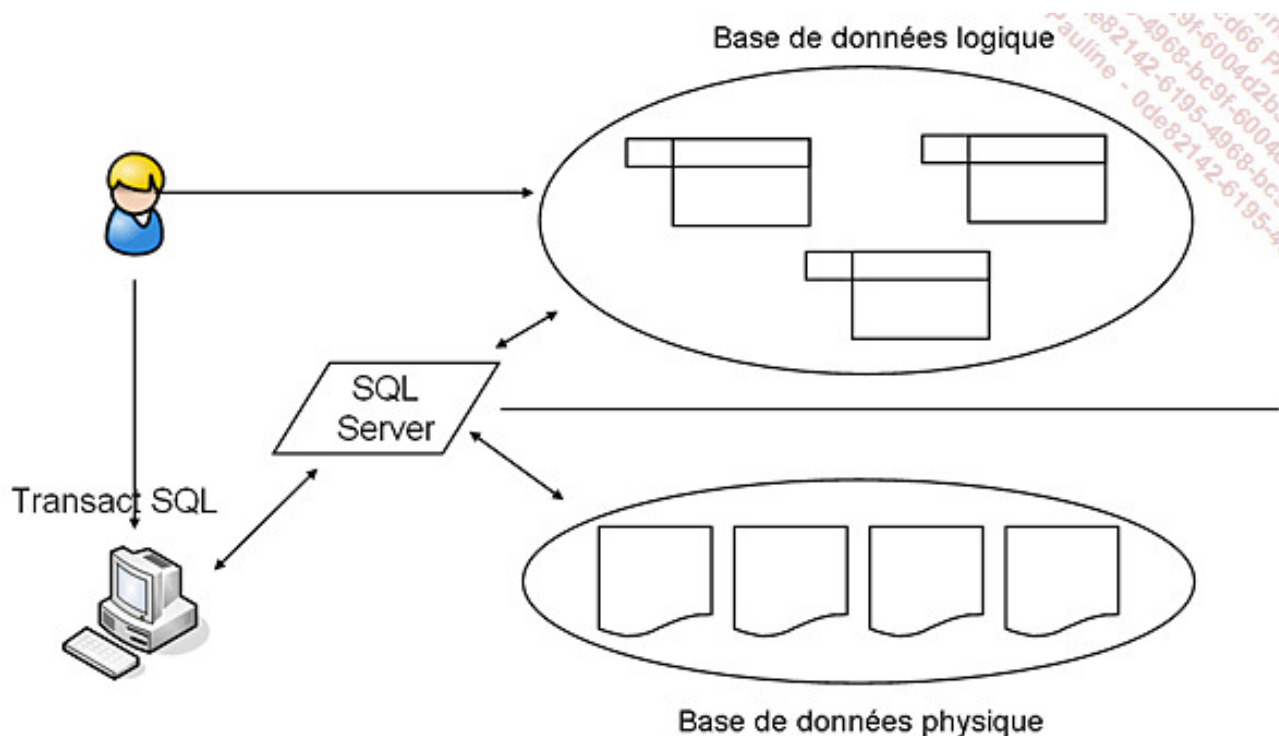
Notions générales

L'installation du serveur SQL réalisée, il convient de définir des espaces logiques de stockage afin de regrouper sous un même nom l'ensemble des données correspondant à un même projet. Cet ensemble est la **base de données**, elle va nous permettre de travailler logiquement avec des objets tels que les tables sans jamais avoir à se soucier du stockage physique. SQL Server permet de réaliser des associations entre les fichiers physiques et les bases de données. Dans ce chapitre, la création et la gestion des fichiers physiques seront abordées en même temps que les bases de données.

1. Liens entre base de données et organisation physique

Lors de la création d'une base de données, il est nécessaire de préciser au moins deux fichiers. Le premier servira à stocker les données, le deuxième sera utilisé par le journal des transactions afin de stocker les informations nécessaires à la gestion des transactions lors des modifications sur les données.

Ces deux fichiers sont obligatoires et sont propres à chaque base. Dans SQL Server, il n'est pas possible de partager un fichier de données ou le journal entre plusieurs bases.



Séparation entre les schémas logique et physique

2. La notion de transaction

a. Qu'est-ce qu'une transaction ?

Une transaction est un ensemble indivisible d'ordres Transact SQL. Soit la totalité peut s'exécuter, soit aucun ordre ne peut s'exécuter. Le moteur SQL doit être capable, tant que la transaction n'est pas terminée, de remettre les données dans l'état initial. Durant toute l'exécution de la transaction, des verrous sont positionnés sur les données afin d'empêcher des conflits d'accès de se produire avec d'autres utilisateurs. Ces verrous peuvent être gérés automatiquement par SQL Server. Cependant, il sera souvent nécessaire de reprendre la main pour les gérer, soit avec la commande **SET TRANSACTION ISOLATION LEVEL**, soit avec les indicateurs de tables dans les requêtes.

L'exemple suivant posera des verrous de modification à la place des verrous de lecture (verrous par défaut).

```
select * from dbo.CLIENTS
with (updlock)
```

Exemple de transaction : un exemple connu mais représentatif de la notion de transaction est celui du retrait d'argent auprès d'un distributeur automatique. La transaction est alors constituée de deux opérations : le débit du compte et la distribution d'argent. S'il n'est pas possible de réaliser l'une des deux opérations, c'est l'ensemble des opérations qui devra être annulé. Il paraît déraisonnable de débiter le compte si la somme correspondante n'a pas été distribuée à l'utilisateur.

b. Les ordres Transact SQL

Ils sont au nombre de quatre, `BEGIN TRAN`, `COMMIT TRAN`, `ROLLBACK TRAN`, `SAVE TRAN` qui indiquent respectivement le début de la transaction, la fin avec succès, la fin avec échec et la définition des points d'arrêt.

`BEGIN TRAN[SACTION]`

Cette instruction permet de démarrer de façon explicite une transaction. En l'absence de cette commande, toute instruction SQL est une transaction implicite qui est validée (`COMMIT`) aussitôt la modification effectuée sur les données. On parle alors de mode autocommit. Lors du début de la transaction, il est possible de nommer la transaction, mais aussi de marquer le début de la transaction dans le journal de la base de données. Cette marque pourra être exploitée lors d'un processus de restauration des données pour restaurer la transaction.

```
BEGIN { TRAN | TRANSACTION } [nomTransaction]
    [ WITH MARK [ 'description' ] ]
[;]
```

Exemple

Dans l'exemple suivant, une nouvelle transaction est démarrée.

```

Begin tran
Select * from dbo.CLIENTS
with (updlock)

```

SAVE TRAN

Cette instruction permet de définir des points d'arrêt et donc donne la possibilité d'annuler une partie de la transaction en cours. Il est possible de définir plusieurs points d'arrêt sur une même transaction. De façon à permettre l'annulation jusqu'à un point d'arrêt précis, ils sont généralement identifiés par un nom.

```
SAVE { TRAN | TRANSACTION } {nomPointArret}[;]
```

Exemple

Dans l'exemple suivant, le point d'arrêt P1 est défini, après l'ajout d'un nouveau client.

```

Begin tran ;
insert into clients (numero,nom, prenom, adresse, codepostal,
ville,telephone, coderep)
values(26,'Zazou','Zoé','rue des Zinnia','59123','Zuydcoote','05 59 17
18 19','CD');
save tran P1;

```

ROLLBACK TRAN[SACTION]

L'instruction **ROLLBACK** permet d'annuler une partie ou la totalité de la transaction, c'est-à-dire des modifications intervenues sur les données. L'annulation partielle d'une transaction n'est possible que si des points d'arrêt ont été définis à l'aide de l'instruction **SAVE TRAN**. Il n'est pas possible d'arrêter l'annulation entre deux points d'arrêt.

```

ROLLBACK { TRAN | TRANSACTION }
[nomTransaction[nomPointArret]][;]

```

Exemple

Dans l'exemple ci-dessous, le premier client inséré sera bien validé contrairement au deuxième, car l'instruction **INSERT** correspondante est annulée par l'instruction **ROLLBACK** qui annule toutes les modifications effectuées sur les données depuis la définition du point d'arrêt P1.

```

Begin tran ;
insert into clients (numero,nom, prenom, adresse, codepostal,
ville,telephone, coderep)
values(26,'Zazou','Zoé','rue des Zinnia','59123','Zuydcoote','05 59 17
18 19','CD');
save tran P1;
insert into clients (numero,nom, prenom, adresse, codepostal,
ville,telephone, coderep)
values(25,'Yvres','Yann','rue des yuccas','43200','Yssingaux','04 43 16
17 18','CD');
rollback tran;

```

COMMIT

Cette instruction permet de mettre fin avec succès à une transaction, c'est-à-dire de conserver l'ensemble des modifications effectuées dans la transaction. C'est simplement à l'issue de la transaction que les modifications sont visibles par les autres utilisateurs de la base de données.

```
COMMIT { TRAN | TRANSACTION } [nomTransaction] [;]
```

Pour SQL Server, si la transaction n'est pas commencée explicitement par la commande **BEGIN TRAN**, alors toute instruction SQL constitue une transaction qui est comitée (validée) automatiquement.



Si au cours d'une transaction, le client à l'origine de la transaction rompt brutalement sa connexion avec le serveur, alors la transaction est annulée (**ROLLBACK**) automatiquement.

Après l'exécution de la commande `COMMIT`, les données concernant la transaction sont enregistrées dans la mémoire vive du serveur et les informations sur la transaction dans le fichier du journal des transactions. Ce n'est qu'après un certain nombre de transactions sur la base de données que toutes les modifications seront transférées sur le fichier de données. Ceci sera réalisé par l'exécution automatique de la commande `CHECKPOINT`.

Cependant, les valeurs des enregistrements resteront cohérentes, même en cas d'arrêt du serveur, car SQL Server utilisera les deux fichiers (données et journal) pour obtenir les valeurs actuelles des données.

3. Les fichiers journaux

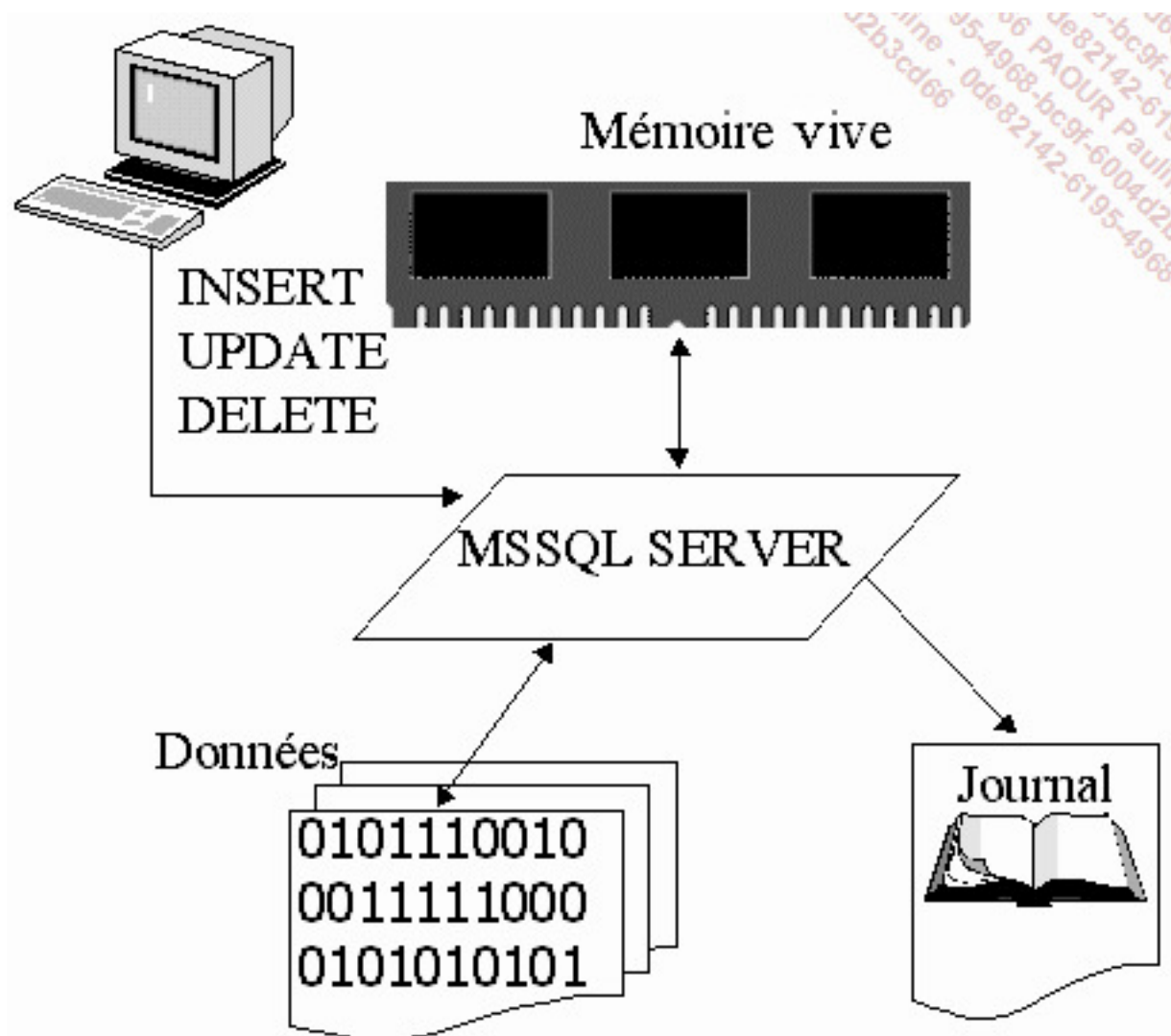
a. Rôle

Les fichiers journaux permettent de stocker les informations nécessaires à la cohérence des modifications ayant eu lieu sur les données contenues dans la base. Seules les opérations du DML (langage de manipulation de données), soit les ordres SQL `INSERT`, `UPDATE` et `DELETE`, provoquent une journalisation des opérations qu'elles effectuent sur les données de la base. Les opérations de grande envergure, comme la création d'un index, sont mentionnées dans le journal. Le journal sera utilisé principalement lors des opérations de restauration automatique suite à un arrêt brutal du serveur, ou bien lors des opérations de sauvegardes lorsque ces dernières s'appuient sur le journal. Il sera également utilisé lorsque la base participe à la réplication.

Le but du journal est de permettre au serveur de toujours garantir la cohérence des données, c'est-à-dire que toutes les transactions validées (`COMMIT`) persistent même s'il arrive un gros problème causant l'arrêt brutal du serveur.

Lors de chaque redémarrage du serveur, SQL Server exécute un point de synchronisation (`CHECKPOINT`) afin de finaliser le traitement de toutes les transactions validées (`COMMIT`), tandis que toutes les transactions non validées sont annulées (`ROLLBACK`).

b. Fonctionnement



Fonctionnement du journal

Lorsqu'un ordre SQL est transmis au serveur SQL, ce dernier va chercher à l'exécuter le plus rapidement possible. Après analyse de l'ordre et mise en place du plan d'exécution, si les données concernées par l'ordre ne sont pas déjà présentes en mémoire, alors le moteur SQL va lire les fichiers sur le disque dur afin d'y trouver les informations nécessaires. Une fois présentes en mémoire, les modifications peuvent être apportées aux données. La modification est toujours enregistrée dans le journal avant d'être réellement effectuée sur les données de la base. Un tel journal est appelé journal à écriture anticipée.

D'une façon logique toutes les informations sont enregistrées les unes à la suite des autres dans le journal. Chaque information est parfaitement identifiée par son LSN (*Log Sequence Number*) ou numéro séquentiel d'enregistrement dans le journal. Chaque

enregistrement dans le journal contient également l'identifiant de la transaction à l'origine de la modification des données.



Les fichiers journaux sont situés, par défaut, dans le répertoire C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\Data, et portent l'extension *.ldf. Il s'agit d'une extension recommandée qui n'est nullement obligatoire. Le journal peut être constitué d'un ou plusieurs fichiers, la taille de ces derniers pouvant être fixe ou variable automatiquement ou de façon manuelle. La gestion des fichiers sera abordée dans ce chapitre dans la section Création, gestion et suppression d'une base de données.

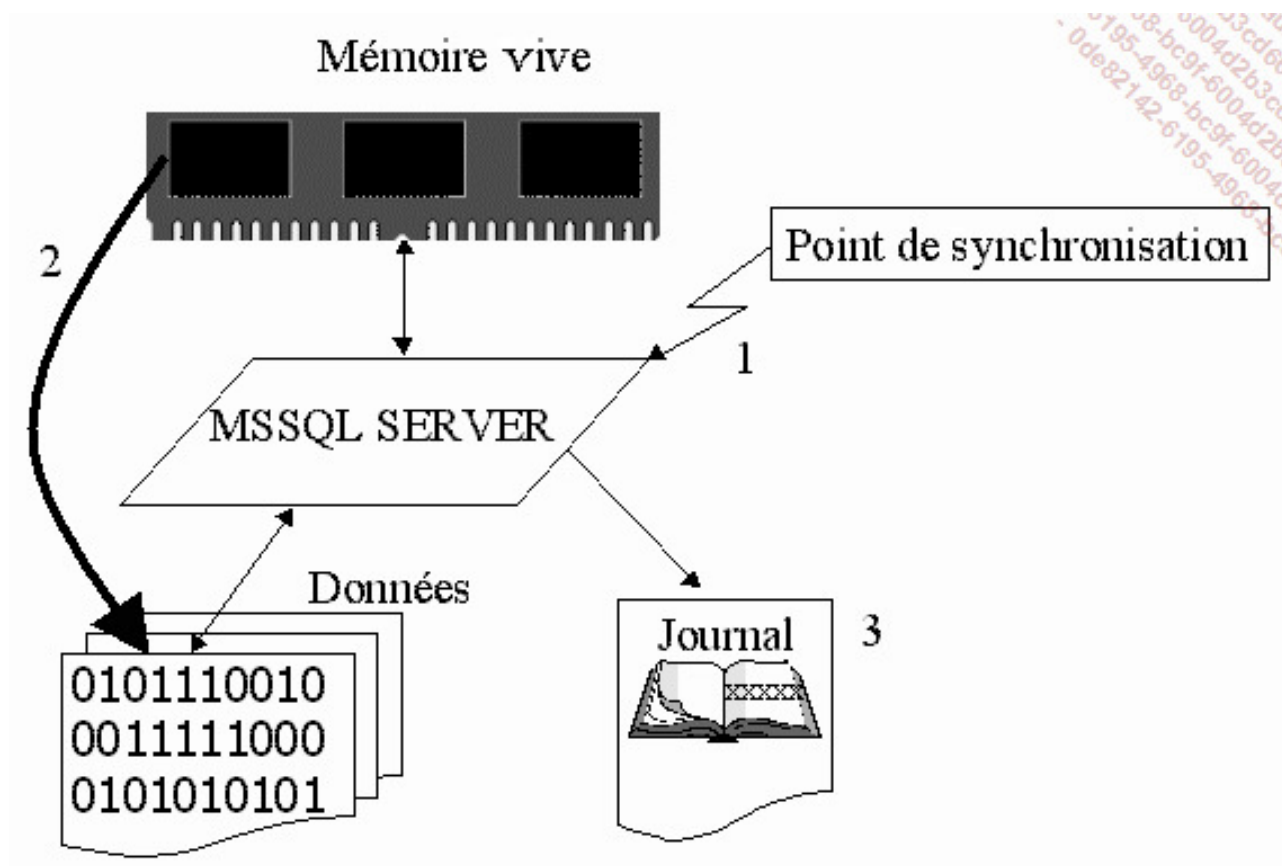
Bien que le journal des transactions puisse être constitué de plusieurs fichiers physiques, ceci n'est pas recommandé (sauf cas particulier, par exemple si le volume disque contenant le fichier journal est plein). En effet, le journal est rempli séquentiellement : le premier fichier est rempli, puis on passe au deuxième fichier, etc. Ceci n'améliorerait donc pas les performances et compliquerait la gestion.



Pour connaître avec précision les performances des disques, Microsoft propose l'utilitaire Diskspd. Cet utilitaire remplace SQLIO et il est disponible depuis le site Microsoft.

c. Les points de synchronisation

Régulièrement, SQL Server va déclencher un point de synchronisation. Il consiste à écrire sur le fichier des données toutes les données modifiées des transactions validées. Les points de synchronisation sont également appelés CHECKPOINT. Le nombre de données touchées entre deux points de synchronisation va déterminer le temps de restauration automatique suite à un arrêt brutal du serveur.



Principe de fonctionnement d'un point de synchronisation

SQL Server optimise les points de synchronisation de façon à garantir la meilleure gestion des données sans détériorer les temps de réponse du serveur. Il est toutefois possible d'intervenir sur cette optimisation par l'intermédiaire de l'instruction **CHECKPOINT**.

CHECKPOINT [tempsRealisation]

tempsRealisation

Permet de préciser le temps accordé en secondes pour terminer le point de synchronisation. C'est SQL Server qui se charge de déclencher le point de synchronisation de façon à avoir terminé dans les temps.



Seul un arrêt de l'instance par l'intermédiaire de l'instruction Transact SQL `SHUTDOWN WITH NOWAIT` permet d'arrêter l'instance sans déclenchement d'un point de synchronisation.

La lecture du journal est possible par l'intermédiaire de la fonction système `sys.fn_dblog`. Ce type de fonction peut se montrer particulièrement intéressant pour identifier le LSN (*Log Sequence Number*) ou bien la date et heure de réalisation d'une transaction. Ce type d'information sera nécessaire lors d'une restauration jusqu'à un point bien déterminé.

```
select * from sys.fn_dblog(null,null)
```

4. Les fichiers de données

a. Rôle

Chaque base de données possède au moins un fichier de données. Ce fichier va contenir l'ensemble des données stockées dans la base. Chaque fichier de données ne peut contenir que des données en provenance d'une seule base, il y a donc spécialisation des fichiers par rapport à la base de données.

b. Structure des fichiers de données

Les fichiers de données sont structurés pour répondre de manière optimum à toutes les sollicitations de la part du moteur et surtout pour être capables de stocker plus de données en optimisant l'espace disque utilisé.

Pour optimiser l'espace dont il dispose, le serveur va formater les fichiers de données de façon à maîtriser leur structure.

Le travail réalisé par SQL Server sur ces fichiers de données est semblable au travail fait par le système d'exploitation sur les disques disponibles sur la machine. Il est tout à fait

admis que le système d'exploitation formate l'espace disque dont il dispose afin de le diviser en blocs. Par la suite ce sont ces blocs qui vont être accordés aux fichiers. SQL Server réalise le même type de travail sur les fichiers de données, puis accorde l'espace disponible aux différentes tables et index.

Les pages

Avant de pouvoir travailler avec un fichier de données, SQL Server va structurer le fichier en le découpant en pages de 8 ko. La taille de 8 ko (non paramétrable) est fixée par SQL Server, de façon à réduire les pertes d'espace tout en simplifiant les opérations d'allocation, mais aussi de lecture et d'écriture. La page représente l'unité de travail de SQL Server. C'est toujours une page entière de données qui est remontée en mémoire et c'est toujours une page qui est écrite sur le disque. Il n'est pas possible de remonter en mémoire cache, une partie d'une page. Bien entendu, une page peut contenir plusieurs lignes d'une table ou bien plusieurs entrées d'index et toutes les informations sont remontées en une seule lecture disque.

Cette taille de 8 ko permet également de gérer des bases de données de plus grande taille avec un nombre de blocs moindre.

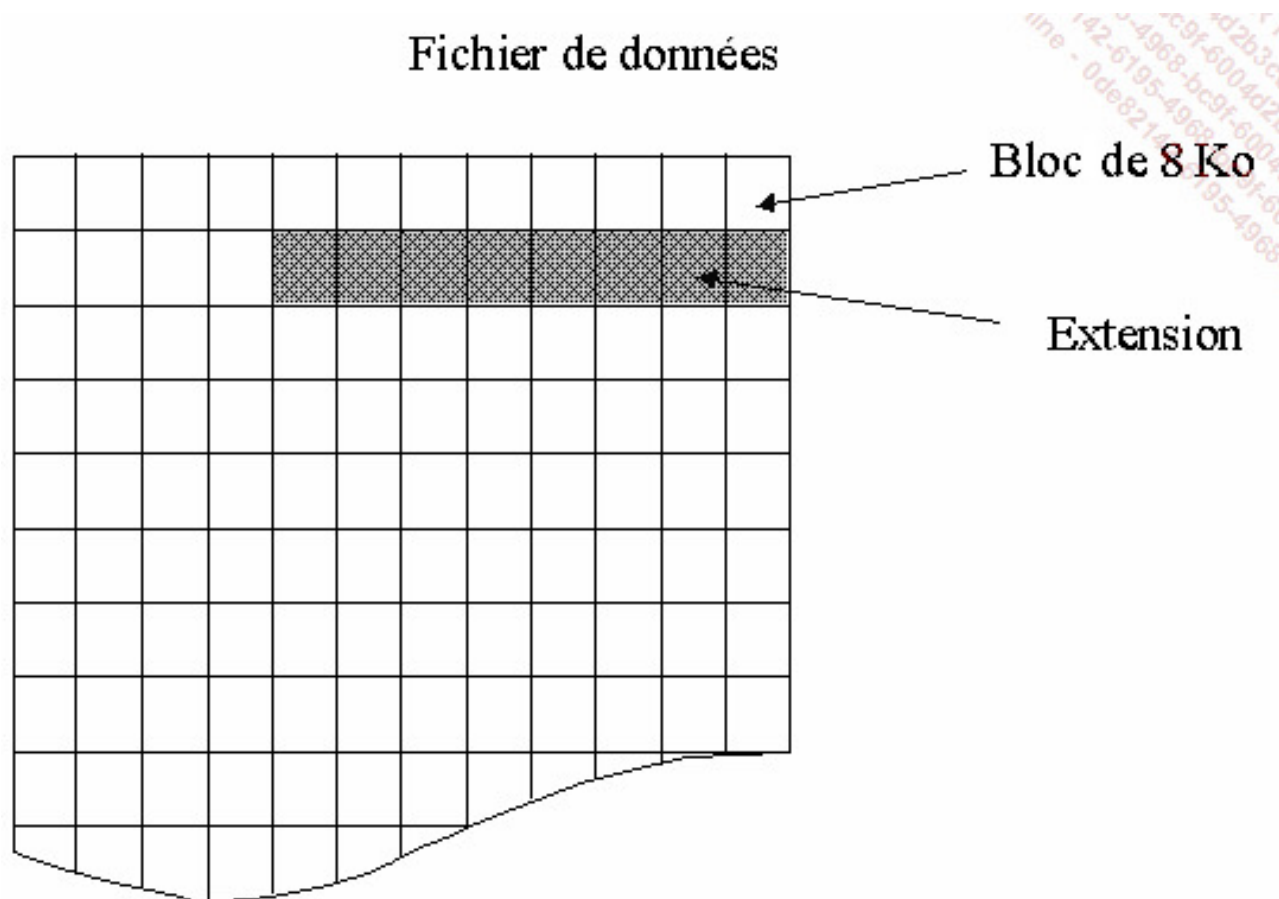
Enfin, pour éviter la fragmentation des lignes de données sur plusieurs pages, une ligne doit toujours être entièrement contenue (hors type varchar/nvarchar(max), varbinary(max) et XML) dans une page. La taille maximale d'une ligne est donc de 8060 octets. Cette valeur de 8060 octets est légèrement inférieure à 8 ko car SQL Server réserve quelques octets pour gérer l'en-tête de la page afin d'en maîtriser la gestion.

Si plusieurs lignes sont stockées dans la même page, elles le sont de façon séquentielle. En cas de changement de taille des lignes, SQL Server gère dynamiquement le déplacement des lignes dans la page ou bien la mise en place d'un pointeur vers une autre page afin de lire la fin de la ligne de données.

Étant donné que la page est l'unité de travail de SQL Server, chaque page contient un type bien précis de données. Il est possible de distinguer les types de page suivants :

- Données : ces pages contiennent des informations au format numérique, texte ou bien date.
- Texte/image : ces pages contiennent soit des textes volumineux, soit des objets au format binaire.

- Index : ces pages contiennent les entrées des index.
- GAM/SGAM (ou *Global Allocation Map* et *Shared Global Allocation Map*) : ces pages contiennent des informations relatives à l'allocation des extensions.
- PFS (ou *Page Free Space*) : ces pages contiennent les informations relatives à l'allocation des pages et l'espace disponible sur ces pages.
- IAM (ou *Index Allocation Map*) : ces pages contiennent les informations relatives à l'utilisation de pages par les tables et les index.
- BCM (ou *Bulk Change Map*) : ces pages contiennent la liste des extensions modifiées par des opérations de copie en bloc depuis la dernière sauvegarde du journal.
- DCM (ou *Differential Change Map*) : ces pages contiennent la liste de toutes les extensions modifiées depuis la dernière sauvegarde de la base.



Le découpage en blocs des fichiers de données

Les extensions

Les extensions sont des regroupements logiques de 8 pages contiguës, elles ont donc une taille de 64 ko (8 x 8 ko). Le rôle des extensions est d'éviter une trop grande dispersion des données pour un même objet au sein des fichiers de données.

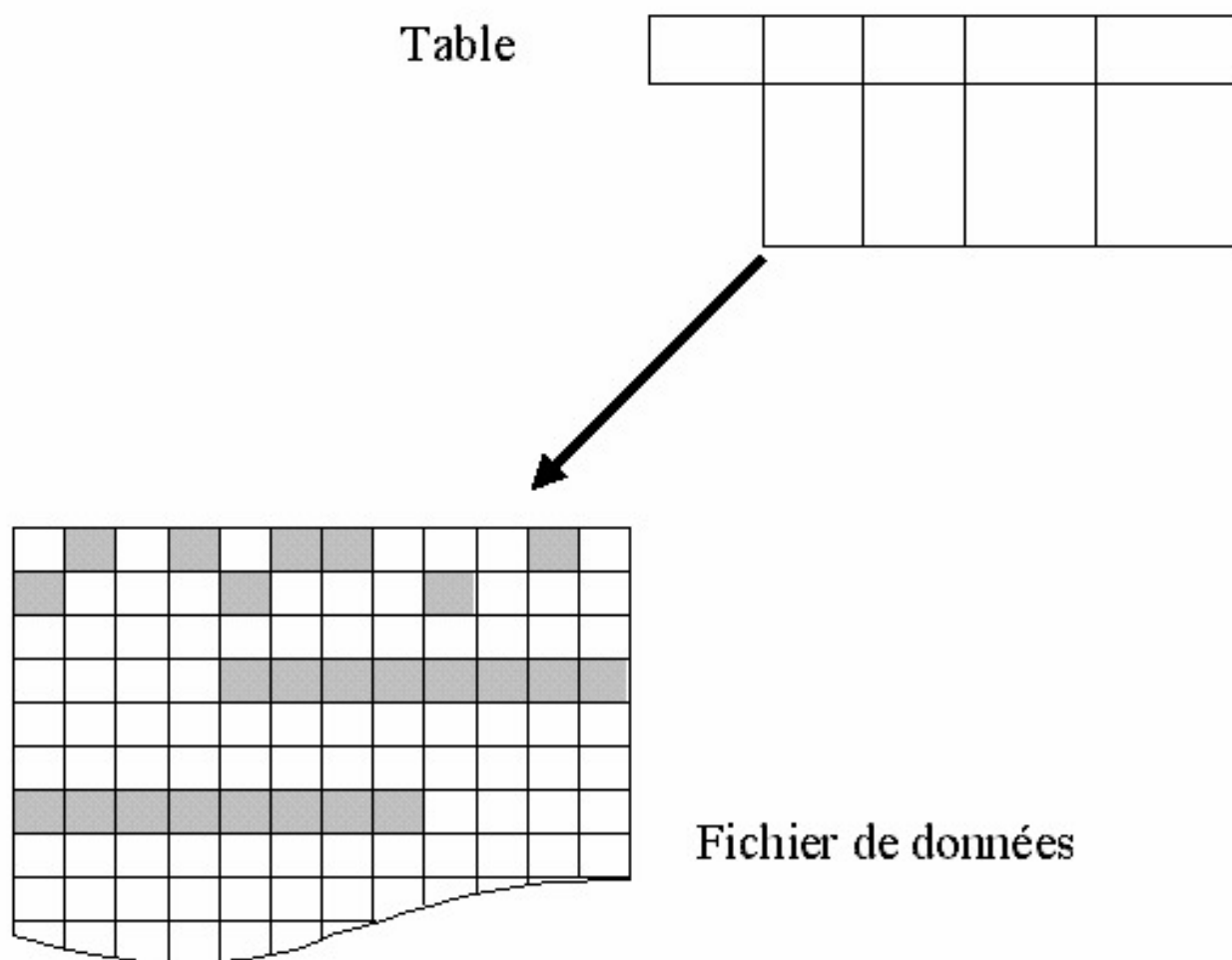
Il existe deux types d'extension : les extensions mixtes et les extensions uniformes. Ces deux types d'extension permettent de limiter au mieux la consommation d'espace disque par une table en fonction du volume de données à stocker.

Les extensions mixtes

Au départ, les extensions sont communes à plusieurs tables ou plusieurs index. Lorsqu'un des objets définis sur l'extension demande de la place, SQL Server octroie de la place page par page tant que l'objet n'utilise pas 8 pages. Dès qu'un objet franchit cette barrière, SQL Server lui octroie de la place sur des extensions uniformes. L'avantage de cette méthode est d'éviter de perdre de la place inutilement avec les tables ou index contenant peu de données.

Les extensions uniformes

Si un objet occupe plus de 64 ko d'information, alors la place lui sera accordée extension par extension. Chacune des extensions étant spécialisée pour un objet, les lectures disque seront d'autant plus rapides car toutes les données sont stockées de façon contiguë par paquets de 64 ko.



L'allocation des extensions

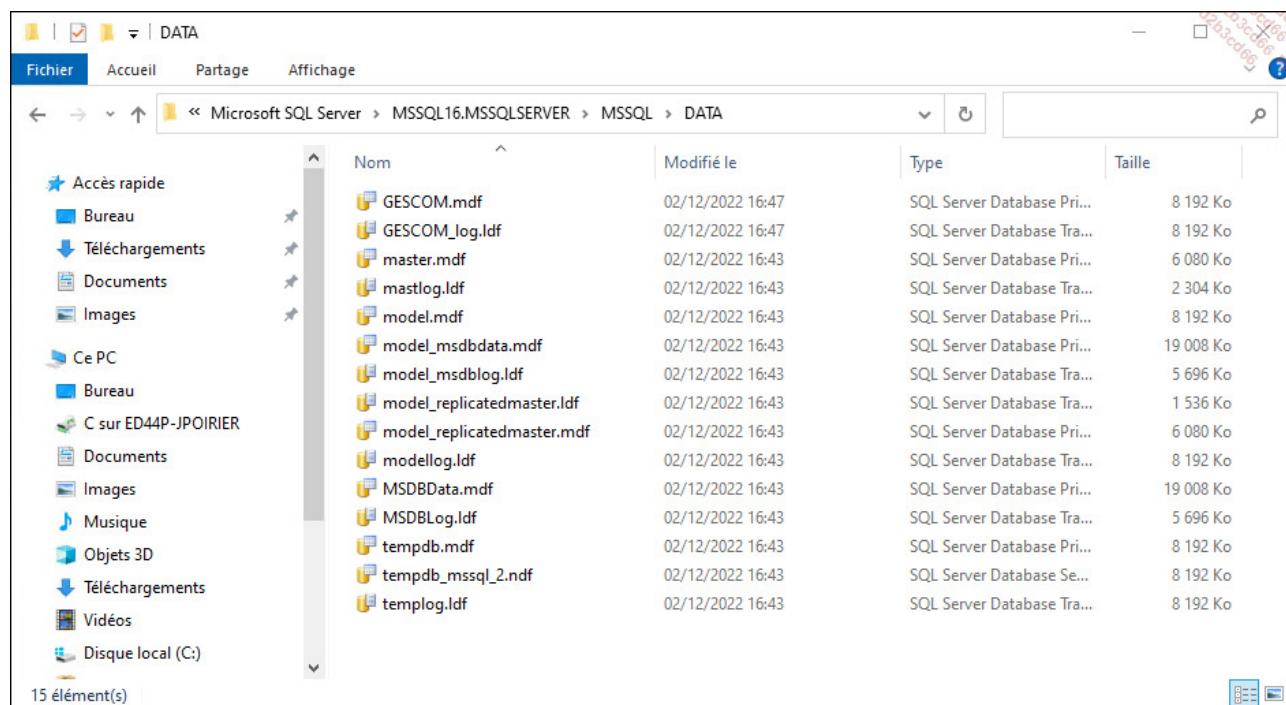
Ainsi, afin d'optimiser le fonctionnement de SQL Server, il convient de placer les fichiers de données sur des volumes formatés avec des unités d'allocation de 64 ko (4 ko par défaut).

c. Fonctionnement

Chaque base possède au moins un fichier de données, ce fichier est spécifique à une base. Son chemin par défaut est C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\Data. Le premier fichier de données d'une base porte l'extension mdf, tous les suivants portent l'extension ndf. Ces extensions sont recommandées mais ne sont pas obligatoires.

Les fichiers des bases système master, model et msdb peuvent rester sur le répertoire par défaut sur C:. Pour les fichiers des bases de données utilisateurs et tempdb, comme

indiqué ci-dessus, il est souhaitable d'avoir un autre volume formaté avec des unités d'allocation de 64 ko.



Création, gestion et suppression d'une base de données

Une base de données gère un ensemble de tables système et des tables utilisateurs. Les informations contenues dans ces tables système représentent, entre autres, la définition des vues, des index, des procédures, des fonctions, des utilisateurs et des privilèges. Les tables utilisateurs vont contenir les informations saisies par les utilisateurs.



Une instance SQL Server ne peut pas contenir plus de 32 767 bases de données.

1. Créer une base de données

La création d'une base de données est une étape ponctuelle, réalisée par un administrateur SQL Server. Avant de tenter de créer une base de données, il est important de définir un certain nombre d'éléments de façon précise :

- Le nom de la base de données qui doit être unique sur le serveur SQL.
- La taille de la base de données.
- Les fichiers utilisés pour le stockage des données.

Pour créer une nouvelle base de données, SQL Server s'appuie sur la base Model. Cette base Model contient tous les éléments qui vont être définis dans les bases utilisateurs. Par défaut, cette base Model contient les tables système. Il est cependant tout à fait possible d'ajouter des éléments dans cette base. Toutes les bases utilisateurs créées par la suite disposeront de ces éléments supplémentaires.

Une base peut être créée de deux façons différentes :

- Par l'intermédiaire de l'instruction Transact SQL `CREATE DATABASE`.
- Par l'intermédiaire de SQL Server Management Studio.

Une base de données est toujours composée au minimum d'un fichier de données principal (extension mdf) et d'un fichier journal (extension ldf). Des fichiers de données secondaires (ndf) peuvent être définis lors de la création de la base ou bien ultérieurement.

Les informations concernant les fichiers de données et les fichiers journaux sont enregistrées dans la base Master.

a. Syntaxe Transact SQL

```
CREATE DATABASE nomBaseDeDonnées
[ ON
  [PRIMARY] [ <spécificationFichier> [,n]]
  [LOG ON < spécificationFichier > [,n]]
]
[ COLLATE classement ]
[;]
```

Avec pour `spécificationFichier` les éléments de syntaxe suivants :

```
(NAME = nomLogique,
FILENAME = 'cheminEtNomFichier'
[.SIZE = taille [KB|MB|GB|TB]]
[.MAXSIZE={tailleMaximum[KB|MB|GB|TB]|UNLIMITED}]
[.FILEGROWTH = pasIncrement [KB|MB|GB|TB|%]]
) [,n]
```

PRIMARY

Permet de préciser le premier groupe de fichiers de la base. Ce groupe est obligatoire, car l'ensemble des tables système est obligatoirement créé dans le groupe primary. Si le mot-clé `PRIMARY` est omis, le premier fichier de données précisé dans la commande `CREATE DATABASE` correspond obligatoirement au fichier primaire, il porte normalement l'extension mdf.

NAME

Cet attribut, obligatoire, permet de préciser le nom logique du fichier. Ce nom sera utilisé pour faire des manipulations sur le fichier via des commandes Transact SQL ; on pense notamment aux commandes DBCC ou à la gestion de la taille des fichiers.

FILENAME

Spécification du nom et emplacement physique du fichier sur le disque dur. Le fichier de données est toujours créé sur un disque local du serveur.

SIZE

Lorsque la taille n'est pas précisée pour le fichier principal de base de données (mdf), c'est alors la taille du fichier principal de la base Model qui est reprise, à savoir 8 Mo. En aucun cas une base de données ne peut avoir un fichier principal dont la taille serait inférieure à celle du fichier de la base Model lors de sa création.

Pour les fichiers secondaires et les fichiers journaux, la taille minimale est de 1 Mo, mais la taille par défaut est de 8 Mo. Cette même taille par défaut est également affectée lors de la création d'un fichier journal.

La taille d'un fichier est toujours exprimée à l'aide d'un nombre entier. Si l'on souhaite exprimer une valeur non entière, il est nécessaire de changer d'unité. Par exemple, il n'est pas possible de dire qu'un fichier possède une taille de 2,5 Mo : il faudra préciser que la taille du fichier est de 2560 ko.

MAXSIZE

Cet attribut optionnel permet de préciser la taille maximale en mégaoctets (par défaut) que peut prendre le fichier. Si rien n'est précisé, le fichier grossira jusqu'à saturation du disque dur, dans la limite de 16 To pour un fichier de données et de 2 To pour un fichier journal.

FILEGROWTH

Il s'agit de préciser le facteur d'extension du fichier. La taille de chacune des extensions peut correspondre à un pourcentage (%), à une taille en mégaoctets (par défaut) ou en kilo-octets. Si on précise la valeur zéro, le facteur d'extension automatique du fichier est nul et donc la taille du fichier ne change pas automatiquement. Le pas d'accroissement

défini par défaut est de 64 Mo. Cette valeur s'applique aussi bien aux fichiers de données qu'aux fichiers journaux.

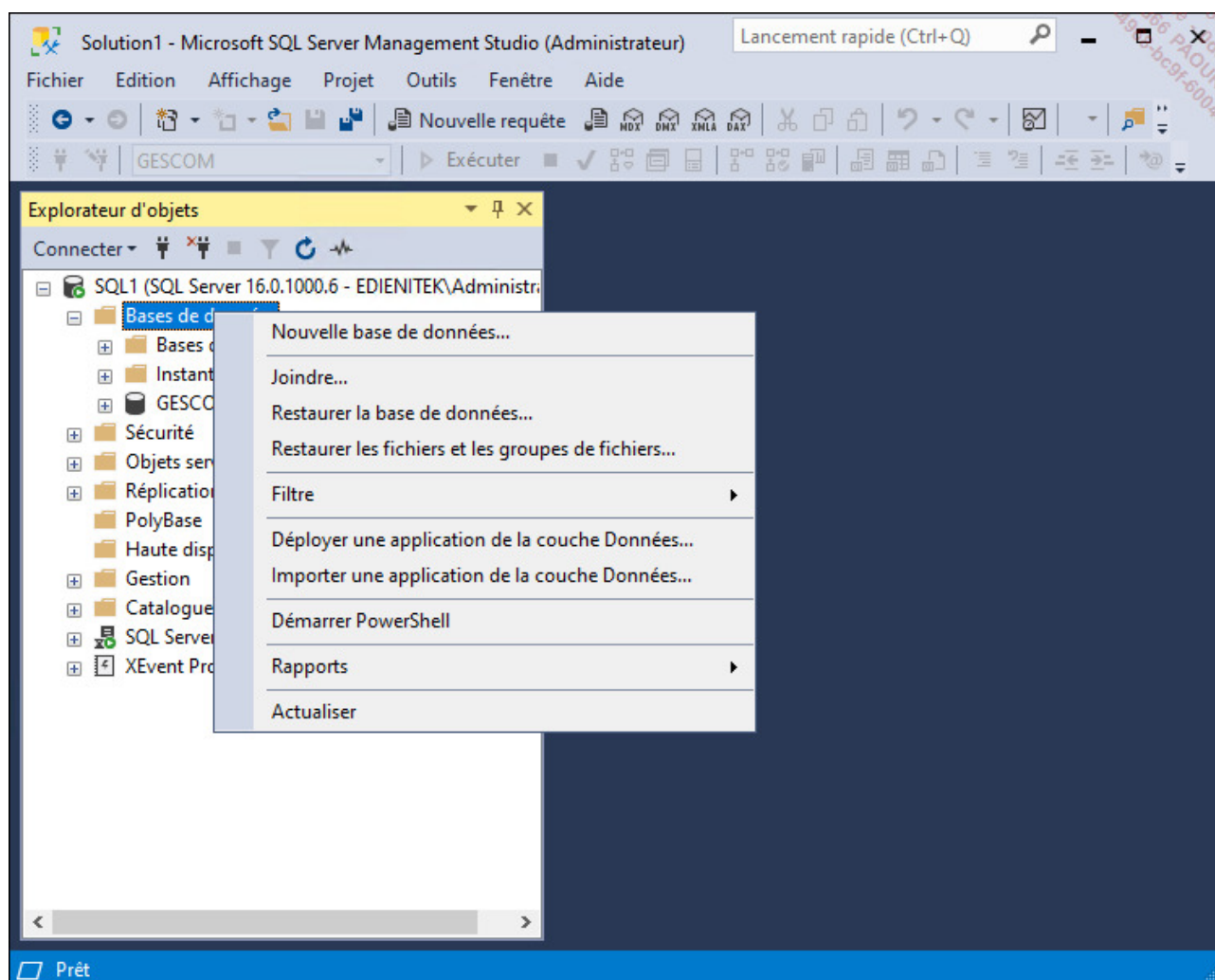
```
CREATE DATABASE [GESCOM]
CONTAINMENT = NONE
ON PRIMARY
( NAME = N'GESCOM',
FILENAME = N'C:\Program Files\Microsoft SQL
Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\GESCOM.mdf' ,
SIZE = 8192KB , MAXSIZE = UNLIMITED , FILEGROWTH = 65536KB )
LOG ON
( NAME = N'GESCOM_log',
FILENAME = N'C:\Program Files\Microsoft SQL
Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\GESCOM_log.ldf' ,
SIZE = 8192KB , MAXSIZE = 2048GB , FILEGROWTH = 65536KB )
WITH CATALOG_COLLATION = DATABASE_DEFAULT
GO
```



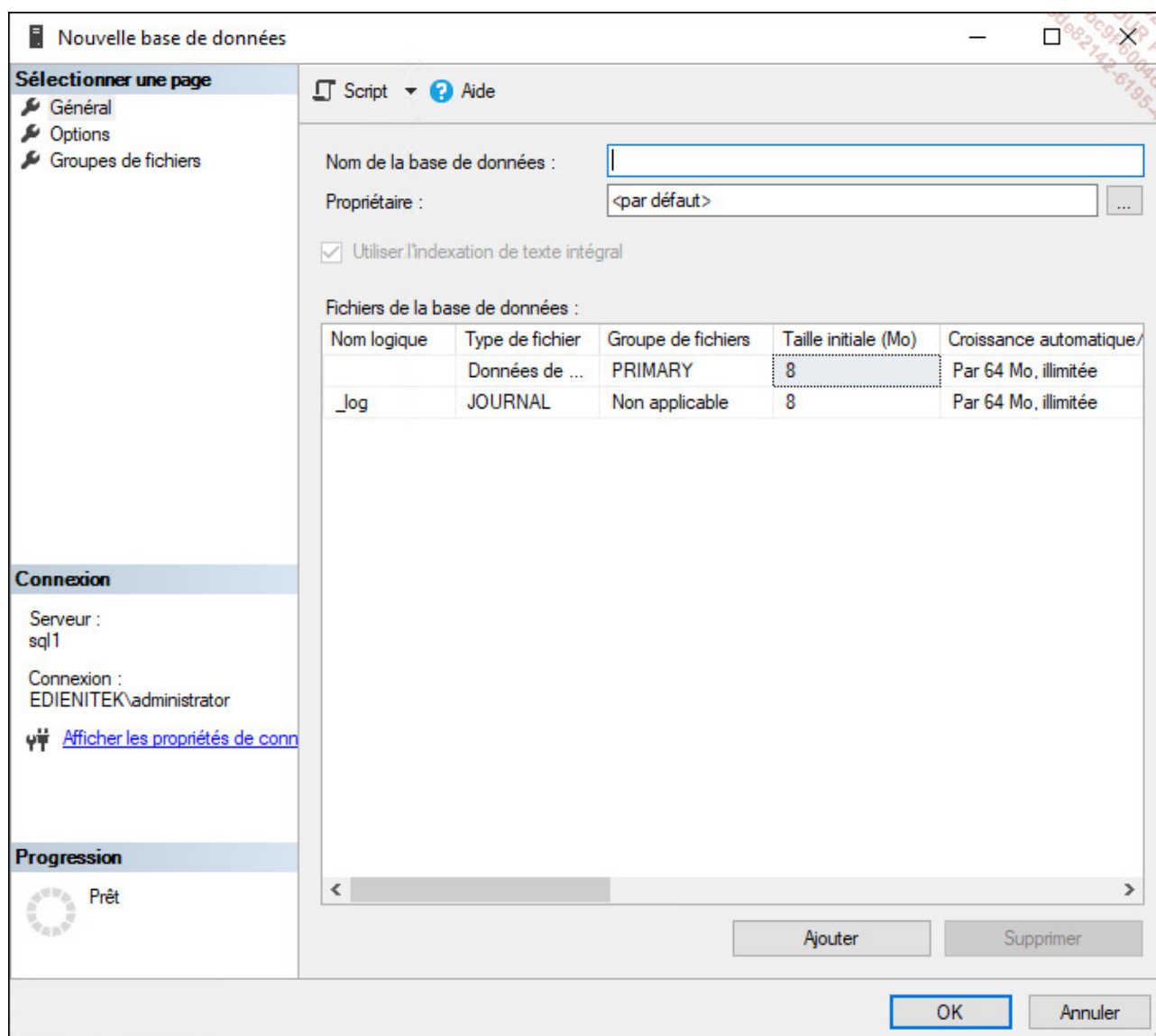
Toutes les options de l'instruction `CREATE DATABASE` ne sont pas exposées ici. Seules les options les plus couramment utilisées lors de la création d'une nouvelle base le sont.

b. Utilisation de SQL Server Management Studio

Il est également possible de créer une base de données de façon graphique depuis SQL Server Management Studio. Il faut alors sélectionner **Nouvelle base de données** depuis le menu contextuel associé au nœud **Bases de données** depuis l'explorateur d'objets, comme illustré ci-après.



SQL Server Management Studio présente alors la boîte de dialogue des propriétés de la base en mode création. Après avoir complété les options comme le nom de la base et le nom et l'emplacement des fichiers, il est possible de demander la création de la base.



Cette boîte de dialogue permet à un utilisateur averti de créer rapidement et simplement une base de données, tout en conservant la maîtrise de tous les paramètres.

Les fichiers de base de données ne doivent pas être créés sur des partitions compressées ou chiffrées.

2. Gérer une base de données

Lors de la gestion d'une base de données, plusieurs critères sont à prendre en compte. Dans cette section, la gestion de l'espace utilisé par les fichiers physiques qui constituent

la base de données est abordée. Les points principaux qui concernent la gestion des fichiers sont : l'accroissement dynamique ou manuel des fichiers, l'ajout de nouveaux fichiers, la réduction de la taille des fichiers.



Sur une base de données, il y a au maximum 32 767 groupes de fichiers et pas plus de 32 767 fichiers par groupe de fichiers.

a. Augmenter l'espace disque disponible pour une base de données

Les fichiers de données et les fichiers journaux stockent de l'information. Comme la base contient normalement de plus en plus d'informations, à un moment donné ces fichiers seront complets. Il se posera alors le problème de savoir où prendre la place.

Les différentes méthodes exposées ci-dessous pour augmenter l'espace de stockage dont dispose la base sont complémentaires car chaque méthode possède ses avantages et ses inconvénients.

Fichier à accroissement dynamique

Lors de la création de la base, il est possible de fixer un certain nombre de critères concernant la taille maximale de fichiers (**MAXSIZE**) et le taux d'accroissement (**FILEGROWTH**). Si ces options sont omises la taille maximale est infinie et le taux d'accroissement est 64 Mo.

En utilisant des fichiers à croissance dynamique, le serveur ne sera jamais bloqué par la taille du fichier sauf saturation du disque ou taille maximale atteinte.

Le facteur d'accroissement permet de fixer la taille de tous les ajouts. Cette taille doit être fixée de façon optimale, en prenant en compte le fait qu'un facteur d'accroissement trop petit introduit beaucoup de fragmentations physiques du fichier et les demandes d'extension du fichier sont fréquentes, tandis qu'un facteur d'accroissement trop grand nécessite une charge de travail importante de la part du serveur lorsque celui-ci met en place la structure de blocs de 8 ko à l'intérieur du fichier.



Si le taux d'accroissement est fixé à zéro, le fichier ne pourra pas grandir dynamiquement.



L'accroissement du fichier possède toujours une taille qui est multiple de 64 ko (taille d'une extension).

Si le paramétrage des fichiers permet de gérer automatiquement la croissance des fichiers, cette solution présente tout de même le désavantage du fait qu'il n'est pas possible de maîtriser quand cet accroissement aura lieu. Si cette opération intervient en pleine charge de travail, elle risque de ralentir le serveur.

Par contre, si le paramétrage de l'accroissement automatique des fichiers est réalisé, cela permet en cas de problème, que les fichiers adaptent leur taille en fonction du volume de données, sans bloquer les utilisateurs.

Fichier à accroissement manuel

La croissance manuelle des fichiers permet de maîtriser le moment où le fichier va grossir et donc le moment où le serveur va subir une charge de travail supplémentaire pour mettre en forme le fichier s'il s'agit d'un fichier de données.

Ajout de fichiers

Ceci peut s'avérer intéressant pour les très grosses bases de données sur les fichiers de données. Ainsi, il deviendra possible de partitionner des tables ou de faire des sauvegardes par fichiers ou groupes de fichiers afin d'en réduire la durée.

Préconisation

L'idéal sera de prévoir un système de surveillance des serveurs afin d'anticiper les phases d'augmentation de taille des fichiers manuellement et ainsi ne pas surcharger le serveur pendant les périodes de travail des utilisateurs.

De plus, paramétrer chaque fichier avec un taux de croissance automatique d'environ 10 % évitera de bloquer les utilisateurs en cas de fichiers remplis complètement.

Modifier un fichier en Transact SQL

C'est l'instruction **ALTER DATABASE** qui permet d'effectuer toutes les opérations relatives aux manipulations des fichiers de base de données, aussi bien pour les fichiers de données que les fichiers du journal des transactions.

Toutes les caractéristiques des fichiers peuvent être modifiées, mais les modifications apportées doivent suivre certaines règles comme, par exemple, la nouvelle taille du fichier qui doit être supérieure à la taille initiale.

```
ALTER DATABASE nomBaseDeDonnées
MODIFY FILE (spécificationFichier)[,]
```

Avec pour **spécificationFichier** les éléments de syntaxe suivants :

```
(NAME = nomLogique,
NEWNAME = nouveauNomLogique,
FILENAME = 'cheminEtNomFichier'
[,SIZE = taille [KB|MB|GB|TB]]
[,MAXSIZE={tailleMaximum[KB|MB|GB|TB]]|UNLIMITED})
[,FILEGROWTH = pasIncrement [KB|MB|GB|TB|%]]
)
```

```
USE [master]
GO
ALTER DATABASE [GESCOM]
MODIFY FILE ( NAME = N'GESCOM', SIZE = 51200KB )
GO
```

Ajouter un fichier en Transact SQL

C'est la commande **ALTER DATABASE** associée à l'option **ADD FILE** qui va permettre de rajouter un fichier de données ou un fichier journal.

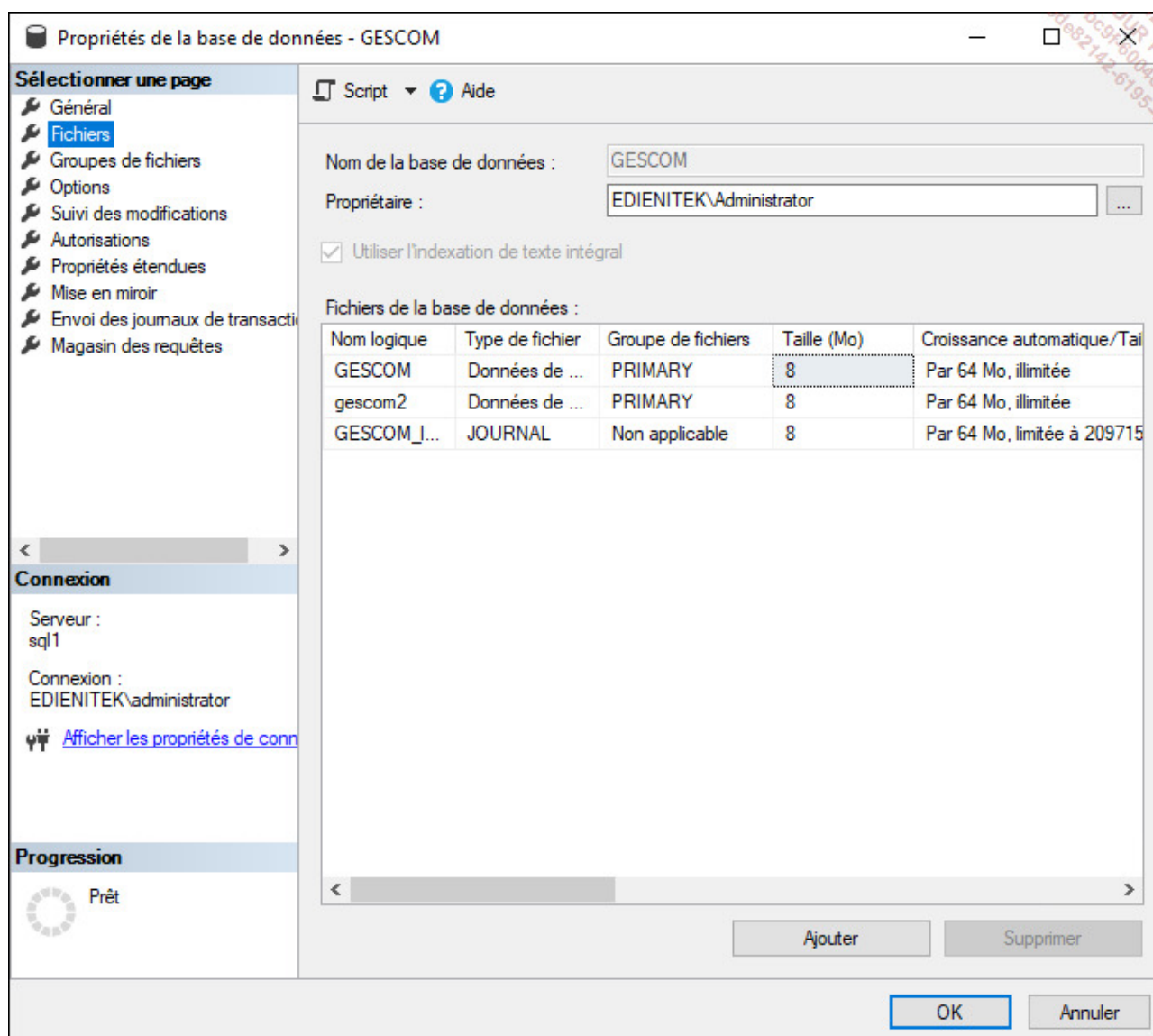
Syntaxe


```
ALTER DATABASE nomBaseDeDonnées  
ADD FILE spécificationFichier[;]
```

```
USE [master]  
GO  
ALTER DATABASE [GESCOM]  
ADD FILE ( NAME = N'gescom2',  
FILENAME = N'C:\Program Files\Microsoft SQL  
Server\MSSQL16.MSSQLSERVER\MSSQL\DATA\gescom2.ndf' ,  
SIZE = 8192KB , FILEGROWTH = 65536KB )  
TO FILEGROUP [PRIMARY]  
GO
```

Modifier et ajouter des fichiers depuis SQL Server Management Studio

C'est par l'intermédiaire de la boîte de dialogue qui indique les propriétés de la base de données qu'il est possible de gérer les opérations sur les fichiers et leur taille.



Emplacement de la base tempdb

La base système tempdb sera d'une taille comparable à celle des bases utilisateurs (prévoir une taille initiale d'environ un tiers de la totalité des bases utilisateurs). Ainsi, il est nécessaire de ne pas la laisser sur le volume C: car elle pourrait en remplir l'espace disponible et empêcher Windows de fonctionner correctement.

Dès l'installation, cette base peut être installée sur un autre volume avec le nombre de fichiers de données nécessaires (1 par cœur de processeur, 8 au maximum). Cependant, si ce n'est pas le cas, il est toujours possible de le faire après l'installation de SQL Server via les commandes suivantes :

```

USE master;
GO
ALTER DATABASE tempdb
MODIFY FILE (NAME = tempdev, FILENAME = 'D:\donnees\tempdb.mdf');
GO
ALTER DATABASE tempdb
MODIFY FILE (NAME = templog, FILENAME = 'D:\donnees\templog.ldf');
GO

```

À noter qu'il y a une commande par fichier (données et journal), donc si votre base comporte déjà plusieurs fichiers de données, il faudra exécuter cette commande pour chacun d'eux.

Suite à l'exécution des commandes, le service SQL Server doit être redémarré pour prendre en compte le nouvel emplacement et recréer la base tempdb. Les anciens fichiers ne seront pas supprimés, c'est à l'administrateur de le faire manuellement.

b. Libérer de l'espace disque utilisé par des fichiers de données vides

Lorsque les tables sont vidées de leurs données à l'aide des commandes **DELETE** ou **TRUNCATE TABLE**, les extensions occupées par les tables et index sont alors libérées. Par contre, la taille des fichiers n'est pas réduite. Pour réaliser une telle opération il est important de s'assurer que la totalité de l'espace libre est regroupée en fin de fichier. Une fois cette opération réalisée, il est possible de tronquer le fichier sans jamais redescendre en dessous de la taille initiale.

Avant de réduire la taille des fichiers, il est important de s'assurer que la base de données n'aura pas besoin de cet espace d'ici quelques temps. Par exemple, si on sait qu'en fin de mois ou trimestre une clôture quelconque permet de supprimer de nombreuses lignes de données, il ne faut pas pour autant libérer l'espace disque. En effet, au cours du prochain mois ou trimestre, de nouvelles informations vont être enregistrées. Ces informations vont nécessiter de l'espace disque et donc les fichiers vont de nouveau être utilisés. Si les fichiers ne contiennent pas d'informations, ils ne dégradent pas les temps de sauvegarde au niveau de SQL Server.

La mise en place de la réduction des fichiers se fait au moyen de deux commandes DBCC.

SHRINKDATABASE

Cette instruction permet de compacter l'ensemble des fichiers constituant la base de données (journaux et données). Pour les fichiers de données toutes les extensions utilisées sont stockées de façon contiguë en haut du fichier. Pour les fichiers journaux, cette opération de compactage intervient en différé et SQL Server essaie de rendre aux fichiers journaux une taille aussi proche possible que celle de la taille cible lorsque les journaux sont tronqués.

Syntaxe :

```
DBCC SHRINKDATABASE (nom_base_données|id_base_données|0)  
[ ,pourcentage_cible]  
[ ,{NOTRUNCATE|TRUNCATEONLY}])
```

nom_base_données

Nom de la base de données sur laquelle va porter l'exécution de la commande.

id_base_données

Identifiant de la base de données. Cet identifiant peut être connu en exécutant la fonction `db_id()` ou bien en interrogeant la vue `sys.databases` depuis la base master.

0

Permet de préciser que l'exécution portera sur la base courante.

pourcentage_cible

Permet de préciser en pourcentage l'espace libre souhaité dans le fichier après compactage.

NOTRUNCATE

Permet de ne pas rendre au système d'exploitation l'espace libre obtenu après compactage. Par défaut, l'espace ainsi obtenu est libéré.

TRUNCATEONLY

Permet de libérer l'espace inutilisé dans les fichiers de données et compacte le fichier à la dernière extension allouée. Aucune réorganisation physique des données n'est envisagée : déplacement des lignes de données afin de compléter au mieux les extensions utilisées par l'objet. Dans un tel cas, le paramètre `pourcentage_cible` est ignoré.

SHRINKFILE

Cette instruction, semblable à `DBCC SHRINKDATABASE` permet de réaliser les opérations de compactage et réduction de fichier de données par fichier.

Syntaxe :

```
DBCC SHRINKFILE ([nom_fichier|id_fichier]
{[[taille_cible]
[{NOTRUNCATE|TRUNCATEONLY}]]|EMPTYFILE}
```

taille_cible

Permet de préciser la taille finale souhaitée exprimée en mégaoctets sous forme de nombre entier. Si aucune taille n'est spécifiée, la taille du fichier est réduite à son maximum.

EMPTYFILE

Cette commande permet de réaliser la migration de toutes les données contenues dans le fichier vers les autres fichiers du même groupe. De plus, SQL Server n'utilise plus ce fichier. Il est alors possible de le supprimer de la base de données à l'aide d'une commande `ALTER DATABASE`.

NOTRUNCATE

Permet de ne pas rendre au système d'exploitation l'espace libre obtenu après compactage. Par défaut, l'espace ainsi obtenu est libéré.

TRUNCATEONLY

Permet de libérer l'espace inutilisé dans les fichiers de données et compacte le fichier à la dernière extension allouée. Aucune réorganisation physique des données n'est envisagée : déplacement des lignes de données afin de compléter au mieux les extensions utilisées par l'objet. Dans un tel cas le paramètre `taille_cible` est ignoré.

Exemple

```
USE [GESCOM]
GO
DBCC SHRINKDATABASE(N'GESCOM', 10 )
GO
```

La taille finale doit être supérieure à la taille de la base **Model** plus la taille des données.



Avant de réaliser une opération de compactage, il est prudent de réaliser une sauvegarde complète de la base de données à compacter ainsi que de la base **Master**.

Les instructions `DBCC SHRINKDATABASE` et `DBCC SHRINKFILE` s'exécutent en mode différé, il est donc possible que la taille des fichiers ne diminue pas de façon instantanée.

Seule l'instruction `DBCC SHRINKFILE` permet de réduire la taille d'un fichier à une taille inférieure à celle précisée lors de la création du fichier. Evidemment, il n'est pas possible de descendre en dessous de la taille occupée par les données.

c. Configuration de la base de données

Il est possible de paramétrer de nombreuses options au niveau de la base de données. Ce paramétrage est possible soit avec l'instruction `ALTER DATABASE` en Transact SQL, soit depuis la fenêtre des propriétés de la base dans SQL Server Management Studio. Tous les

paramètres listés ci-dessous sont à fixer pour chaque base de données. Il n'est donc pas possible de fixer des options sur plusieurs bases de données en une seule commande, tandis qu'il est possible de préciser plusieurs options d'une base en une commande.

Si certains choix doivent être adoptés par toutes les bases utilisateur, il est préférable de changer les options de la base **Model**, ainsi toute nouvelle base utilisateur, fonctionnera avec les paramètres définis dans **Model**.

```
ALTER DATABASE nomBaseDeDonnees  
SET option [:]
```

Parmi toutes les options disponibles, il est possible d'en isoler quelques-unes :

Option	Descriptif
<code>AUTO_SHRINK {ON OFF}</code>	Si cette option est activée les fichiers sont réduits dès qu'ils disposent de plus de 25 % d'espace libre.
<code>READ_ONLY</code>	Permet de positionner la base de données en mode lecture seule.
<code>READ_WRITE</code>	La base de données est positionnée en mode lecture/ écriture.
<code>SINGLE_USER</code>	Seul un utilisateur peut travailler sur la base de données.
<code>RESTRICTED_USER</code>	Seuls les utilisateurs membres des rôles db_owner , db_creator et sysadmin peuvent se connecter à la base de données.
<code>MULTI_USER</code>	C'est le mode de fonctionnement standard d'une base, en autorisant plusieurs utilisateurs à travailler simultanément.
<code>AUTO_CREATE_STATISTICS { ON OFF }</code>	Lorsque cette option est positionnée à ON les statistiques manquantes lors de l'optimisation de la requête, sont calculées de façon automatique.
<code>AUTO_UPDATE_STATISTICS { ON OFF }</code>	Lorsque cette option est positionnée à ON, les statistiques obsolètes sont calculées de façon automatique.

L'utilisation de l'option `AUTO_SHRINK` peut conduire à de trop nombreuses opérations de modification de la taille des fichiers de données, aussi, si la base est administrée de près, il est préférable de laisser cette option désactivée.

Les options `AUTO_CREATE_STATISTICS` et `AUTO_UPDATE_STATISTICS`, actives

par défaut, sont très utiles car elles permettent de maintenir à jour les statistiques mesurant la pertinence des index. Lorsque l'optimiseur de requête dresse un plan d'exécution, il va utiliser les statistiques afin de déterminer si l'utilisation de l'index permet ou non de gagner du temps. Si les statistiques ne sont pas à jour, l'optimiseur risque de choisir un plan non optimum car les informations dont il dispose au départ sont mauvaises. Il est possible d'imager cela avec un GPS pour automobile. Lorsque l'on souhaite se rendre d'un point A à un point B, le GPS décide de l'itinéraire à suivre. La première génération de GPS met en place cet itinéraire sans tenir compte des difficultés possibles sur un axe donné. L'itinéraire indiqué, même s'il est valide, n'est pas forcément le plus rapide car il ne dispose pas des statistiques d'utilisation du réseau. Si maintenant une seconde génération de GPS connaît en temps presque réel les statistiques d'utilisation des routes, il va alors être en mesure de choisir l'itinéraire le plus court en temps pour arriver à destination. Bien entendu, cela a un coût car votre appareil passe du temps à recevoir, analyser et émettre ces informations. Au niveau SQL Server, c'est presque la même chose : le fait d'activer les options de création et de mise à jour des statistiques de façon automatique consomme des ressources sur le serveur mais permet d'autre part de dresser de meilleurs plans d'exécution.



Les options `AUTO_CREATE_STATISTICS` et `AUTO_UPDATE_STATISTICS` ne concernent pas les tables système ou bien à usage interne à SQL Server comme par exemple les index XML, les index de texte intégral, les files d'attentes Service Broker. Pour toutes ces tables les statistiques sont créées et mises à jour quelle que soit la valeur des options `AUTO_CREATE_STATISTICS` et `AUTO_UPDATE_STATISTICS`.

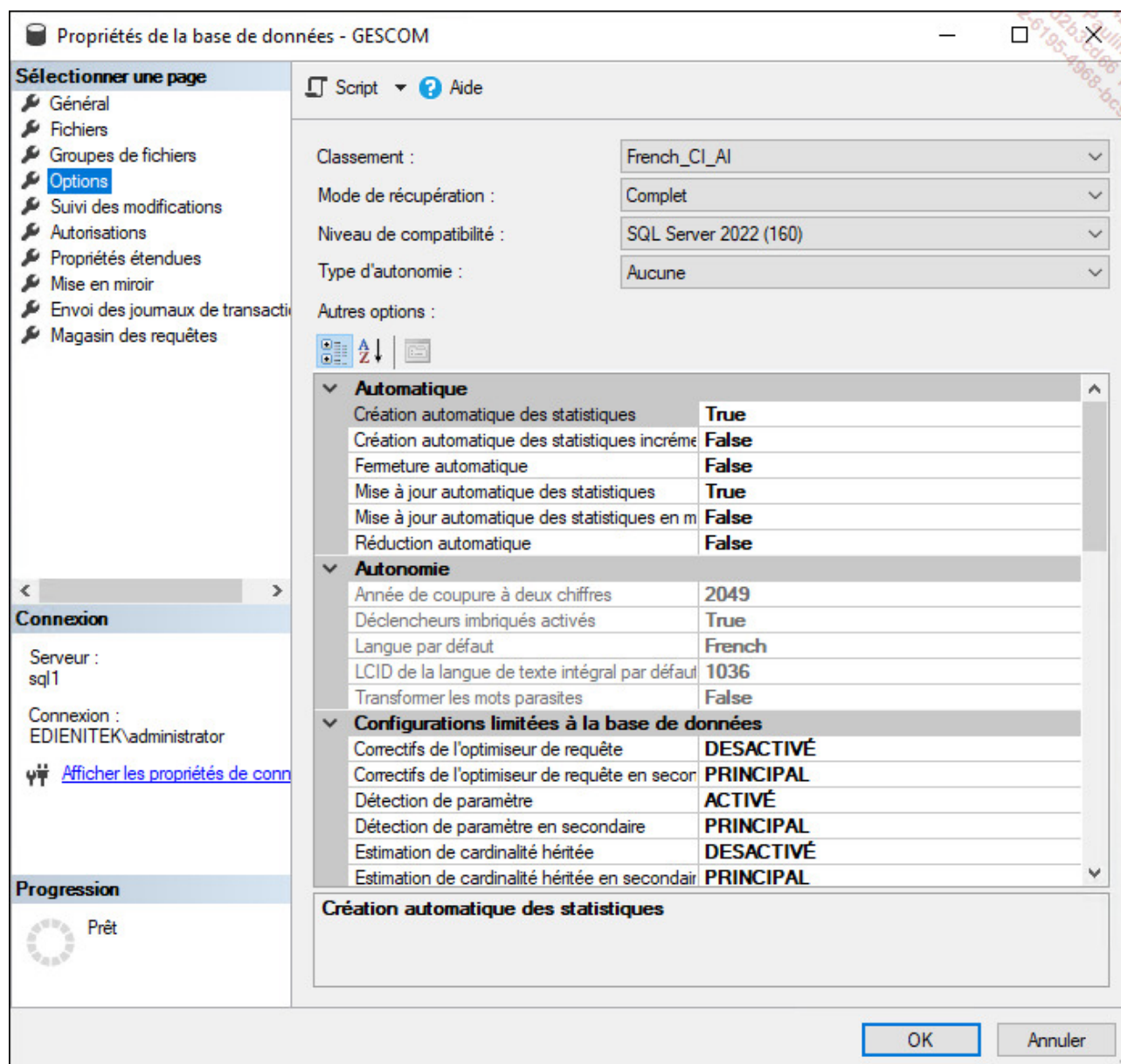


La fonction `DATABASEPROPERTYEX` permet de connaître la valeur actuelle de l'option qui lui est passée en paramètre.

Pour régler les paramètres de la base, il est possible de passer par :

[SQL Server Management Studio](#)

Pour connaître et éventuellement modifier les paramètres d'une base de données, il faut afficher la fenêtre présentant les propriétés de la base. Cette fenêtre est affichée en sélectionnant **Propriétés** depuis le menu contextuel associé à la base de données.



ALTER DATABASE

```
USE [master]
```

```
GO
```

```
ALTER DATABASE [GESCOM] SET RESTRICTED_USER WITH NO_WAIT
```

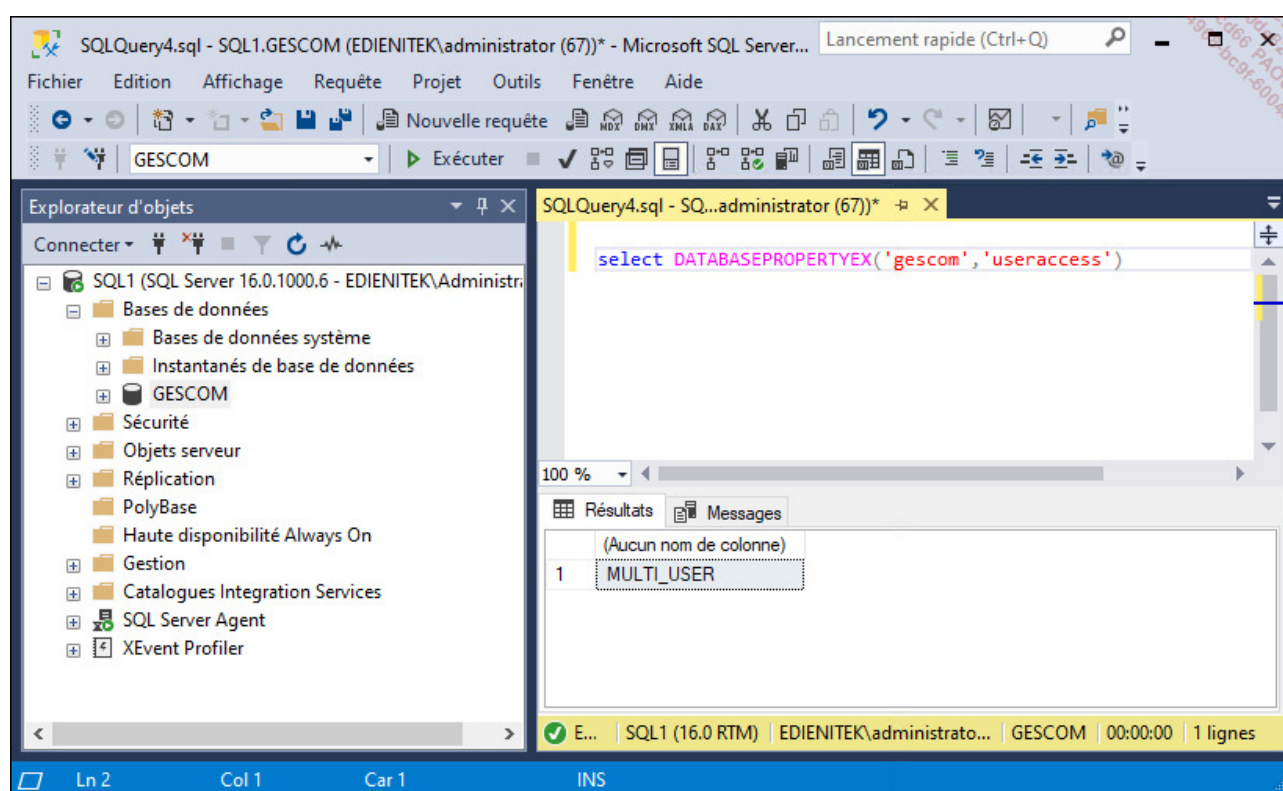
```
GO
```

Options actuellement gérées

Avant de fixer un nouveau paramétrage de la base de données, il est important de connaître le paramétrage actuel. La connaissance de ce paramétrage peut également aider à la compréhension du fonctionnement de la base. Il est possible de lire les valeurs des différentes options depuis SQL Server Management Studio, mais la connaissance du paramétrage de la base peut également être faite sous la forme de script Transact SQL.

Databasepropertyex

Pour connaître la valeur d'un paramètre.



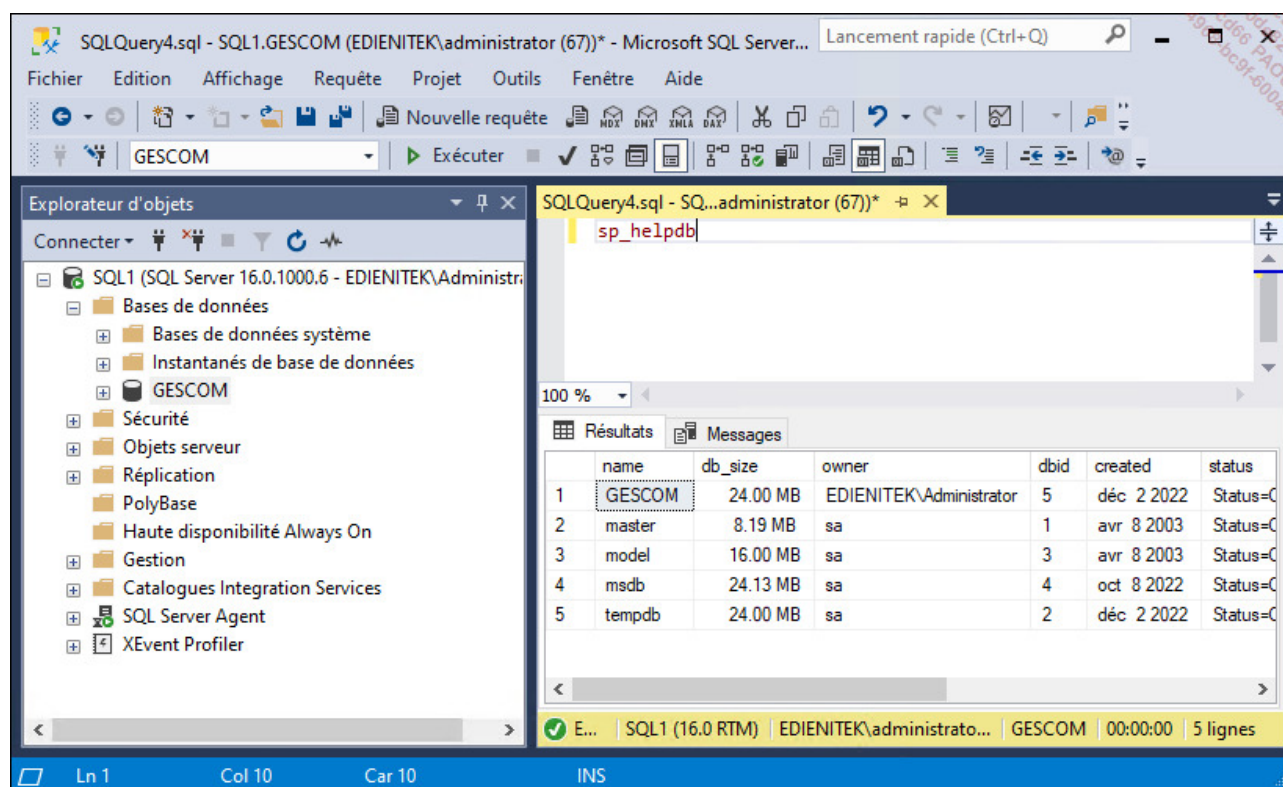
sp_helpdb

Si aucun paramètre n'est passé, cette procédure permet de connaître l'ensemble des bases qui existent sur le serveur. Les renseignements fournis sont le nom, la taille, le propriétaire, l'identificateur, la date de création et les options.

Si un nom de base est passé en paramètre, la procédure permet de connaître les

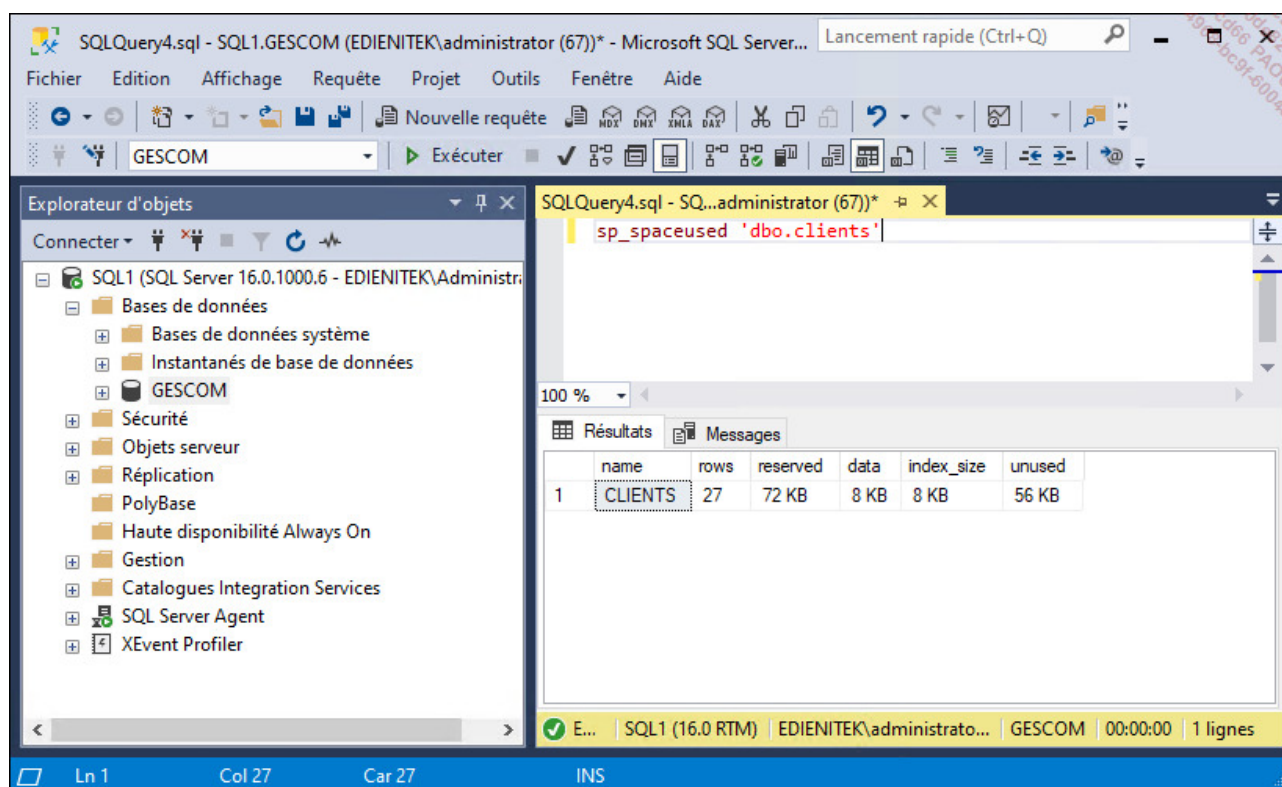
informations générales de la base ainsi que les nom et emplacement des fichiers de données et des fichiers journaux.

La procédure `sp_helpdb` utilise la table **sys.databases** pour établir la liste des bases et les différentes options de chacune d'elles.



sp_spaceused [nom_objet]

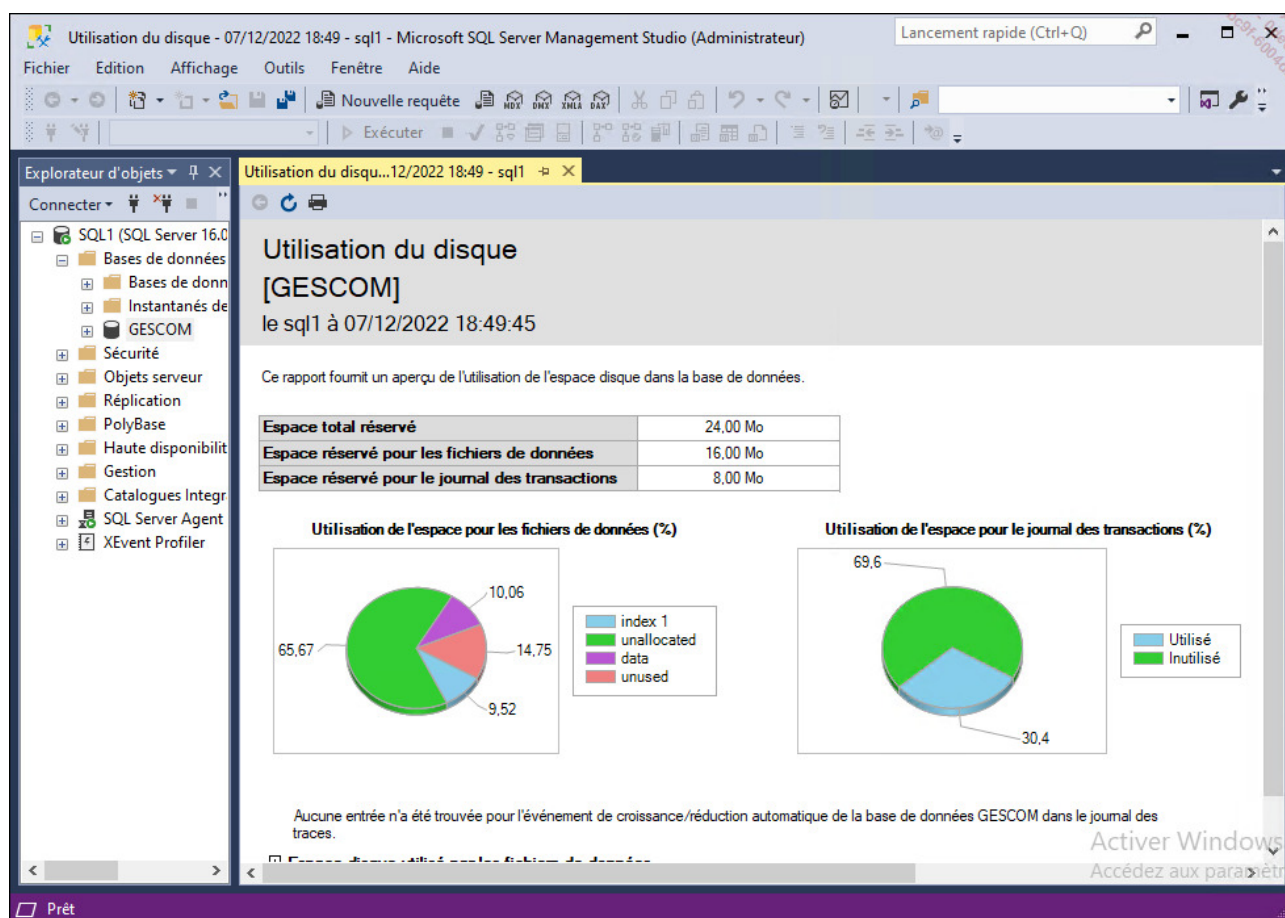
Permet de connaître l'espace de stockage utilisé par une base de données, un journal ou des objets de la base (table...).



Il est possible au niveau de SQL Server Management Studio d'obtenir un rapport sur l'utilisation de l'espace disque par les différentes tables de la base de données en cours. Pour cela, après s'être positionné sur la base de données cible, il faut sélectionner l'option **Rapports - Rapports standards**. À ce niveau, la liste des rapports prédéfinis est affichée. Ils sont nombreux mais seuls quelques-uns concernent l'utilisation de l'espace disque. Si la notion de rapport est disponible sur des nœuds plus précis comme **Tables** ou bien également au niveau de chaque table, les rapports prédéfinis ne sont disponibles qu'au niveau des bases de données et des services.

Exemple

Le rapport ci-dessous présente l'utilisation du disque par la base de données Gescom.



3. Supprimer une base de données

La suppression d'une base de données utilisateur est une opération ponctuelle qui peut être réalisée, comme la plupart des opérations d'administration soit par SQL Server Management Studio, soit en Transact SQL. La suppression de la base de données a pour conséquence de supprimer tous les fichiers qui correspondent à cette base ainsi que toutes les données contenues dans cette base. Cette opération est irréversible et en cas de mauvaise manipulation il est nécessaire de remonter une sauvegarde.



Après suppression d'une base de données, chaque connexion qui utilisait cette base par défaut se retrouve sans base par défaut.

Les limites

Il n'est bien sûr pas toujours possible de supprimer une base, les principales limites sont :

- Lorsqu'elle est en cours de restauration.
- Lorsqu'elle est ouverte par un utilisateur, en lecture ou en écriture.
- Lorsqu'elle participe à une publication dans le cadre de la réplication.



Il n'est pas possible de supprimer les bases de données système.

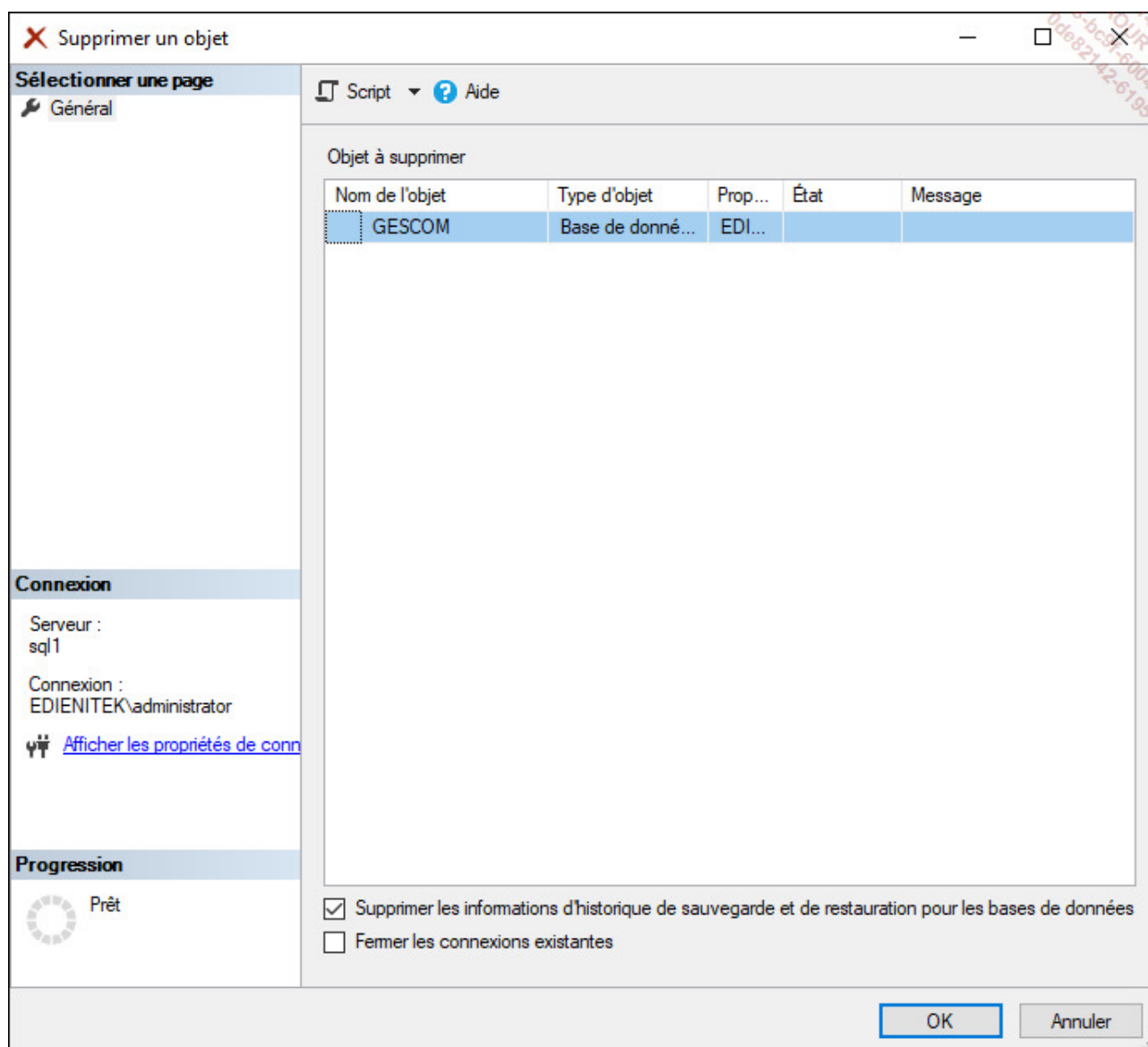
a. Transact SQL

C'est par l'intermédiaire de la commande `DROP DATABASE` que seront supprimées les bases de données.

```
use master;  
go  
drop database GESCOM;
```

b. SQL Server Management Studio

Par un simple clic droit sur la base de données, vous avez accès au menu **Supprimer** dans le menu contextuel. Par cette méthode, il n'est possible de supprimer les bases qu'une par une.



Dans la boîte de dialogue qui demande la confirmation de la suppression, la case à cocher **Fermer les connexions existantes** n'est pas activée. Aussi, si une personne est connectée à la base de données, alors la suppression n'est pas possible.

L'activation de cette option permet au contraire de forcer la suppression de la base de données même si des connexions sont actives lorsque la commande est exécutée.

Mise en place de groupes de fichiers

Il est possible de préciser les fichiers de données utilisés par la base, mais il est malheureusement impossible de préciser sur quel fichier est créé un objet particulier. Pour résoudre ce problème, il existe la possibilité de créer une table ou un index sur un ensemble de fichiers. Cet ensemble de fichiers, également appelé **groupe de fichiers**, est géré assez simplement.

Pourquoi est-il nécessaire de travailler avec plusieurs groupes de fichiers ? Parce qu'il est normal sur un système d'exploitation de séparer les fichiers système des programmes et des données utilisateur ; pourquoi en serait-il autrement dans une base de données ? Il convient donc de regrouper sur un même groupe de fichiers des données de même type. Par exemple, les données stables (comme les clients, les articles) peuvent être distinguées des données de type historique tout simplement parce que les volumes ne sont pas les mêmes, la façon de travailler avec non plus. Il peut être également intéressant de définir les index et les tables sur des groupes de fichiers distincts afin de réduire les temps de mise à jour des données et des index. Dans le cas de données sensibles, la répartition par groupe de fichiers peut également être conduite par la politique de sauvegarde adoptée.

1. Création d'un groupe de fichiers

Avant de pouvoir utiliser les groupes de fichiers, il faut les créer. Lors de la création de la base, le groupe de fichiers **PRIMARY** a été créé.

C'est par l'intermédiaire d'une commande **ALTER DATABASE** que l'on va créer un groupe de fichiers. Et cette commande va permettre d'ajouter des fichiers à l'intérieur du groupe.

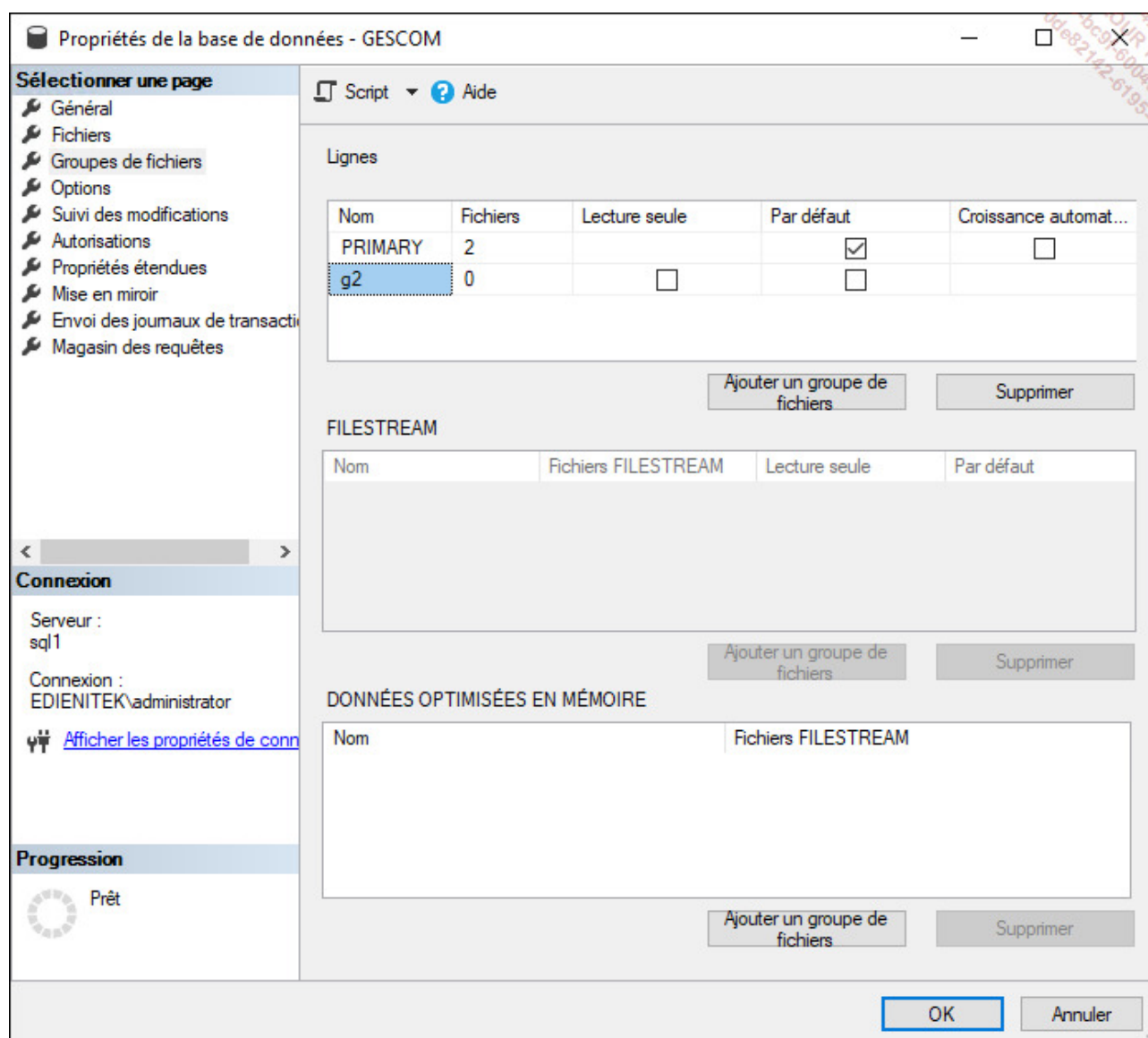
Syntaxe

```
ALTER DATABASE nom_base_données  
ADD FILEGROUP nom_groupe_fichiers[;]
```

Exemple

```
USE [master]
GO
ALTER DATABASE [GESCOM] ADD FILEGROUP [g2]
GO
```

Depuis SQL Server Management Studio, la gestion des groupes de fichiers est réalisée par l'intermédiaire de la fenêtre des propriétés de la base.



2. Ajout de fichiers

L'ajout de fichiers est réalisé par la commande `ALTER DATABASE`.

Un fichier ne peut appartenir qu'à un seul groupe de fichiers. Par défaut, si aucun groupe de fichiers n'est précisé lors de l'ajout du fichier à la base, il est ajouté au groupe `PRIMARY`.

Syntaxe

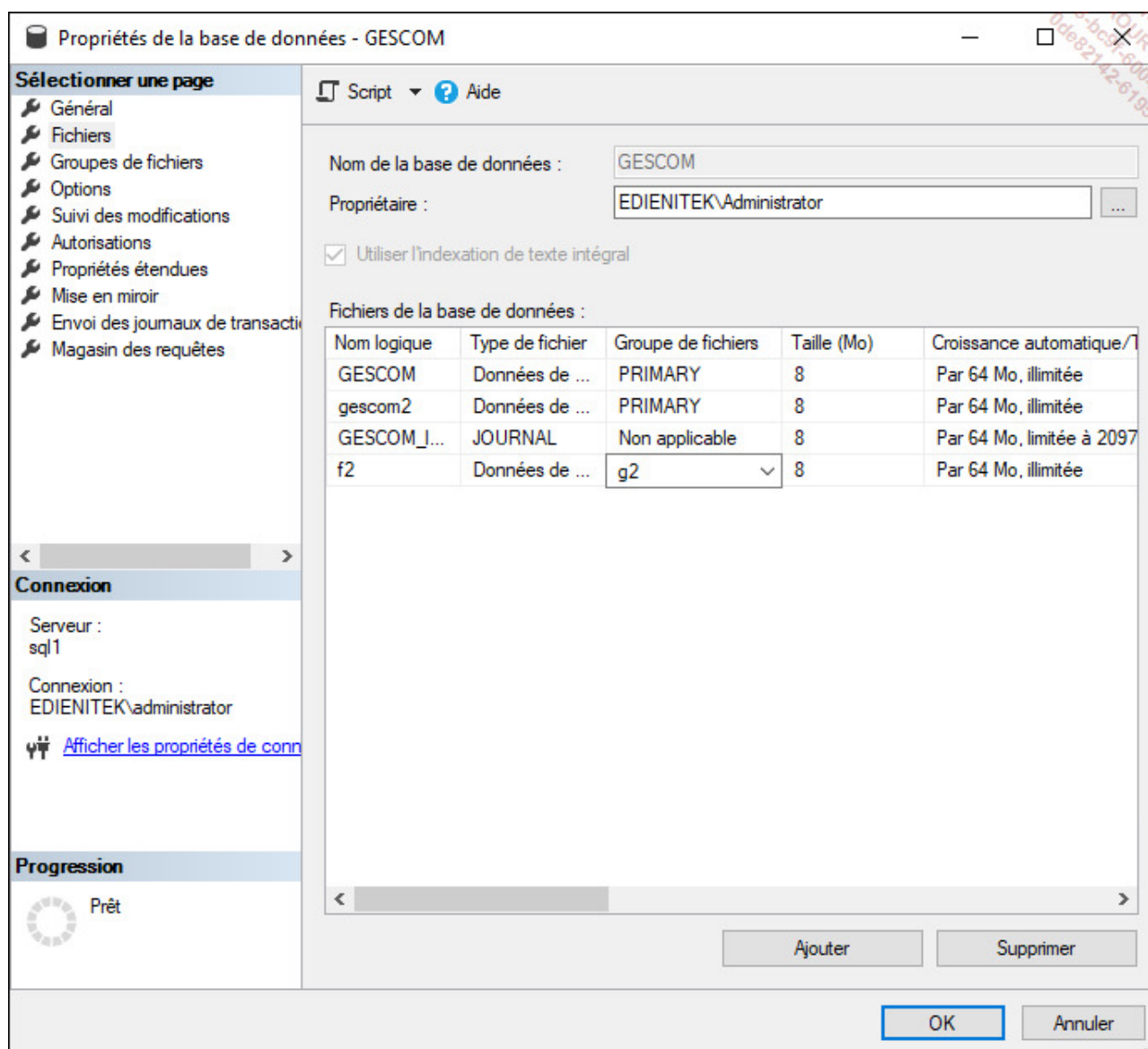
```
ALTER DATABASE nom_base_données  
ADD FILE spécification fichier  
TO FILEGROUP nom_groupe_fichiers
```

Exemple

```
USE [master]  
GO  
  
ALTER DATABASE [GESCOM]  
ADD FILE ( NAME = N'f2',  
FILENAME = N'd:\donnees\f2.ndf' ,  
SIZE = 8192KB , FILEGROWTH = 65536KB ) TO FILEGROUP [g2]  
GO
```

Les fichiers qui participent à un groupe de fichiers sont gérés de la même façon que ceux qui participent au groupe de fichiers `PRIMARY`.

Depuis SQL Server Management Studio, la gestion des fichiers est possible depuis la fenêtre des propriétés de la base.



3. Utilisation d'un groupe de fichiers

L'utilisation d'un groupe de fichiers par un objet doit être précisée dès la création de ce dernier. Seuls les objets contenant de l'information sont concernés, soit les tables et les index. L'instruction `ON nom_groupe_fichiers` est simplement ajoutée à la fin de la commande SQL et avant son exécution. Un fichier ne peut appartenir qu'à un et un seul groupe de fichiers.

```
CREATE TABLE [dbo].[CATEGORIES](  
    [code_cat] [int] IDENTITY(100,1) NOT NULL,  
    [libelle_cat] [nvarchar](200) NULL  
) ON [g2]  
GO
```



Par défaut, les objets sont créés sur le groupe **PRIMARY**.

La procédure **sp_help** permet de connaître les détails de la table et entre autres le groupe de fichiers associé.

Il est également possible de connaître le groupe de fichiers utilisé par une table en affichant la page **Stockage** depuis la boîte de dialogue présentant les propriétés d'une table.

Instructions INSERT, SELECT ... INTO

L'instruction `SELECT INTO` permet d'insérer les données résultat d'une commande `SELECT` dans une table qui sera elle-même créée par cette requête `SELECT`.



La table créée de la sorte ne dispose d'aucune contrainte d'intégrité.

```
select nom,prenom  
into cli  
from dbo.CLIENTS;
```

La commande `INSERT` est également capable d'insérer le résultat d'une commande `SELECT` dans une table qui a été créée auparavant à l'aide de l'instruction SQL DDL `CREATE TABLE`.

Cette fois-ci, il est tout à fait possible d'inclure la définition de contraintes d'intégrité lors de la création de la table.

```
insert into cli(nom,prenom)
select nom,prenom
from dbo.clients;
```

Structure des index

SQL Server propose deux types d'index :

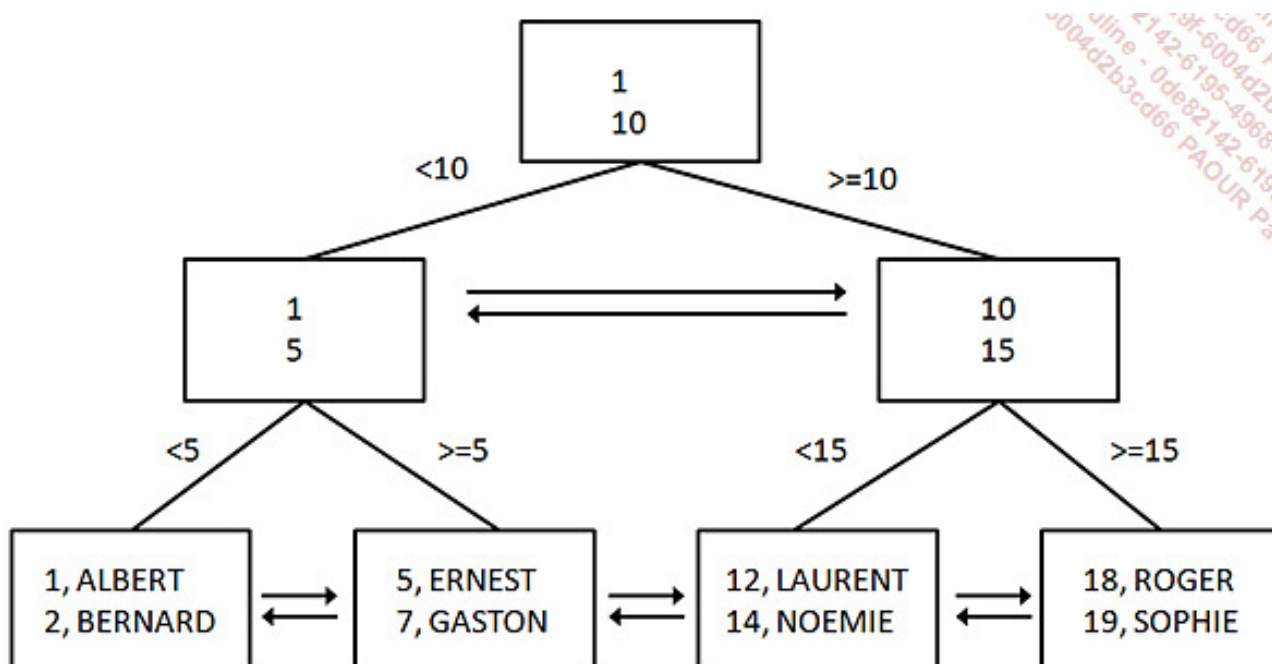
- Les index ordonnés ou clustered.
- Les index non ordonnés ou non clustered.

1. Les index ordonnés

Ces index sont constitués d'un arbre binaire (b-tree) dans lequel les pages de niveau feuille contiennent les données de la table sous-jacente. Lors de l'ajout d'une ligne d'information, cette ligne est insérée en fonction de la valeur de sa clé. Il ne peut donc y avoir qu'un seul index ordonné par table.

Étant donné que la clé de l'index organise physiquement la table, il est nécessaire de baser cet index sur une valeur stable et c'est pourquoi la clé primaire, qui de préférence doit être un entier compteur, est traditionnellement retenue.

Le schéma ci-dessous illustre de façon synthétique la structure d'un index ordonné. En plus des chaînages permettant de parcourir l'arbre de bas en haut (depuis la racine vers les feuilles), il existe un double chaînage permettant de parcourir toutes les pages d'un même niveau.



L'index ordonné est créé par défaut lorsqu'une contrainte de clé primaire est définie sur une table. Si l'administrateur souhaite que la définition de la contrainte ne s'accompagne pas de la création d'un tel index il lui est nécessaire de spécifier le mot-clé **NONCLUSTERED** lors de la définition de la contrainte.

Syntaxe

```

ALTER TABLE nomTable
ADD CONSTRAINT nomContrainte PRIMARY KEY
[CLUSTERED|NONCLUSTERED] (listeColonnes);
  
```

La seconde possibilité est de définir un index avec l'option **CLUSTERED**.

Syntaxe

```

CREATE [UNIQUE] [CLUSTERED|NONCLUSTERED] INDEX nomIndex
ON nomTable (listeColonnes)
[ON groupeDeFichiers];
  
```

2. Les index non ordonnés

L'autre type d'index qu'il est possible de définir au niveau de SQL Server concerne les index dits **NONCLUSTERED**, c'est-à-dire que la définition de ces index ne réorganise pas physiquement la table. De ce fait, il est possible de définir plusieurs index de ce type sur une même table.



Il n'est pas possible de définir plus de 999 index non ordonnés sur une même table.

Si la création d'un index ordonné (**CLUSTERED**) est planifiée pour une table, il est souhaitable que cette définition intervienne avant la définition des index non ordonnés (**NONCLUSTERED**).

Comme pour les index ordonnés, ces index peuvent contenir une ou plusieurs colonnes, avec soit un tri ascendant (par défaut), soit descendant pour chaque colonne. L'ordre de tri est spécifié à l'aide des caractères **ASC** pour ascendant ou bien **DESC** pour descendant derrière le nom de chaque colonne.

Contrairement à l'index ordonné qui doit être posé sur des données relativement stables, l'index non ordonné peut être défini sans prendre en compte la stabilité de valeurs indexées. C'est plutôt l'utilisation qui est faite des données qui va permettre de définir ces index. Les opérations de tri et de jointure peuvent être très nettement accélérées grâce à la définition de l'index.

Si un index ordonné est défini sur une table qui possède des index non ordonnés, alors les index non ordonnés sont reconstruits.



Les critères permettant de définir ou non les index peuvent être définis au travers de l'assistant paramétrage de base de données. L'utilisation de cet outil est détaillée au chapitre sur l'optimisation.

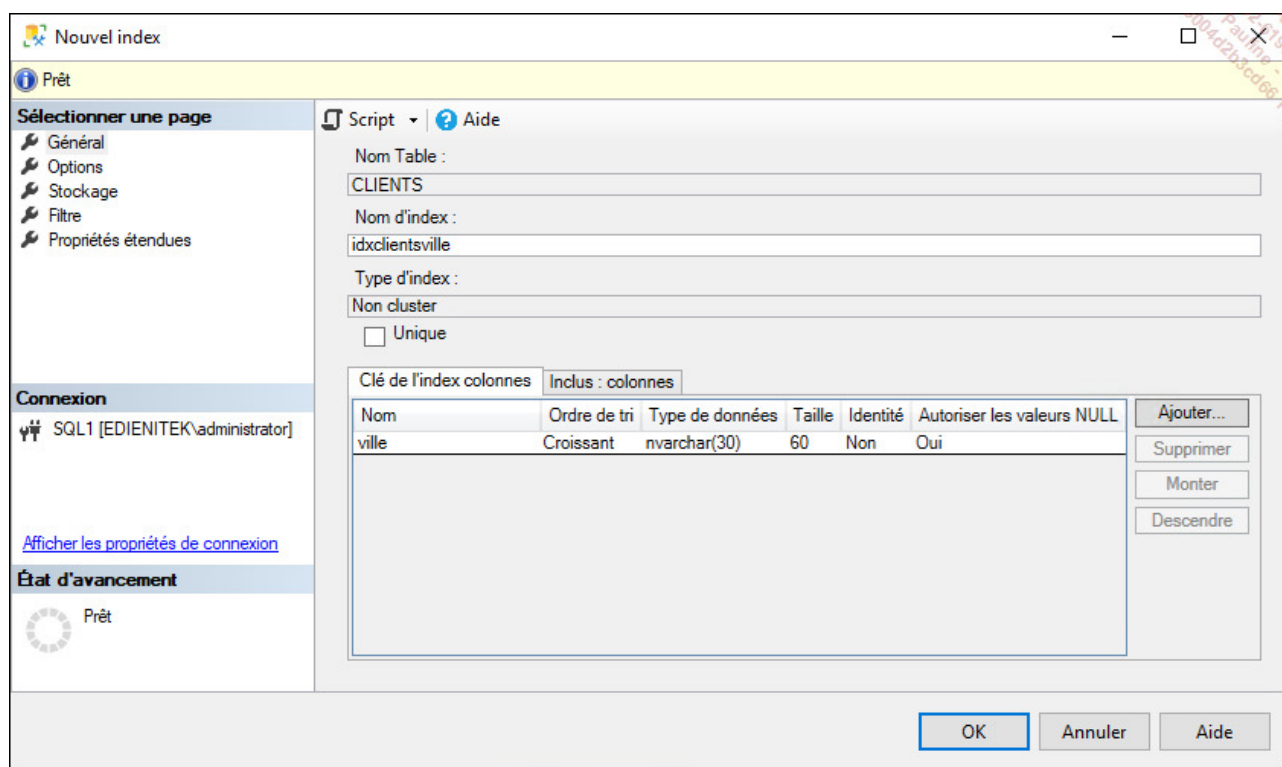
Exemple

L'exemple suivant illustre la définition d'un index sur la colonne ville de la table des clients.

```
create index idxclientsville on dbo.clients(ville);
```

Il est également possible de définir ce type d'index directement depuis SQL Server Management Studio.

Il faut sélectionner le menu **Nouvel index - Index non cluster** depuis le menu contextuel associé au nœud index dans le descriptif de la table fait par l'explorateur d'objets.



3. Les index couvrants

Il s'agit cette fois d'une particularité de SQL Server, qui consiste à définir des index qui vont contenir au niveau feuille la clé de l'index ainsi que les valeurs issues d'une ou de plusieurs colonnes. L'objectif de ces index est de permettre au moteur SQL Server de ne parcourir que l'index, sans qu'il soit nécessaire d'accéder à la table pour répondre aux besoins de

données d'une requête.

En termes de volume de données manipulées, le gain peut être conséquent. Toutefois, il est à pondérer par rapport au volume disque occupé mais aussi par le temps supplémentaire nécessaire pour mener à bien les actions de mise à jour (**INSERT**, **UPDATE** et **DELETE**).

De tels index sont définis à l'aide de l'instruction **CREATE INDEX** suivie du mot-clé **INCLUDE** afin de préciser la ou les colonnes à inclure au niveau feuille de l'arbre d'index.

Syntaxe

```
CREATE [UNIQUE] [CLUSTERED|NONCLUSTERED] INDEX nomIndex
ON nomTable (listeColonnes)
INCLUDE (listeColonnes)
[ON groupeDeFichiers];
```

Exemple

Dans l'exemple suivant, un index est défini sur le prénom du client et le nom est inclus au niveau feuille.

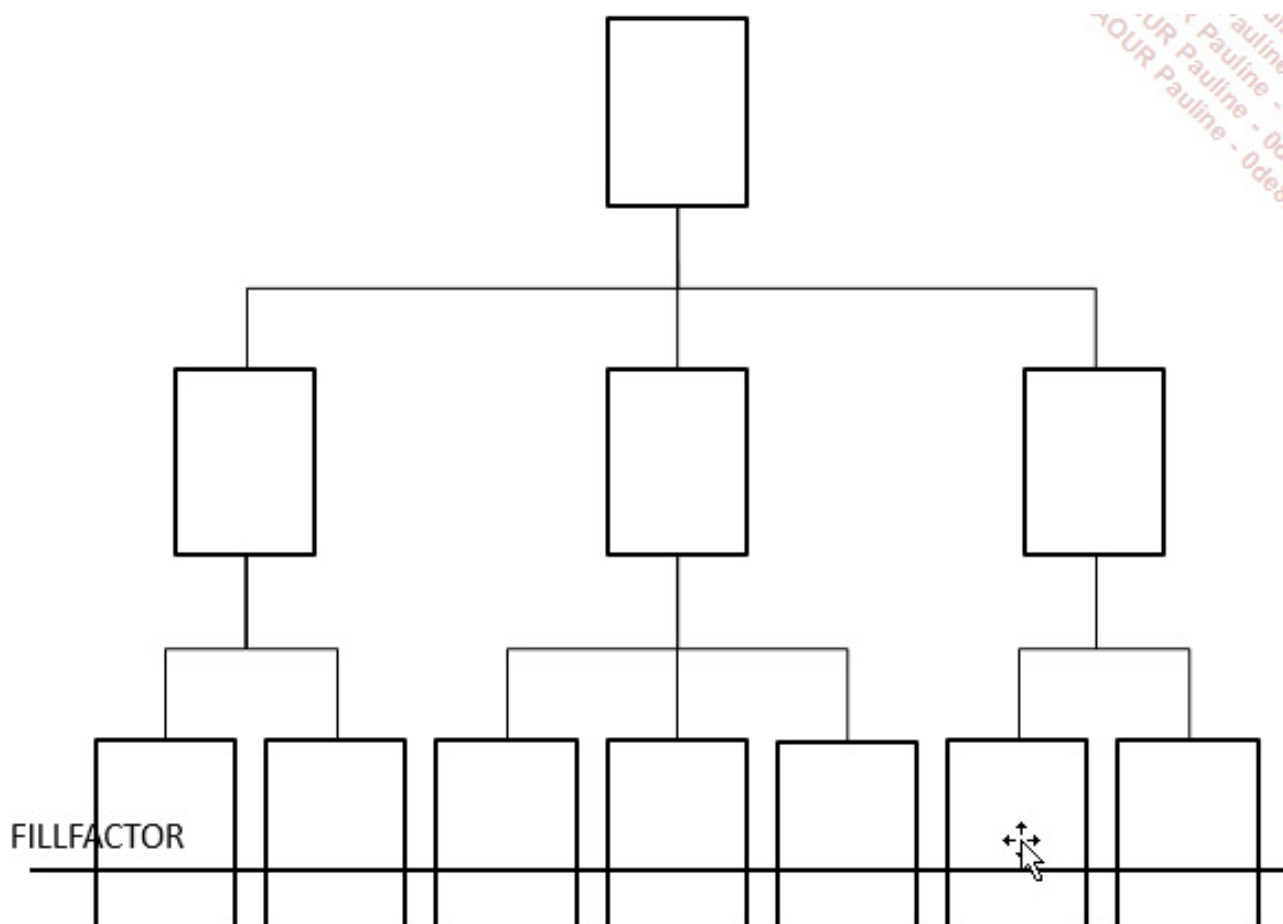
```
create index idxclientsprenomnom
on dbo.clients(prenom)
include (nom);
```

4. Fixer le paramètre FILLFACTOR

Lors de la construction ou de la reconstruction des index, le paramètre **FILLFACTOR** est présent. Ce paramètre permet de définir le taux d'occupation des pages lors de la mise en place de l'index. Le paramètre **FILLFACTOR** (également présent sous la dénomination « taux d'occupation » dans SQL Server Management Studio) ne s'applique qu'aux pages de niveau feuille. Les autres pages constitutives de l'index ne sont pas concernées par ce taux de remplissage. Le taux de remplissage est exprimé sous forme de pourcentage. Par

exemple, une valeur de 70 passée au paramètre **FILLFACTOR** signifie que la page est remplie à 70 % et donc qu'il reste 30 % d'espace libre. Cet espace libre va être utilisé pour l'ajout de données dans la table, donc dans l'index, sans avoir besoin de déplacer physiquement des données. La durée d'exécution des requêtes de mise à jour s'en trouve réduite. La valeur du paramètre **FILLFACTOR** doit être choisie en fonction du taux de mise à jour.

Le schéma ci-dessous illustre la structure d'un index et le positionnement de **FILLFACTOR** au niveau feuille.



Une valeur forte de **FILLFACTOR** signifie que les pages de niveau feuille vont contenir de nombreuses informations ; l'index va donc être plus compact car moins de pages seront nécessaires et l'utilisation de l'index sera d'autant plus rapide. Par contre, l'ajout d'informations dans la table indexée va rapidement déséquilibrer l'index (plus exactement, la structure de B-arbre de l'index) et il sera donc nécessaire de le reconstruire.

Une valeur faible de **FILLFACTOR** va avoir pour conséquence d'accroître le nombre de

pages qui composent l'index et l'utilisation de l'index sera donc moins performante car elle nécessitera de manipuler un nombre plus élevé de pages. Par contre, ce type d'index sera résistant face aux ajouts/modifications de données et le risque de déséquilibre est très faible.

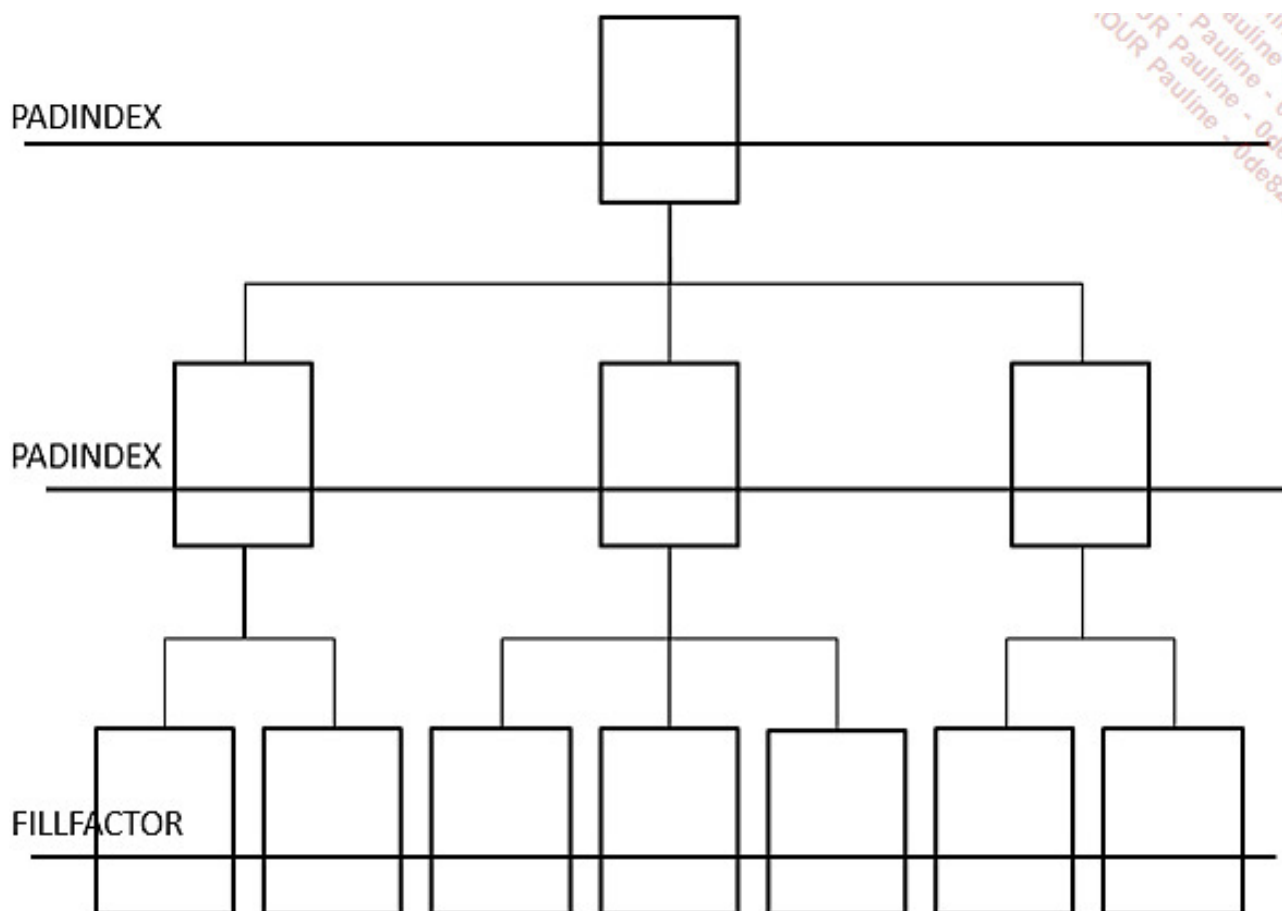
Enfin, pour apporter plus de paramétrage au niveau des index, le taux d'occupation défini à l'aide du paramètre `FILLFACTOR` peut être appliqué sur les pages de niveau feuille par l'intermédiaire du paramètre `PADINDEX`.

Ce paramètre `FILLFACTOR` peut être utilisé lors de la reconstruction d'index à l'aide de l'instruction SQL `ALTER INDEX`.



L'instruction `DBCC DBREINDEX` est maintenue pour des raisons de compatibilité, mais il ne faut plus l'utiliser car elle va être supprimée dans les prochaines versions de SQL Server.

Le schéma ci-dessous illustre la structure d'un index et le positionnement de `FILLFACTOR` au niveau feuille et de `PADINDEX` sur les autres pages de l'index.❖



Syntaxe

```
ALTER INDEX { nom_index | ALL } ON nom_table
REBUILD WITH ([PAD_INDEX=ON,]FILLFACTOR=valeur);
```

Exemple

L'exemple ci-dessous illustre la reconstruction de tous les index sur la table des clients en appliquant une valeur de 40 % au paramètre `FILLFACTOR`.

```
alter index all on dbo.clients rebuild with (fillfactor=40);
```

5. Indexer des colonnes calculées

Toujours dans l'objectif de répondre rapidement aux utilisateurs, il est possible de définir des colonnes calculées dans une table. Toutefois, pour pouvoir être défini, le calcul devra être élémentaire, c'est-à-dire portant sur chaque ligne de données et non pas issu d'un regroupement. Les données doivent toutes provenir de la même table et la fonction de calcul doit être déterministe.

Dans ce cas, il est possible de définir un index sur ces colonnes calculées.

Exemple

```
alter table dbo.articles
add ttc as (prixht_art*1.2) persisted;
go
create index idxarticlesttc
on dbo.articles(ttc);
```

6. Indexer les vues

Les vues sont fréquemment utilisées dans les requêtes d'extraction car elles permettent, entre autres, de simplifier l'écriture des requêtes. Pour améliorer les performances des requêtes qui utilisent les vues, il est possible de définir un ou plusieurs index sur les vues.

Le point de départ consiste à définir un index ordonné (**CLUSTERED**) unique afin de matérialiser la vue. Par la suite des index non ordonnés peuvent être définis sur la vue. Même les colonnes présentant le résultat d'un calcul peuvent être indexées.

La définition de la vue ne doit cependant pas contenir de fonction non déterministe. Le mot-clé **SCHEMABINDING** doit également faire partie de sa définition.

La syntaxe de définition d'un index sur une table ou bien sur une vue est identique.

Exemple

```
create view dbo.clientsdeParis
with schemabinding as
```



```

select numero,nom,prenom,adresse,codepostal,ville
from dbo.CLIENTS
where ville='Paris' ;
go
create unique clustered index idxclientsdeparisadresse
on dbo.clientsdeparis(nom,prenom,adresse) ;

```

7. Les index filtrés

Les index filtrés ne sont pas un nouveau type d'index, mais plutôt une solution fournie par SQL Server pour permettre de définir des index tout en limitant l'espace occupé sur le disque dur par ces index et donc les temps de mise à jour de l'index. Lors de la définition de l'index filtré, une clause **WHERE** est ajoutée à la définition de l'index afin que l'index ne porte pas sur toutes les lignes de la table mais simplement celles qui satisfont au(x) critère(s) de sélection. Ainsi, il est possible de définir des index uniquement dans certains cas où les regroupements sont nombreux.

Exemple

Pour la table des clients, un index sur les noms et prénoms est défini uniquement pour les clients qui habitent Paris.

```

create index idxclientsparis
on dbo.clients(nom,prenom)
where ville='Paris';

```

8. Les index ColumnStore

Les index vus précédemment sont également appelés index rowbased ou rowstore. C'est-à-dire que leur stockage est basé sur les lignes (ou enregistrements) des tables. Un nouveau type d'index est apparu avec SQL Server 2012 : les index ColumnStore, avec un stockage basé sur les colonnes : une page de 8 ko ne contient que des données d'une

seule colonne.

Ce type d'index a été créé pour optimiser les bases de données servant d'entrepôt de données (datawarehouse), car leur utilisation est bien différente des autres bases, mais à leur apparition, ces index étaient uniquement en lecture seule. À partir de SQL Server 2016, ils existent en lecture/écriture et peuvent être associés à des index rowbased.

Contrairement aux index rowbased, les index columnstore ne possèdent pas de clé d'index, on leur associe juste une liste des colonnes de la table qu'ils doivent stocker.

Par analogie avec les index rowbased, il existe deux types d'index columnstore : **CLUSTERED** et **NONCLUSTERED**. Les index **CLUSTERED COLUMNSTORE** contiennent donc également toute la table (c'est-à-dire toutes les colonnes) alors que les index **NONCLUSTERED COLUMNSTORE** ne contiennent qu'une partie des colonnes de la table qu'il faudra indiquer à la création de l'index.

Une table ne pourra avoir qu'un seul index **CLUSTERED**, qu'il soit rowbased ou columnstore, et pourra en plus avoir plusieurs index **NONCLUSTERED** (rowbased ou columnstore).

Ces index peuvent être intéressants dans le cas de requêtes portant sur peu de colonnes d'une table et beaucoup de lignes.

Exemple

Création d'un index **NONCLUSTERED COLUMNSTORE** pour la table des clients et les colonnes code postal et ville :

```
CREATE COLUMNSTORE INDEX idx_col_clients ON
dbo.clients(codepostal,ville)
```

9. Les index XML

Comme pour les autres colonnes, il est possible d'indexer les colonnes de type XML. Cependant, l'indexation des données XML est particulière en fonction de la structure même des données. Lors du traitement d'une requête, les informations XML sont

analysées au niveau de chaque ligne, ce qui peut entraîner des traitements longs et coûteux lorsque le nombre de lignes est important et/ou quand les informations au format XML sont nombreuses.

Le mécanisme habituel d'indexation qui repose sur un arbre balancé va être utilisé pour définir l'index dit principal sur la colonne de type XML. Mais les index qui vont effectivement permettre d'accélérer le traitement des requêtes sont les index qui vont reposer sur cet index principal. Ces index secondaires sont définis par rapport aux types de requêtes fréquemment exécutées :

- Index **PATH** pour des requêtes portant sur le chemin d'accès.
- Index **PROPERTY** pour des requêtes portant sur les propriétés.
- Index **VALUE** pour des requêtes portant sur des valeurs.



Il est également possible de définir un index de type texte intégral sur les colonnes de type XML.

a. Index principal

L'index principal représente le point de départ à toute indexation de la colonne de type XML. Il ne peut être défini que sur une table qui possède une contrainte de clé primaire associée à un index qui organise physiquement la table, ce qui est fréquemment le cas.

Syntaxe

```
CREATE PRIMARY XML INDEX nomIndex ON table(colonneXML)[;]
```

Exemple

Un index principal est défini sur la colonne page de la table catalogue.

```
create table dbo.catalogue
(numero int constraint PK_catalogue Primary Key,
```

page XML);

b. Index secondaire

La définition d'un index secondaire n'est possible que si et seulement si un index primaire est déjà défini sur la colonne.

Le document XML ne peut contenir que 128 niveaux au maximum. Les documents qui possèdent une hiérarchie plus complexe sont rejetés lors de l'insertion ou de la modification des colonnes.

L'indexation, quant à elle, porte sur les 128 premiers octets du nœud. Les valeurs plus longues ne sont pas prises en compte dans l'index.

Syntaxe

```
CREATE XML INDEX nomIndex ON table(colonneXML)
USING XML INDEX nomIndexXMLPrincipal
FOR {PATH|PROPERTY|VALUE}[,]
```

PATH

Permet de construire un index sur les colonnes **path** et **value** (chemin et valeur) de l'index XML principal. Un tel index peut participer à améliorer sensiblement les temps de réponse lors de l'utilisation de la méthode **exist()**, dans une clause where par exemple.

PROPERTY

Permet de construire un index sur les colonnes PK, path et value de l'index XML principal. Le symbole PK correspond à la clé primaire de la table. Ce type d'index est donc utile lors de l'utilisation de la méthode **value()** dans les requêtes SQL de manipulation de données.

VALUE

Permet de construire un index sur les colonnes `value` et `path` de l'index XML principal. Ce type d'index est utilisé dans les requêtes pour lesquelles la valeur du nœud est connue indépendamment du chemin d'accès, ce qui peut être le cas lors de l'utilisation de la méthode `exist()` par exemple.

Exemple

Dans l'exemple présenté ci-après, les trois types d'index sont créés par rapport à l'index XML principal défini sur la colonne `page` de type XML de la table `Catalogue`.

```
create primary xml index pidx_page on dbo.catalogue(page);
go
create xml index idx_page_path on dbo.catalogue(page)
    using xml index pidx_page for path;
go
create xml index idx_page_property on dbo.catalogue(page)
    using xml index pidx_page for property;
go
create xml index idx_page_value on dbo.catalogue(page)
    using xml index pidx_page for value;
go
```

10. Les index spatiaux

SQL Server permet de stocker des données spatiales de type `geometry` ou `geography`. Comme pour toutes les informations conservées par SQL Server, le moteur se doit de fournir un accès rapide aux informations. Dans cet objectif les index jouent un rôle crucial. Il est donc tout à fait logique que SQL Server propose d'indexer les colonnes de type spatial.

La structure de ces index diffère des index classiques. Pour ce type d'index, SQL Server définit une structure compatible avec les données spatiales en définissant un maillage sur quatre niveaux afin d'accéder rapidement à la cellule souhaitée. Chaque niveau correspond à un maillage défini sur 16, 64 ou 256 cellules. Chaque cellule d'un niveau est détaillée par une grille de niveau inférieur. Par exemple, si le choix est fait de travailler avec une grille de 16 cellules au niveau 1 alors le niveau 4 (le plus détaillé) comportera 65 536 cellules. Le

nombre de cellules définies dans les grilles des différents niveaux représente la densité de l'index. Cette densité peut prendre les valeurs **LOW** (16 cellules), **MEDIUM** (64 cellules) ou bien **HIGH** (256 cellules).

Les zones géographiques contenues dans la table sont ainsi indexées et le parcours des différents niveaux de l'index permet de localiser rapidement la ou les cellules qui contiennent la zone cible de la recherche.

Afin de limiter l'étendue des grilles d'index, lors de la définition de l'index, la zone géographique à prendre en compte pour l'indexation est définie à l'aide de l'option **BOUNDING BOX**.

Syntaxe

```
CREATE SPATIAL INDEX nomIndex
ON nomTable(nomColonne) WITH (
  BOUNDING_BOX=(xMin, yMin, xMax, yMAX),
  GRIDS(LEVEL_1=densité1, LEVEL_2=densité2, LEVEL_3=densité3,
  LEVEL_4=densité4)
);
```

ou

```
CREATE SPATIAL INDEX nomIndex
ON nomTable(nomColonne) USING
{GEOMETRY_GRID|GEOGRAPHY_GRID}
WITH (
  BOUNDING_BOX=(xMin, yMin, xMax, yMAX),
  GRIDS(densité));
```

L'option **USING** permet de spécifier lors de la création de l'index si les données indexées sont de type géométrique ou bien géographique.

Comme précisé précédemment, la densité peut prendre les valeurs **LOW**, **MEDIUM** ou **HIGH**. Si le niveau de densité n'est pas spécifié lors de la création de l'index alors un index est défini avec une densité de type **MEDIUM** par défaut.

Exemple

```
alter table dbo.stocks  
add position geography;  
go  
create spatial index idxstockposition  
on dbo.stocks(position)  
using geography_grid;
```



Les index de types spatiaux peuvent être définis sur un groupe de fichiers différents de celui qui contient la table indexée.

Partitionnement des tables et des index

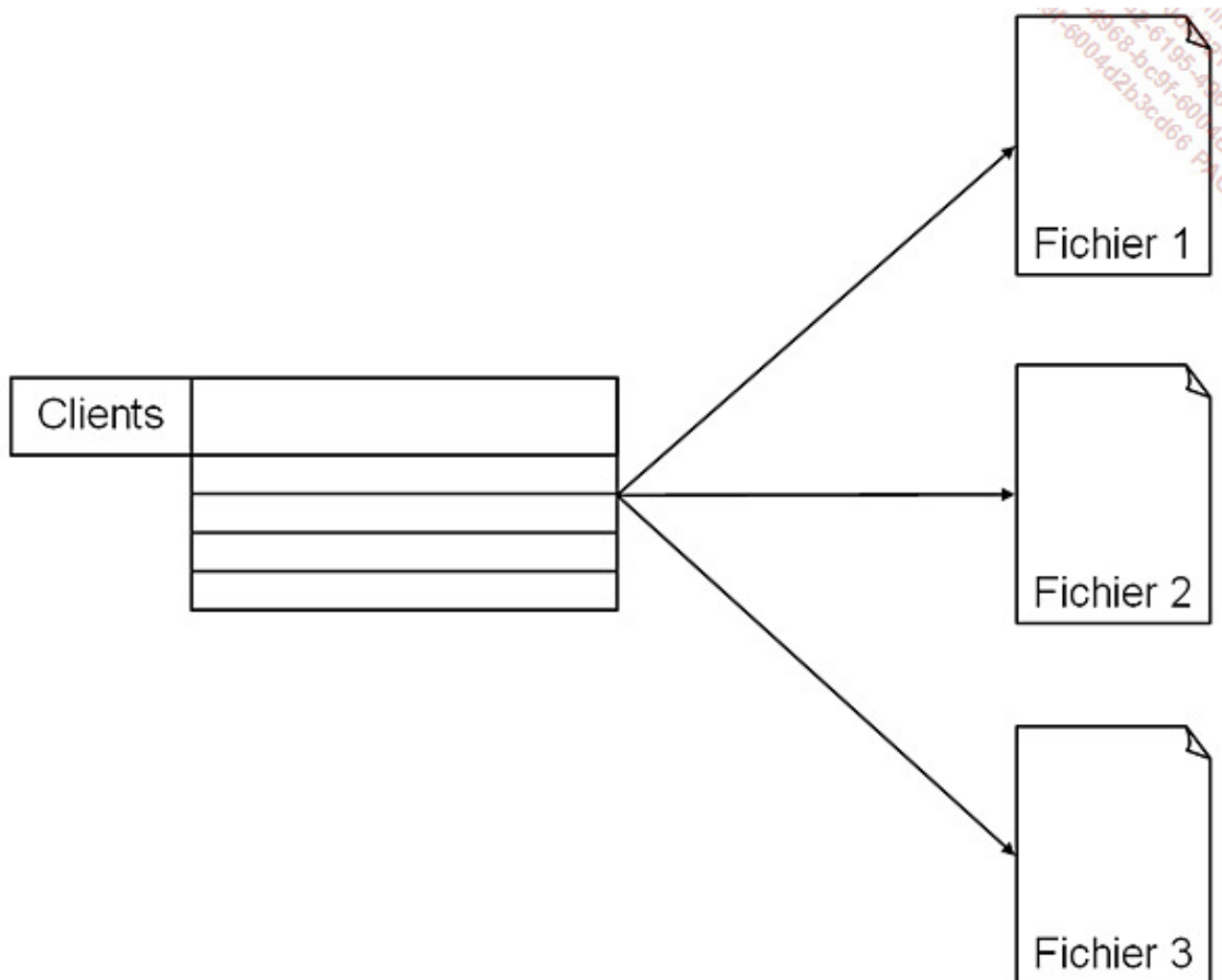
L'objectif du partitionnement est d'offrir une meilleure montée en charge sur les tables très volumineuses en termes de données et accédées par de nombreux utilisateurs.

Le partitionnement peut être utile pour diviser la méthode d'accès aux données. Par exemple, sur la table des commandes, il n'est possible de modifier (update) que les commandes de l'année comptable en cours. Les commandes des années précédentes doivent être en lecture seule.

Le partitionnement d'une table permet de stocker une table de grande dimension sur plusieurs fichiers/ groupes de fichiers. Chacune de ces partitions est plus petite que la table initiale et donc plus facile à gérer pour SQL Server.

Le partitionnement se fait selon un critère lié aux colonnes de la table ; par exemple, sur la date de commande avec le 1^{er} janvier de l'année en cours pour valeur de délimiteur.

La table partitionnée permet d'optimiser le stockage des informations, sans que le nombre de tâches administratives supplémentaires soit important. En effet, des éléments tels les contraintes d'intégrités, les déclencheurs sont définis au niveau de la table sans tenir compte de l'espace de stockage physique utilisé.



Pour partitionner la table, il est donc nécessaire de définir une fonction et un schéma de partitionnement qui vont indiquer une clé de partitionnement qui va être utilisée pour sélectionner la partition, donc le groupe de fichiers où vont être stockées les données.

Si deux tables utilisent la même fonction de partitionnement et le même schéma de partitionnement, alors les données en relations seront stockées sur le même groupe de fichiers. Dans un tel cas de figure, les données sont dites alignées.

Il est possible de créer des index sur une table partitionnée. L'index créé est alors partitionné. Comme sur toute table, il est possible de définir un index ordonné (**CLUSTERED**). Si le choix de définir un tel type d'index est fait, alors l'organisation physique des données doit être en relation avec les éléments passés à la fonction de partitionnement. C'est-à-dire que les colonnes passées en paramètre à la fonction de partitionnement doivent être présentes dans la clé de l'index. Ainsi, l'organisation physique de la table suit la répartition des données entre les différents groupes de fichiers.

1. La fonction de partitionnement

C'est la fonction de partitionnement qui va permettre d'orienter les données sur un groupe de fichiers ou bien un autre. Pour répartir les données entre les différentes partitions, la fonction utilise des plages de valeurs. Chaque plage est bornée par des valeurs. Seules les valeurs frontières sont indiquées dans la fonction de partitionnement. Dans le cas où la valeur frontière doit être incluse dans la plage de valeurs inférieure, il faut utiliser le mot-clé `LEFT`. Le mot-clé `RIGHT` permet d'inclure la valeur frontière dans l'espace suivant.



SQL trie toujours les données de façon ascendante.

Syntaxe

```
CREATE PARTITION FUNCTION nomfonction ( parametre)  
AS RANGE [ LEFT | RIGHT ] FOR VALUES ( [ valeurLimite [ .... ] ] )
```

`parametre`

Colonne de tous types sauf timestamp, varchar(max), nvarchar(max) et varbinary utilisé pour calculer la clé de partitionnement.

`valeurLimite`

Valeur marquant la frontière de chaque partition.

Exemple

Dans l'exemple suivant, une fonction de partitionnement est définie sur une valeur de type entier. Cette fonction définit quatre partitions différentes :

- Première partition pour les valeurs inférieures ou égale à 10 000.
- Deuxième partition pour les valeurs strictement supérieures à 10 000 et inférieures ou

égale à 20 000.

- Troisième partition pour les valeurs strictement supérieures à 20 000 et inférieures ou égale à 30 000.
- Quatrième partition pour les valeurs strictement supérieures à 30 000.

```
create partition function pfclients(int)
as range left for values(10000,20000,30000);
```

2. Le schéma de partitionnement

Le schéma de partitionnement va permettre d'établir une table de correspondance entre les valeurs retournées par la fonction de partitionnement et l'utilisation d'une partition ou bien d'une autre. Chaque partition va être affectée à un groupe de fichiers. Il est possible, bien que non recommandé, d'utiliser le même groupe de fichiers pour toutes les partitions.

Il est également possible de préciser plus de groupes de fichiers que ne réclame la fonction de partitionnement. Dans ce cas, le premier groupe de fichiers supplémentaire sera marqué **NEXT USED**, c'est-à-dire comme le prochain groupe de fichiers à utiliser si la fonction de partitionnement définit une nouvelle partition.

Syntaxe

```
CREATE PARTITION SCHEME nomSchemaPartition
AS PARTITION nomFonctionPartition
[ ALL ] TO ( { groupeDeFichiers | [ PRIMARY ] } [...])
[;]
```

nomSchemaPartition

Identifiant du schéma de partitionnement.

nomFonctionPartition

Nom de la fonction de partitionnement relative au schéma. Un schéma ne peut faire relation qu'à une et une seule fonction. Par contre, une même fonction peut être utilisée par plusieurs schémas.

`groupeDeFichiers`

Nom du ou des groupes de fichiers utilisés par les différentes partitions.

Exemple

Dans l'exemple ci-dessous, un schéma de partitionnement est défini par rapport à la fonction de partitionnement définie lors de l'exemple précédent.

```
create partition scheme schemaclients
as partition pfclients
to (g1,g2,g3,g4);
```

3. La table partitionnée

La répartition des données entre les différentes partitions de la table va être une opération transparente pour les utilisateurs de la table. Par contre, lors de la création de la table, il faut indiquer le schéma de partitionnement à utiliser ainsi que la colonne qui sera utilisée par la fonction de partitionnement, pour décider de l'affectation de la partition à chaque ligne d'information. Le type de cette colonne doit parfaitement correspondre à celui du paramètre de la fonction. Dans le cas où la colonne utilisée pour le calcul du partitionnement est une colonne calculée, alors elle doit être de type `PERSISTED`.

Syntaxe

```
CREATE TABLE nomTable(
  definitionColonne [...]
) ON
nomSchemaPartition(colonneUtiliséePourCalculerLaPartition)[,]
```



Les informations données ici pour les tables sont également valables pour les index.

Exemple

Dans l'exemple suivant, la table *TCLIENTS* est définie en utilisant le schéma de partitionnement défini lors de l'exemple précédent.

```
create table dbo.Tclients
numero int identity(1,1) constraint pk_tclients primary key,
nom nvarchar(80),
prenom nvarchar(80),
adresse nvarchar(100),
codepostal char(5),
cille nvarchar(80),
telephone char(14))
on schemaclients(numero);
```

4. Les index partitionnés

Comme pour les tables, il est tout à fait possible de partitionner les index. Le processus de partitionnement est le même, c'est-à-dire qu'il s'appuie sur un schéma et une fonction de partitionnement. Il est toutefois fortement recommandé que l'index partitionné soit défini sur une table déjà partitionnée et qu'il s'appuie sur les mêmes schémas et fonctions de partitionnement.

Si la colonne de partitionnement ne fait pas partie des colonnes indexées alors elle est incluse dans l'index comme une colonne incluse qui permet de définir des index couvrants.

Syntaxe

```
CREATE INDEX nomIndex  
ON nomTable(colonne1,...)  
ON nomSchemaPartition(colonneDePartition);
```

Exemple

Un index est défini sur la colonne nom de la table partitionné TCLIENTS.

```
create index idx_tclients_nom  
on dbo.tclients(nom)  
on schemaclients(numero);
```

Compression des données

SQL Server donne la possibilité d'activer la compression au niveau des tables et des index. Si la compression peut être définie sur des tables et index existants, il ne sera pris en compte qu'après la reconstruction de la table (`ALTER TABLE nomTable REBUILD`) ou bien de l'index concerné. Si la compression de la table entraîne la compression de l'index ordonné (`CLUSTERED`), les index non ordonnés ne sont pas affectés, et il est nécessaire d'activer la compression sur chacun d'eux un par un. Dans le cas des tables partitionnées, la compression peut être mise en place partition par partition.



La compression n'est possible que sur les données utilisateur. Les tables système ne peuvent pas être compressées.

L'objectif de la compression est de réduire l'espace disque utilisé par les données de la table. La compression des données va permettre de stocker plus de lignes d'informations sur le même bloc de 8 ko. La compression ne permet pas d'augmenter la taille maximale des lignes. En effet, le mécanisme doit être réversible.

Sur des tables valorisées, il est possible de connaître l'impact de la compression des données en exécutant la procédure stockée `sp_estimate_data_compression_savings`.

La mise en place de la compression est une opération ponctuelle. Aussi, est-il préférable de passer par l'assistant proposé par SQL Server Management Studio. Au niveau Transact SQL, la compression est mise en place avec les instructions `CREATE TABLE / CREATE INDEX` et `ALTER TABLE / ALTER INDEX`.

L'assistant de compression des données est lancé depuis le menu contextuel associé à la table en sélectionnant l'option **Stockage - Gérer la compression**.

L'assistant permet lui aussi de visualiser le résultat estimé en termes de gain d'espace de la compression grâce au bouton **Calculer** qui permet d'avoir une bonne estimation en fonction du type de compression choisi.

Bien évidemment, le nombre de lignes présentes dans la table doit être conséquent pour que le gain de compression soit intéressant. En fonction des types de données choisis pour les différentes colonnes de la table, la compression peut se révéler plus ou moins intéressante.

Assistant Compression de données - ARTICLES

Sélectionner le type de compression

Sélectionnez un type de compression pour chaque partition.

☐ Utiliser le même type de compression pour toutes les partitions

	Numéro de partition	Type de compression	Limite	Nombre de lignes	Espace actuel	Espace compressé demandé
▶	1	None		25		

Calculer

Aide < Précédent Suivant > Terminer >> Annuler

Cryptage des données

SQL Server permet de crypter les fichiers de données et les journaux. Ce cryptage est dynamique et est effectué lors de chaque écriture sur le disque. Il est en est de même pour l'opération de décryptage. Cette fonctionnalité de cryptage/décryptage transparent est identifiée sous le nom TDE ou *Transparent Data Encryption*.

La mise en place de cette opération de cryptage permet de garantir une opacité plus grande des fichiers de données et journaux face aux différents outils système d'analyse des fichiers, ou bien pour éviter un détachement/attachement de base de données non autorisé. Cependant cette opération de cryptage n'apporte aucune garantie supplémentaire en ce qui concerne la communication entre le processus client et le serveur. Lorsque le cryptage de la base de données est actif, les sauvegardes sont également cryptées avec la même clé. Il est donc nécessaire de posséder cette clé pour être en mesure de restaurer les données.

Le cryptage des données s'appuie sur une clé de cryptage (DEK : *Database Encryption Key*) qui est enregistrée dans la base master par exemple sous la forme d'un certificat.

Avant de pouvoir mettre en place le cryptage sur une base, il faut commencer par définir une clé principale afin d'obtenir un certificat valide. Le certificat est alors utilisé pour définir la clé de cryptage. La génération de cette clé peut être réalisée par différents algorithmes. Le nom de l'algorithme choisi est passé en paramètre à l'instruction `CREATE DATABASE ENCRYPTION KEY`.

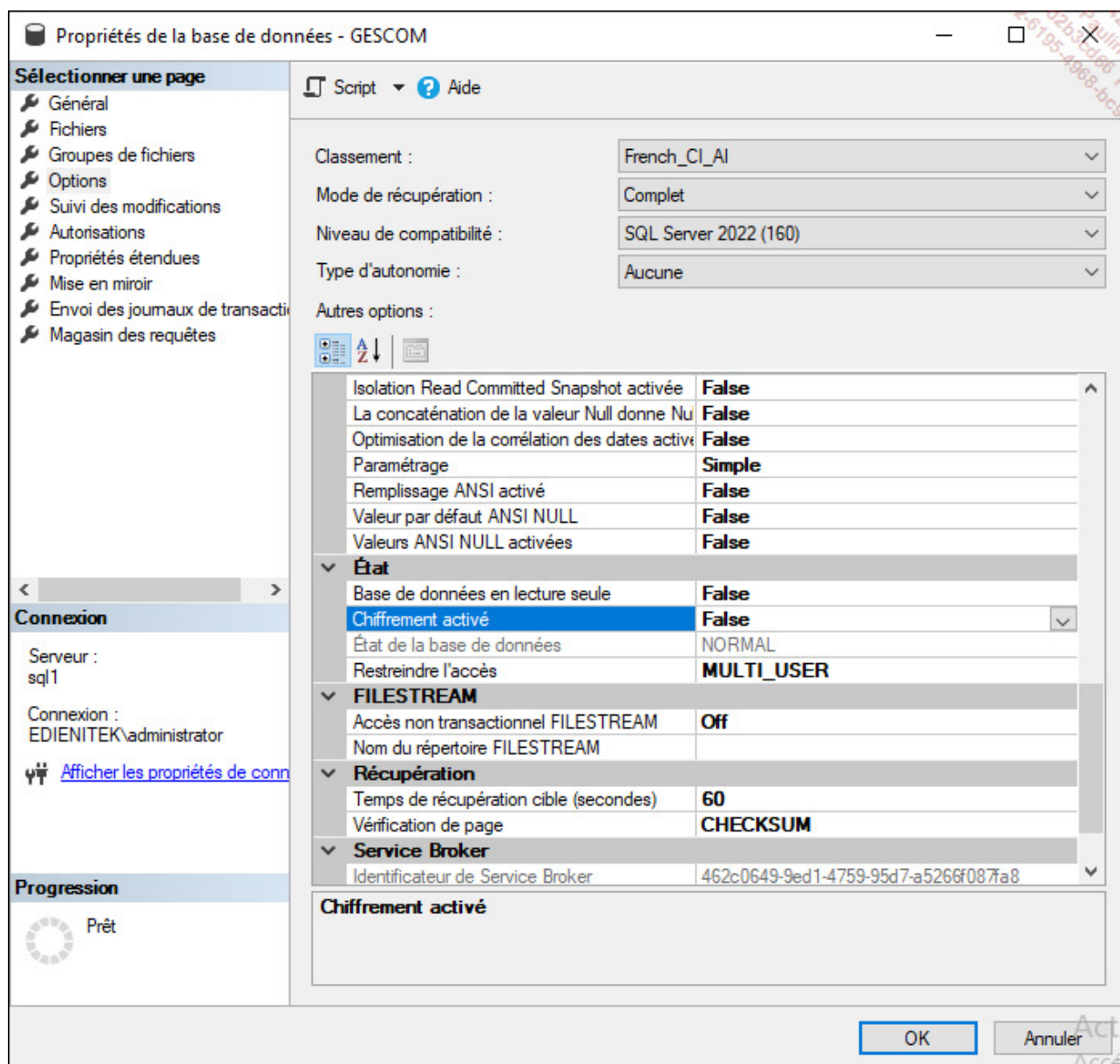
Exemple

Dans l'exemple ci-dessous, le cryptage est défini sur la base GESCOM.

```
USE master;
GO
CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'Pa$$w0rd';
go
CREATE CERTIFICATE Cert_gescom WITH SUBJECT = 'Certificat pour Gescom';
go
USE GESCOM;
GO
CREATE DATABASE ENCRYPTION KEY
```

```
WITH ALGORITHM = AES_128  
ENCRYPTION BY SERVER CERTIFICATE Cert_gescom;  
GO  
ALTER DATABASE GESCOM  
SET ENCRYPTION ON;  
GO
```

À partir de SQL Server Management Studio, il est possible d'activer ou non la prise en charge du cryptage au niveau de la base de données depuis la page **Options** des propriétés de la base de données. Ceci correspond uniquement à la dernière commande de l'exemple précédent.



L'activation du cryptage sur une base de données affecte tous les fichiers de données et si un groupe de fichiers est positionné en lecture seule alors cette activation échoue.

Les tables temporelles

SQL Server propose de suivre automatiquement les différentes versions des informations stockées dans une table. C'est-à-dire que SQL Server va garder une version de chaque ligne avant et après modification.

Le nom exact de ce mécanisme proposé par SQL est « tables temporelles à système par version » car la période de validité de chaque ligne est gérée par SQL Server.

Pour pouvoir gérer cette notion de temporalité, SQL Server s'appuie sur :

- deux colonnes de type `datetime2`. Ces deux colonnes correspondent aux dates et heures de début et de fin de validité des données,
- une table miroir (en termes de structure) à la table qui contient les données actives afin de conserver l'historique. On nomme souvent cette

table « table d'historique ». Elle peut être créée automatiquement par SQL Server ou bien par l'administrateur.

Ces tables, bien que particulières dans leur système de gestion de l'information, peuvent s'administrer comme n'importe quelle autre table de la base de données.

Syntaxe

```
CREATE TABLE nomTable(  
  nomColonne type, ...  
  debutValidite datetime2 GENERATED ALWAYS AS ROW START NOT NULL,  
  finValidite datetime2 GENERATED ALWAYS AS ROW END NOT NULL,  
  PERIOD FOR SYSTEM_TIME (debutValidite, finValidite)  
) WITH (SYSTEM_VERSIONING=ON (HISTORY_TABLE= nomTableHistorique)) ;
```



Cette table nécessite une clé primaire.

Exemple

Dans l'exemple présenté ci-dessous, une table des

produits est générée avec une notion de temporalité.

```
CREATE TABLE dbo.produits(  
reference char(8) constraint pk_produits primary key,  
designation varchar(200),  
debutValidite datetime2 GENERATED ALWAYS AS ROW START NOT NULL,  
finValidite datetime2 GENERATED ALWAYS AS ROW END NOT NULL,  
PERIOD FOR SYSTEM_TIME (debutValidite, finValidite)  
) WITH (SYSTEM_VERSIONING=ON (HISTORY_TABLE= dbo.produitsHistorique)) ;
```

Il est ensuite possible d'interroger ces tables en fonction de la période souhaitée. Par exemple, lister tous les produits à la date du 15 avril 2018 :

```
select * from dbo.produits  
FOR SYSTEM_TIME AS OF '2018-04-15'
```

Planification

1. Dimensionner les fichiers

Afin d'évaluer la taille des fichiers nécessaires au stockage des informations contenues dans la base il faut prendre en compte de nombreux critères.

Pour les fichiers de données

- Distinguer les tables système et utilisateur.
- Prendre en compte le nombre de lignes dans les tables.
- Recenser les valeur indexées (clé, nombre de ligne, facteur de remplissage).

C'est après une fine évaluation de la quantité d'espace occupée qu'il est possible de fixer la taille initiale des fichiers de données. La méthode la plus simple est d'évaluer la longueur moyenne d'une ligne, de calculer combien de lignes peuvent être stockées dans un bloc de 8 ko, et enfin de trouver le nombre de blocs

nécessaires pour stocker toutes les lignes de la table. À partir de ce nombre de blocs utilisés par la table, il convient de prendre le multiple de 8 immédiatement supérieur puis de diviser par 8 pour obtenir le nombre d'extensions.

Pour les fichiers journaux

- L'activité.
- La fréquence.
- La taille des transactions.
- Les sauvegardes.

C'est la prise en compte de ces critères, ainsi que la consultation des options de la base, qui vont permettre de fixer une taille optimale pour le fichier journal. Au départ il peut être utile de fixer la taille du journal entre 10 % et 25 % de la taille des données dans la base. Ce pourcentage est à diminuer si la base supporte principalement des requêtes de sélection `SELECT`, qui n'utilisent pas le journal.

2. Nommer la base et les fichiers de façon explicite

Il est important de prévoir le nom de la base, les noms logiques, physiques et la taille des fichiers ainsi que la gestion dynamique ou non de l'accroissement.

3. Emplacement des fichiers

L'emplacement des fichiers données et journaux doit être spécifié avec précision afin de réduire au maximum les conflits d'accès au disque, et de faire travailler tous les disques de la machine de façon équitable.

4. Utilisation des groupes de fichiers

Pour les très grosses bases de données, il peut être intéressant de créer plusieurs groupes de fichiers sur

le serveur et de spécialiser chacun d'eux. Ceci peut être intéressant pour créer des tables partitionnées ou de gérer les sauvegardes de bases par fichiers/groupes de fichiers.

5. Niveau de compatibilité

Un serveur peut héberger différentes bases de données utilisateur avec différents niveaux de compatibilité. Une base créée avec SQL Server 2022 possède un niveau de compatibilité de 160. Une instance SQL Server 2022 peut gérer des bases depuis le niveau 100 pour SQL Server 2008.

Toutes les instructions Transact SQL sont interprétées au sein d'une base de données par rapport au niveau de compatibilité fixé.

Il est possible de modifier le niveau de compatibilité d'une base à l'aide de l'instruction `ALTER DATABASE` et de l'option `SET COMPATIBILITY LEVEL`.

Syntaxe

```
ALTER DATABASE nomBaseDeDonnées SET COMPATIBILITY LEVEL = { 100 | 110 |  
120 | 130 | 150 | 160 }
```

Exercice : créer une base de données

1. Énoncé

La première étape consiste donc à créer la base de données LivreTSQL, et comme son nom l'indique, elle devra être créée à l'aide d'un script Transact SQL. Une seconde base, nommée LivreSSMS, sera créée depuis l'interface graphique de Management Studio.

Les fichiers de données vont être définis dans un répertoire spécifique. Il convient de créer le dossier c:\donnees. Bien entendu, stocker les données directement sur le disque c:\ relève de l'exemple pédagogique.

Paramètres de la base LivreTSQL

Fichier de données

- Taille : 10 Mo
- Nom physique : c:\donnees\LivreTSQL.mdf
- Nom logique : LivreTSQL

Fichier journal

- Taille : 8 Mo
- Nom physique : c:\donnees\LivreTSQL_log.ldf
- Nom logique : LivreTSQL_log

Paramètres de la base LivreSSMS

Fichier de données

- Taille : 15 Mo
- Nom physique : c:\donnees\LivreSSMS.mdf
- Nom logique : LivreSSMS

Fichier journal

- Taille : 8 Mo
- Nom physique : c:\donnees\LivreSSMS_log.ldf
- Nom logique : LivreSSMS_log

2. Corrigé

La base de données LivresTSQL peut être créée à l'aide du script suivant :

```
CREATE DATABASE LivresTSQL
ON PRIMARY (
    NAME=LivresTSQL,
    FILENAME='c:\donnees\LivresTSQL.mdf',
    SIZE=10Mb
)
LOG ON (
    NAME=LivresTSQL_log,
    FILENAME='c:\donnees\LivresTSQL_log.ldf',
    SIZE=8Mb
);
```

Pour créer la base LivreSSMS, il faut, depuis l'explorateur d'objets de SQL Server Management Studio, afficher le menu contextuel associé au nœud VotreServeur - Base de données et sélectionner l'option **Nouvelle base de données**. L'écran ci-dessous est alors présenté. Il faut simplement veiller au dossier de création des fichiers ainsi qu'au nom physique des fichiers.

Nouvelle base de données

Sélectionner une page

- Général
- Options
- Groupes de fichiers

Script ? Aide

Nom de la base de données :

Propriétaire :

☒ Utiliser l'indexation de texte intégral

Fichiers de la base de données :

Nom logique	Type de fichier	Groupe de fichiers	Taille initiale (Mo)	Croissance automatique
LivreSSMS	Données de ...	PRIMARY	8	Par 64 Mo, illimitée
LivreSSMS...	JOURNAL	Non applicable	8	Par 64 Mo, illimitée

Connexion

Serveur : sql1

Connexion : EDIENITEK\administrator

[Afficher les propriétés de connexion](#)

Progression

Prêt

Ajouter Supprimer

OK Annuler

Exercice : ajouter un groupe de fichiers

1. Énoncé

Sur la base LivreTSQL, ajoutez le groupe de fichiers Data. Ce groupe de fichiers est composé de deux fichiers : data1.ndf et data2.ndf.

Le fichier data1.ndf possède une taille fixe de 50 Mo tandis que le fichier data2.ndf possède une taille initiale de 10 Mo puis peut croître jusqu'à la taille de 50 Mo par pas de 10 Mo.

2. Corrigé

Pour ajouter un groupe de fichiers et définir les fichiers présents dans ce groupe, il est possible de passer par la boîte de dialogue présentant les propriétés de la base depuis SSM, mais il est

également possible de passer par un script Transact SQL. Ce script est présenté ci-dessous. Dans un premier temps, le groupe de fichiers est défini à l'aide de l'instruction `ALTER DATABASE`.

```
ALTER DATABASE LivreTSQL
ADD FILEGROUP Data;
```

Le groupe de fichiers est défini, mais il n'est pas encore possible de l'utiliser car aucun fichier n'appartient à ce groupe. Le premier fichier (data1) va être créé à l'aide du script ci-dessous.

```
ALTER DATABASE LivreTSQL
ADD FILE (
    NAME=data1,
    FILENAME='c:\donnees\data1.ndf',
    SIZE=50Mb,
    MAXSIZE=50Mb
)
TO FILEGROUP Data;
```

Le second fichier (data2) est créé de façon très

similaire mais en ajoutant les paramètres `MAXSIZE` et `FILEGROWTH` afin de contrôler la taille maximale du fichier et son pas d'accroissement.

```
ALTER DATABASE LivreTSQL
ADD FILE (
    NAME=data2,
    FILENAME='c:\donnees\data2.ndf',
    SIZE=10Mb,
    MAXSIZE=50Mb,
    FILEGROWTH=10Mb
)
TO FILEGROUP Data;
```

Introduction

Le contrôle d'accès représente une opération importante au niveau de la gestion de la sécurité sur un serveur de bases de données. La sécurisation des données nécessite une organisation des objets de façon indépendante des utilisateurs, ce qui est possible par les schémas. La sécurité passe également par un meilleur contrôle des autorisations et la possibilité d'accorder juste les privilèges nécessaires à chaque utilisateur pour qu'il puisse travailler de façon autonome.

Pour l'organisation de cette politique de sécurité, il faut prendre en compte l'organisation hiérarchique des éléments de sécurité, de façon à rendre la gestion des droits d'accès simple et efficace.

SQL Server s'appuie sur trois éléments clés qui sont :

- les entités de sécurité,
- les sécurisables,
- les autorisations.

Les entités de sécurité sont des comptes de sécurité qui disposent d'un accès au serveur SQL.

Les sécurisables représentent les objets gérés par le serveur. Ici, un objet peut être une table, un schéma ou une base de données par exemple.

Les autorisations sont accordées aux entités de sécurité afin qu'elles puissent travailler avec les sécurisables.

L'organisation hiérarchique permet d'accorder une autorisation (par exemple `SELECT`) sur un sécurisable de niveau élevé (par exemple le schéma) pour permettre à l'entité de sécurité qui reçoit l'autorisation d'exécuter l'instruction `SELECT` sur toutes les tables contenues dans le schéma.

Les vues du catalogue système permettent d'obtenir un rapport complet et détaillé sur les connexions existantes, les utilisateurs de base de données définis et les privilèges accordés. Quelques-unes de ces vues sont présentées ci-dessous :

- `sys.server_permissions` : liste des permissions

de niveau serveur et de leurs bénéficiaires.

- sys.sql_logins : liste des connexions.
- sys.server_principals : entité de sécurité définie au niveau serveur.
- sys.server_role_members : liste des bénéficiaires d'un rôle de serveur.
- sys.database_permissions : liste des permissions et de leur bénéficiaire au niveau base de données.
- sys.database_principals : entité de sécurité de niveau base de données.
- sys.database_role_members : liste des bénéficiaires d'un rôle de base de données.

Pour simplifier la gestion des droits d'accès, il est possible d'utiliser trois types de rôles. Les **rôles de serveur** qui regroupent des autorisations au niveau du serveur, ces autorisations sont valables pour toutes les bases installées. Les **rôles de base de données**, regroupent quant à eux des droits au niveau de la base de données sur laquelle ils sont définis. Et enfin les **rôles d'applications**, définis sur les bases de

données utilisateur, permettent de regrouper les droits nécessaires à la bonne exécution d'une application cliente.

Gestion des accès serveur

Avant de pouvoir travailler avec les données gérées par les bases, il faut dans un premier temps se connecter au serveur SQL. Cette étape permet de se faire identifier par le serveur SQL et d'utiliser par la suite tous les droits qui sont accordés à notre connexion. Il existe dans SQL Server deux modes de gestion des accès au serveur de base de données.

Attention : dans cette section, seule la partie connexion au serveur est abordée. Il est important de bien distinguer la connexion au serveur et l'utilisation de bases de données. La connexion au serveur permet de se faire identifier par le serveur SQL comme un utilisateur valide, pour utiliser, par la suite, une base de données : les données et les objets. L'ensemble de ces droits seront définis ultérieurement. Ces droits sont associés à un utilisateur de base de données, pour lequel correspond une connexion.



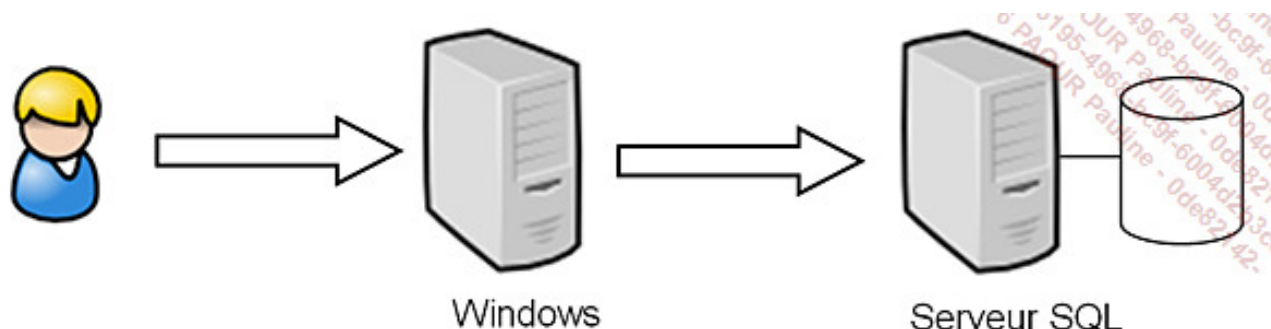
On parlera de connexion au serveur ou de logins.

1. Mode de sécurité Windows

Ce type de gestion de la sécurité permet de s'appuyer sur les utilisateurs et les groupes Windows pour le domaine et le poste local. SQL Server utilise la gestion des utilisateurs de Windows (gestion des mots de passe...) et récupère uniquement les noms pour créer des connexions au serveur.

Une fonctionnalité très importante de SQL Server est de pouvoir autoriser des groupes Windows à venir se connecter. La gestion des groupes permet de simplifier grandement la gestion de l'accès aux ressources. Les mêmes groupes Windows peuvent donc être utilisés pour donner accès à des fichiers ou à des objets de bases de données SQL Server.

Avec une telle méthode de fonctionnement, comme un utilisateur Windows peut appartenir à plusieurs groupes, il peut posséder plusieurs droits de connexion à SQL Server.



Authentication Windows

En mode sécurité Windows, seuls les noms d'utilisateurs sont stockés. La gestion des mots de passe et de l'appartenance aux différents groupes est laissée à Windows. Ce mode de fonctionnement permet de bien discerner les tâches de chacun et de spécialiser SQL Server sur la gestion des données en laissant Windows gérer l'authentification des utilisateurs, ce qu'il sait bien faire. De plus, avec un tel schéma de fonctionnement, il est possible d'appliquer la politique suivante : un utilisateur = un mot de passe. L'accès au serveur SQL est transparent pour les utilisateurs approuvés par Windows.



SQL Server s'appuie sur les groupes auxquels appartient l'utilisateur lors de la connexion au serveur. Si des modifications d'appartenance aux groupes sont effectuées depuis Windows, ces modifications ne seront prises en compte que lors de la prochaine connexion de l'utilisateur au serveur SQL.

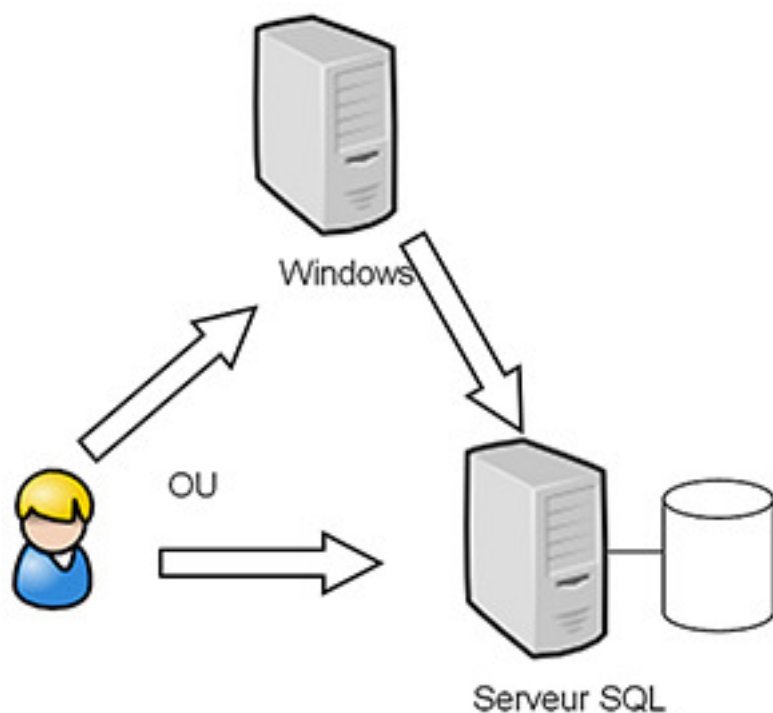


SQL Server s'appuie sur le SID de Windows pour identifier le groupe. Si un groupe est supprimé puis recréé dans Windows, il est important de réaliser la même opération en ce qui concerne la connexion du groupe dans SQL Server.

2. Mode de sécurité mixte

Le mode de sécurité mixte repose sur une authentification Windows puis sur une authentification SQL Server. C'est ce mode d'authentification qui va être détaillé ici.

Dans un tel mode de fonctionnement, c'est SQL Server qui se charge de vérifier que l'utilisateur qui demande à se connecter est bien défini puis il se charge également de vérifier le mot de passe.

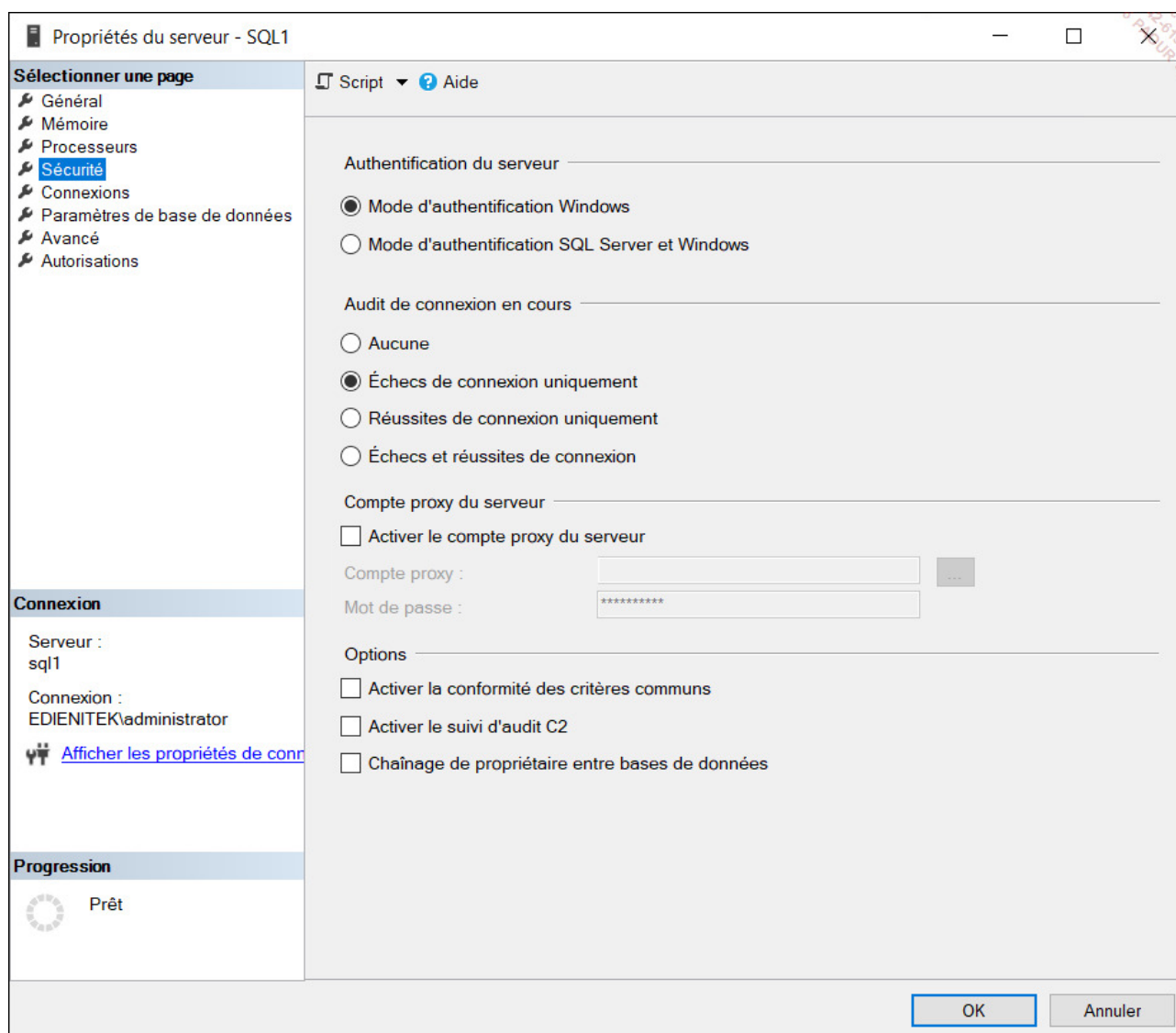


Mode de sécurité mixte

Tous les utilisateurs sont entièrement gérés par SQL Server (nom et mot de passe). Ce type de gestion des connexions est bien adapté pour des clients qui ne peuvent pas s'identifier auprès de Windows.

3. Comment choisir un mode de sécurité ?

Il est possible de modifier le mode de sécurité utilisé par le serveur SQL directement depuis SQL Server Management Studio en allant modifier les propriétés de l'instance comme ci-après.



Lors de l'installation du serveur SQL, des connexions administrateurs SQL (**sysadmins**) sont créées. En mode Windows, le groupe local des Administrateurs peut être autorisé à se connecter, et en mode de sécurité SQL Server, l'utilisateur **sa** est déjà créé, seul un mot de passe est à paramétrer.

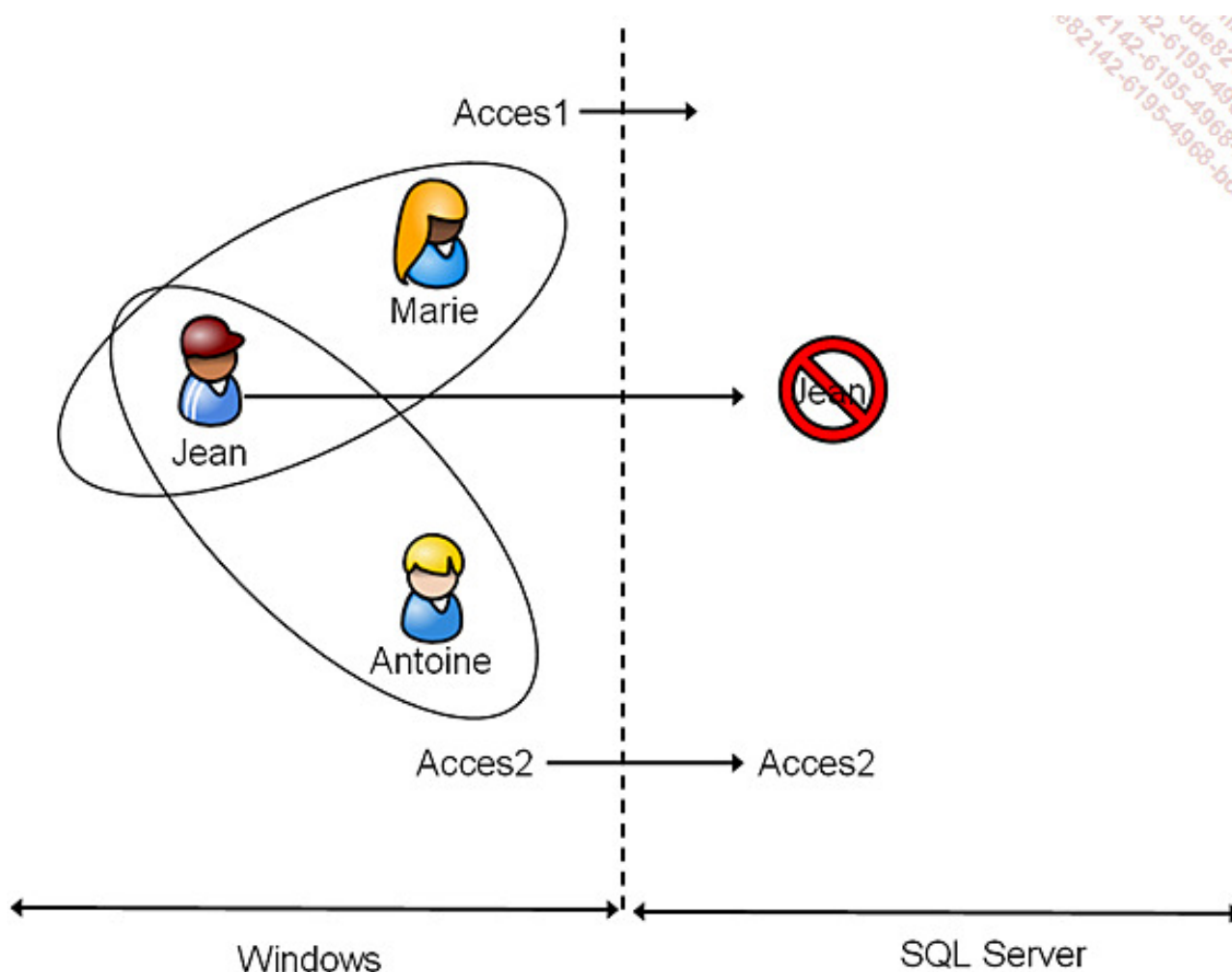


Il est recommandé d'utiliser la sécurité Windows, qui offre une plus grande souplesse au niveau de la gestion des utilisateurs.

Les changements de mode entre mode mixte et mode Windows ne sont pris en compte qu'après redémarrage du service SQL Server.

4. Gérer une connexion à SQL Server

Étant donné qu'un utilisateur Windows peut appartenir à plusieurs groupes, il peut se voir accorder plusieurs fois le droit de connexion au serveur SQL. Ceci peut poser un problème lorsqu'un utilisateur ou un groupe d'utilisateurs ne doit jamais pouvoir se connecter au serveur SQL. Afin de remédier à ce souci, Microsoft fournit, parmi les ordres Transact SQL pour la gestion des connexions, l'ordre **DENY** qui permet de refuser explicitement un utilisateur ou un groupe Windows. Le **DENY** constitue un refus explicite et est prioritaire par rapport aux différentes autorisations de connexion.



Création d'une connexion

Dans le schéma ci-dessus, il y a trois utilisateurs Windows et deux groupes. Une connexion SQL a été mise en place pour le groupe Acces2, le groupe Acces1 ne possède

pas de connexion, mais pour que l'utilisateur Jean soit interdit d'accès à SQL Server, une connexion a été créée sur SQL Server avec la propriété `DENY`.



Il faut posséder une permission d'administrateur (**sysadmin**) ou une permission de gestionnaire de sécurité (**securityadmin**) pour pouvoir réaliser les différentes opérations relatives à la gestion des connexions.

Les connexions, qu'elles soient de type SQL Server ou bien Windows, doivent être définies au niveau de l'instance SQL Server pour permettre aux utilisateurs de se connecter. Elles sont stockées dans la base de données système **Master**.

La gestion des connexions peut être réalisée de façon graphique par l'intermédiaire de l'explorateur d'objets depuis SQL Server Management Studio ou bien sous forme de script Transact SQL. Toutes les opérations de gestion des connexions sont réalisables en Transact SQL avec l'aide des instructions `CREATE LOGIN`, `ALTER LOGIN` et `DROP LOGIN`. Chaque solution possède ses avantages et ses inconvénients.



Les procédures `sp_addlogin` et `sp_grantlogin` ne doivent plus être utilisées. Elles sont encore présentes dans SQL Server pour assurer la compatibilité des scripts.

a. En mode de sécurité Windows

Les noms des groupes ou des utilisateurs devront correspondre à ceux définis dans Windows.

SQL Server Management Studio

Il est possible de créer les connexions depuis l'interface graphique de SQL Server Management Studio, en procédant de la sorte :

- Depuis l'explorateur d'objets, se placer sur le nœud **Sécurité - Connexions**.

- Depuis le menu contextuel associé au nœud connexion, faire le choix **Nouvelle connexion**.

L'écran suivant apparaît alors. Il ne reste plus qu'à sélectionner le compte d'utilisateur Windows ou bien le groupe qui est autorisé à se connecter.

Nouvelle connexion

Sélectionner une page

- Général
- Rôles du serveur
- Mappage d'utilisateur
- Éléments sécurisables
- État

Script ? Aide

Nom d'accès : EDIENTEK\AMARTIN Rechercher...

☒ Authentification Windows
☐ Authentification SQL Server

Mot de passe :

Confirmer le mot de passe :

☐ Spécifier l'ancien mot de passe

Ancien mot de passe :

☒ Appliquer la stratégie de mot de passe
☒ Appliquer l'expiration du mot de passe
☒ L'utilisateur doit changer de mot de passe à la prochaine connexion

☐ Mappé au certificat
☐ Mappé à la clé asymétrique
☐ Mapper aux informations d'identification Ajouter

Informations d'...	Fournisseur

Supprimer

Base de données par défaut : master

Langue par défaut : <par défaut>

OK Annuler

Connexion

Serveur : sql1

Connexion : EDIENTEK\administrator

[Afficher les propriétés de connexion](#)

Progression

Prêt

L'interface graphique de SQL Server Management Studio permet également de modifier simplement un compte de connexion à partir de la fenêtre des propriétés ou bien de supprimer une connexion.

Transact SQL

```
CREATE LOGIN nom_connexion
FROM WINDOWS
[WITH DEFAULT_DATABASE=baseDeDonnées|DEFAULT_LANGUAGE=langue]
```

nom_connexion

Nom de l'utilisateur ou du groupe Windows à ajouter. Il doit être donné sous la forme domaine\utilisateur.

baseDeDonnées

Précise la base de données qui sera utilisée par défaut. Si aucune base de données n'est précisée, alors la base de données par défaut est la base Master.

langue

Langue de travail par défaut pour cette connexion. Cette option n'est à préciser que si la langue relative à cette connexion est distincte de celle définie par défaut sur le serveur SQL.

Exemple

```
USE [master]
GO
CREATE LOGIN [Edienitek\amartin] FROM WINDOWS WITH DEFAULT_DATABASE=[master]
GO
```

b. En mode de sécurité mixte

Comme pour la sécurité en mode Windows, il existe deux moyens pour gérer les connexions.

Mis à part l'ensemble des règles de gestion des mots de passe, la gestion d'une connexion en mode de sécurité SQL Server est en tout point semblable à celui d'une connexion en mode de sécurité Windows.

Une fois créés, les deux types de connexion sont en tous points identiques. Il n'y a pas une

solution qui présente plus de fonctionnalités que l'autre.

Pour les connexions utilisant une sécurité SQL Server, il est possible de définir des règles de gestion des mots de passe.

Pour les connexions utilisant une sécurité Windows, c'est le système d'exploitation qui gère les règles de sécurité relatives au mot de passe.

SQL Server Management Studio

Il est possible de créer les connexions depuis l'interface graphique de SQL Server Management Studio, en procédant de la sorte :

- Depuis l'explorateur d'objets, se placer sur le nœud **Sécurité - Connexion**.
- Depuis le menu contextuel associé au nœud de connexion, faire le choix **Nouvelle connexion**.

L'écran suivant apparaît alors. Il ne reste plus qu'à définir le nom de la nouvelle connexion ainsi que le mot de passe associé. C'est également à ce niveau que peut être précisée la politique de gestion des passes à mettre en place. Les règles de gestion des mots de passe sont héritées de celles mise en place sur Windows.

Nouvelle connexion

Sélectionner une page

- Général
- Rôles du serveur
- Mappage d'utilisateur
- Éléments sécurisables
- État

Script ? Aide

Nom d'accès : jacques Rechercher...

☐ Authentification Windows
☒ Authentification SQL Server

Mot de passe :

Confirmer le mot de passe :

☐ Spécifier l'ancien mot de passe
 Ancien mot de passe :

☒ Appliquer la stratégie de mot de passe
☒ Appliquer l'expiration du mot de passe
☒ L'utilisateur doit changer de mot de passe à la prochaine connexion

☐ Mappé au certificat
☐ Mappé à la clé asymétrique
☐ Mapper aux informations d'identification

Informations d'identification mappées

Informations d'i...	Fournisseur
---------------------	-------------

Ajouter

Supprimer

Base de données par défaut : master

Langue par défaut : <par défaut>

OK Annuler

Connexion

Serveur : sql1

Connexion : EDIENTEK\administrator

[Afficher les propriétés de connexion](#)

Progression

Prêt

L'interface graphique de SQL Server Management Studio permet également de modifier simplement un compte de connexion à partir de la fenêtre des propriétés ou bien de supprimer une connexion.

Transact SQL

```
CREATE LOGIN nom_connexion
WITH {PASSWORD='motDePasse' | motDePasseHaché HASHED} [MUST_CHANGED]
[,SID=sid |
,DEFAULT_DATABASE=baseDeDonnées |
,DEFAULT_LANGUAGE=langue |
,CHECK_EXPIRATION={ON | OFF}]
```

```
,CHECK_POLICY={ON | OFF}  
,[CREDENTIAL=nom_credit]]
```

nom_connexion

Nom de la connexion qui sera définie dans SQL Server.

Lorsque la connexion correspond à un compte Windows, le nom de la connexion sera au format [nomDomaine\nomCompte], il n'est pas possible d'utiliser un nommage au format UPN, soit nomCompte@nomDomaine. Dans le cadre de la sécurité mixte, lorsqu'une connexion SQL server est créée, le nom de cette connexion doit respecter les règles de nommage des identifiants SQL Server et, à ce titre, il n'est pas possible d'y introduire le caractère \.

motDePasse

Mot de passe associé à la connexion. Ce mot de passe est stocké de façon cryptée dans la base Master et il est vérifié lors de chaque nouvelle connexion au serveur. Il est obligatoire de définir un mot de passe pour chaque connexion.

Les mots de passe possèdent un minimum de 8 caractères et un maximum de 12 ; de plus, la distinction majuscules/minuscules est active au niveau des mots de passe. Pour des raisons évidentes de sécurité, il est très fortement recommandé d'adopter un mot de passe fort.

motDePasseHaché

Il s'agit ici de préciser que la chaîne fournie correspond à la version hachée du mot de passe. Ce n'est donc pas le mot de passe tel que l'utilisateur le saisit, mais le mot de passe tel que SQL le stocke qui est fourni.

MUST_CHANGED

Avec cette option SQL Server demande à l'utilisateur de saisir un nouveau mot de passe lors de sa première connexion au serveur. Cette option n'est possible que si `CHECK_EXPIRATION` et `CHECK_POLICY` sont activées (positionnées à `ON`).

baseDeDonnØes

Précise la base de données qui sera utilisée par défaut. Si aucune base de données n'est précisée alors la base de données par défaut est la base Master.

langue

Langue de travail par défaut pour cette connexion. Cette option n'est à préciser que si la langue relative à cette connexion est distincte de celle définie par défaut sur le serveur SQL.

CHECK_EXPIRATION

Si cette option est positionnée à **ON** (**OFF** par défaut), elle indique que le compte de connexion va suivre la règle relative à l'expiration des mots de passe définie sur le serveur SQL. L'activation de cette option n'est possible que si **CHECK_POLICY** est également activée.

CHECK_POLICY

Activée par défaut, cette option permet de répercuter au niveau de SQL Server les règles de gestion des mots de passe définies au niveau de Windows sur le poste qui héberge l'instance SQL Server.

CREDENTIAL

Permet de relier la connexion à un credential créé précédemment par l'intermédiaire de l'instruction **CREATE CREDENTIAL** .

Exemple

Dans l'exemple suivant, la connexion PIERRE est créée avec le mot de passe. La base de données par défaut est également configurée.

```
USE [master]
GO
CREATE LOGIN Pierre
WITH PASSWORD=N'Pa$Sw0rd', DEFAULT_DATABASE=Gescom
```

GO

En utilisant un certificat

SQL Server donne la possibilité de créer des connexions au serveur basées sur des certificats (CERTIFICATE). Ces certificats permettent d'identifier de façon sûre un utilisateur en se basant sur différentes informations telles que :

- Une clé publique.
- Une identification telle que le nom et l'adresse e-mail.
- Une période de validité.

Pour être tout à fait précis, les certificats de SQL Server sont conformes au standard X.509.

La création d'un certificat dans SQL Server peut s'appuyer sur un certificat défini dans un fichier ou un assembly ou bien demander à SQL Server de générer les clés publiques et privées.

Les certificats de sécurité vont permettre à des applications et/ou des services, d'ouvrir une session sécurisée sur le serveur. Ce type d'ouverture de session par un service est utilisé par le service Service Broker qui utilise un certificat pour poster un message sur une base distante.

Dans le cas où le certificat est créé sans s'appuyer sur un fichier externe, alors la clé privée est encryptée en utilisant la clé principale de la base de données.

5. Base de données par défaut

Dans les deux cas de types de connexion (Windows et SQL Server) existe la propriété de « base de données par défaut ». C'est sur cette base que sera positionné tout utilisateur qui utilise cette connexion si aucune base n'est spécifiée au moment de la connexion. Attention à bien prendre en compte le fait que définir une base de données par défaut ne donne pas de droits d'utilisation sur cette base.

Nouvelle connexion

Sélectionner une page

- Général
- Rôles du serveur
- Mappage d'utilisateur
- Éléments sécurisables
- État

Script Aide

Nom d'accès : jacques Rechercher...

☐ Authentification Windows
☒ Authentification SQL Server

Mot de passe :

Confirmer le mot de passe :

☐ Spécifier l'ancien mot de passe
 Ancien mot de passe :

☒ Appliquer la stratégie de mot de passe
☒ Appliquer l'expiration du mot de passe
☒ L'utilisateur doit changer de mot de passe à la prochaine connexion

☐ Mappé au certificat
☐ Mappé à la clé asymétrique
☐ Mapper aux informations d'identification

Informations d'identification mappées

Informations d'i...	Fournisseur
---------------------	-------------

Ajouter

Supprimer

Base de données par défaut : GESCOM

Langue par défaut : <par défaut>

OK Annuler

Connexion

Serveur : sql1
 Connexion : EDIENTEK\administrator
[Afficher les propriétés de connexion](#)

Progression

Prêt

Il est possible de connaître la base de données par défaut pour chaque connexion en interrogeant la vue `sys.syslogins` de la façon suivante :

```
select name, dbname
from sys.syslogins
```

Dans le cas où la base de données par défaut associée à la connexion n'est plus valide ou bien lorsque l'on souhaite modifier la base de données par défaut, il est nécessaire de revenir sur les propriétés de la connexion depuis SQL Server Management Studio ou bien d'utiliser l'instruction **ALTER LOGIN** (instruction présentée plus loin dans ce chapitre).

6. Informations d'identification

Ces objets permettent à des connexions en mode de sécurité SQL Server d'accéder à des ressources externes au serveur de base de données. Les informations d'identification (credentials) vont donc contenir un nom de compte Windows et son mot de passe. Les connexions SQL Server sont reliées à un credential par l'intermédiaire de l'instruction `CREATE LOGIN` ou bien `ALTER LOGIN`.

La modification et la suppression sont possibles par l'intermédiaire des instructions `ALTER CREDENTIAL` et `DROP CREDENTIAL`.

Syntaxe

```
CREATE CREDENTIAL nomDuCredit  
WITH IDENTITY = 'identité' [, SECRET = 'secret'];
```

`nomDuCredit`

Permet d'identifier le credential de façon unique au niveau du serveur. Cet identifiant ne peut pas commencer par le caractère `#`. Les credentials système commencent toujours par les caractères `##`.

`identitØ`

Nom du compte Windows qui sera associé au credential et permettra l'accès à des ressources en dehors de SQL Server.

`secret`

Optionnel, ce paramètre permet de spécifier le mot de passe associé au compte Windows.

Exemple

Un credential est créé et il correspond au compte d'utilisateur Antoine défini au niveau de Windows.

```
USE [master]
GO
CREATE CREDENTIAL [Antoine] WITH IDENTITY = N'Edienitek\Antoine', SECRET =
N'Pa$$w0rd'
GO
```

Il est possible d'avoir toutes les informations relatives aux credentials déjà définis sur le serveur en interrogeant la vue **sys.credentials**.

Exemple

Le script présenté ci-dessous permet de connaître les credentials définis et la connexion associée.

```
select * from sys.credentials
```

SQL Server Management Studio

La gestion d'un credential de façon graphique est possible en procédant de la façon suivante :

- Depuis l'explorateur d'objets, se positionner sur le nœud **Sécurité - Informations d'identification**.
- Depuis le menu contextuel associé au nœud **Informations d'identification**, demander la création d'un credential en choisissant l'option **Nouvelles informations d'identification....**

Nouvelles informations d'identification

Sélectionner une page

Général

Script ? Aide

Nom d'identification : Antoine

Identité : edienitek\antoine

Mot de passe :

Confirmer le mot de passe :

☐ Utiliser le fournisseur de services de chiffrement

Fournisseur

Connexion

Serveur : sql1

Connexion : EDIENITEK\administrator

[Afficher les propriétés de connexion](#)

Progression

Prêt

OK Annuler

Fenêtre de création d'un nouveau credential

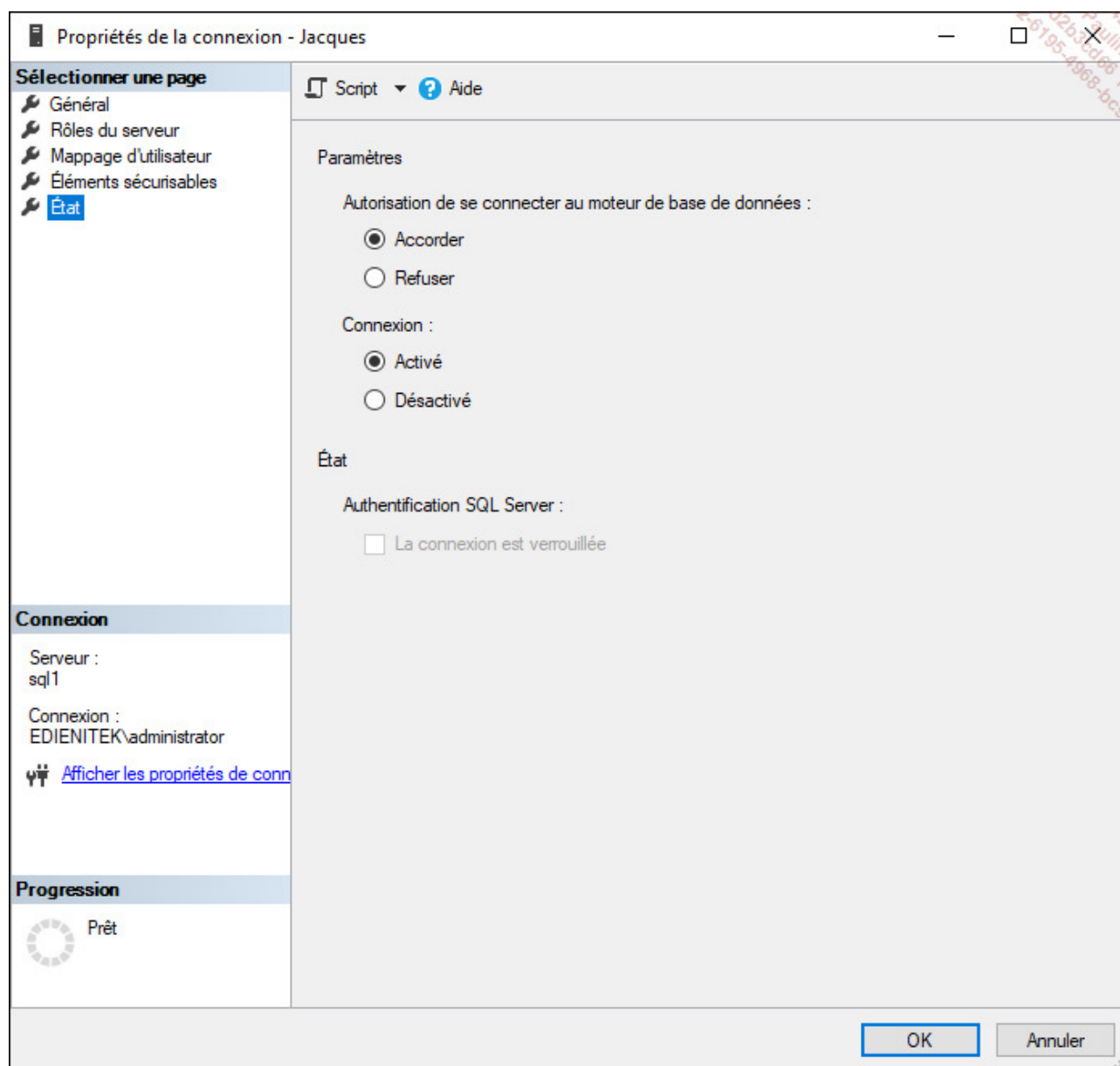
7. Activer et désactiver une connexion

Sur une connexion définie, il est possible de la désactiver afin de désactiver temporairement son utilisation. Si toutes les informations et propriétés de la connexion sont conservées, il n'est pas possible d'utiliser la connexion désactivée pour se connecter à SQL Server. La désactivation de connexion est utile pour renforcer la sécurité sur des

comptes de connexion qui, bien que définis, ne sont pas utilisés ou bien lorsque l'administrateur définit au préalable des connexions. Il ne lui restera plus qu'à activer la ou les connexions lorsque le besoin sera présent.

SQL Server Management Studio

Depuis la fenêtre des propriétés de la connexion sur la page **État**, il est possible d'activer ou de désactiver la connexion.



Dans cette même page de propriétés, il est également possible de refuser explicitement un accès (commande **DENY**). Ceci peut être utile par exemple lorsqu'un utilisateur est

membre de plusieurs groupes.

Transact SQL

L'activation et la désactivation d'une connexion s'effectuent par l'intermédiaire de l'instruction **ALTER LOGIN**.

Syntaxe

```
ALTER LOGIN nomConnexion {ENABLE|DISABLE};
```

Exemple

La connexion Pierre est désactivée.

```
alter login Pierre disable
```

8. Les informations relatives aux connexions

Toutes les informations relatives aux connexions sont enregistrées dans les tables système de la base master. Il est possible de connaître ces informations en interrogeant les vues système présentées ci-dessous. Le recours direct aux tables est fortement déconseillé afin de garantir la bonne exécution des scripts sur les différentes versions de SQL Server. Un script testé et validé fonctionnera sur les versions suivantes de SQL Server.

sys.server_principals

Liste des connexions définies sur le serveur.

sys.sql_logins

Liste des connexions définies avec le mode de sécurité SQL Server.

sys.server_permissions

Liste des autorisations accordées aux connexions directement sur le serveur.

`sys.server_role_member`

Liste des rôles de serveur et des connexions qui en bénéficient.

`sys.system_components_surface_area_configuration`

Retourne la liste des objets système susceptibles d'être visibles ou non en fonction de la configuration effectuée. Ainsi, si la visibilité est activée, un utilisateur peut afficher les métadonnées d'un objet même s'il ne dispose pas d'autorisation dessus.

Gestion des utilisateurs de base de données

Après la définition des connexions (login) au niveau du serveur, il est nécessaire de définir des utilisateurs dans les différentes bases de données.

C'est au niveau des utilisateurs de base de données que seront attribués les droits d'utilisation des objets définis dans la base de données. Lors de la définition d'une connexion, la base de données par défaut permet de positionner le compte de connexion sur une base de données pour commencer à travailler. Cependant, la connexion ne pourra réellement travailler sur la base que s'il existe un compte d'utilisateur défini au niveau base et associé à la connexion. C'est un point de passage obligatoire, sauf si la connexion s'est vue attribuée des privilèges de haut niveau.



Si aucune base de données par défaut n'est définie au niveau de la connexion, alors c'est la base Master qui est considérée comme base par défaut.

Les utilisateurs de base de données sont, en général, associés à une connexion au niveau du serveur. Cependant, l'utilisateur **guest** (qui est désactivé par défaut) n'est mappé à aucune connexion. Comme à n'importe quel compte utilisateur de base de données, des droits d'accès aux objets peuvent lui être accordés.

Le compte invité (**guest**) est activé sur les bases master et tempdb. Cela fait partie du mode de fonctionnement normal de SQL Server et il n'est pas possible de déroger à cette règle.

Si un utilisateur dispose d'une connexion à SQL Server mais s'il n'existe pas d'utilisateur de base de données lui permettant d'intervenir sur les bases, l'utilisateur ne peut réaliser que quelques opérations très limitées :

- Sélectionner les informations contenues dans les tables système et exécuter certaines procédures stockées.
- Accéder à toutes les bases de données utilisateur munies d'un compte

d'utilisateur **guest** activé.

- Exécuter les instructions qui ne nécessitent pas d'autorisation telles que la fonction **PRINT**.

Il existe deux types de droits : les droits d'utilisation des objets définis dans une base de données et les droits d'exécuter des instructions SQL qui vont ajouter ou modifier des objets à l'intérieur de la base.

Les utilisateurs de base de données sont liés à une connexion. Ce sont les utilisateurs de base de données qui reçoivent les différents droits qui permettent d'accéder aux objets des bases.

Pour être capable de gérer les accès à la base, il faut posséder des droits de propriétaire de base (**db_owner**) ou de gestionnaire des accès (**db_accessadmin**).



Les utilisateurs de base de données sont stockés dans la table système **sysusers** de la base de données sur laquelle l'utilisateur est défini.



À sa création un utilisateur de base de données ne dispose d'aucun privilège. Il est nécessaire d'accorder tous les droits dont l'utilisateur a besoin.

1. Créer un utilisateur

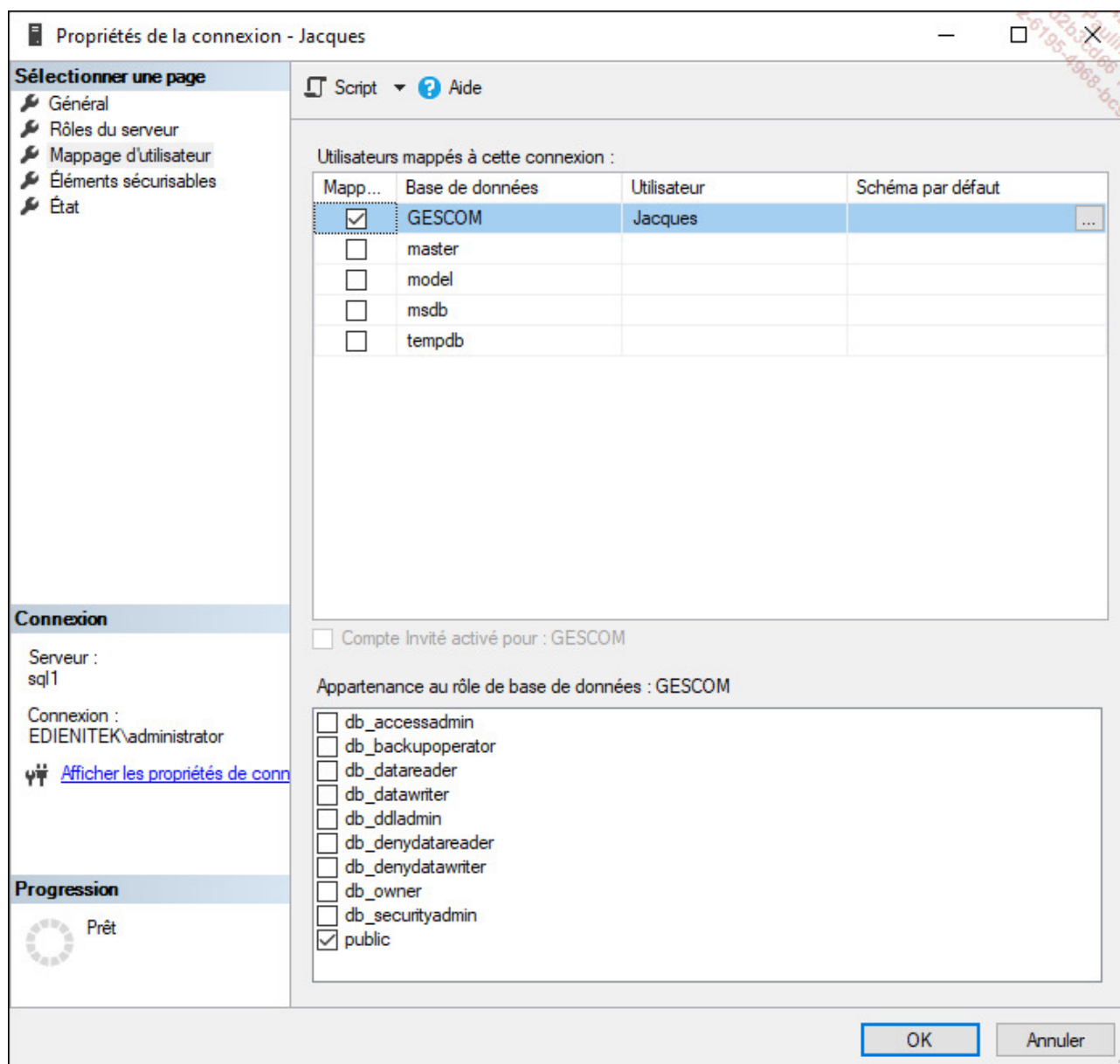
La création d'un utilisateur de base de données va permettre de lier une connexion avec un utilisateur, et donc autoriser l'utilisation de cette base.

SQL Server Management Studio

Pour créer un utilisateur de base de données, il y a plusieurs possibilités. L'une d'entre elles est de se positionner sur le compte de connexion associé et d'ouvrir sa fenêtre **Propriétés** :

- Aller sur la page **Mappage d'utilisateur**.

- Cocher toutes les bases auxquelles cette connexion aura accès : ainsi, un compte utilisateur de base de données sera créé pour chacune d'elles.
- Enfin, il est possible de préciser les rôles de base de données accordés à cet utilisateur.



Dans l'exemple précédent, l'utilisateur Jacques est créé dans la base de données GESCOM. Cet utilisateur est mappé à la connexion Jacques.

Transact SQL

C'est par l'intermédiaire de l'instruction `CREATE USER` qu'il est possible de définir des

comptes utilisateurs au niveau de la base de données.

Syntaxe

```
CREATE USER nomUtilisateur  
[ FOR {LOGIN nomConnexion |  
    CERTIFICATE nomCertificat|  
    ASYMMETRIC KEY nomCléAsymétrique}]  
[ WITH DEFAULT_SCHEMA = nomSchema ]
```

nomUtilisateur

Nom du nouvel utilisateur de base de données.

nomConnexion

Nom de la connexion à laquelle l'utilisateur de Base de données est associé. Si aucun nom de connexion n'est précisé, alors SQL Server résout le problème de la façon suivante :

- 1 : SQL Server tente d'associer le compte d'utilisateur de base de données à une connexion qui porte le même nom.
- 2 : si l'étape précédente échoue, si le nom d'utilisateur est **guest** et s'il n'existe pas encore de compte de **guest** sur la base de données, alors le compte d'utilisateur **guest** (invité) est créé.
- 3 : dans tous les autres cas l'instruction échoue.

nomCertificat

Nom du certificat à associer à l'utilisateur de base de données.

nomCléAsymétrique

Nom de la clé asymétrique associée à l'utilisateur de base de données.

nomSchema

Nom du schéma associé à cet utilisateur de base de données. Il est possible d'associer plusieurs utilisateurs au même schéma.

Exemple

```
USE [GESCOM]
GO
CREATE USER [Jacques] FOR LOGIN [Jacques]
GO
```



Les procédures `sp_grantdbaccess` et `sp_adduser` sont maintenues dans SQL Server uniquement pour des raisons de compatibilité. Il est recommandé de ne plus l'utiliser.

2. Information

SQL Server Management Studio

Pour obtenir l'ensemble des informations relatives à un utilisateur de base de données, il faut procéder de la façon suivante :

- Depuis l'explorateur d'objets, se positionner sur la base de données.
- Développer les nœuds **Sécurité - Utilisateurs**.
- Se positionner sur l'utilisateur pour lequel on souhaite obtenir les informations et demander l'affichage des propriétés à partir du menu contextuel associé.

Utilisateur de la base de données - Jacques

Sélectionner une page

- Général
- Schémas appartenant à un rôle
- Appartenance
- Éléments sécurisables
- Propriétés étendues

Script ? Aide

Type d'utilisateur :
Utilisateur SQL avec connexion

Nom d'utilisateur :
Jacques

Nom de connexion :
Jacques

Schéma par défaut :
dbo

Connexion

Serveur :
sql1

Connexion :
EDIENITEK\administrator

[Afficher les propriétés de connexion](#)

Progression

Prêt

OK Annuler

Transact SQL

Il est possible d'obtenir l'ensemble des informations relatives aux utilisateurs de base de données en interrogeant la vue **sys.database_principals**.

```
USE [GESCOM]
GO
select * from sys.database_principals
```



La procédure stockée `sp_helpuser` est maintenue pour des raisons de compatibilité mais il faut lui préférer la vue **`sys.database_principals`**.

La procédure `sp_who` permet de connaître les utilisateurs actuellement connectés et les processus en cours.

Pour chaque connexion, la procédure `sp_who` permet, entre autres, de connaître :

- Le nom de connexion utilisé (loginame).
- Le nom de l'ordinateur à partir duquel la connexion est établie (hostname).
- Le nom de la base de données courante (dbname).

The screenshot shows the Microsoft SQL Server Management Studio interface. The query editor contains the command `sp_who`. The Results pane displays the following data:

	spid	ecid	status	loginame	hostname	blk	dbname	cmd	request_id
1	1	0	background	sa		0	NULL	PARALLEL REDO TASK	0
2	2	0	background	sa		0	NULL	PARALLEL REDO TASK	0
3	3	0	background	sa		0	NULL	XIO_LEASE_RENEWAL_WORKER	0
4	4	0	background	sa		0	NULL	XIO_RETRY_WORKER	0
5	7	0	background	sa		0	NULL	XTP_CKPT_AGENT	0
6	8	0	background	sa		0	NULL	RECOVERY WRITER	0
7	9	0	background	sa		0	NULL	PVS_PREALLOCATOR	0
8	10	0	background	sa		0	NULL	LAZY WRITER	0
9	11	0	background	sa		0	NULL		0
10	12	0	background	sa		0	NULL	LOG WRITER	0
11	13	0	background	sa		0	NULL	LOCK MONITOR	0
12	14	0	background	sa		0	master	SIGNAL HANDLER	0
13	15	0	background	sa		0	master	BRKR TASK	0
14	16	0	sleeping	sa		0	master	TASK MANAGER	0
15	17	0	background	sa		0	NULL	RECEIVE	0
16	20	0	background	sa		0	master	TRACE QUEUE TASK	0

The status bar at the bottom indicates: "Exécution de requête réussie. | SQL1 (16.0 RTM) | EDIENITEK\administrato... | master | 00:00:00 | 44 lignes".

3. Établir la liste des connexions et des utilisateurs associés

Pour garantir la sécurité de l'instance SQL Server, il est bien sûr nécessaire de contrôler qui peut avoir accès à l'information. Très rapidement, la situation devient complexe car le nombre d'utilisateurs et de bases de données va en grandissant. Il est donc souvent intéressant d'établir la liste des connexions, des utilisateurs de base de données correspondant à ces connexions, mais aussi des privilèges accordés à ces différents utilisateurs.

Pour établir cette liste, le plus simple consiste à utiliser les vues système afin de rendre ce processus automatique. Cette requête va donc s'appuyer sur les vues système **sys.database_principals** à l'intérieur d'une base de données pour obtenir la liste des utilisateurs qui y sont définis et la vue **sys.server_principals** qui fournit la liste des connexions définies.

Exemple

La requête suivante permet d'établir la liste des utilisateurs de la base de données Gescom ainsi que la connexion qui leur est associée.

```
Use GESCOM ;
GO
SELECT s.name as "Connexion", p.name as "Utilisateur"
FROM sys.database_principals p
INNER JOIN sys.server_principals s
ON s.sid=p.sid;
```

La vue **sys.databases** sera également utilisée afin de parcourir toutes les bases présentes sur l'instance SQL Server.

Le script ci-dessous permet de faire cette liste complète. Pour cela, un curseur est utilisé pour parcourir toutes les bases. Afin de limiter la liste retournée et donc de gagner en clarté, seules les bases de données utilisateurs sont prises en compte, ce qui explique l'exclusion des bases master, model, tempdb et msdb. Pour chaque valeur retournée par le curseur, la requête vue précédemment est exécutée. Cependant, comme une partie de la requête est dynamique (le nom de la base), il est nécessaire de construire la requête sous

forme de chaîne de caractères (ce qui est fait en valorisant la variable `@rqt`).

Pour l'exécution de cette requête dynamique, la procédure stockée **sp_executesql** est mise à contribution.

```

DECLARE cLesBases cursor for
select name
from sys.databases
where name not in('master','model','tempdb','msdb');

DECLARE @nomBase sysname;
DECLARE @rqt nvarchar(500)
BEGIN
    OPEN cLesBases;
    FETCH cLesBases INTO @nomBase;
    WHILE (@@FETCH_STATUS=0) BEGIN
        set @rqt='SELECT '''+@nomBase+''' as Base,'
        set @rqt=@rqt+'s.name as Connexion, p.name as Utilisateur '
        set @rqt=@rqt+'FROM master.sys.server_principals p '
        set @rqt=@rqt+'INNER JOIN '''+@nomBase+'''.sys.database_principals s '
        set @rqt=@rqt+'ON s.sid=p.sid'
        exec sp_executesql @rqt
        FETCH cLesBases INTO @nomBase;
    END;
    CLOSE cLesBases;
    DEALLOCATE cLesBases
END;

```



Ces scripts, comme tous ceux présentés dans ce livre, sont disponibles en téléchargement depuis la page Informations générales.

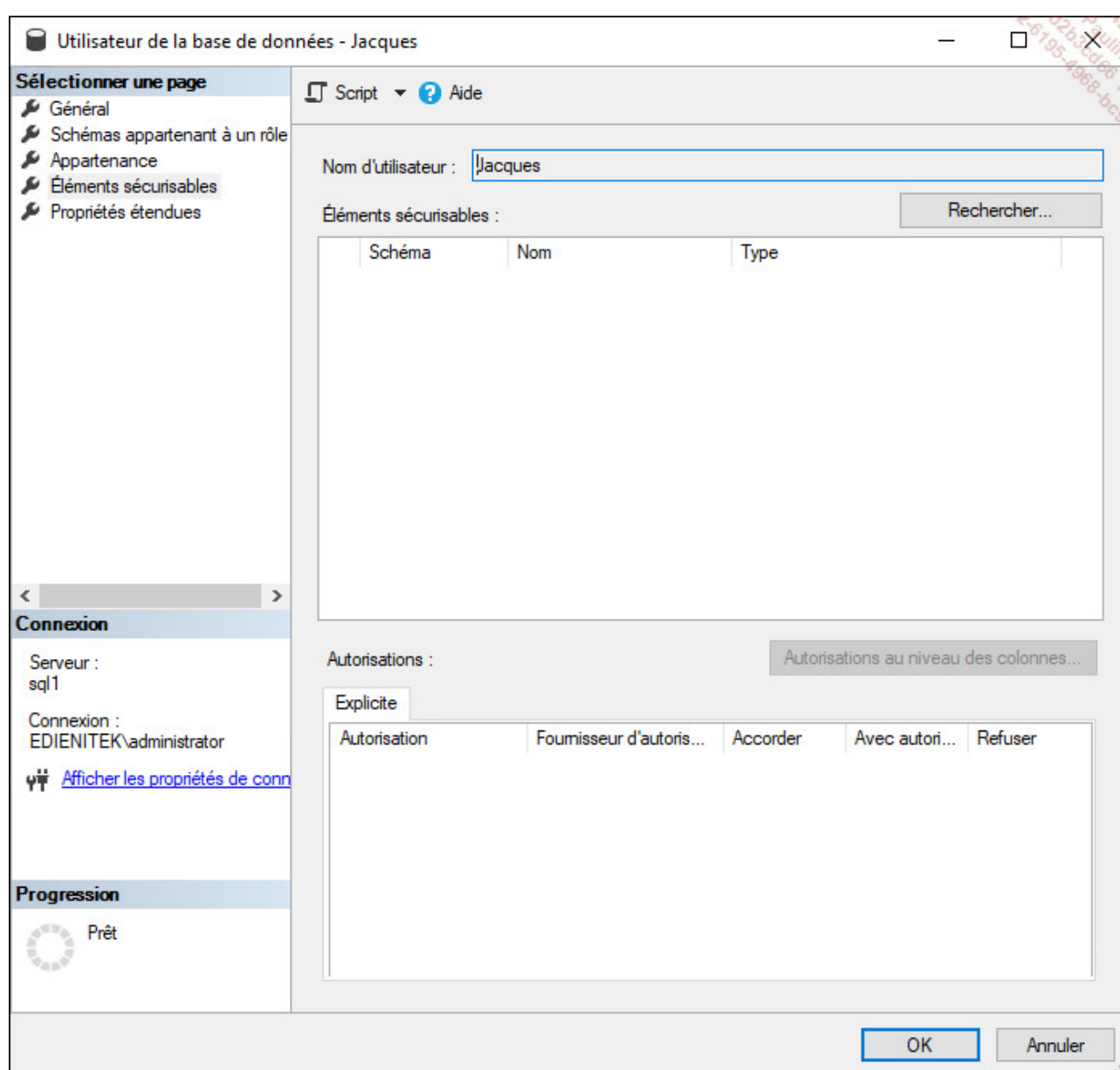
4. Modification

Il est possible de modifier un utilisateur de base de données afin de modifier son nom ou bien le schéma associé à cet utilisateur.

SQL Server Management Studio

Il faut se positionner depuis l'explorateur d'objets sur la base de données concernée par la modification du compte d'utilisateur et réaliser les manipulations suivantes :

- Développer le nœud **Sécurité** puis **Utilisateurs**.
- Se positionner sur le compte d'utilisateur à modifier.
- Afficher la boîte de propriétés depuis l'option **Propriétés** disponible dans le menu contextuel associé à l'utilisateur.



Transact SQL

L'instruction `ALTER USER` permet de réaliser cette opération.

Syntaxe

```
ALTER USER nomUtilisateur  
WITH NAME=nouveauNomUtilisateur,  
      DEFAULT_SCHEMA = nomNouveauSchema
```

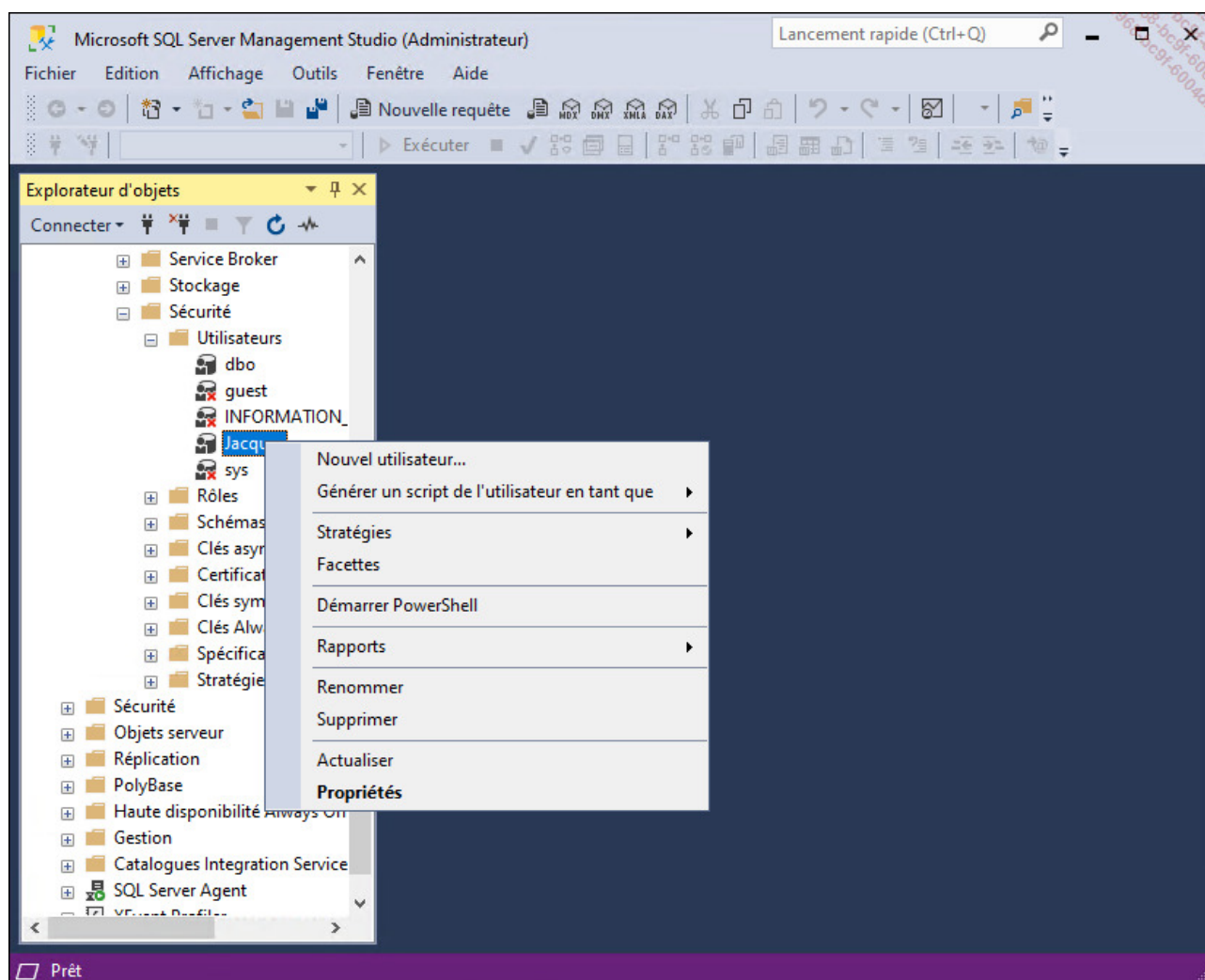
5. Suppression

Compte tenu que les utilisateurs ne possèdent pas d'objets, leur suppression est toujours possible et sans aucun risque de perte de données.

SQL Server Management Studio

Il faut se positionner depuis l'explorateur d'objets sur la base de données concernée par la suppression du compte d'utilisateur et réaliser les manipulations suivantes :

- Développer le nœud **Sécurité** puis **Utilisateurs**.
- Se positionner sur le compte d'utilisateur à supprimer.
- Sélectionner le choix **Supprimer** depuis le menu contextuel associé au compte d'utilisateur.



La suppression n'est pas possible sur les comptes d'utilisateur prédéfinis à savoir : dbo, guest, INFORMATION_SCHEMA et sys.

Transact SQL

L'instruction `DROP USER` permet de réaliser cette opération.



Les procédures `sp_dropuser` et `sp_revokedbaccess` sont maintenues dans SQL Server uniquement pour des raisons de compatibilité.

Syntaxe

```
DROP USER nomUtilisateur
```

Exemple

```
use gescom;
go
drop user Jacques;
```

6. Les comptes utilisateurs sans connexion

Afin de rendre les bases de données indépendantes du serveur sur lequel elles se trouvent, SQL Server propose de créer des comptes utilisateurs de bases de données n'ayant pas besoin de comptes de connexion associés. Ainsi, il devient beaucoup plus facile et donc rapide de transférer la base de données d'un serveur vers un autre : plus besoin de créer les comptes de connexion et de les associer aux comptes utilisateurs. Pour rendre possible cette opération, il est nécessaire d'activer l'option 'contained database authentication' au niveau du serveur :

```
sp_configure 'contained database authentication',1
reconfigure
```

Ensuite, il faut modifier l'option **CONTAINMENT** de la base de données vers la valeur **PARTIAL** :

```
USE [master];
GO
ALTER DATABASE [GESCOM] SET CONTAINMENT = PARTIAL WITH NO_WAIT ;
GO
```

Par défaut, les bases de données créées avec SQL Server ou bien issues d'une version antérieure le sont avec la valeur **NONE**.

Il ne reste plus qu'à créer les comptes utilisateurs :

```
use GESCOM ;  
go  
-- utilisateur SQL  
CREATE USER test with PASSWORD='Pa$$w0rd' ;  
GO  
-- utilisateur Windows  
CREATE USER [Edienitek\amartin] ;  
GO
```

Par contre, lors de l'utilisation de ce type de compte utilisateur, il faudra absolument spécifier une base de données dans la chaîne de connexion, car la connexion ne s'effectue pas sur le serveur SQL Server, mais directement sur la base de données. Dans la fenêtre de connexion de SQL Server Management Studio, cliquez sur le bouton **Options** en bas à droite, puis sur l'onglet **Propriétés de connexion**, et spécifiez la base dans la liste de sélection :

Se connecter au serveur

SQL Server

Connexion Propriétés de connexion Always Encrypted Paramètres de connexion su

Tapez ou sélectionnez le nom de la base de données pour la connexion.

Se connecter à la base de données :

Réseau

Protocole réseau :

Taille du paquet réseau : octets

Connexion

Délai de connexion : secondes

Délai d'exécution : secondes

☐ Chiffrer la connexion

☐ Faire confiance au certificat du serveur

☐ Utiliser une couleur personnalisée : Sélectionner...

Réinitialiser tout

Connexion Annuler Aide Options <<

Gestion des schémas

L'objectif des schémas est de faciliter la gestion de la sécurité d'accès des utilisateurs vers les objets (tables, vues, procédures stockées, fonctions). Un peu comme sur un serveur de fichiers avec des dossiers et sous-dossiers où les fichiers vont être stockés, les droits d'accès sont posés en priorité sur les dossiers, ce qui sera plus simple que fichier par fichier. Pour le cas de SQL Server, les droits peuvent être posés sur les bases de données, les schémas et éventuellement sur les objets pour gérer une exception.

Dans le cas où on ne trouve pas de découpage logique de schémas pour une base de données, les objets seront placés dans le schéma par défaut à savoir dbo.

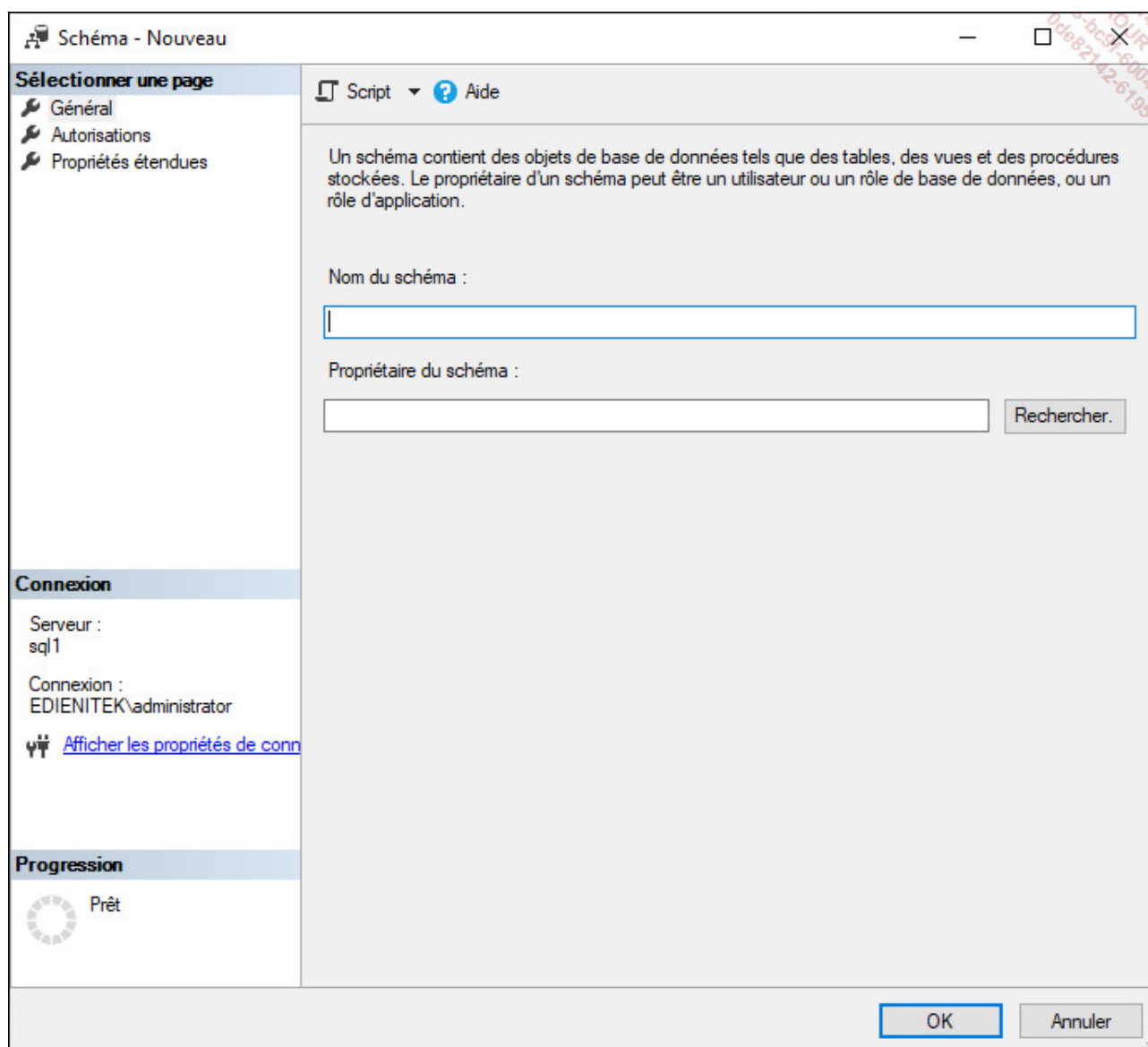
Pour accéder aux objets, même s'ils sont stockés dans le schéma par défaut de l'utilisateur, il est préférable de les nommer en utilisant leur schéma, c'est-à-dire nomSchema.nomObjet. Lors de l'utilisation d'un nom court (simplement le nom de l'objet sans le préfixer par le nom du schéma), SQL Server recherche l'existence de l'objet dans le schéma par défaut de l'utilisateur et ensuite dans le schéma dbo, donc si le nom de schéma est spécifié, cette étape n'a pas lieu d'être et cela optimise le temps de réponse des requêtes.

1. Création

SQL Server Management Studio

Pour créer un schéma de base de données, il faut se positionner sur la base de données concernée par l'ajout et depuis l'explorateur d'objets réaliser la manipulation suivante :

- Développer le nœud **Sécurité** et se positionner sur le nœud **Schémas**.
- Depuis le menu contextuel associé au nœud **Schémas**, sélectionner **Nouveau schéma**.



Transact SQL

```
CREATE SCHEMA nomSchema
AUTHORIZATION nomPropriétaire[
definitionDesTables |
definitionDesVues |
gestionDePrivilèges]
```

Plus simplement, il est possible de dire qu'un schéma est caractérisé par son nom. L'option **AUTHORIZATION** permet de définir l'utilisateur de base de données qui est propriétaire du schéma. Un même utilisateur de base de données peut être le propriétaire de plusieurs

schémas. Le propriétaire d'un schéma ne doit pas nécessairement avoir comme schéma par défaut un schéma dont il est propriétaire. Il est également possible de créer les tables et les vues d'un schéma dès la construction du schéma.

Dans ce cas, et comme pour toutes les opérations du DDL, l'action de l'instruction **CREATE** est indivisible dans le sens où soit tous les objets sont créés, soit aucun. Ainsi, un problème de création sur une table ou une vue va empêcher la création entière du schéma.

Exemple

Dans l'exemple suivant, le schéma RI est créé.

```
use gescom;
go
create schema RI
authorization dbo;
```

Dans ce second exemple, le schéma SchemaExemple est défini ainsi que des tables et des vues spécifiques à ce schéma.

```
use gescom;
go
create schema schemaexemple
authorization dbo
create table articles
(reference nvarchar(8) constraint pk_articles primary key,
designation nvarchar(200),
prixht money,
tva numeric(4,2))
create view catalogue as
select reference, designation, prixht, tva, prixht*(1+tva)/100 as ttc
from articles ;
```

2. Modification

La modification d'un schéma consiste à modifier les objets contenus dans ce schéma. Lorsqu'un objet est transféré d'un schéma à un autre, les autorisations relatives à cet objet sont perdues. Le transfert d'un objet entre deux schémas est possible à l'intérieur d'une même base de données.

Le transfert d'un objet d'un schéma à un autre entraîne néanmoins quelques conséquences. Si le propriétaire n'était pas explicitement nommé, c'est-à-dire que l'objet appartenait au propriétaire du schéma (SCHEMA OWNER), alors après transfert c'est le propriétaire du schéma cible qui devient le propriétaire de l'objet. D'autre part, si des autorisations d'utilisation de l'objet avaient été définies avant le transfert vers le nouveau schéma, ces autorisations sont supprimées lors du transfert.

Cette action n'est possible qu'en Transact SQL.

Transact SQL

```
ALTER SCHEMA nomSchema TRANSFER nomObjet
```

`nomObjet`

Représente le nom de l'objet qui va être transféré dans le schéma courant. Le nom de cet objet est au format suivant `nomSchema.nomCourtObjet`.

Exemple

Dans l'exemple suivant, la table catalogue présente dans le schéma schemaexemple est transférée vers le schéma ri.

```
use gescom;  
go  
alter schema ri  
transfer schemaexemple.catalogue;
```

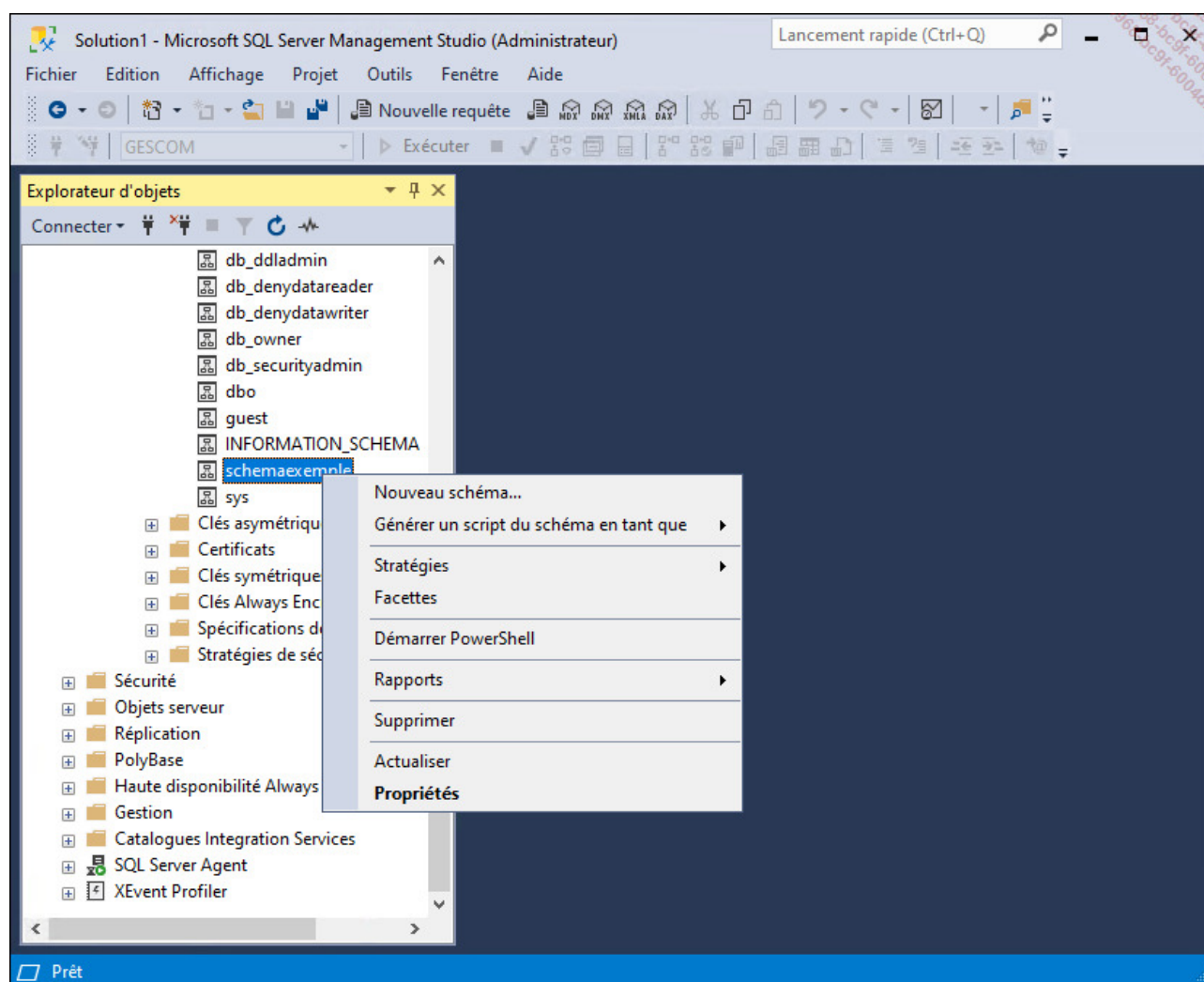
3. Suppression

La suppression d'un schéma est possible pour les schémas qui ne contiennent pas d'objet. Par exemple, si le schéma contient des tables ou des vues, il est nécessaire de supprimer ou de transférer vers un autre schéma l'ensemble des tables et des vues avant de pouvoir supprimer le schéma courant.

SQL Server Management Studio

Pour supprimer un schéma de base de données, il faut se positionner sur la base de données concernée par la suppression et depuis l'explorateur d'objets réaliser la manipulation suivante :

- Développer le nœud **Sécurité** et se positionner sur le nœud **Schéma**.
- Se positionner sur le schéma à supprimer.
- Sélectionner le choix **Supprimer** depuis le menu contextuel associé au schéma.



Transact SQL

```
DROP SCHEMA nomSchema
```

Exemple

```
USE [GESCOM]  
GO  
DROP SCHEMA [RI]  
GO
```

4. Les informations relatives aux schémas

Il est possible de retrouver les informations relatives à chaque schéma en interrogeant la vue **sys.schemas**. Cette vue est structurée avec les colonnes suivantes :

Name

Représente le nom du schéma. Il ne peut y avoir deux schémas portant le même nom dans une même base de données.

Schema_id

Identifiant numérique attribué au schéma.

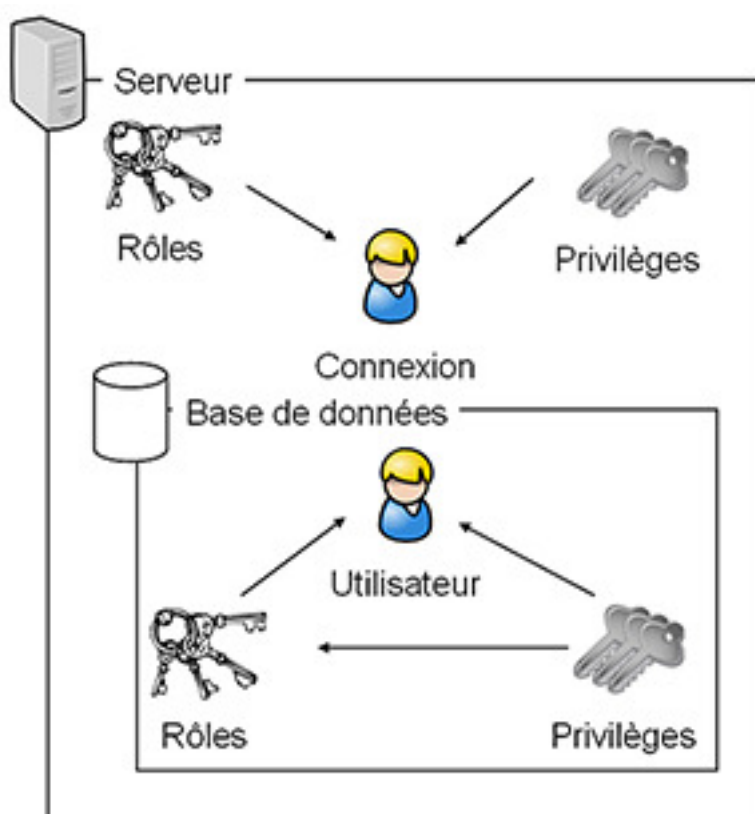
Principal_id

Identifiant numérique correspondant au propriétaire du schéma.

Gestion des droits

Tous les utilisateurs de base de données, y compris **guest** (l'invité), appartiennent au groupe **public**. Les droits qui vont être détaillés ci-dessous peuvent bien sûr être accordés directement à **public**.

Les droits sont organisés de façon hiérarchique par rapport aux éléments sécurisables du serveur.



Il est possible de gérer l'attribution de privilèges au niveau du serveur, de la base de données, du schéma ou bien directement de l'objet. Ainsi, les privilèges peuvent être accordés soit à un utilisateur de base de données, soit à une connexion.

Les droits accordés au niveau du serveur sont particuliers et relèvent plus de droits pour des administrateurs délégués. Au niveau bases de données, il existe également des droits concernant des accès administratifs, mais aussi des droits concernant l'accès des utilisateurs aux objets. Ces mêmes droits peuvent également être posés sur les schémas ou directement sur les objets (il y a héritage des droits sur ces trois niveaux de la même manière que sur un serveur de fichiers entre les dossiers et les fichiers).

SQL Server gère les privilèges avec les trois commandes suivantes :

- GRANT
- REVOKE
- DENY

C'est-à-dire qu'un privilège peut être accordé (GRANT) ou bien retiré (REVOKE) s'il a été accordé.

L'instruction DENY permet d'interdire l'utilisation d'un privilège particulier même si le privilège en question a été accordé soit directement, soit par l'intermédiaire d'un rôle.

1. Droits d'utilisation d'instructions

Les droits d'utilisation des instructions SQL pour créer de nouveaux objets au sein de la base sont des autorisations pour réaliser l'exécution de certains ordres SQL. Un utilisateur qui dispose de tels droits est capable par exemple de créer ses propres tables, ses procédures... L'accord de ces droits peut être dangereux et comme pour tous les droits, doivent être accordés uniquement lorsque cela est nécessaire.

Les droits principaux d'instructions disponibles sont :

- CREATE DATABASE
- CREATE PROCEDURE
- CREATE FUNCTION
- CREATE TABLE
- BACKUP DATABASE
- CREATE VIEW
- BACKUP LOG



Il est possible d'obtenir une vue graphique de toutes les permissions disponibles sur SQL Server en téléchargeant le document au format PDF disponible sur le site de Microsoft à l'URL suivante : <https://github.com/Microsoft/sql-server-samples/tree/master/samples/features/security/permissions-posters>

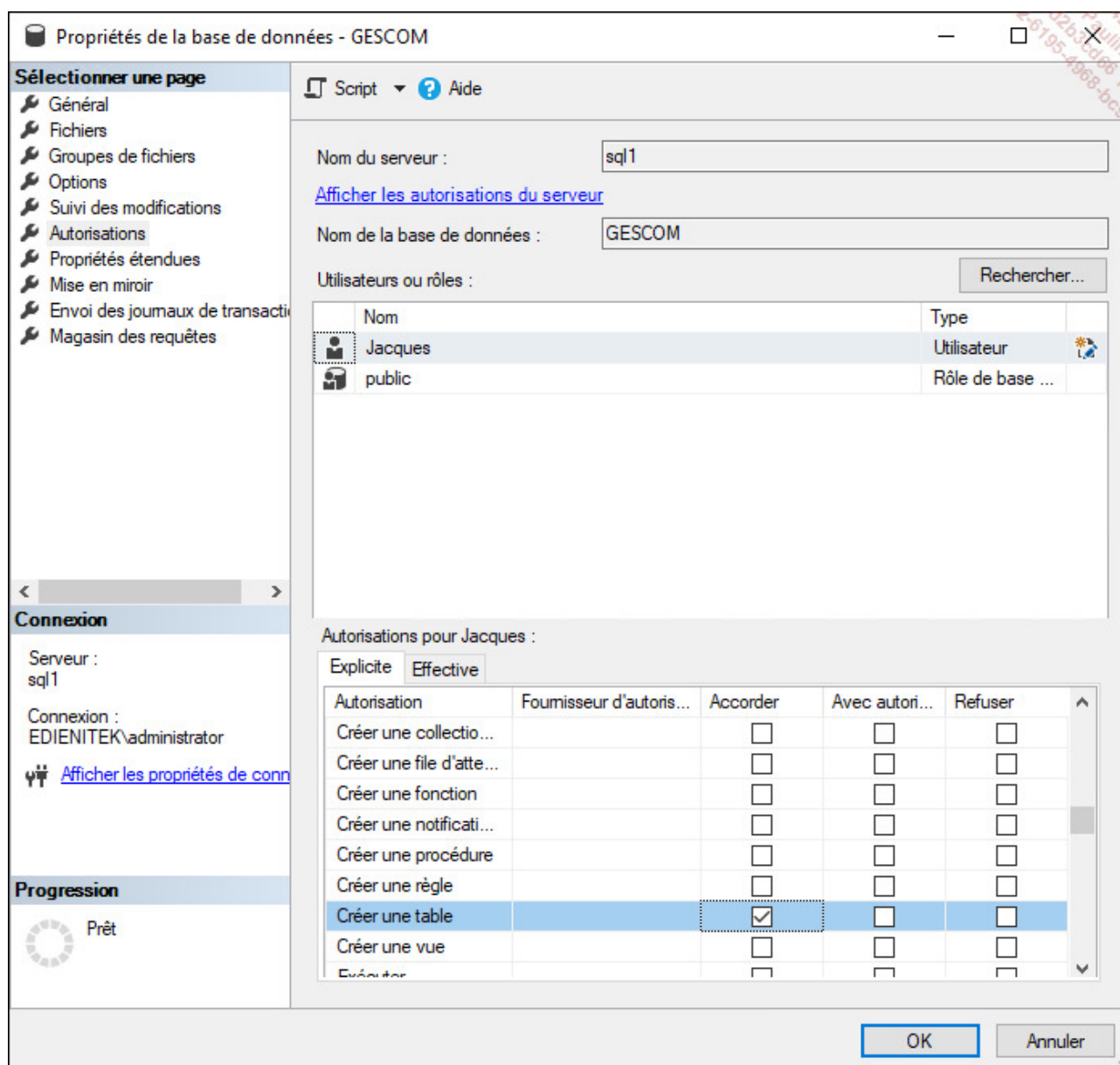
a. Autoriser

SQL Server Management Studio

Ces droits sont administrés au niveau de la base de données par l'intermédiaire de la fenêtre des propriétés.

Exemple

Le privilège **CREATE TABLE** est accordé à l'utilisateur de base de données Jacques par l'intermédiaire de la boîte de propriétés de la base GESCOM.



Transact SQL

L'accord de privilèges s'effectue en utilisant l'instruction **GRANT** dont la syntaxe est détaillée ci-dessous.

```
GRANT permission [...]  
TO utilisateur[...]  
[ WITH GRANT OPTION ]
```

permission

Nom de la ou des permissions concernées par cette autorisation. Il est également possible d'utiliser le mot-clé **ALL** à la place de citer explicitement la ou les permissions accordées. Toutefois ce terme **ALL** ne permet pas d'accorder des privilèges d'exécution de toutes les instructions mais simplement sur les instructions pour créer des bases de données, des tables, des procédures, des fonctions, des tables vues ainsi que d'effectuer des sauvegardes de la base et du journal.

utilisateur

Nom de l'utilisateur ou des utilisateurs de base de données qui reçoivent les permissions.

WITH GRANT OPTION

Si la permission est reçue avec ce privilège, alors l'utilisateur peut accorder la permission à d'autres utilisateurs de base de données.

Exemple

Dans l'exemple suivant, le privilège **CREATE VIEW** est accordé à l'utilisateur de base de données Jacques.

```
use GESCOM
GO
GRANT CREATE VIEW TO Jacques
GO
```

b. Retirer

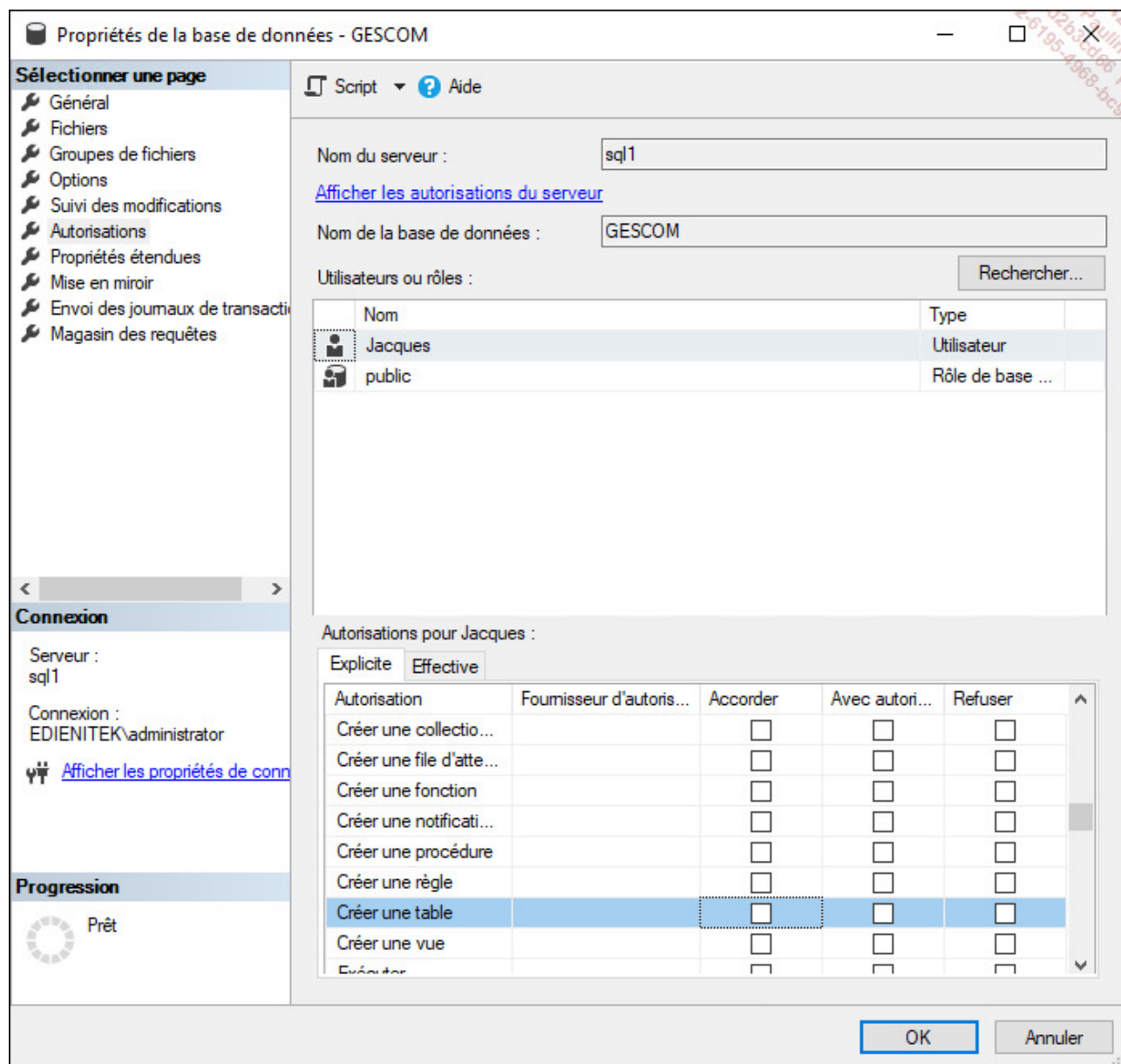
Il est possible de retirer un privilège qui a été accordé à une entité de sécurité. Si le privilège n'a pas été accordé à l'entité de sécurité, l'instruction est sans effet.

SQL Server Management Studio

La gestion des privilèges s'effectue au niveau des propriétés de la base de données.

Exemple

La permission **CREATE TABLE** est retirée à l'utilisateur Jacques.



Transact SQL

```
REVOKE [GRANT OPTION FOR]
    permission [...]
FROM utilisateur [...]
[CASCADE]
```

GRANT OPTION FOR

Si la permission a été accordée avec le privilège d'administration `GRANT OPTION`, il est possible par l'intermédiaire de cette option de retirer simplement le paramètre d'administration.

`permission`

Liste des permissions à retirer.

`utilisateur`

Liste des utilisateurs concernés par cette suppression.

`CASCADE`

Si la permission retirée a été accordée avec le privilège d'administration `WITH GRANT OPTION`, l'option `CASCADE` permet de retirer le privilège aux utilisateurs qui l'ont reçu par l'intermédiaire de l'utilisateur qui est concerné par la suppression initiale de privilège.

Exemple

Le privilège `CREATE VIEW` est retiré à l'utilisateur Jacques.

```
use GESCOM
GO
REVOKE CREATE VIEW TO Jacques
GO
```

c. Interdire

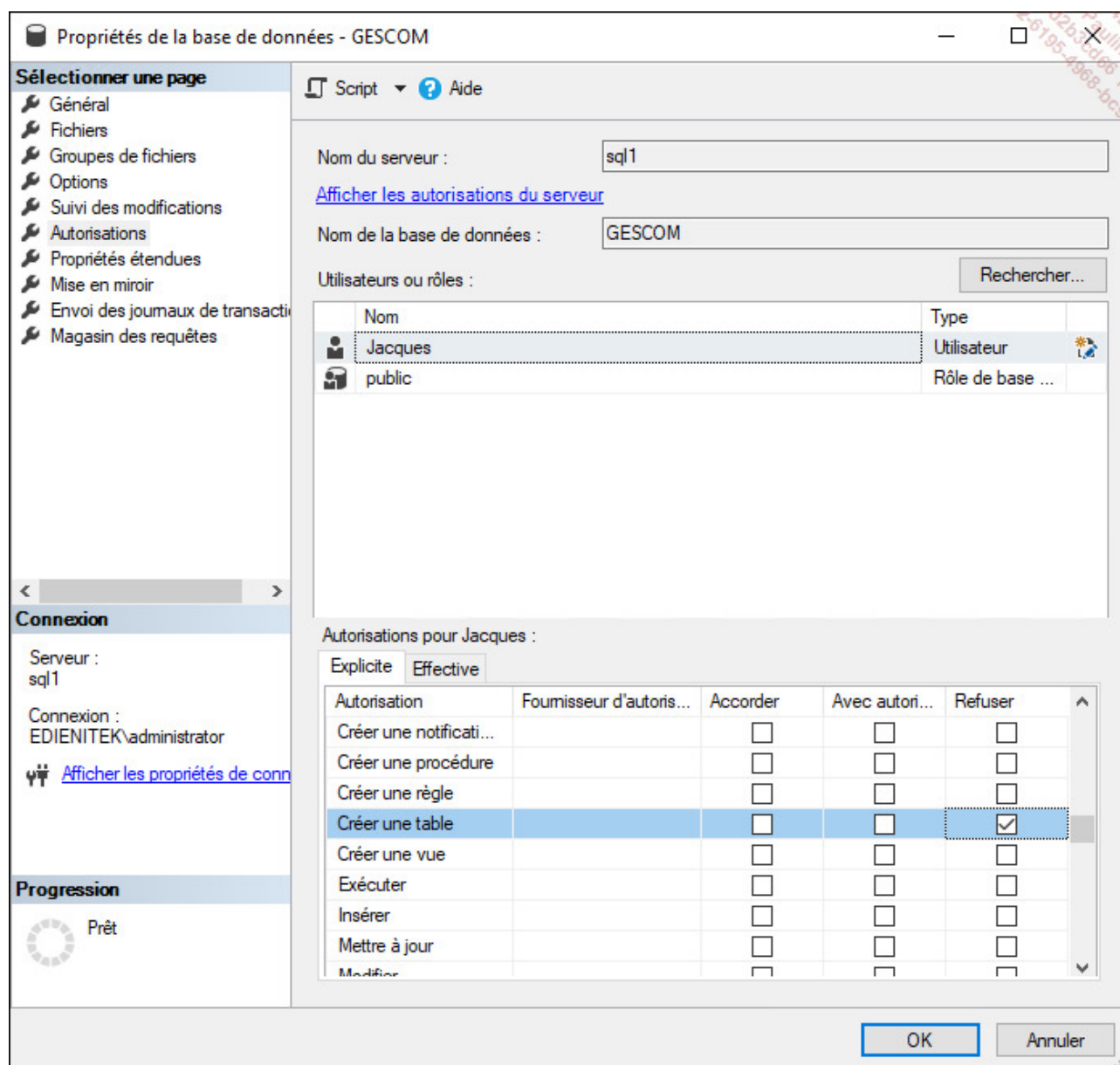
L'instruction `DENY` permet d'interdire à un utilisateur l'utilisation d'un privilège, même s'il en reçoit la permission soit directement, soit par son appartenance à un groupe.

SQL Server Management Studio

Comme pour l'accord et la suppression, ce type de privilège est géré de façon centralisée au niveau des propriétés de la base de données.

Exemple

Par l'intermédiaire des propriétés de la base de données, l'utilisateur Jacques se voit interdire la création de table.



Transact SQL

DENY permission [...]
 TO utilisateur [...]
 [CASCADE]

Exemple

L'utilisateur Jacques se voit interdire la possibilité de créer des vues.

```
use GESCOM
GO
DENY CREATE VIEW TO Jacques
GO
```

2. Droits d'utilisation des objets

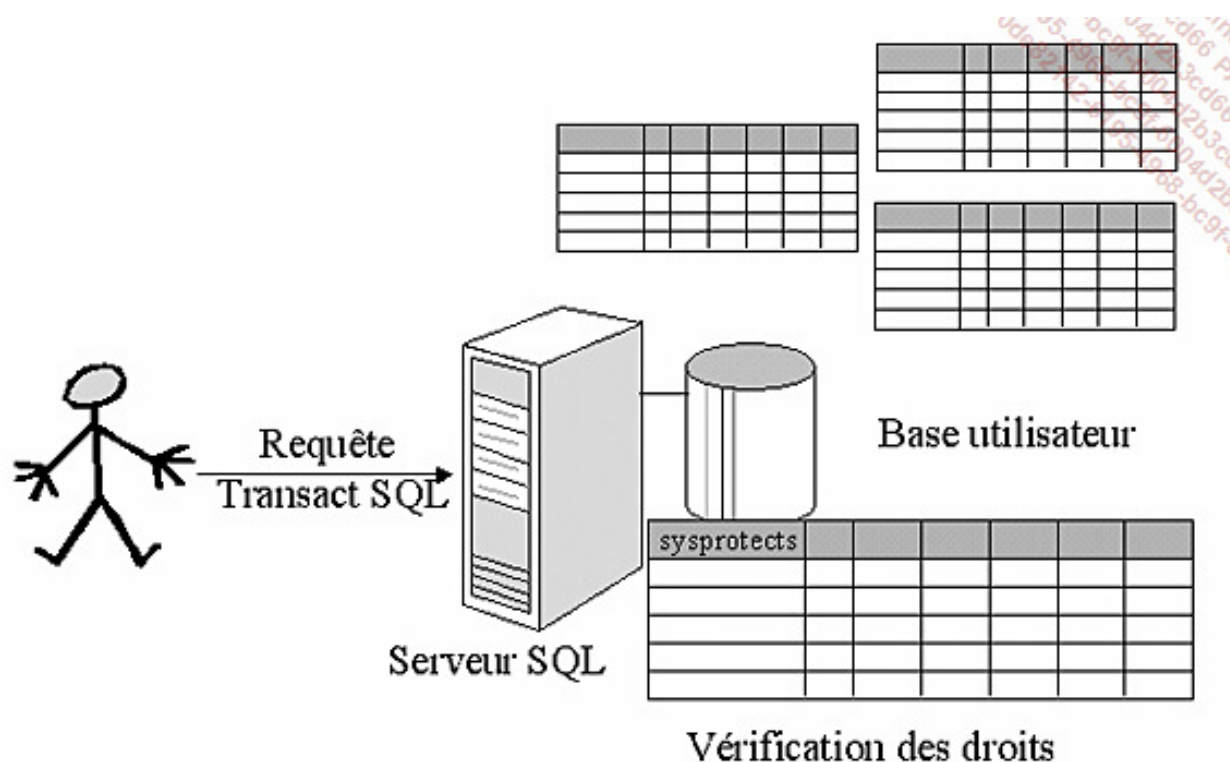
Ces droits permettent de fixer quelles opérations (lecture, modification, ajout ou suppression) l'utilisateur peut réaliser sur des données contenues dans une table, ou bien donner le droit d'exécution d'une procédure stockée.

En ce qui concerne le droit de lecture des données contenues dans une table (utilisation de l'instruction **SELECT**), il est possible de préciser quelles colonnes l'utilisateur peut visualiser. Par défaut, il s'agit de toutes les colonnes.

Les principaux droits d'utilisation des objets concernant les tables, vues, procédures stockées correspondent aux ordres :

- **INSERT**
- **UPDATE**
- **DELETE**
- **SELECT** (qui s'utilise pour les tables, les vues et les fonctions table)
- **EXECUTE** (qui s'utilise pour les procédures stockées et les fonctions scalaires).

Les instructions **SELECT** et **UPDATE** peuvent être limitées à certaines colonnes de la table ou de la vue. Il est cependant préférable de ne pas trop explorer cette possibilité car il en résulte un plus grand travail administratif au niveau de la gestion des droits. Il est préférable de passer par une vue ou bien une procédure stockée pour limiter l'usage de la table.



Principe de fonctionnement des autorisations

a. Autoriser

SQL Server Management Studio

Les privilèges d'utilisation des objets peuvent être gérés à deux emplacements dans SQL Server Management Studio :

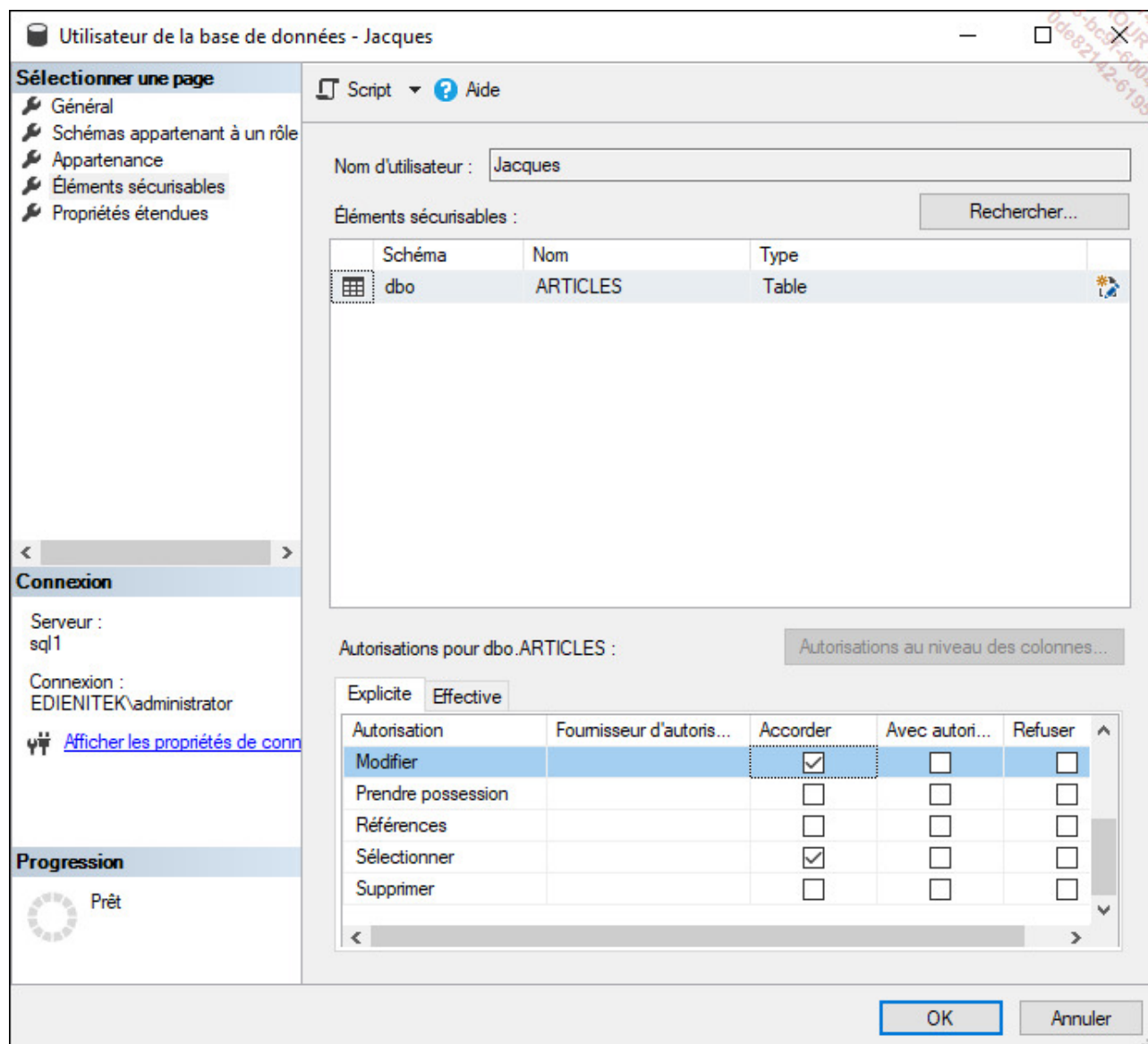
- Au niveau utilisateur, ce qui permet de connaître les possibilités de travail que possède chaque utilisateur.
- Au niveau des objets, afin de connaître quels sont les utilisateurs qui peuvent utiliser l'objet en question et comment ils peuvent l'utiliser.

Dans chacun des deux cas, les autorisations sont gérées par l'intermédiaire de la fenêtre des propriétés.

Exemple

Dans l'exemple suivant, l'utilisateur Jacques se voit accorder la possibilité d'exécuter les

instructions **UPDATE** et **SELECT** sur la table `dbo.Articles`.



Transact SQL

```
GRANT {ALL [PRIVILEGES]}permission[(colonne,...)] [...]}
ON objet[...]
TO utilisateur [...]
[WITH GRANT OPTION ]
```

ALL

Permet d'accorder tous les privilèges d'utilisation de l'objet. Les privilèges accordés sont toujours fonction du type de l'objet concerné par cet accord de privilège.

PRIVILEGES

Le mot-clé a été ajouté pour le respect de la norme ANSI-92. Il n'apporte aucune modification quant à l'utilisation du mot-clé **ALL**.

permission

Permet de spécifier la ou les opérations qui seront accordées aux utilisateurs.

objet

Correspond au nom complet de l'objet ou des objets sur lesquels porte l'accord de privilège d'utilisation.

utilisateur

Correspond à l'utilisateur, ou plus exactement à l'entité de sécurité qui va pouvoir bénéficier du ou des privilèges.

WITH GRANT OPTION

Le privilège est accordé avec une option d'administration qui autorise le bénéficiaire du privilège à accorder ce même privilège à d'autres utilisateurs de la base de données.

Exemple

Dans l'exemple ci-dessous, l'utilisateur Jacques reçoit tous les privilèges d'utilisation sur la table des commandes ainsi que la possibilité de mener des requêtes d'interrogation sur la table des articles.

```
use [GESCOM]
GO
grant all on dbo.commandes to Jacques;
grant select on dbo.articles to Jacques;
```

b. Retirer

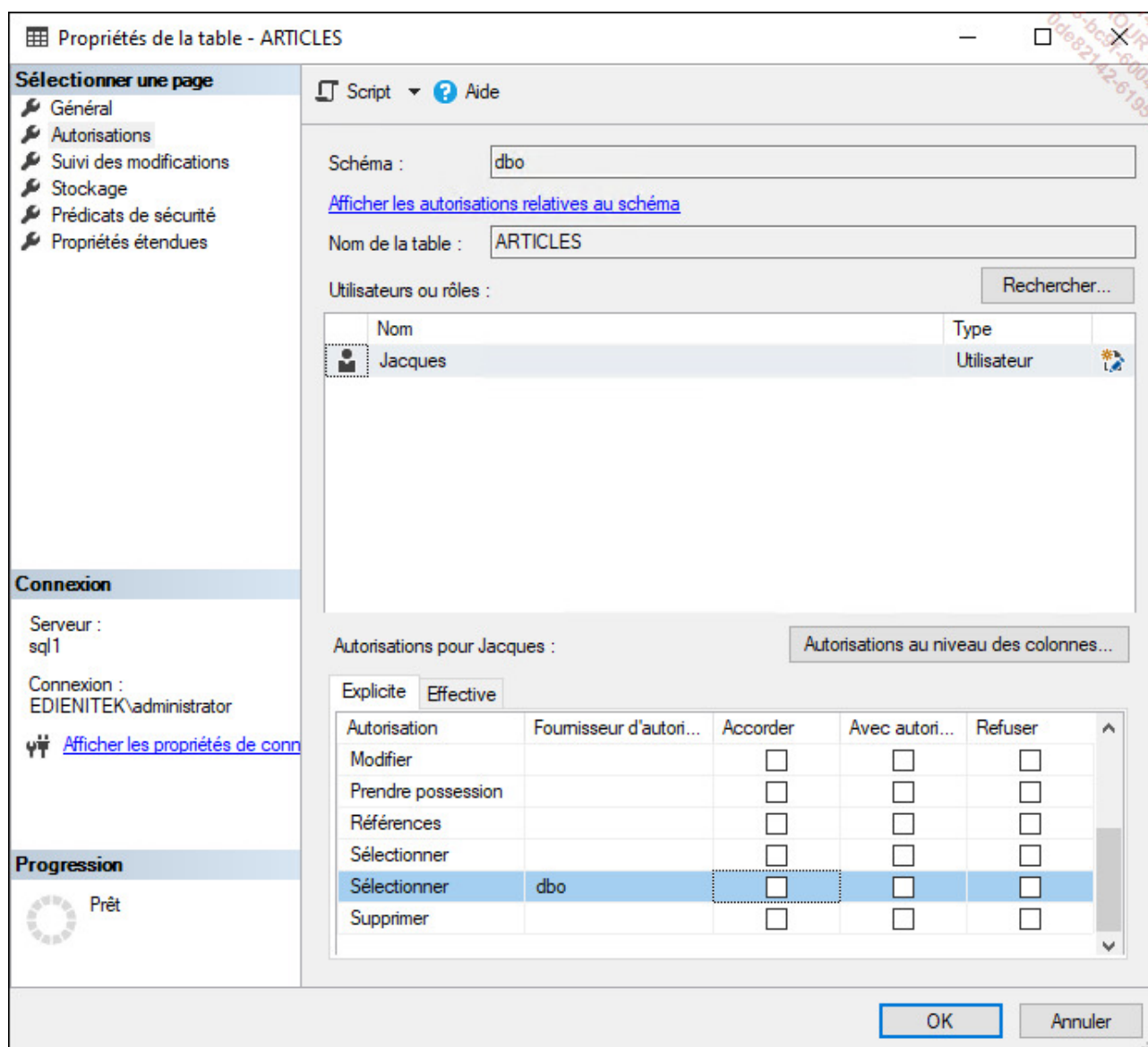
Lorsqu'une permission a été accordée, il est possible de retirer cet accord. Il n'est pas possible de retirer l'utilisation d'une permission à une entité de sécurité si cette permission n'a pas été accordée à cette même entité de sécurité.

SQL Server Management Studio

C'est par l'intermédiaire de la fenêtre des propriétés de l'entité de sécurité ou bien de l'objet, qu'il est possible de retirer une permission qui a été accordée précédemment.

Exemple

À partir des propriétés de la table `dbo.ARTICLES`, l'utilisateur Jacques se voit retirer la possibilité d'effectuer des requêtes de type **SELECT** sur cette table.



Transact SQL

```

REVOKE [GRANT OPTION FOR ]
    { ALL [PRIVILEGES] | permission[(colonne[,...])] [...] }
    ON object [(colonne [,...])]
    { FROM | TO } utilisateur [...]
    [ CASCADE ]
  
```

GRANT OPTION FOR

Avec cette option, il est possible de retirer simplement le privilège d'administration du

privilège.

permission

Permet de préciser la ou les permissions concernées par le retrait. Comme pour l'accord, il est possible de préciser les colonnes concernées par le retrait, mais la gestion d'un tel niveau de droit est lourde.

objet

Il faut préciser le nom complet de l'objet au sein de la base de données, c'est-à-dire nomSchema.nomObjet.

FROM, TO

Ces deux termes sont synonymes. L'usage habituel consiste à utiliser **TO** lors de l'accord d'un privilège et **FROM** lors du retrait d'un privilège.

utilisateur

Il s'agit très exactement de l'entité de sécurité à qui le privilège est retiré. Cette entité de sécurité est le plus souvent un utilisateur, mais il peut s'agir d'un rôle par exemple.

CASCADE

Lors du retrait d'une permission accordée avec le privilège d'administration, cette option permet de cascader le retrait, c'est-à-dire d'effectuer le retrait de la permission à toutes les entités de sécurité qui l'ont reçue de la part de celle qui est concernée initialement par le retrait.

Exemple

Dans l'exemple suivant, l'utilisateur Jacques se voit retirer la possibilité de lister des articles.

```
use [GESCOM]
GO
REVOKE SELECT ON dbo.ARTICLES TO [Jacques]
```

GO

c. Interdire

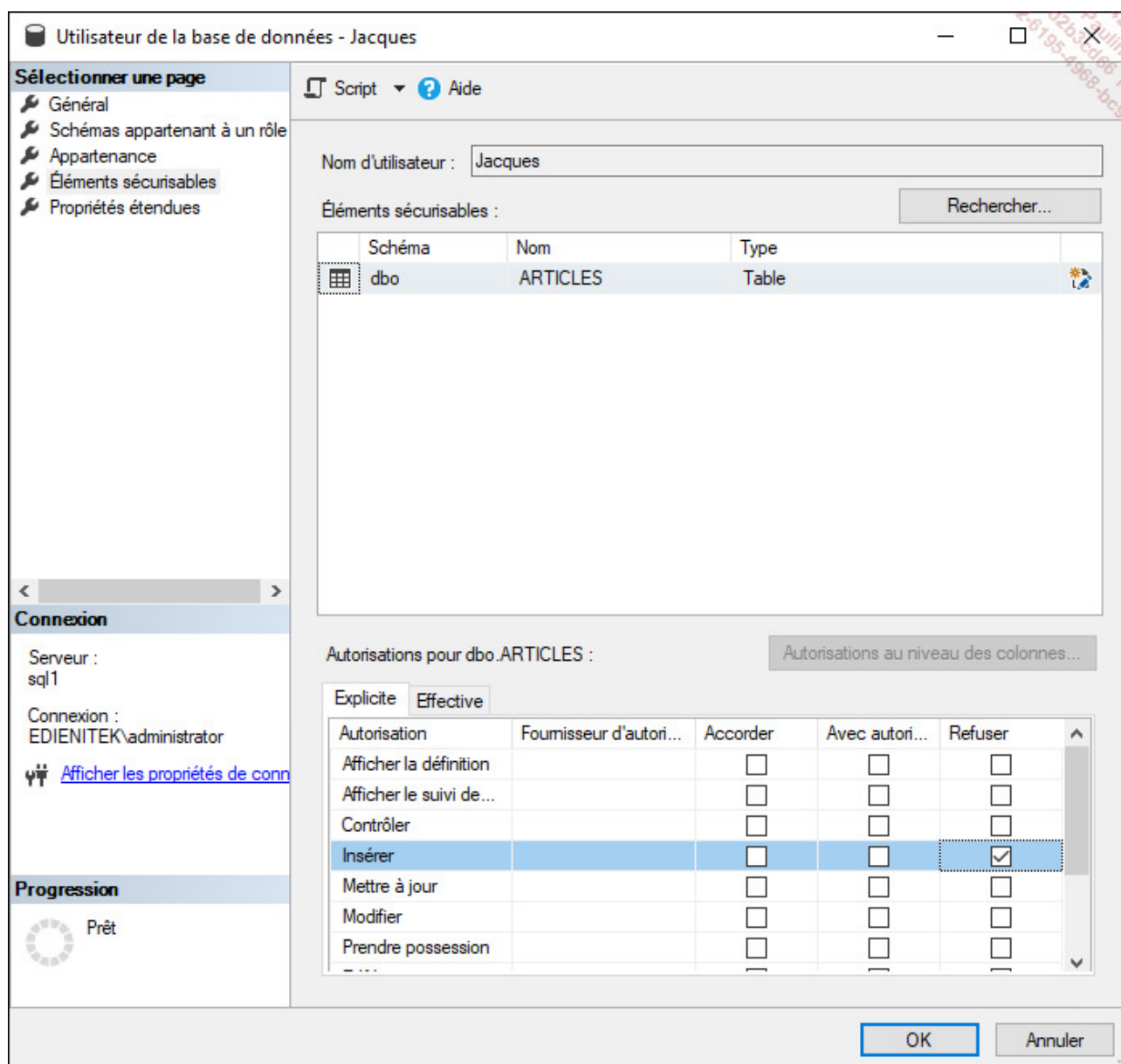
L'interdiction d'utiliser une permission d'utilisation d'un objet est une instruction plus forte que la suppression car elle peut être appliquée à une entité de sécurité, même si cette dernière n'a pas encore reçu de privilège d'utilisation de l'objet de façon directe ou non.

SQL Server Management Studio

Comme pour l'ajout ou bien le retrait, l'interdiction va être gérée par les fenêtres de propriétés, soit celle de l'entité de sécurité, soit par celle de l'objet. Le choix d'une solution par rapport à l'autre dépend du type de vue que l'on souhaite adopter et du type d'opération que l'on souhaite faire. Est-ce que l'on souhaite, par exemple, mieux contrôler l'utilisation d'une table, ou bien les actions que peut mener un utilisateur de base de données ?

Exemple

Dans l'exemple suivant, l'utilisateur Jacques se voit interdire la possibilité d'insérer des informations sur la table dbo.ARTICLES, par l'intermédiaire de la fenêtre de propriétés de l'utilisateur.



Transact SQL

```
DENY {ALL [PRIVILEGES]}permission[(colonne,[...])] [...]}
ON object [(colonne [...])]
TO utilisateur [...]
[CASCADE]
```

colonne

L'interdiction au niveau colonne est sans effet sur une autorisation accordée au niveau

de la table.

CASCADE

L'interdiction est transmise aux entités de sécurité qui ont reçu la permission d'utiliser ce privilège par l'intermédiaire de l'entité de sécurité à qui l'utilisation de ce privilège est interdite.

Exemple

Dans l'exemple suivant, l'utilisateur de base de données Jacques se voit interdire la possibilité d'insérer des informations dans la table `dbo.ARTICLES`.

```
use [GESCOM]
GO
DENY INSERT ON dbo.ARTICLES TO Jacques
GO
```

3. Droits au niveau de la base de données

Les privilèges accordés au niveau de la base de données donnent la possibilité à l'utilisateur qui reçoit ce privilège de réaliser des actions qui ont une portée sur l'ensemble de la base de données. Par exemple, le privilège `ALTER ANY USER` permet à un utilisateur de créer, supprimer et modifier n'importe quel compte de la base de données.

Au niveau base de données, il est possible d'accorder des privilèges à un utilisateur mais également à un schéma, un assembly et à un objet service broker.

La gestion de ces privilèges utilise soit les instructions Transact SQL `GRANT`, `REVOKE` et `DENY`, soit peut être effectuée de façon graphique à partir des propriétés de la base de données.

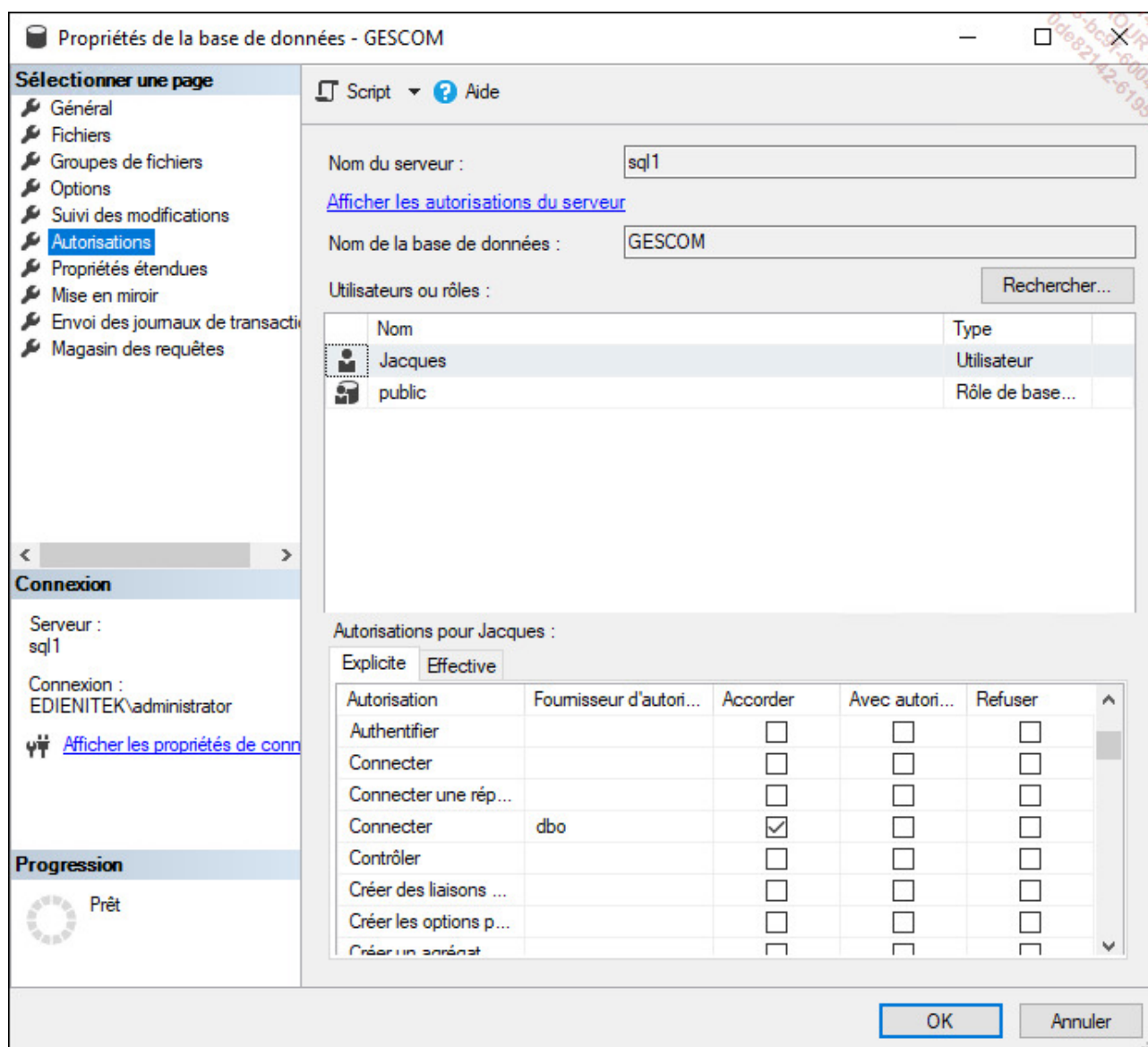


Seules les méthodes pour accorder des privilèges sont illustrées ici, les opérations de suppression et d'interdiction sont très semblables à celles illustrées lors des deux points précédents.

SQL Server Management Studio

Les droits relatifs à l'utilisation peuvent être attribués de la façon suivante depuis SQL Server Management Studio :

- Depuis l'explorateur d'objets, développer le nœud représentant la base de données concernée par la modification de privilège.
- Afficher les propriétés de la base en effectuant le choix **Propriétés** depuis le menu contextuel.



Il est également possible d'accorder ces permissions de façon graphique en allant modifier les propriétés des utilisateurs de base de données.

Transact SQL

La même opération peut être réalisée en Transact SQL à l'aide de l'instruction **GRANT**.

```
GRANT permissionBaseDeDonnées [ ,... ]
TO utilisateur [ ,...n ]
[ WITH GRANT OPTION ]
[ AS { group | role } ]
```

permissionBaseDeDonnées

La ou les permissions de base de données accordées.

Utilisateur

Compte d'utilisateur de base de données qui reçoit le privilège.

AS {group|role}

Contexte de sécurité utilisé pour pouvoir accorder le privilège.

Exemple

Dans l'exemple suivant, le privilège de sauvegarder la base de données GESCOM est accordé à l'utilisateur Jacques.

```
use GESCOM;
go
grant backup database to Jacques;
```

Les privilèges qu'il est possible d'accorder à ce niveau sont :

ALTER	CREATE DATABASE
ALTER ANY APPLICATION ROLE	CREATE DATABASE DDL EVENT NOTIFICATION
ALTER ANY ASSEMBLY	CREATE DEFAULT
ALTER ANY ASYMMETRIC KEY	CREATE FULLTEXT CATALOG
ALTER ANY CERTIFICATE	CREATE FUNCTION

ALTER ANY CONTRACT	CREATE MESSAGE TYPE
ALTER ANY DATABASE DDL TRIGGER	CREATE PROCEDURE
ALTER ANY DATABASE EVENT NOTIFICATION	CREATE QUEUE
ALTER ANY DATASPACE	CREATE REMOTE SERVICE BINDING
ALTER ANY FULLTEXT CATALOG	CREATE ROLE
ALTER ANY MESSAGE TYPE	CREATE ROUTE
ALTER ANY REMOTE SERVICE BINDING	CREATE RULE
ALTER ANY ROLE	CREATE SCHEMA
ALTER ANY ROUTE	CREATE SERVICE
ALTER ANY SCHEMA	CREATE SYMMETRIC KEY
ALTER ANY SERVICE	CREATE SYNONYM
ALTER ANY SYMMETRIC KEY	CREATE TABLE
ALTER ANY USER	CREATE TYPE

AUTHENTICATE	CREATE VIEW
BACKUP DATABASE	CREATE XML SCHEMA COLLECTION
BACKUP LOG	DELETE
CHECKPOINT	EXECUTE
CONNECT	INSERT
CONNECT REPLICATION	REFERENCES
CONTROL	SELECT
CREATE AGGREGATE	SHOWPLAN
CREATE ASSEMBLY	SUBSCRIBE QUERY NOTIFICATIONS
CREATE ASYMMETRIC KEY	TAKE OWNERSHIP
CREATE CERTIFICATE	UPDATE
CREATE CONTRACT	VIEW DATABASE STATE
ALTER ANY DATABASE EVENT SESSION	VIEW DEFINITION

4. Droits au niveau du serveur

SQL Server permet d'attribuer des privilèges au niveau du serveur. Ces privilèges ne sont pas accordés à un utilisateur de base de données mais à une connexion.

Comme pour les droits d'utilisation des objets et des instructions, il est possible d'accorder ces privilèges avec une option d'administration. Ainsi, la connexion qui possède ce droit peut accorder ce même droit à une ou plusieurs autres connexions.

Les différents droits qu'il est possible d'accorder au niveau serveur sont :

ADMINISTER BULK OPERATIONS	CONNECT SQL
ALTER ANY CONNECTION	CONTROL SERVER
ALTER ANY CREDENTIAL	CREATE ANY DATABASE
ALTER ANY DATABASE	CREATE DDL EVENT NOTIFICATION
ALTER ANY ENDPOINT	CREATE ENDPOINT
ALTER ANY EVENT NOTIFICATION	CREATE TRACE EVENT NOTIFICATION
ALTER ANY LINKED SERVER	EXTERNAL ACCESS ASSEMBLY
ALTER ANY LOGIN	IMPERSONATE ANY LOGIN
ALTER RESOURCES	SELECT ALL USERS SECURABLES
ALTER SERVER STATE	SHUTDOWN
ALTER SETTINGS	UNSAFE ASSEMBLY
ALTER TRACE	VIEW ANY DATABASE
AUTHENTICATE SERVER	VIEW ANY DEFINITION
CONNECT ANY DATABASE	VIEW SERVER STATE



Toutes les informations relatives aux privilèges accordés au niveau du serveur sont visibles au travers de la vue **sys.server_permissions**.



Pour pouvoir accorder de tels privilèges, il faut travailler sur la base Master.

5. Interroger les vues système

Il est possible, en interrogeant les vues système du dictionnaire et plus exactement celles traitant de la sécurité, de connaître les différentes autorisations et interdictions relatives aux différentes entités de sécurité.

Les principales vues sont :

- `sys.database_permissions` qui permet de lister les différentes autorisations d'utilisation des privilèges.
- `sys.database_principals` qui permet de lister les différents comptes de sécurité.

`sys.database_principals`

Cette vue recense toutes les entités de sécurité définies localement à la base de données. Les principales colonnes de cette vue sont :

- **name** : nom de l'entité de sécurité. Ce nom est unique à l'intérieur de la base de données.
- **principal_id** : identifiant numérique qui permet d'identifier de façon unique une entité de sécurité dans la base de données.
- **type** : code relatif au type du contexte de sécurité : utilisateur SQL, utilisateur Windows, rôle, groupe Windows...
- **type_desc** : description en toute lettre pour comprendre le type mentionné à la colonne précédente.
- **default_schema_name** : nom du schéma associé par défaut à l'entité de sécurité.

- **create_date** : date de création de l'entité de sécurité.
- **modify_date** : date de la dernière modification sur l'entité de sécurité.
- **owning_principal_id** : identifiant du contexte de sécurité qui possède le compte en question. Tous, à l'exception des rôles de base de données, doivent être possédés par **dbo**.
- **sid** : ou Security Identifier. Ce champ est renseigné si l'entité de sécurité est liée à un external.
- **is_fixed_role** : cet attribut est à 1 lorsque l'entité de sécurité est liée à un rôle fixe prédéfini sur le serveur.

Exemple

La requête suivante permet de connaître les comptes d'utilisateurs définis dans la base GESCOM.

```
select * from sys.database_principals
where type in ('S','U')
```

sys.database_permissions

Cette vue permet d'obtenir l'ensemble des informations relatives aux différentes permissions accordées dans la base. Il existe une ligne d'information pour chaque permission accordée à chaque entité de sécurité. Dans le cas de la gestion des permissions au niveau de la colonne, il existe une ligne d'information pour chaque colonne concernée par la permission.

Les principales colonnes de cette vue sont :

- **class** : code relatif à la classification du privilège. Est-ce un privilège de niveau base, qui porte sur l'utilisation d'un objet ou d'une colonne...
- **class_desc** : description de la classification du privilège.
- **major_id** : identifiant de l'objet sur lequel porte la permission.
- **minor_id** : identifiant de la colonne sur laquelle porte la permission.
- **grantee_principal_id** : identifiant du contexte de sécurité concerné par la permission.

- **grantor_principal_id** : identifiant du contexte de sécurité à l'origine de la permission.
- **type** : type du privilège.
- **permission_name** : nom du privilège concerné par la permission.
- **state** : code relatif à l'état de la permission.
- **state_desc** : description de l'état actuel de la permission (GRANT, REVOKE, DENY ou bien GRANT_WITH_GRANT_OPTION).

Exemple

À l'aide de la requête suivante, il est facile de connaître les permissions que possède l'utilisateur de base de données Jacques.

```
select *
from sys.database_permissions
where grantee_principal_id=user_id('Jacques');
```

Voici les autres vues système qu'il est possible d'interroger pour obtenir des informations relatives aux différents types d'autorisation accordés :

- sys.database_role_members : liste des bénéficiaires (principal_id) d'un rôle de base de données.
- sys.master_key_passwords : informations relatives à chaque clé principale de base de données créée avec la procédure sp_control_dbmasterkey_password.

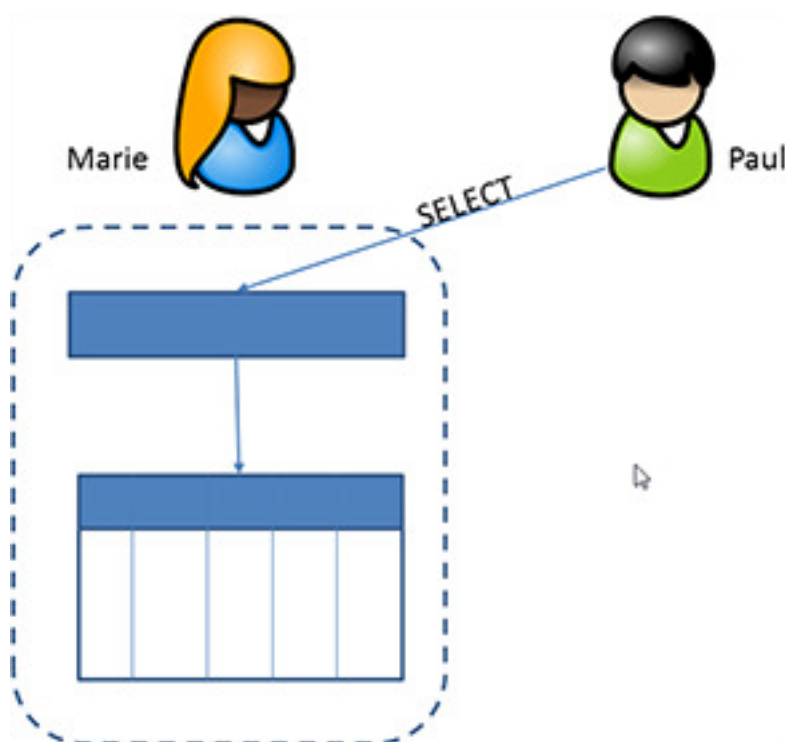
Contexte d'exécution

Le contexte d'exécution est directement lié à la connexion et à l'utilisateur de base de données associé. Le contexte d'exécution permet d'établir la liste des actions possibles et celles qui ne le sont pas. Cette liste est établie à partir des privilèges accordés à l'utilisateur soit directement, soit par l'intermédiaire de rôles.

Dans certains cas, il peut être nécessaire et souhaitable de modifier le contexte d'exécution afin de profiter de privilèges étendus, mais uniquement dans le cadre d'un script, d'une procédure ou d'une fonction.

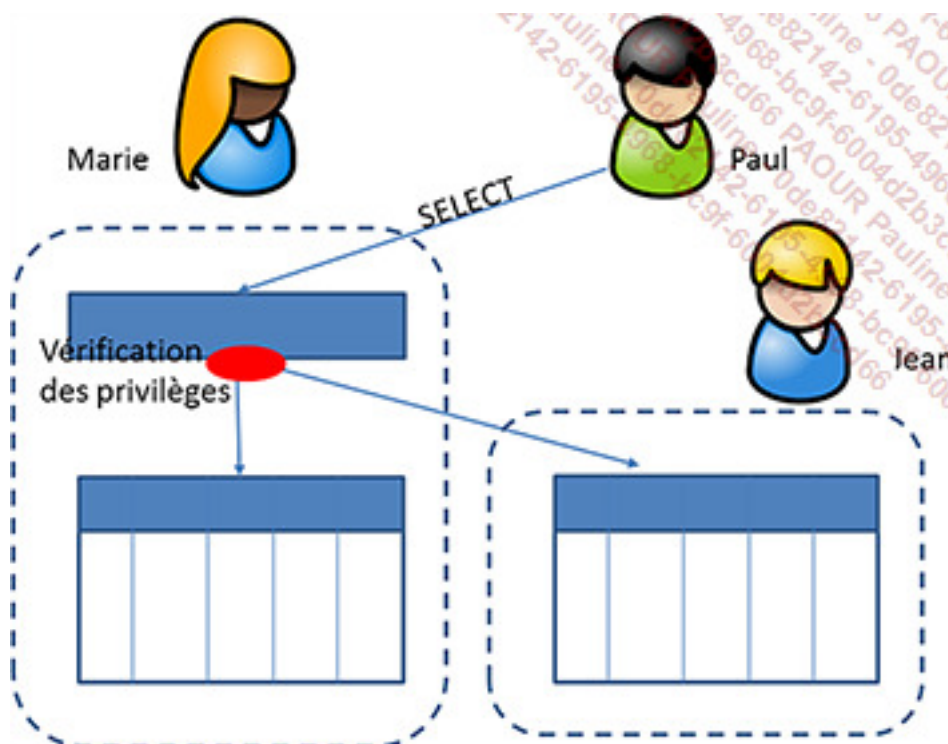
Associée au contexte d'exécution, il est nécessaire de bien comprendre la notion de chaîne de propriétés dans SQL Server.

Par exemple, l'utilisateur Paul a reçu de Marie le droit d'utiliser (**SELECT**) une vue que possède Marie. La requête **SELECT**, à l'origine de cette vue, fait référence à une table également possédée par Marie. Paul ne dispose d'aucun privilège sur cette table, pourtant il sera en mesure d'utiliser la vue sans soucis car les deux objets (vue et table) ont le même propriétaire.



Dans le cas où la vue de Marie que Paul interroge accède à une table dont Marie n'est pas

la propriétaire, alors les privilèges accordés à Paul sont vérifiés afin de savoir si Paul possède les privilèges nécessaires pour exécuter la requête.



EXECUTE AS

À l'aide de cette instruction, il est possible de demander à se connecter sur la base en utilisant une connexion différente de celle en cours. Cette instruction peut être exécutée de façon autonome dans un script Transact SQL ou bien être utilisée en tant que clause lors de la création d'une procédure, fonction ou trigger.

Pour les procédures, fonctions et triggers, la clause `EXECUTE AS` donne beaucoup de souplesse en termes de programmation et permet à un utilisateur de réaliser des actions pour lesquelles il ne possède pas de privilèges. Pour le développeur, le changement de contexte d'exécution permet également de garantir que la bonne exécution du code ne sera pas entravée par un problème de droits d'accès aux données.

Lors de l'exécution d'un script Transact SQL, l'instruction `EXECUTE AS` permet de réaliser par exemple de façon ponctuelle des opérations qui nécessitent des privilèges élevés alors que le reste du script ne le nécessite pas.

Le contexte d'exécution ainsi choisi doit être le plus étroit possible pour permettre simplement l'instruction spécifiée. Par exemple, il n'est pas concevable de se connecter en

tant que propriétaire de la base de données si l'opération demandée est une simple création de table. En exécutant une requête avec des privilèges trop importants, c'est une faille de sécurité qui est ouverte.

Il est donc possible de préciser soit le nom d'une connexion (**LOGIN**), soit celui d'un utilisateur de la base de données courante (**USER**). L'option **CALLER** n'est utile que lors de l'exécution d'une procédure ou d'un module pour spécifier que les privilèges à utiliser sont ceux accordés au contexte qui exécute le module.

Si l'option **WITH NO REVERT** est spécifiée lors de l'exécution de l'instruction **EXECUTE AS**, il n'est pas possible de quitter le nouveau contexte d'exécution par une instruction **REVERT** ou **EXECUTE AS**. Il est nécessaire de se déconnecter.

Syntaxe

```
EXECUTE AS {LOGIN|USER}='identifiant'|CALLER ;
EXECUTE AS {LOGIN|USER}='identifiant'|CALLER WITH NO REVERT ;
```

Exemple

Dans cette première partie, le contexte est préparé en définissant les comptes des utilisateurs Marie et Paul et des droits d'impersonnalisation sont accordés à Paul sur Marie.

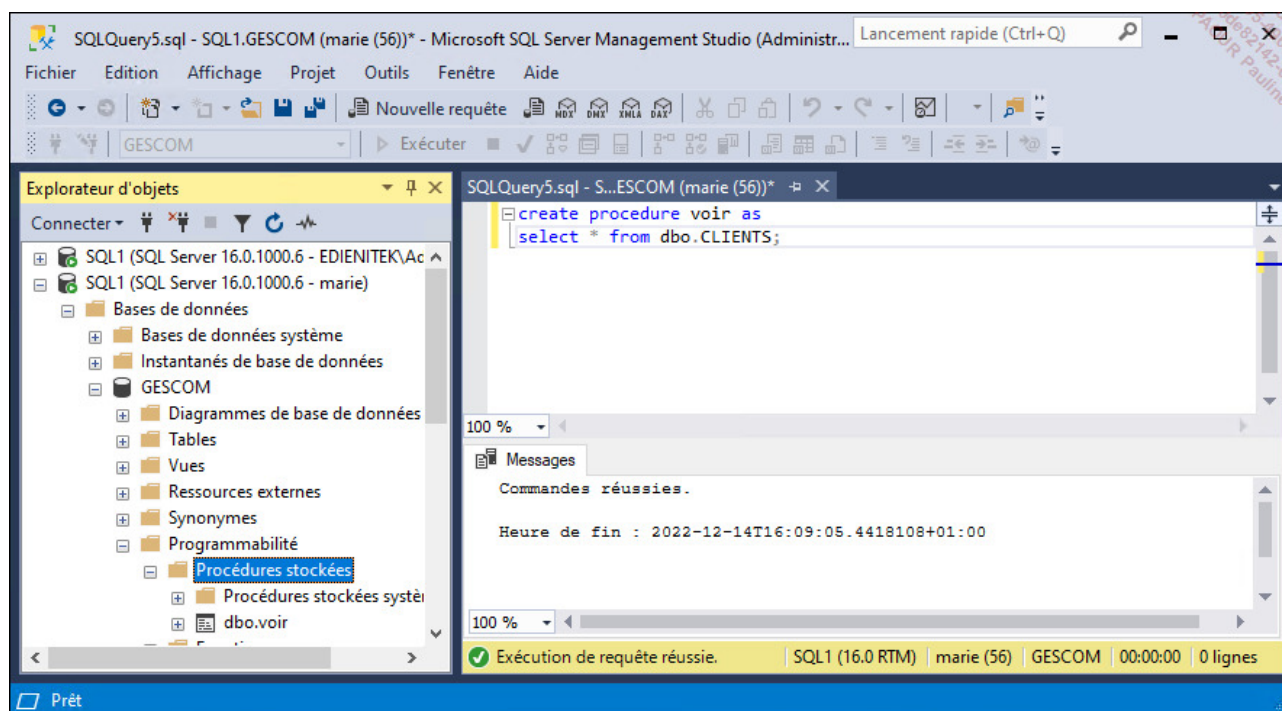
```
use master;
create login marie with password='Pa$$w0rd', default_database=gescom;
create login paul with password='Pa$$w0rd', default_database=gescom;
go
use GESCOM;
go
create user marie for login marie;
create user paul for login paul;
grant create procedure to marie;
exec sp_addrolemember 'db_datareader','marie'
grant select on schema:: dbo to marie;
grant alter on schema:: dbo to marie;
grant execute on schema:: dbo to marie;
go
grant impersonate on user::marie to paul;
```

L'utilisatrice Marie définit sur son schéma la procédure `voir` qui permet de visualiser le contenu de la table des CLIENTS.

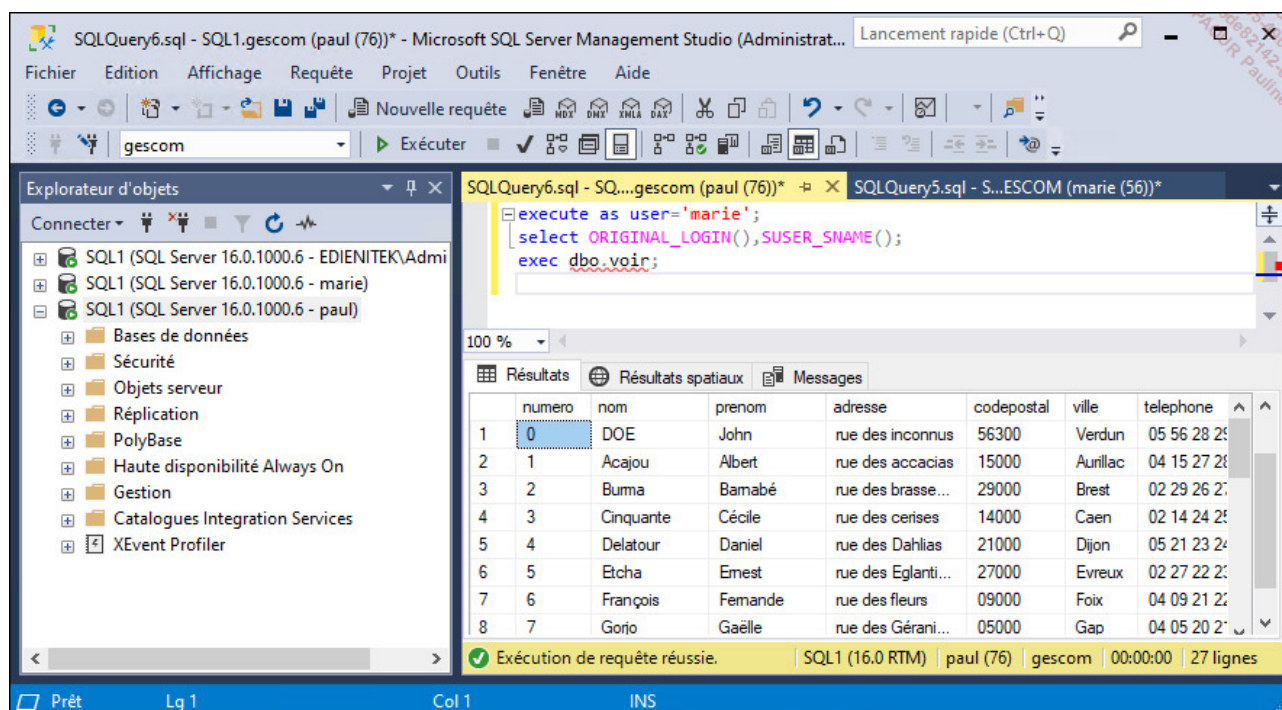
Avant d'exécuter le script ci-dessous, il est nécessaire de se connecter avec les identifiants de l'utilisatrice Marie.



Le nom de la connexion utilisée pour un script est indiqué dans la barre d'état de la requête.



Enfin, l'utilisateur Paul tente d'utiliser la procédure `voir` de Marie. Dans un premier temps, Paul demande à prendre l'identité de Marie avec l'instruction `EXECUTE AS`. Les identités sont vérifiées à l'aide des fonctions `ORIGINAL_LOGIN` et `SUSER_NAME`, puis l'utilisateur Paul peut exécuter sans difficulté la procédure `voir` créée par Marie.



Une autre version de la commande **EXECUTE AS** permet de préciser le contexte d'exécution d'une procédure, d'une fonction ou d'un déclencheur de base de données. La portée de ce changement de contexte d'exécution est limitée à l'exécution du module.

Syntaxe

procédure Ou fonction Ou Trigger

EXECUTE AS {CALLER|SELF|OWNER|'nomUtilisateur'}

...

CALLER

C'est le contexte de celui qui exécute le module qui est pris en compte dans le module.

SELF

C'est le contexte de l'utilisateur qui crée ou modifie le module qui est pris en compte.

OWNER

C'est le contexte du propriétaire du module qui est utilisé.

nomUtilisateur

Permet de préciser le contexte d'utilisateur à utiliser.

SETUSER

Contrairement à l'instruction `EXECUTE AS` qui ne modifie pas la connexion initiale au serveur, l'instruction `SETUSER` permet de changer de connexion au cours d'un script Transact SQL. C'est-à-dire que le contexte actuel d'exécution est clôturé et un nouveau contexte d'exécution est ouvert. Il n'est pas possible de revenir au premier contexte d'exécution.

Original_Login

Cette fonction permet de déterminer le nom exact de la connexion utilisée initialement pour se connecter au serveur. La connaissance de ce nom ne présente un intérêt que lorsque le contexte d'exécution diffère de la connexion initiale. Le contexte d'exécution peut être modifié par l'intermédiaire de l'instruction `EXECUTE AS`.

REVERT

Suite au changement de contexte d'exécution à l'aide de l'instruction `EXECUTE AS`, l'instruction `REVERT` permet de revenir au contexte d'exécution présent lors du changement de contexte avec `EXECUTE AS`.

Rôles

Les rôles sont des ensembles de permissions. Ces ensembles existent à trois niveaux distincts : serveur, base de données et application. Les rôles permettent de regrouper les droits et de gérer plus facilement les différents utilisateurs et les connexions. Il est en effet toujours préférable d'attribuer des droits à des rôles puis d'accorder les rôles aux utilisateurs. Avec une telle structure, l'ajout et la modification de droits ou d'utilisateur sont beaucoup plus simples.

Il est possible de définir un rôle comme un ensemble nommé de privilèges. Pour faciliter la gestion des droits, SQL Server propose des rôles prédéfinis aussi appelés fixes car il n'est pas possible d'ajouter ou bien de supprimer des privilèges dans ces rôles.

Ces rôles fixes sont définis à deux niveaux :

- Serveur
- Base de données

En plus de ces rôles fixes, il est possible de gérer des rôles. Il convient alors de définir un nom unique pour définir un rôle, puis d'accorder un ou plusieurs privilèges en respectant une procédure en tout point similaire à celle utilisée pour accorder des permissions à des utilisateurs. Ces rôles peuvent être définis à trois niveaux :

- Serveur
- Base de données
- Application

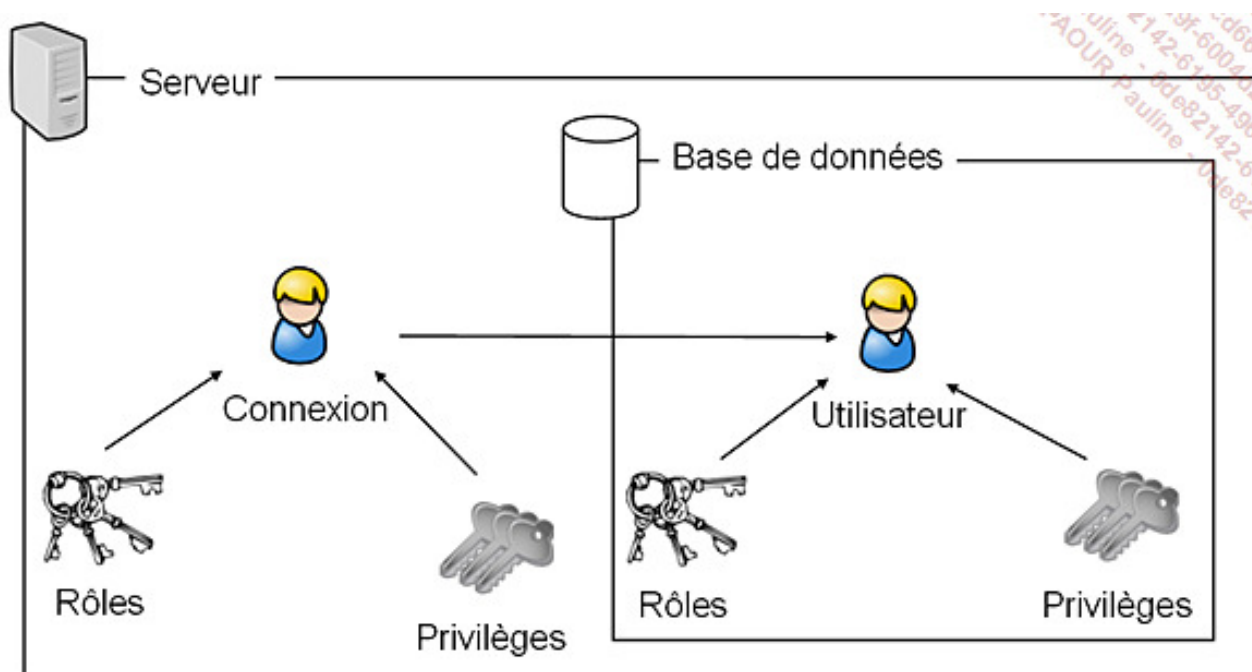
Les rôles permettent une gestion simplifiée des privilèges puisqu'il est possible ainsi de définir des profils types de privilèges, puis d'accorder à chaque utilisateur de base de données, un ou plusieurs profils de façon à lui fournir toutes les autorisations dont il a besoin pour travailler sur la base.

L'utilisateur dispose au final de l'ensemble des privilèges qui sont accordés :

- directement à la connexion utilisée,
- indirectement par l'intermédiaire d'un rôle de serveur accordé à la connexion,
- indirectement par l'intermédiaire des rôles accordés à l'utilisateur de base de

données,

- directement à l'utilisateur de base de données.



En complément de ces différents rôles, il existe le rôle **public**. Ce rôle est à part car tous les utilisateurs reçoivent le rôle public et ils ne peuvent pas s'y soustraire. Ce rôle est également particulier car il est possible de lui accorder, retirer ou bien interdire des permissions.

Toutes les modifications au niveau des permissions faites sur le rôle public sont valables pour tous les utilisateurs. Il n'est donc pas recommandé de travailler avec ce rôle mais, dans certains cas, il apporte une souplesse de gestion des permissions qui est intéressante.

1. Rôles de serveur

Ce sont des rôles prédéfinis qui donnent aux connexions un certain nombre de fonctionnalités. Leur utilisation facilite la gestion en cas de nombreux utilisateurs.

a. Les rôles prédéfinis

Les rôles de serveur sont au nombre de huit :

sysadmin

Exécute n'importe quelle opération sur le serveur, c'est l'administrateur du serveur.

serveradmin

Permet de configurer les paramètres au niveau du serveur.

setupadmin

Permet d'ajouter/supprimer des serveurs liés et d'exécuter certaines procédures stockées système telles que `sp_serveroptions`.

securityadmin

Permet de gérer les connexions d'accès au serveur.

processadmin

Permet de gérer les traitements s'exécutant sur SQL Server.

dbcreator

Permet de créer et modifier les bases de données.

diskadmin

Permet de gérer les fichiers sur le disque.

bulkadmin

Peut exécuter l'instruction `BULK INSERT` pour une insertion des données par blocs. L'intérêt de ce type d'insertion réside dans le fait que l'opération n'est pas journalisée et donc les temps d'insertion sont beaucoup plus rapides.



Seuls les membres du rôle **sysadmin** peuvent accorder un rôle de serveur à une connexion.

Le rôle **public** reçoit le privilège `VIEW ANY DATABASE`. Ainsi, les utilisateurs qui se connectent à SQL peuvent lister les bases définies sur l'instance. Ils ne pourront y accéder que si un compte d'utilisateur de base de données est mappé à leur connexion ou bien si l'utilisateur **guest** (invité) est activé au niveau de la base.

b. Créer un rôle de serveur

SQL Server donne la possibilité de créer ses propres rôles au niveau serveur. Ces rôles ne peuvent contenir que des privilèges disponibles au niveau serveur et vont bien entendu être accordés aux connexions de la même façon que les rôles prédéfinis.

Avant d'effectuer la création d'un rôle, il est nécessaire d'identifier précisément les autorisations que l'on souhaite y positionner. Pour identifier ces permissions, le plus simple est d'interroger la fonction `sys.fn_builtin_permissions` en lui passant le paramètre `SERVER`. Cette requête est illustrée par l'exemple présenté ci-dessous. Attention toutefois à ne pas mésestimer les rôles prédéfinis, car ils permettent de répondre à la plus grande majorité des cas d'utilisation.

Exemple

```
select * from sys.fn_builtin_permissions('server');
```

Le rôle est créé en utilisant l'instruction `CREATE SERVER ROLE`. Comme pour tous les objets SQL Server, les instructions `ALTER` et `DROP` permettent la modification et la suppression des rôles de serveur.

Syntaxe

```
CREATE SERVER ROLE nomRoleServer ;
```

Exemple

Création d'un rôle au niveau du serveur :

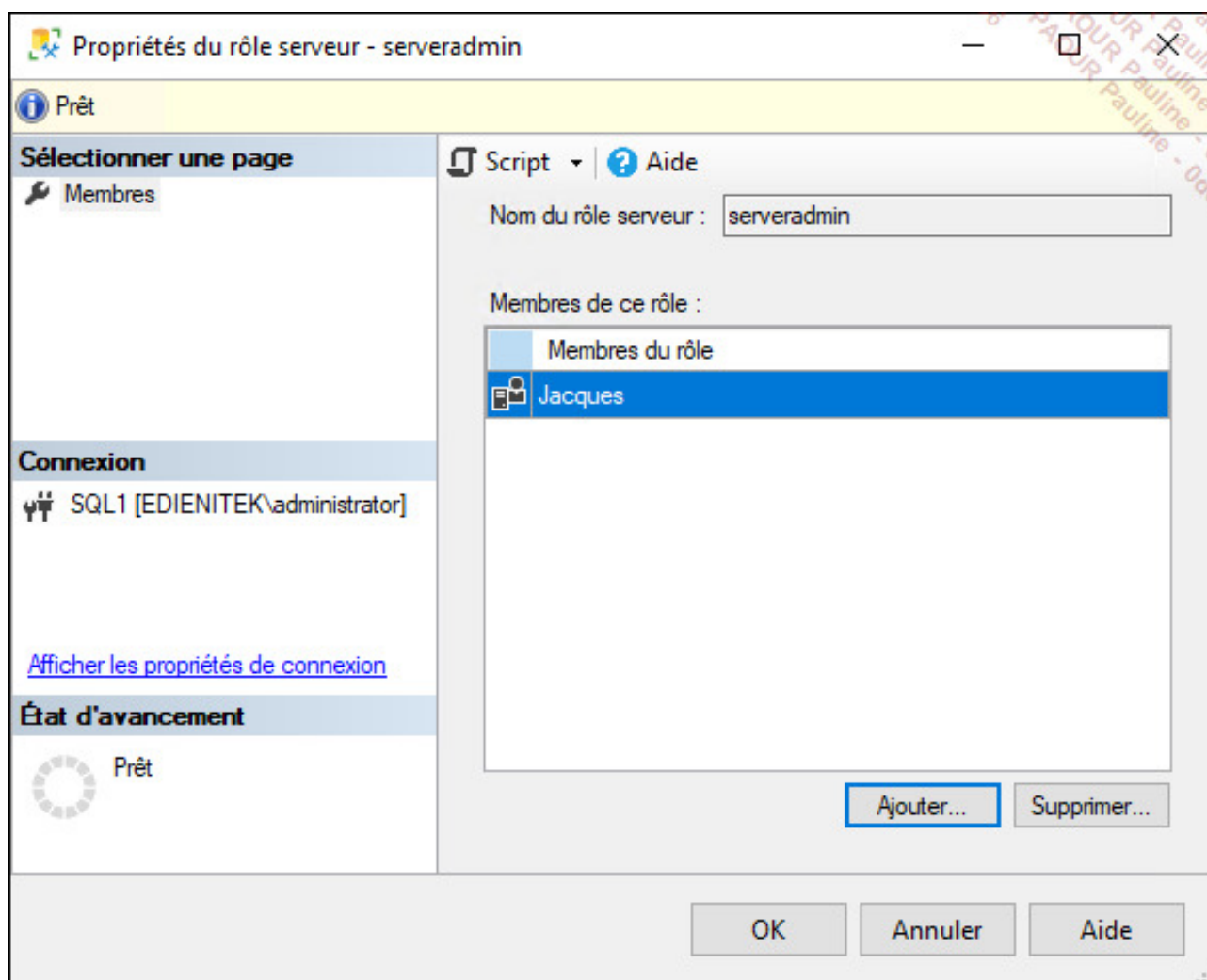
```
create server role roletest;
```

c. Accorder les rôles

Les procédures `sp_helpsrvrole` et `sp_helpsrvrolemember` permettent d'obtenir toutes les informations nécessaires sur les différents rôles fixes de serveur et leur attribution.

SQL Server Management Studio

La gestion des rôles fixes de serveur et plus exactement la gestion des connexions au sein du rôle est gérée depuis la fenêtre des propriétés du rôle.



Transact SQL

Pour ajouter une connexion à un rôle de serveur, il faut utiliser l'instruction **ALTER ROLE**. Le fait de faire bénéficier une connexion d'un rôle ou bien de retirer ce bénéfice est en effet considéré comme une modification.

Syntaxe

```
ALTER SERVER ROLE nomRoleServer ADD MEMBER nomConnexion ;
ALTER SERVER ROLE nomRoleServer DROP MEMBER nomConnexion ;
```

Exemple

```
ALTER SERVER ROLE serveradmin ADD MEMBER Jacques;
```

Dans ce second exemple, la connexion Jacques ne va plus bénéficier du rôle serveradmin.

```
ALTER SERVER ROLE serveradmin DROP MEMBER Jacques;
```



Les procédures stockées `sp_addsrvrolemember` et `sp_dropsrvrolemember` sont conservées pour des raisons de compatibilité mais ne sont plus à utiliser car Microsoft ne garantit pas leur présence dans les prochaines versions de SQL Server.

2. Rôles de base de données

Les rôles de bases de données permettent toujours de regrouper les différentes autorisations ou refus d'instructions ou d'objets et ainsi de faciliter la gestion des droits. Il existe des rôles prédéfinis et il est possible de définir ses propres rôles.

Les modifications sur les rôles sont dynamiques et les utilisateurs n'ont pas besoin de se déconnecter/reconnecter de la base pour bénéficier des changements.



Les rôles prédéfinis ne peuvent pas être supprimés.

Seuls les membres des rôles fixes de base de données **db_owner** et **db_securityadmin** sont à même de gérer l'appartenance des utilisateurs à un rôle fixe ou non de la base de données.

a. Le rôle public

Il s'agit d'un rôle prédéfini, bien particulier, car tous les utilisateurs de la base possèdent ce rôle. Ainsi, lorsque qu'une autorisation d'instruction ou d'objet est accordée, supprimée ou interdite à **public**, instantanément tous les utilisateurs de la base bénéficient des modifications.

Le rôle **public** contient toutes les autorisations par défaut dont bénéficient les utilisateurs de la base.

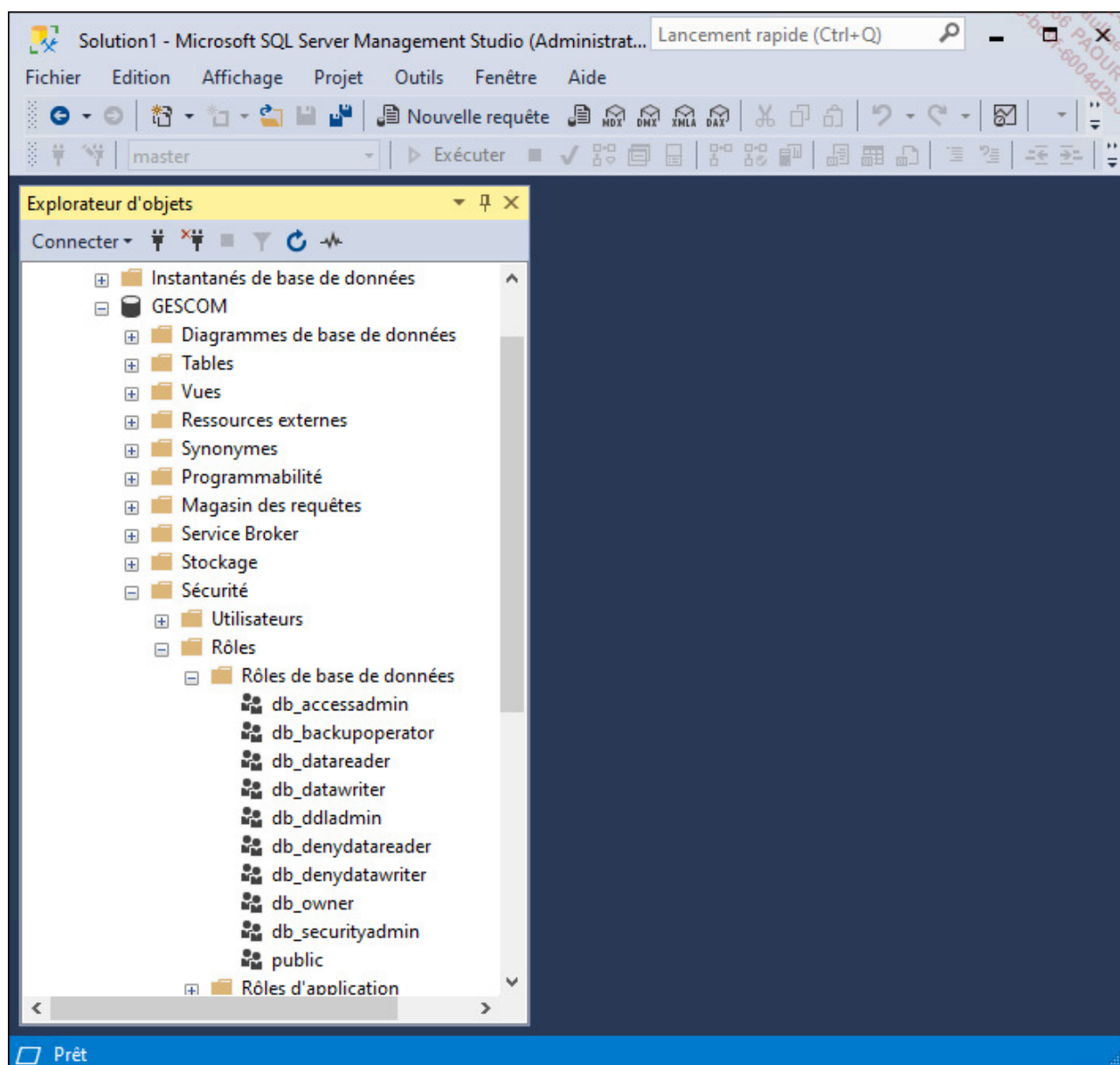
Il n'est pas possible d'attribuer des membres à ce rôle puisque tout le monde le possède implicitement. Ce rôle est présent dans toutes les bases du serveur, aussi bien les bases système que les bases utilisateur.



À sa création, un utilisateur ne dispose d'aucune autorisation sauf celles accordées à public.

b. Les rôles prédéfinis

Mis à part le rôle **public**, il existe des rôles prédéfinis dans la base de données.



db_owner

Ensemble de droits équivalents au propriétaire de la base. Ce rôle contient toutes les activités possibles dans les autres rôles de la base de données, ainsi que la possibilité d'exécuter certaines tâches de maintenance et de configuration de la base. Tous les membres de ce groupe vont créer des objets dont le propriétaire est **dbo**. Il s'agit du propriétaire par défaut de tous les objets.

db_accessadmin

Permet d'ajouter ou de supprimer des utilisateurs dans la base de données. Ces utilisateurs correspondent à des connexions SQL Server ou bien à des groupes ou utilisateurs Windows.

db_datareader

Permet de consulter (**SELECT**) le contenu de toutes les tables de la base de données.

db_datawriter

Permet d'ajouter (**INSERT**), modifier (**UPDATE**) ou supprimer (**DELETE**) des données dans toutes les tables utilisateur de la base de données.

db_ddladmin

Permet d'ajouter (**CREATE**), modifier (**ALTER**) ou supprimer (**DELETE**) des objets de la base de données.

db_securityadmin

Permet de gérer les rôles, les membres des rôles, et les autorisations sur les instructions et les objets de la base de données.

db_backupoperator

Permet d'effectuer une sauvegarde de la base de données.

db_denydatareader

Interdit la visualisation des données de la base.

db_denydatawriter

Interdit la modification des données contenues dans la base.



Tous les rôles prédéfinis sont présents dans toutes les bases système.

c. Les rôles de base de données définis par les utilisateurs

Il est possible de définir ses propres rôles afin de faciliter l'administration des droits à l'intérieur de la base.

Le rôle SQL Server va être mis en place lorsque plusieurs utilisateurs Windows souhaitent effectuer les mêmes opérations dans la base et qu'il n'existe pas de groupe Windows correspondant ou bien lorsque des utilisateurs authentifiés par SQL Server et des utilisateurs authentifiés par Windows doivent partager les mêmes droits.

Pour les rôles définis au niveau base de données, il est possible de savoir quels utilisateurs en bénéficient, en demandant l'affichage de la fenêtre des propriétés du rôle depuis SQL Server Management Studio.

Nouveau rôle de base de données

Script ? Aide

Sélectionner une page

- Général
- Éléments sécurisables
- Propriétés étendues

Connexion

Serveur : sql1

Connexion : EDIENITEK\administrator

[Afficher les propriétés de connexion](#)

Progression

Prêt

Nom du rôle : role_livre

Propriétaire : ...

Schémas appartenant à ce rôle :

Schémas appartenant à un rôle

- ☐ db_accessadmin
- ☐ dbo
- ☐ schemaexemple
- ☐ sys
- ☐ db_owner
- ☐ db_backupoperator

Membres de ce rôle :

Membres du rôle

- ☐ marie
- ☐ paul

Ajouter... Supprimer

OK Annuler

Les rôles peuvent être accordés soit directement à un utilisateur, soit à un autre rôle. Cependant, l'imbrication répétée des rôles peut nuire à la performance. De plus, il n'est pas possible de créer des rôles récursifs.



L'utilisation de rôles peut paraître parfois lourde lors de la création mais facilite grandement les évolutions qui peuvent intervenir (modification des autorisations, des utilisateurs).



Pour définir un rôle il faut être membre du rôle **sysadmin**, ou bien membre de **db_owner** ou **db_securityadmin**.

L'interrogation des tables système **sys.database_principals** et **sys.database_role_members** permet de connaître la liste des utilisateurs qui appartiennent à chaque rôle de base de données. L'exemple suivant illustre ce propos :

```
select utilisateurs.name, roles.name
from sys.database_role_members as drm
join sys.database_principals as utilisateurs
on drm.member_principal_id=utilisateurs.principal_id
join sys.database_principals as roles
on drm.role_principal_id=roles.principal_id
where roles.type='R';
```

Comme l'ensemble des opérations d'administration, il est possible de gérer les rôles de deux façons différentes.

d. Création d'un rôle de base de données

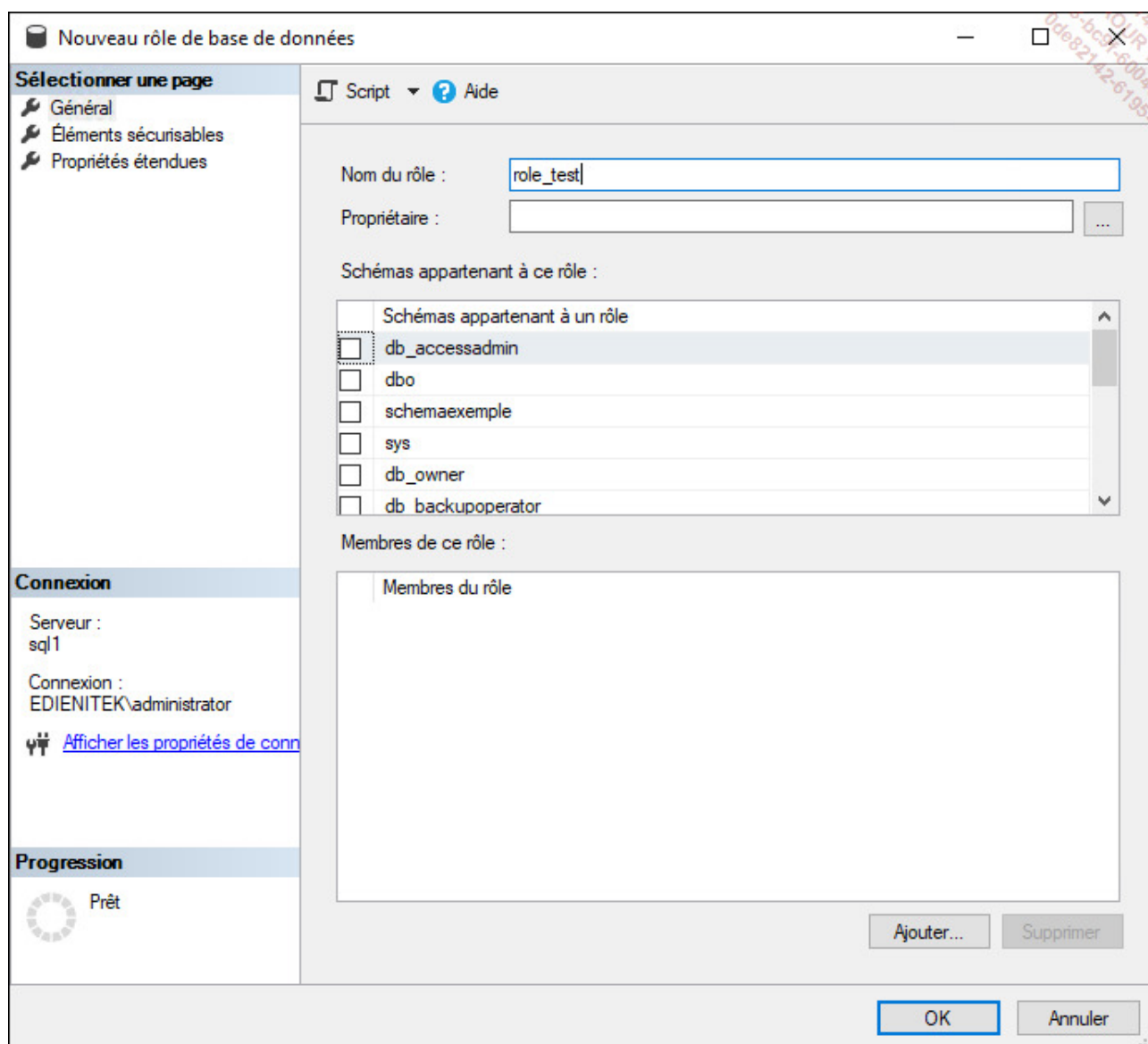
SQL Server Management Studio

La création des rôles de base de données est possible en réalisant les opérations suivantes :

- Depuis l'explorateur d'objets, se positionner sur la base de données utilisateur concernée par cet ajout.
- Se positionner sur le nœud **Sécurité - Rôles - Rôles de base de données**.
- Depuis le menu contextuel associé au nœud **Rôles de base de données**, faire le choix **Nouveau rôle de base de données**.

Exemple

Dans l'écran suivant, le rôle `role_test` est créé.



C'est par l'intermédiaire de la fenêtre présentant les propriétés du rôle qu'il est possible de le modifier.

Transact SQL

Syntaxe

```
CREATE ROLE nomRole
[ AUTHORIZATION propriétaire ]
```

nomRole

Nom du rôle qui vient d'être créé.

propriétaire

Il est possible de préciser un propriétaire pour le rôle.

Exemple

```
USE [GESCOM]
GO
CREATE ROLE [role_test]
GO
```

e. Gestion des membres d'un rôle

La gestion des membres d'un rôle s'effectue en Transact SQL par l'intermédiaire de l'instruction **ALTER ROLE**.

Syntaxe

```
ALTER ROLE nomRoleServer ADD MEMBER nomUtilisateur;
ALTER ROLE nomRoleServer DROP MEMBER nomUtilisateur ;
```

Exemple

```
USE [GESCOM] ;
GO
ALTER ROLE [role_test] ADD MEMBER [marie] ;
GO

USE [GESCOM] ;
GO
ALTER ROLE [role_test] DROP MEMBER [Pierre] ;
```

GO



Les procédures stockées `sp_addrolemember` et `sp_droprolemember` sont conservées pour des raisons de compatibilité, mais il est dorénavant recommandé de ne plus les utiliser.

f. Suppression d'un rôle

L'option de suppression est disponible depuis le menu contextuel associé au rôle à supprimer.

Transact SQL

Syntaxe

```
DROP ROLE nomRole
```

Exemple

```
USE [GESCOM] ;  
GO  
DROP ROLE [role_test] ;
```

3. Rôles d'application

Les rôles d'application sont des rôles définis au niveau de la base de données sur laquelle ils portent. Comme tous les rôles, les rôles d'application permettent de regrouper des autorisations d'objets et d'instructions. Cependant, les rôles d'application se distinguent car ils ne possèdent aucun utilisateur et ils sont protégés par un mot de passe.